Protocol compiler (protocol-compiler)

Contract, version 0.24.0

This module compiles a protocol specification into the finite protocol declaration the key-rate engine consumes. The specification has five parts: Alice's source as the transmitter module emits it, the receiver's measurement, a reduction chain in two segments (source links that replace Alice's signal ensemble, then receiver links that replace Bob's measurement), the public announcements with sifting and key rule, and the constraint entries; the declaration comes with an index map fixing the order of every alphabet and of the sectors. Operators on the measured system are lists of sector blocks against a declared sector map, every numerical condition is decided exactly on the declared binary64 numbers at no tolerance, and the compiler passes declared data through unchanged, constructing only the raw-key map, the index map and the measurement families that carry the weights of public choices. Its main results are three. A public choice of known bias, such as a basis choice, enters as a weight inside the measurement operators and never as a separate fraction: the weighted family stays a measurement, a weight that depends on no private setting is reproduced exactly as the probability of its choice value wherever the weighted family is the one emitted, and a weight that depends on the private setting of a measuring station, of either party, is applied after that station's reduction chain and commutes with it (Theorem 4.4). Passing the validator's eight checks establishes every target condition of a well-formed declaration, and every check except the feasibility check is complete, so a well-formed declaration is refused only when neither a default candidate nor a supplied witness state is verified to be feasible (Theorem 6.2). Gates at emission decide whether a well-formed declaration may back a key-length claim, and every gate failure has one fixed outcome: a refusal, or an emission flagged as not eligible to back such a claim (Theorem 9.4). The compiler decides what the record of the reduction chain and the transmitter's output make decidable and names the rest: the soundness of each reduction link, the licence that restricts the attacks considered, and the physical fidelity of the declared data are obligations it records and never establishes.

Grey text, here and throughout, is non-normative: it explains, and nothing in it is assumed or proved.

The design notes (non-normative) explain why the module is built as it is and draw its processing flowchart. The dependencies between the sections are drawn in the section dependency matrix, and the graph index opens every section's dependency graph. The statements digest holds every definition, assumption, result and requirement without the proofs. How to read a contract explains this layout and its links.

Contents

Main results 3

Assumptions at a glance 3

Notation 4

Terms 5

I Mathematics 16

1 The protocol declaration 16

2 Feasible states and licensed constraints 27

3 Specification, compilation map and typing 35

4 Public choices as measurement weights 52

5 Sifting, key computability and feasibility 71

6 The emitted validator 80

7 Consuming the reduction chain 95

8 The Band-A route 112

9 The emission gates 117

10 Assumptions 129

II Engine requirements 137

11 Requirements: the emitted declaration 137

12 Requirements: choice weighting 142

13 Requirements: validation, feasibility and the chain 147

14 Requirements: the emission gates 157

Interface 160

Claim index 165

Index of defined terms 170

References 171

Main results

The module's most important results, those its overview names as what it establishes, with every assumption each rests on, directly or through the claims it uses. Each links to its statement and proof; the statements digest holds them in full.

Theorem 4.4 Encoding of public choices in the measurement operators

A public choice of known bias enters as weights inside the measurement operators: the family stays a POVM, fixed weights are reproduced exactly as choice probabilities where the weighted family is emitted, and setting-dependent weights commute with the chain. A fraction outside them is refused.

Rests on A-COIN, A-LINK, A-SPEC

Theorem 6.2 Soundness, completeness and decidability of the validator

Passing the validator's eight checks establishes every target condition of a well-formed declaration, and every check but the feasibility check is complete, so a well-formed declaration is refused only when no default or supplied candidate is verified feasible. Every decision is finite and exact.

Rests on A-SPEC

Theorem 9.4 The emission gates and their outcomes

Each gate is decidable or a presence test, and each failure has one fixed outcome, a refusal or an emission flagged as backing no key length, never a default, dropped entry, changed source mode or licence the compiler writes. Passing asserts nothing about the apparatus or attack-set inclusion.

Rests on A-ATK, A-RHOM, A-SEQ, A-SPEC, A-SRC

Assumptions at a glance

Every assumption this contract states or its claims assume, with its category and the claims of this contract that assume it directly. An assumption of another contract links into that contract's PDF.

Assumption	Category	Title	Stated as	Assumed directly by
A-IDEAL	receiver	Ideal qubit receiver on the Band-A route	Assumption 10.8	Thm. 8.2
A-MEAS	receiver	Each measuring party's device as one POVM per round	reductions/ass: meas	Req. 13.10
A-ALICE-MEAS	protocol structure	Alice's declared measurement where no source certificate is demanded	Assumption 10.2	—

Assumption	Category	Title	Stated as	Assumed directly by
A-RHOM	protocol structure	Round homogeneity of the declaration	Assumption 10.3	Thm. 9.4
A-SPEC	protocol structure	Specification faithful to the apparatus	Assumption 10.1	Thm. 3.4, Thm. 3.7, Thm. 5.1
A-COIN	local randomness	Public choices as private coins of declared bias	Assumption 10.4	Thm. 4.4, Rem. 4.5, Rem. 4.8
A-ATK	supplied artefact	Attack-set licence	Assumption 10.7	Thm. 9.4
A-LINK	supplied artefact	Soundness of every reduction link	Assumption 10.6	Thm. 4.4, Thm. 7.5

Notation

The symbols of the repository's notation catalogue (spec/notation.md) that this contract uses, in the catalogue's order, with their meaning.

Symbol	Meaning
$\mathcal{H}(A)$	Hilbert space of the system label A
$\mathcal{L}(A)$	Bounded linear operators on $\mathcal{H}(A)$
$\mathrm{Pos}(A)$	Positive semidefinite operators
$S_=(A)$	Normalized states
$\mathbb{1}$, $\mathbb{1}_C$	Identity operator (contextual / named)
id	Identity channel
$\leq N$, $> N$, $= n$	Block indices for `\blk`
$\mathfrak{D}$	Protocol declaration $\big(A, \tau_A;\, Q, \mathbb{S};\, \{M_x\};\, \{F_v\};\, \mathfrak{P};\, a_A, a_B;\, \phi;\, r, \mathsf{h};\, \mathfrak{L}\big)$ — the tuple of meat-core/def:declaration: the party partition $\mathfrak{P}$ is a slot, and the marginal slot τ_A takes a state or the no-marginal value (trusted- vs untrusted-source mode)
$\mathbb{S}$	Sector index set of Q
Q	The declared sector-carrying measured system
$\mathfrak{P}$	The declared party partition: the readout maps π_{A}, π_{B} onto the alphabets $\mathcal{O}_{\mathsf{A}}$, $\mathcal{O}_{\mathsf{B}}$, with a party label for A and, for each sector s, a declared tensor factorisation of $\mathcal{H}(Q_s)$ with a party label per factor
$\mathcal{X}$	Outcome alphabet of the measurement on A — Alice's under the default partition
$\mathcal{V}$	The measuring parties' declared joint outcome alphabet — Bob's alone under the default partition

Symbol	Meaning
$\mathcal{C}_A$, $\mathcal{C}_B$	The alphabets of the first and second message of the transcript, which is the $L = 2$ case
$\mathcal{K}$	Transcript alphabet $\prod_{m=1}^{L} \mathcal{C}_m$, which at $L = 2$ is $\mathcal{C}_A \times \mathcal{C}_B$
$\widehat{C}$, $\widehat{\mathcal{C}}$	The acceptance-statistic register $\widehat{C}$ and its alphabet $\widehat{\mathcal{C}}$
$\widehat{\mathcal{C}}_{\text{key}}$	Key-live acceptance values
$\mathcal{Z}$, $\mathcal{Z}_\perp$	Key alphabet $\{0, \ldots, d_Z - 1\}$ and $\mathcal{Z} \sqcup \{\perp\}$
h	Declared key holder, $\mathsf{h} \in \{\mathsf{A}, \mathsf{B}\}$
$\mathfrak{L}$	Declared constraint list $\{(\hat{O}_k, \bowtie_k, \Theta_k)\}$
$\mathbf{\Phi}$	The test-statistics map $\mathcal{L}(AQ) \to \mathbb{R}^{\widehat{\mathcal{C}}}$
$\mathcal{W}$	Feasible set of sector-block-diagonal marginals: $\operatorname{tr} \omega = 1$ always; $\omega_A = \tau_A$ in trusted-source mode only; $\mathfrak{L}$ if declared (meat-core/def:feasible)
C, P	Channel input; probe input

Terms

The phrases this contract defines, and the terms of other contracts it uses, each with a one-line gloss and the definition that fixes it. In the text, a linked term opens its definition.

Term	Meaning	Defined in
announcement map	A map from one party's readout alphabet to the alphabet of what that party announces.	Definition 1.1
attack sequence	one channel per round whose output is the per-round system, with Alice's system where the adversary supplies it, and Eve's new memory	foundations/def:round-model
attack set	the channels whose adjoint maps every listed operator into the stated relation with its constant	foundations/def:round-model
Band A	The direct route: a finite measurement declared on one sector, an empty source segment and one identity receiver link, given as data, with empty slots, that delivers the declared measurement.	Definition 8.1
basic format	The format with one Alice sector, one measured system, Alice's measurement as a POVM, two messages neither reading the other, and no party-local entry.	Definition 1.1

Term	Meaning	Defined in
binary64 numbers	the finite values of the IEEE 754 binary64 format, each a dyadic rational	foundations/def: binary64
block form	The certificate form declared as positive blocks, one per sector, on matched sector structures of Alice's system and the virtual system.	transmitter/def: srcert
block list	one square complex matrix per block of a sector map, of that block's dimension, identified with their direct sum	foundations/def: sector-map
choice alphabet	The finite alphabet, with a declared order, of a public choice whose weight depends on the private setting of the station that measures the family.	Definition 4.1
choice-weighted family	The measurement whose elements are declared measurement elements multiplied by the declared weight of a public choice value.	Definition 4.1
compilation map	The map from a protocol specification to the protocol declaration it fills, slot by slot.	Definition 3.3
computed sequential sum	the left-to-right accumulation of a finite sequence from zero with every addition rounded by fl, entrywise, with no re-ordering and no re-association	foundations/def: declared-order
consumer	a module other than a supplier that reads declared data	foundations/def: declared-data
data sector	A sector of a reduction instance's target whose blocks are the datum's matrices.	reductions/def: reduction-instance
declared coarse-graining	A surjection from the raw click patterns onto the declared emitted alphabet, supplied as data by the protocol declaration.	receiver/def: coarse
declared data	the data an instance states and the data a producer emits as its own declaration, with the sets such data define	foundations/def: declared-data
declared datum	The datum the emission normalizer outputs: exactly admissible on the declared lattice, and carried entry by entry in the emitted target.	reductions/def: emitnorm

Term	Meaning	Defined in
declared model	The handed model with its datum replaced by the declared datum, as the instance's model map defines it.	reductions/def: emitnorm
declared order	a total order on a finite alphabet, fixed together with the alphabet	foundations/def: declared-order
declared value	a binary64 number stated as a datum, which denotes exactly that dyadic rational and approximates nothing	foundations/def: binary64
declarer	the party whose declaration declared data are: the instance that states them or the producer that emits them	foundations/def: declared-data
default party partition	The party partition in which Alice reads x and holds her system, and Bob reads v and holds every sector.	Definition 1.5
defects	the entrywise maximum moduli of the differences that the predicates of data name: the Hermiticity defect and the completeness, column, trace and sum residues	foundations/def: tolerance
ensemble form	The trusted-source form of Alice's measurement slot that holds positive blocks summing to the marginal, and denotes a POVM-form declaration.	Definition 1.1
ensemble presentation	The certificate form that declares only the weighted virtual ensemble in sector blocks and defines the certificate from it.	transmitter/def: srcert
entrywise enclosure	a Gaussian-rational Hermitian centre together with symmetric non-negative rational radii that bound, entry by entry, the modulus of the difference between a matrix and that centre	foundations/def: exact-reference

Term	Meaning	Defined in
exact island	the computations of the engine, its instances' declarations included, by which a consumer of data or the declarer about to declare them decides whether they satisfy the conditions their declaration states, a declarer computes and decides the reference of a measurement family, a consumer's replacement of an emitted scalar bound is decided to lie on the bound's weakening side, a producer constructs the operator data it emits, the acceptance side of the optimisation is constructed and certified, or a lower bound on a linear minimum over the feasible side of the optimisation is certified	foundations/def: exact-island
exactly admissible	said of a declared operator family whose sector map agrees with the given one, whose blocks are exactly Hermitian and positive semidefinite, and whose sum in the declared order equals the constant times the identity on every block and rounds nowhere; the constant is one where none is named	foundations/def: exact-admissibility
factor	One entry of a factorised sector map: a system tag, a party label and a sector map of its own.	Definition 1.1
factor enclosure	a matrix box of columns together with an exact positive diagonal core, such that the matrix is the product of a matrix L the box contains, the core and the adjoint of L	foundations/def: factor-enclosure
feasible set	The set of normalised block-list states on A and the measured system that meet the declared marginal and every constraint entry, each read exactly.	Definition 2.1
gate outcome table	The table that assigns to each gate failure its outcome, a refusal or an emission that is not key-length-eligible, or none for the two certificate-absent rows, the refusal taking precedence where a declaration meets both kinds of row.	Definition 9.3

Term	Meaning	Defined in
general	Said of a constraint entry declaring a Hermitian matrix on Alice's system as its right-hand side.	Definition 2.5
homogeneous	Said of a constraint entry, not declared party-local, whose right-hand side is a real scalar; in trusted-source mode it denotes that scalar times the marginal.	Definition 2.5
index map	The emitted choice of a bijection from each alphabet and from the sector set onto an initial segment of the non-negative integers, through which every emitted list is indexed.	Definition 3.3
instance	the party that fixes one protocol's device data, protocol data and run configuration	foundations/def: declared-data
interface conditions	The conditions (S1) to (S4) a source segment satisfies: descriptor match, index preservation, the source identity, and the slots each link carries.	transmitter/def: source-segment
interface conditions on a chain	The conditions a reduction chain presented to the compiler must satisfy, on each of its two segments and between them.	Definition 7.1
joint alphabet	The product of the outcome alphabet and the settings, ordered with the outcome varying fastest.	reductions/def: joint-family
joint family	The POVM over the joint alphabet whose element at an outcome and a setting is the class weight times the conditional element of that outcome given that setting.	reductions/def: joint-family
joint sector	A pair of an Alice sector and a sector of the measured system; the joint sectors are ordered with the Alice sector slowest.	Definition 1.1
key-length-eligible	An emitted declaration that may back a key-length claim: every check passes, the source-replacement licence record is present and re-decided where required, and a non-empty constraint list is licensed and carries an attack-set licence record.	Definition 9.1

Term	Meaning	Defined in
key-live	Said of an acceptance value that some announcement class passing sifting maps to.	Definition 1.1
lattice exponent	an integer k with $1 \leq k \leq 53$	foundations/def: binary64
licence record	The record of the attack-set licence a link carries exactly when its constraint list is non-empty.	reductions/def: link
licensed for the declared source mode	Said of a constraint list every entry of which is in the form the declared source mode admits, or is party-local.	Definition 2.5
lifted operator	The operator on the whole measured system that a scoped entry's operator denotes: the identity on every other factor.	Definition 1.1
marginal slot	The slot of a declaration that holds Alice's marginal state, or the value marking untrusted-source mode.	Definition 1.1
matrix box	a matrix of Gaussian-rational centres together with non-negative rational radii that bound, entry by entry, the modulus of the difference between a rectangular complex matrix and that centre	foundations/def: factor-enclosure
measured factor	A factor tagged Q: a system one party measures, with its own sector map, alphabet and family.	Definition 1.1
measured system	The quantum system the measuring parties receive in one round, and no other system; Bob's alone where he alone measures.	reductions/def: protocol
measurement channel	The channel that measures a POVM on a quantum input and writes the outcome into a classical register.	reductions/def: msm
measuring parties	The parties a declaration names as measuring; their measurements are the factors of the round's measurement channel.	reductions/def: protocol
message	One entry of a transcript protocol: a sender, a finite alphabet and a total table.	Definition 1.1
module boundary	the passage of declared data from their declarer to a consumer	foundations/def: declared-data

Term	Meaning	Defined in
obligation record	What a theorem-supplied source link carries in place of a decided identity: its kind, the soundness claim, the assumptions the obligation carries, and the family member its descriptors are.	transmitter/def: source-link
outcome alphabet	The finite ordered set of device outcomes that a private setting is joined to.	reductions/def: joint-family
party partition	The readout maps of the two parties, with a party label for Alice's system and for each tensor factor of every sector.	Definition 1.1
party-local	Said of a scoped constraint entry, tagged so, whose condition holds against the marginal of the state on the systems other than its factor.	Definition 2.5
party-local form	The form of a constraint entry scoped to one factor of a product measured system: tagged party-local and read as an operator inequality against the marginal actually present.	reductions/def: licensed
POVM form	The form of Alice's measurement slot that holds the POVM itself.	Definition 1.1
producer	a module other than a supplier that emits numerical data, constructed from declared data it read, to another module as its own declaration	foundations/def: declared-data
protocol declaration	The finite object the compiler emits and the key-rate engine consumes: systems, sector map, marginal slot, measurement families, public maps, raw-key map and constraint list.	Definition 1.1
protocol specification	The compiler's input: a source part, a receiver part, a reduction chain, a public part and a constraint part.	Definition 3.1
public-choice alphabet	The finite alphabet of values of a per-round public choice of known bias, each value with a declared positive weight.	Definition 4.1
pure form	The certificate form declared as a rank-one projector, on the one-sector structure.	transmitter/def: srcert

Term	Meaning	Defined in
reachable-set statement	The set of reachable squashed states of a link, for its channel witness and the declared family of inputs.	reductions/def: link
readout map	A total map from the outcome pairs to one party's declared finite alphabet, saying what that party reads.	Definition 1.1
receiver segment	The list of receiver-stage typed links of a reduction chain.	reductions/def: chain
reduction chain	A pair of a source segment and a receiver segment of typed links.	reductions/def: chain
round model	n rounds of a per-round system S, with the attack sequences and attack sets below	foundations/def: round-model
round-homogeneous	one fixed list of objects describes every round, and every round behaves as they describe whatever the other rounds do	foundations/def: round-homogeneity
rounds nowhere	said of an accumulation every one of whose binary64 additions is exact	foundations/def: declared-order
scope	The measured factor that an entry of the constraint list is declared on.	Definition 1.1
sector map	a block kind together with a non-empty, ordered list of block identifiers, each with a dimension	foundations/def: sector-map
sector set	The finite ordered list of a reduction instance's target sectors, data and scalar, each with a declared dimension.	reductions/def: reduction-instance
sector structure	A decomposition of Alice's retained system into orthogonal sectors, each spanned by declared basis vectors.	transmitter/def: source-spec
setting alphabet	The finite alphabet of the private per-round setting of the station that measures the family, onto which the declared setting map sends each outcome.	Definition 4.1
setting component	The value of the measuring station's private setting that an outcome records, read by the declared setting map.	Definition 4.1

Term	Meaning	Defined in
setting-dependent form	The choice-weighted family of a measured system after its chain: each delivered element times a stochastic weight read through the measuring station's setting.	Definition 4.1
setting-independent form	The choice-weighted family whose weights are one declared number per choice value, the same in every round.	Definition 4.1
soundness condition of a party-local entry	The condition a party-local entry is sound under: every attack reaches its scoped factor through a channel on that factor alone whose adjoint, applied to the entry's operator, stands in the entry's relation to its scalar times the identity.	Definition 2.5
soundness registry	The closed list, emitted with every artefact, of admitted source-link kinds with their soundness claims and carried assumptions.	transmitter/def: source-segment
source descriptor	The transmitted system and signal ensemble a source link starts from.	transmitter/def: source-link
source family	The system and POVM a receiver-stage link starts from.	reductions/def: link
source link	A link of a reduction chain's source stage that maps one Alice-side ensemble to another with the same labels and weights.	transmitter/def: source-link
source model	A submodule's named definition of the signal ensemble as mathematical objects, possibly infinite dimensional, rather than as matrices.	transmitter/def: source-spec
source segment	The ordered, possibly empty list of source links leading from the virtual ensemble to the specified one.	transmitter/def: source-segment
source specification	The declared description of Alice's side of a protocol: alphabet, retained system with sectors, measurement, signal ensemble and source mode.	transmitter/def: source-spec

Term	Meaning	Defined in
source-replacement certificate	An operator on Alice's retained system and the virtual transmitted system whose marginal on her side is her declared marginal and which, measured on her side by her POVM, leaves the weighted virtual members, with retention attested.	transmitter/def: srcert
specified signal ensemble	The weights and signal states, one per letter, that Alice's source is specified to emit.	transmitter/def: source-spec
squashed sequence	The attack sequence followed, in every round, by the squashing channel on the measured system and the identity on everything else.	reductions/def: roundmodel
squashing relation	The relation in which the adjoint of a channel takes every target POVM element to the physical one and every carried operator to its prescribed physical operator.	reductions/def: squashing
sum in the declared order	the sequential sum of a family indexed by an alphabet, taken in the alphabet's declared order	foundations/def: declared-order
support of the weighting	The pairs of an outcome and a choice value whose weight under the declared stochastic matrix is positive.	Definition 4.1
system tag	The label A or Q of a factor, saying whether it is Alice's system or a measured system.	Definition 1.1
target descriptor	The transmitted system and signal ensemble a source link produces.	transmitter/def: source-link
target family	The target POVM of a reduction instance at a datum, assigned one sector block at a time from the datum and the scalar values.	reductions/def: reduction-instance
target POVM	The POVM on the squashed system that a squashing relation pulls back to the physical POVM.	reductions/def: squashing
theorem constructor	A named claim of a submodule that supplies a source link's channel for each member of a declared family of descriptors.	transmitter/def: source-link
total weight	The sum of the certified weights over all outcomes.	reductions/ wpfss/def:targets

Term	Meaning	Defined in
transcript protocol	The announcement slot of the general format: an ordered list of messages, each a function of its sender's readout and of the earlier messages.	Definition 1.1
transmitted system	The system Alice sends into the channel, carrying the specified signal ensemble.	transmitter/def: source-spec
trusted-source mode	The source mode in which Alice's retained system never reaches the adversary, with a declared marginal and, above dimension one, a certificate.	transmitter/def: source-spec
typed link	A link of a reduction chain, at the source stage or the receiver stage, carrying what is needed to re-check it.	reductions/def: link
untrusted-source mode	The source mode with no certificate and no marginal, in which the adversary may have prepared Alice's retained system.	transmitter/def: source-spec
virtual ensemble	The finite-dimensional ensemble the source segment starts from and the certificate is written against.	transmitter/def: source-segment
virtual transmitted system	The transmitted system of the virtual ensemble, where the source segment starts.	transmitter/def: source-segment
weighted members	The positive operators, weight times state, one per letter, through which an ensemble is declared and read.	transmitter/def: source-spec
well-formed declaration	A declaration, together with its index map, that meets the eight target conditions D(i) to D(viii).	Definition 1.5
well-formed specification	A protocol specification with finite alphabets and systems, total tables, exactly admissible declared families, the presented physical one included, normalised weights and Hermitian, tagged constraint blocks.	Definition 3.2

Part I
Mathematics

1 The protocol declaration

The object the compiler emits is a protocol declaration: finite systems, a sector map, Alice's marginal slot, the two measurement families, the public maps, the raw-key map and a typed constraint list, every operator on the measured system being given as the list of its sector blocks. In its general format the sector map is an ordered list of factors, Alice's system may carry sectors, each measured factor declares its own family and no entry of their product is declared, Alice's measurement may be declared in ensemble form, as positive blocks summing to her marginal, and the public discussion is a transcript of messages. Declared operator data are exactly admissible when five conditions hold of the declared numbers, one of them that every binary64 addition forming a completeness sum is exact, and the eight target conditions state when a declaration, with the index map that orders its alphabets and sectors, is well-formed. The consuming engine decides the same five conditions, and the declarations admitted here are a subset of those it admits (Remark 1.6).

Imports foundations/def:binary64, foundations/def:exact-admissibility, foundations/lem:exact-accumulation

A protocol declaration fixes the systems, sector map, marginal slot, both measurements, public maps, raw-key map and a typed constraint list, operators on the measured system being block lists. The general format adds factors, Alice sectors, an ensemble form, a transcript and scoped entries.

The key-rate engine consumes a protocol declaration. We state it here as this contract's own definition, with its conditions in Definitions 1.3 and 1.5, so that every theorem of this contract is proved from text inside it. Whether restricting a state to the block lists of the general format is sound is the subject of Remark 2.4. How an entry's form reaches the emitted declaration is Requirement 11.3.

Definition 1.1 (Protocol declaration)**.** A *protocol declaration* is a tuple

$$\mathfrak{D} = \Big(A,\ \tau_A;\ Q,\ \mathbb{S};\ \{M_x\}_{x\in\mathcal{X}};\ \{F_v\}_{v\in\mathcal{V}};\ \mathfrak{P};\ a_A,\ a_B;\ \phi;\ r,\ \mathsf{h};\ \mathfrak{L}\Big), \quad (1.1)$$

with components as follows. A and Q are finite-dimensional systems, $\mathcal{H}(Q) = \bigoplus_{s\in\mathbb{S}} \mathcal{H}(Q_s)$ with a finite index set $\mathbb{S}$ of sectors, sector projectors Π_s and dimensions $d_s = \dim \mathcal{H}(Q_s)$, $d_Q = \sum_s d_s$, $d_A = \dim \mathcal{H}(A)$. The sector map $\mathfrak{s}$ is the ordered list $\big((s, d_s)\big)_{s\in\mathbb{S}}$ of block identifiers with their dimensions, together with its block kind, a label naming what the identifiers mean (Definition "Sector maps and block-list operators" (foundations/def:sector-map)).

It is one object, carried with the declaration beside the slot $Q, \mathbb{S}$ of (1.1), and it is what fixes $\mathbb{S}$, its total order and $(d_s)_s$. The *marginal slot* holds either a state $\tau_A \in S_=(A)$, in trusted-source mode, or the value none, in untrusted-source mode. $\{M_x\}_{x\in\mathcal{X}}$ is a POVM on A, given as one matrix per x, since A carries no sector decomposition. The words "state" and "POVM" in the two sentences above name the intended objects, and they are also exactly the conditions this contract decides: what a declaration is required to satisfy is D(i)–D(viii) of Definition 1.5, every operator predicate of which is decided *exactly* on the declared binary64 numbers, together with the summation condition of Definition 1.3, without which a computed sum would not be the sum it is read as.

Operators on Q are block lists. An operator on Q is given as one array $F^{(s)} \in \mathcal{L}(Q_s)$ per $s \in \mathbb{S}$, against the sector map $\mathfrak{s}$, and that list is the operator: it is a block list against $\mathfrak{s}$, the operator it denotes is $\bigoplus_s F^{(s)}$, and the declaration holds no array larger than a single sector block. Where $|\mathbb{S}| = 1$, say $\mathbb{S} = \{s_0\}$, that one block *is* the whole of Q and is held and emitted as such, $d_{s_0} = d_Q$, as on the Band-A route (Theorem 8.2 (i)). Sector block-diagonality is therefore a property of the *type*: there is no off-sector entry an operator of this form could carry, and correspondingly nothing to verify. So $\{F_v\}_{v\in\mathcal{V}}$ is a family of such block lists over a finite alphabet, one list per v; $F_v^{(s)}$ denotes its s-block. A state ω on AQ is likewise a list, one array $\omega^{(s)} \in \mathcal{L}(AQ_s)$ per s, and $\omega = \bigoplus_s \omega^{(s)}$ in the same sense. $\mathfrak{P}$ is a *party partition*: two total maps $\pi_{\mathsf{A}} : \mathcal{X} \times \mathcal{V} \to \mathcal{O}_{\mathsf{A}}$ and $\pi_{\mathsf{B}} : \mathcal{X} \times \mathcal{V} \to \mathcal{O}_{\mathsf{B}}$ onto declared finite alphabets, each a *readout map*, together with a party label for A and, for each s, a declared tensor factorization of $\mathcal{H}(Q_s)$ with a party label per factor. $a_A : \mathcal{O}_{\mathsf{A}} \to \mathcal{C}_A$ and $a_B : \mathcal{O}_{\mathsf{B}} \to \mathcal{C}_B$, each an *announcement map*, are written jointly

$$a(x, v) \coloneqq \big(a_A(\pi_{\mathsf{A}}(x, v)),\, a_B(\pi_{\mathsf{B}}(x, v))\big) \in \mathcal{K} \coloneqq \mathcal{C}_A \times \mathcal{C}_B, \tag{1.2}$$

$\phi : \mathcal{K} \to \widehat{\mathcal{C}}$ is the acceptance-statistic map, $r : \mathcal{X} \times \mathcal{V} \to \mathcal{Z}_\perp$ the raw-key map with $\mathcal{Z}_\perp = \mathcal{Z} \sqcup \{\perp\}$ and $\mathcal{Z} = \{0, \ldots, d_Z - 1\}$, $\mathsf{h} \in \{\mathsf{A}, \mathsf{B}\}$ the declared key holder, and $\mathfrak{L} = \{(\hat{O}_k, \bowtie_k, \mathrm{rhs}_k)\}_{k=1}^m$, $m \geq 0$, a finite constraint list, each entry an operator $\hat{O}_k$, a relation symbol $\bowtie_k$ and a right-hand side rhs_k (which symbols a well-formed declaration admits is D(vi) of Definition 1.5), with every $\hat{O}_k$ a block list against the same $\mathfrak{s}$, Hermitian block by block.

The right-hand side of an entry carries a declared tag, and the tag is part of the datum rather than a property measured of it. In trusted-source mode an entry is declared either homogeneous, carrying a real scalar c_k, from which the right-hand side it denotes is *constructed* as $\Theta_k \coloneqq c_k \tau_A$, or general, carrying a Hermitian matrix Θ_k on A. In untrusted-source mode only the scalar form exists and the right-hand side is a real number $\Theta_k = c_k$. So an entry is written $(\hat{O}_k, \bowtie_k, c_k)$ exactly when it is declared homogeneous or is untrusted-mode, and $(\hat{O}_k, \bowtie_k, \Theta_k)$ when it is declared

general (Definition 2.5). For these two forms the tag is the type of the right-hand-side datum, a real scalar or a Hermitian matrix, which is why Definition 2.5 calls them the untagged forms; the third form, party-local, exists only in the general format below and is declared by its tag (DX6). We write $\kappa = (c_A, c_B) \in \mathcal{K}$, a value of the joint announcement (1.2), for an announcement class and

$$\widehat{\mathcal{C}}_{\text{key}} := \big\{\hat{c} \in \widehat{\mathcal{C}} : \ \exists\, \kappa \in \phi^{-1}(\hat{c}) \text{ with } \varsigma(\kappa) = \text{pass}\big\} \tag{1.3}$$

for the *key-live* acceptance values. Here ς is the sifting predicate of the specification's public part (Definition 3.1 (d)), read as $\varsigma(\kappa) = \text{fail}$ on every class $\kappa \notin a(\mathcal{X} \times \mathcal{V})$, which no outcome pair realises; on the realised classes it is, under the hypotheses of Theorem 5.1, a witness of D(v) of Definition 1.5 (Theorem 5.1 (ii)). It is *not* a slot of (1.1), so (1.3) is the set of acceptance values to which some realised class passing sifting maps, fixed by the specification and not by the declaration; the key-live set of the consuming engine, Definition "The protocol declaration as the MEAT core reads it" (meat-core/def:declaration), takes the same convention on the classes no outcome pair realises.

The general format. Everything above is written for the format in which A carries one sector, one system is measured, Alice's measurement is declared as the POVM $\{M_x\}$, the public discussion is two messages neither of which reads the other, and no entry is declared party-local; we call it the *basic format*. It is the case $|\mathbb{T}| = 1$, $|\mathbb{F}| = 1$, POVM form, $L = 2$ of the following one, *entry for entry* on every component a derivation reads: at those values every such object defined in (DX1)–(DX6) is the object of the same name above, and nothing else is declared. On the party labels of the tensor factors of each sector block, which no derivation of this module reads and which the consuming engine's laboratory premise reads as naming the parties' laboratories, the general format is narrower than the basic one. In general a declaration is the tuple above with its slots read as follows.

(DX1) *(factorised sector map)* The sector map is an ordered list, each entry of which is a *factor*. Each factor carries a *system tag*, A or Q, a party label, and a sector map of its own, an ordered list of block identifiers with their dimensions; all factors carry one block kind. At most one factor is tagged A, and where there is one it is the first of the list; at least one factor is tagged Q. The factor tagged A fixes a finite ordered index set $\mathbb{T}$ and a decomposition $\mathcal{H}(A) = \bigoplus_{t\in\mathbb{T}} \mathcal{H}(A_t)$ into subspaces spanned by vectors of the declared basis of A, with projectors Π_t^A and dimensions d_{A_t}, $d_A = \sum_t d_{A_t}$; where no factor is tagged A, the system A is still present and is one Alice sector, $\mathbb{T} = \{t_0\}$ and $A_{t_0} = A$. A factor tagged Q is a *measured factor*, and the measured factors form the ordered set $\mathbb{F}$; factor f fixes a system Q_f with $\mathcal{H}(Q_f) = \bigoplus_{s_f\in\mathbb{S}^f} \mathcal{H}(Q_{f,s_f})$, projectors $\Pi_{s_f}^f$ and dimensions

d_{f,s_f}, and for $s = (s_f)_f$

$$Q = \bigotimes_{f\in\mathbb{F}} Q_f, \qquad \mathbb{S} = \prod_{f\in\mathbb{F}} \mathbb{S}^f, \qquad \mathcal{H}(Q_s) = \bigotimes_f \mathcal{H}(Q_{f,s_f}), \qquad \Pi_s = \bigotimes_f \Pi^f_{s_f}, \qquad d_s = \prod_f d_{f,s_f} \tag{1.4}$$

all products in the order of $\mathbb{F}$, and $\mathbb{S}$ ordered with the last factor fastest, a product identifier being the factors' identifiers joined in that order. Each pair $(t, s) \in \mathbb{T}\times\mathbb{S}$ is a *joint sector*; the joint sectors are ordered with t slowest, and $\mathcal{H}(AQ) = \bigoplus_{(t,s)} \mathcal{H}(A_t)\otimes\mathcal{H}(Q_s)$, the A index first inside a block. The party partition labels A with the party of the factor tagged A, where there is one, and otherwise with the party label it declares for A, as in the basic format; and it labels every tensor factor of $\mathcal{H}(Q_{f,s_f})$ with the party of f.

(DX2) *(Alice's data and ω are block lists)* τ_A, every operator of Alice's measurement slot (DX4) and the right-hand side Θ_k of a general entry are lists of one array on $\mathcal{H}(A_t)$ per $t \in \mathbb{T}$, and the list is the operator: $\tau_A = \bigoplus_t \tau_A^{(t)}$, $M_x = \bigoplus_t M_x^{(t)}$, $\Theta_k = \bigoplus_t \Theta_k^{(t)}$. A state ω on AQ is a list of one array $\omega^{(t,s)} \in \mathcal{L}(A_tQ_s)$ per joint sector, $\omega = \bigoplus_{(t,s)} \omega^{(t,s)}$. Block diagonality along $\mathbb{T}$ is a property of the type: no off-sector entry can be written. By itself this typing licenses nothing; whether restricting ω to such lists is sound is not settled by this definition.

(DX3) *(factorised family)* The outcome alphabet is the product $\mathcal{V} = \prod_{f\in\mathbb{F}} \mathcal{V}^f$ of the factors' ordered alphabets, in the product order, the last factor fastest. What is declared is one family of block lists $\{F^f_{v_f}\}_{v_f\in\mathcal{V}^f}$ per measured factor, against that factor's own sector map, together with the rule

$$F_v \coloneqq \bigotimes_{f\in\mathbb{F}} F^f_{v_f}, \qquad F^{(s)}_v = \bigotimes_{f\in\mathbb{F}} F^{f,(s_f)}_{v_f} \qquad \big(v = (v_f)_f,\ s = (s_f)_f\big). \tag{1.5}$$

No entry of a product is declared. That $\{F_v\}_{v\in\mathcal{V}}$ is then a POVM on Q, block diagonal for the product sector map with the blocks of (1.5), follows for the exact numbers the factors declare: a tensor product of Hermitian positive semidefinite operators is Hermitian and positive semidefinite, $\sum_v \bigotimes_f F^f_{v_f} = \bigotimes_f \sum_{v_f} F^f_{v_f} = \mathbb{1}_Q$ by multilinearity, and the tensor product distributes over the direct sums $F^f_{v_f} = \bigoplus_{s_f} F^{f,(s_f)}_{v_f}$.

(DX4) *(the two forms of Alice's measurement slot)* In *POVM form* the slot holds $\{M_x\}_{x\in\mathcal{X}}$. In *ensemble form*, which exists in trusted-source mode only, it holds a family $\{N_x\}_{x\in\mathcal{X}}$ of block lists along $\mathbb{T}$, every block Hermitian and positive semidefinite, with

$$\sum_{x\in\mathcal{X}} N_x^{(t)} = \tau_A^{(t)} \quad \text{and} \quad \tau_A^{(t)} \neq 0 \qquad \text{for every } t \in \mathbb{T}, \tag{1.6}$$

the sum formed entrywise in the declared order of $\mathcal{X}$ and rounding nowhere, and the second condition a comparison of declared entries with the datum 0. An ensemble-form declaration *denotes* the POVM-form declaration with the same remaining slots and

$$M_x^{(t)} \coloneqq \big(\tau_A^{(t)}\big)^{-1/2} N_x^{(t)} \big(\tau_A^{(t)}\big)^{-1/2} + \frac{\operatorname{tr} N_x^{(t)}}{\operatorname{tr} \tau_A^{(t)}} \big(\mathbb{1}_{A_t} - P_t\big), \tag{1.7}$$

where P_t is the projector onto the support of $\tau_A^{(t)}$ and the inverse square root is taken on that support and is zero off it. The operators (1.7) are defined and in general irrational; every statement made of an ensemble-form declaration is made of the declaration it denotes. For a POVM-form declaration in trusted-source mode put $N_x \coloneqq \tau_A^{1/2} M_x \tau_A^{1/2}$; for the declaration an ensemble form denotes, this returns the declared N_x.

(DX5) *(transcript protocol)* The announcement slot holds a *transcript protocol*, an ordered list of $L \geq 1$ entries, each a *message*, message m carrying a sender $\pi_m \in \{\mathsf{A}, \mathsf{B}\}$, a finite alphabet $\mathcal{C}_m$ and a total table $a_m : \mathcal{O}_{\pi_m} \times \mathcal{C}_1 \times \cdots \times \mathcal{C}_{m-1} \to \mathcal{C}_m$. The joint announcement function is *derived* from the tables by composition,

$$\kappa_m(x, v) \coloneqq a_m\big(\pi_{\pi_m}(x, v),\, \kappa_1(x, v), \ldots, \kappa_{m-1}(x, v)\big), \qquad a(x, v) \coloneqq \big(\kappa_m(x, v)\big)_{m=1}^{L} \in \mathcal{K} \coloneqq \prod_{m=1}^{L} \tag{1.8}$$

and the acceptance-statistic map ϕ, the sifting predicate ς of (1.3) and the holder's key map r_{h} of D(viii) of Definition 1.5 are maps on this $\mathcal{K}$. The two maps a_A, a_B of the basic format are the transcript protocol with $L = 2$, $\pi = (\mathsf{A}, \mathsf{B})$, $\mathcal{C}_1 = \mathcal{C}_A$, $\mathcal{C}_2 = \mathcal{C}_B$, $a_1 = a_A$ and $a_2(o, \kappa_1) = a_B(o)$. A message is a function of its sender's readout and of the earlier messages of the same round, and of nothing else. No measurement of a round depends on a message of that round, the measurements being the fixed families of (DX3) and (DX4); processing across rounds is not a round message.

(DX6) *(party-scoped entries)* Each entry of $\mathfrak{L}$ carries the label of one measured factor, its *scope* f_k, and its operator $\hat{O}_k$ is a block list against that factor's own sector map, Hermitian block by block. Unless the entry is declared party-local it denotes the *lifted operator* $\mathbb{1} \otimes \hat{O}_k \otimes \mathbb{1}$ on Q, with $\hat{O}_k$ on the tensor factor Q_{f_k}, whose s-block is $\mathbb{1} \otimes \hat{O}_k^{(s_{f_k})} \otimes \mathbb{1}$, and it is read in the forms stated above for that lifted operator. With one measured factor the scope is that factor and the lifted operator is $\hat{O}_k$ itself. An entry may instead be declared party-local, in

either source mode, carrying a real c_k; its reading is the third form of Definition 2.5.

What this module decides of the general format is Definition 1.3 for the operator data and D(i)–D(viii) of Definition 1.5 for the rest; that the family denoted by an ensemble form is a POVM is Theorem 5.1 (v).

Used by Rem. 1.2, Def. 1.3, Def. 1.5, Def. 2.1, Lem. 2.2, Rem. 2.3, Def. 2.5, Def. 3.1, Def. 3.3, Thm. 3.7, Def. 4.1, Rem. 4.9, Thm. 5.1, Thm. 5.5, Def. 6.1, Thm. 6.2, Def. 7.1, Thm. 7.5, Def. 9.1, Thm. 9.4, Ass. 10.4, Req. 11.1, Req. 11.2, Req. 11.4, Req. 11.8, Req. 13.11, meat-core/def:declaration · newly derived · implemented · *Code* certiqs_key. protocol_compiler.declaration:CandidateDeclaration

Deriving the party labels from the factor list is narrower than the basic format, which may label the tensor factors of one sector differently. Only the premises placing the laboratories read the labels; two parties inside one measured system need two measured factors.

Remark 1.2 (Party labels in the general format). The party labels that (DX1) of Definition 1.1 derives from the factor list are narrower than the party partition of the basic format at one measured factor: there a declared tensor factorization of $\mathcal{H}(Q_s)$ may carry different party labels on different tensor factors, while in the general format every tensor factor of one measured factor's block carries that factor's party. The narrowing is admitted and not repaired. No derivation of the compiler or of the consuming engine reads those labels — what the derivations consume is the readout maps π_{A}, π_{B} — and correspondingly no check compares a declared label with a derived one. The labels are read by the premises that place the parties' laboratories: the consuming engine's laboratory premise (Assumption "Each party's laboratory is sealed" (meat-core/ass:lab)) takes them, with the party label of A, as naming the parties' laboratories, and the source licence (Assumption "Retention of the system A in trusted-source mode" (transmitter/ass:src)) reads the party label of A; so the narrowing restricts which laboratory boundaries a general-format declaration can state, and a declaration that wants two parties inside one measured system declares two measured factors instead. So the general format coincides with the basic format entry for entry on every component a derivation reads, and on the party labels it is a restriction.

Uses Def. 1.1 · newly derived · accepted

Declared binary64 operator data are admitted by sector-map agreement, Hermiticity, positivity, completeness and a completeness sum that rounds nowhere, each decided exactly on the declared numbers. No tolerance is read and no ideal operator is inferred.

Of the five conditions of the definition, four are conditions on an operator family and one, (S), is a condition on the binary64 additions that form a completeness sum. Binary64 addition is not associative, so each such sum comes with the order in which it is accumulated (Remark "Dependence of a computed sum on its order" (foundations/rem:summation-order)).

Deciding every admission predicate on declared data exactly, on the declared values, is what Requirement "Exact decision by every consumer"

(foundations/req:consumer-decides-exactly) requires of every consumer. The conditions the definition states on Alice's data, on $\{M_x\}$ and on τ_A, are producer obligations of this module and not only consumer checks, since the consuming engine decides them exactly and refuses otherwise, so a declaration this module emitted without deciding them would be refused downstream on data this module had passed. For $\{M_x\}$ and for the trace sum of τ_A the obligation is Requirement "Exact verification before emission" (foundations/req:exact-emission) (ii)–(v), and for the Hermiticity, positivity and unit trace of τ_A it is Requirement "Exact admissibility at every module boundary" (foundations/req:boundary-admission) (ii). Each is decided here, before emission, and refused with a named reason otherwise (Requirement 13.1).

Definition 1.3 (Exact admissibility of declared operator data)**.** Declared operator data are binary64 numbers, each the dyadic rational it denotes and not an approximation of anything (Definition "Binary64 numbers, declared values and dyadic lattices" (foundations/def:binary64) (iii)), and every predicate this module decides on them is decided *exactly*, on those numbers. This definition lists those predicates, together with the one condition of the list that is a condition on the arithmetic that forms a sum rather than a predicate on an operator.

The declared summation order. The compilation map emits a total order on every declared alphabet and on $\mathbb{S}$: the label-to-index map idx, which numbers the letters of each declared alphabet, and the sectors, consecutively from 0 (Definition 3.3, Theorem 3.4); the declared order of an alphabet is its idx order. For a finite family $\{a_j\}_{j\in J}$ of $n \times n$ matrices indexed by a declared alphabet J with idx order $j_1 \prec \cdots \prec j_{|J|}$, the sum in the declared order is accumulated as the computed sequential sum (Definition "Sums in a declared order and sums that round nowhere" (foundations/def:declared-order))

$$\Sigma_0 := 0, \qquad \Sigma_i := \Sigma_{i-1} + a_{j_i} \quad (1 \le i \le |J|), \qquad \Sigma := \Sigma_{|J|}, \tag{1.9}$$

formed left to right, entrywise, in binary64, with no re-ordering and no re-association; the trace of a matrix is likewise accumulated over its row index in increasing order, which is the order of the emitted array.

The five conditions. Let $\{a_j\}_{j\in J}$ be a finite family of block lists against a sector map $\mathfrak{s}$, one array $a_j^{(s)}$ on $\mathcal{H}(Q_s)$ per $s \in \mathbb{S}$. The family is exactly admissible for $\mathfrak{s}$ when

(T) *(sector map)* the sector map the supplied family carries is $\mathfrak{s}$, the map this module holds: the same block kind, the same block identifiers in the same order, the same dimensions, so that $a_j = \bigoplus_s a_j^{(s)}$ holds *by construction*. This is a type condition: it is decided, never measured, and an off-block entry is not small but inexpressible;

(S) *(the sum rounds nowhere)* writing $\Sigma^{(s)}$ for the total Σ that (1.9) produces on the block family $\{a_j^{(s)}\}_j$: for each s, every one of the $|J|$ entrywise binary64 additions forming it is exact, so that the computed $\Sigma^{(s)}$ *is* the real sum $\sum_j a_j^{(s)}$;

(H) *(Hermiticity)* $a_j^{(s)} = a_j^{(s)\dagger}$ for every (j, s);

(P) *(positivity)* $a_j^{(s)} \succeq 0$ for every (j, s); the matrix being Hermitian by (H), this says that its spectrum is non-negative;

(C) *(completeness)* $\Sigma^{(s)} = \mathbb{1}_{Q_s}$ for every s.

Each of (H), (P), (C) is an *identity or an order relation between exactly represented numbers* and is decided as such: (H) and (C) entry by entry, (P) by exact symmetric elimination on the block with the zero-pivot row test (a negative pivot fails, and a zero pivot passes only when the rest of its row is zero), cubic in d_s. Together the five conditions are exact admissibility for $\mathfrak{s}$ with constant 1, that is with every completeness sum equal to the identity, in the sense of Definition "Exact admissibility of a declared operator family" (foundations/def:exact-admissibility), the declared order of J being its idx order: (T), (H), (P) and (S) are the conditions of that definition bearing the same names, and (C), which reads the computed sum, is its completeness identity, because under (S) the computed sum $\Sigma^{(s)}$ is the exact sum $\sum_j a_j^{(s)}$ (Lemma "Exactness of a sum that rounds nowhere" (foundations/lem:exact-accumulation)).

The same conditions on Alice's data, where there is one block and no sector map. A carries no sector decomposition, so a family on A is one array per index and (T) is empty for it. $\{M_x\}_{x\in\mathcal{X}}$ is *exactly admissible* when (S), (H), (P) and (C) hold of it on A with $\mathbb{1}_A$ on the right of (C); the declared marginal τ_A satisfies its own three conditions when $\tau_A = \tau_A^\dagger$, $\tau_A \succeq 0$ and $\operatorname{tr}\tau_A = 1$ with the trace accumulated in the row order *without rounding*, the trace being a sum, read below as an identity of real numbers exactly as a completeness sum is.

What is not on this list. A strict inequality on a declared scalar, such as the condition of Definition 4.1 that every public-choice weight be positive, is a domain condition on a declared real number, decided by its sign and not by any of the five. A relation symbol, a declared tag and the type of a right-hand side are likewise decided and never measured (D(vi)). And nothing in this definition is a *norm*: no quantity here is compared against a tolerance window, and no defect, the entrywise size of the difference a condition names, is computed.

The general format. For a declaration in the general format of Definition 1.1 (DX1)–(DX6) the same conditions are read as follows. *Per measured factor:* for every $f \in \mathbb{F}$ the family $\{F_{v_f}^f\}_{v_f\in\mathcal{V}^f}$ is exactly admissible for the

sector map of f — (T) against that factor's map, and (S), (H), (P), (C) on that factor's blocks in that factor's declared order. No condition is decided on a product block (1.5), none being a datum; that the product family then satisfies (H), (P) and (C) on Q as identities of real numbers is shown in (DX3). An entry's operator $\hat{O}_k$ is decided (T) and (H) against the map of its scope. *Alice's data, block by block:* with a factor tagged A, condition (T) is not empty for Alice's data: τ_A, the operators of her measurement slot and a general Θ_k are block lists against that factor's map. In POVM form $\{M_x^{(t)}\}_x$ satisfies (S), (H), (P) and (C) on every A_t with $\mathbb{1}_{A_t}$ on the right of (C); τ_A satisfies (H) and (P) block by block and $\sum_t \operatorname{tr} \tau_A^{(t)} = 1$, the traces accumulated in the row order and then over the blocks in their declared order, rounding nowhere. *Ensemble form:* the family $\{N_x^{(t)}\}_x$ satisfies (S), (H) and (P) on every A_t, and in place of (C) the identity $\Sigma^{(t)} = \tau_A^{(t)}$ of (1.6), entry by entry, between the declared-order sum and the declared block of τ_A; and $\tau_A^{(t)} \neq 0$ for every t, a comparison of declared entries with the datum 0. These are all the conditions of the form: nothing is decided of the denoted $\{M_x\}$ of (1.7), its being a POVM a theorem (Theorem 5.1 (v)). At one Alice sector and one measured factor in POVM form these readings are the paragraphs above.

Uses Def. 1.1, foundations/def:binary64, foundations/def:exact-admissibility, foundations/lem:exact-accumulation · *Used by* Rem. 1.4, Def. 1.5, Rem. 1.6, Def. 2.1, Def. 3.2, Def. 3.3, Thm. 3.7, Def. 4.1, Rem. 4.6, Ex. 4.7, Thm. 5.1, Thm. 5.5, Def. 6.1, Thm. 6.2, Def. 7.1, Thm. 7.5, Thm. 8.2, Def. 9.1, Thm. 9.4, Prop. 9.6, Req. 11.4, Req. 11.6, Req. 13.5 · newly derived · implemented (effective: accepted) · *Code* certiqs_key.foundations.exact:sum_in_declared_order

Completeness alone says that the computed floating-point sum is the identity; the summation condition is what makes it an identity of the declared numbers. A family of exactly representable entries may therefore still be refused, since the condition is on the additions.

Remark 1.4 (The summation condition beside completeness). The total $\Sigma^{(s)}$ of Definition 1.3 is by (1.9) a *floating-point* sum. So condition (C) alone says that the *computed* sum is the identity, and *not* that $\sum_j a_j = \mathbb{1}_Q$ holds of the declared numbers, which is what every derivation of this contract that consumes completeness reads it as, and what a consumer of the emitted declaration reads it as. Condition (S) is exactly what makes the two the same statement (Lemma "Exactness of a sum that rounds nowhere" (foundations/lem:exact-accumulation)), and it is the reason a family whose entries are all exactly representable may still be refused: the condition is on the *additions*, not on the operands. Remark "A complete computed sum from an incomplete family" (foundations/rem:computed-completeness) exhibits a family satisfying (H), (P) and completeness of the computed sum whose true sum is not the identity.

Uses Def. 1.3 · newly derived · accepted

A declaration with its index map is well-formed when D(i) to D(viii) hold: agreeing finite data, one sector order, an admissible marginal slot, exact measurement families, raw-key values in the key alphabet or none, announcement-determined sifting, typed entries, feasibility and party computability.

The sector order a consumer compares against its own sector map and the summation order it reads off the emitted index map must be one order, so D(i) states their agreement as a condition on the declaration together with its index map.

Definition 1.5 (Target conditions of a well-formed declaration)**.** A declaration (1.1), *together with the label-to-index map* idx *emitted alongside it* (Definition 3.3), is a *well-formed declaration* when the following hold, every operator predicate below being decided *exactly* in the sense of Definition 1.3 and every combinatorial condition exactly as well:

D(i) all systems are finite-dimensional, all alphabets are finite and non-empty, all declared maps are total, and *the sector map agrees*: there is one sector map $\mathfrak{s}$; every declared Q-operator is a block list against it, with the same block kind, the same identifiers in the same order and the same dimensions; and $d_Q = \sum_{s\in\mathbb{S}} d_s$. That much is condition (T) of Definition 1.3, stated once for the declaration. *And the order declared on the sectors is one order and not two*: the restriction of idx to $\mathbb{S}$ is the total order $\mathfrak{s}$ fixes. This last clause is a condition on the pair $(\mathfrak{D}, \mathsf{idx})$ and not on a slot of (1.1);

D(ii) the marginal slot holds τ_A with $\tau_A = \tau_A^\dagger$, $\tau_A \succeq 0$ and $\operatorname{tr}\tau_A = 1$, the trace summed in the row order and rounding nowhere — so that $\tau_A \in S_=(A)$ holds of the declared numbers, and a slot holding the zero matrix is inadmissible, its trace being 0 — or the value none;

D(iii) *(typing)* $\{F_v\}$ and every $\hat{O}_k$ are block lists against $\mathfrak{s}$ in the sense of Definition 1.1, so that $\Pi_s F_v \Pi_{s'} = 0$ and $\Pi_s \hat{O}_k \Pi_{s'} = 0$ for $s \neq s'$ hold *by construction* and are not conditions on data; and *(five predicates)* $M_x \succeq 0$ for each x and $\{M_x\}$ complete on A; $F_v^{(s)} \succeq 0$ for each (v, s) and $\{F_v^{(s)}\}_v$ complete on Q_s for each s; and $\hat{O}_k^{(s)}$ Hermitian for each (k, s), each decided exactly, each positivity predicate being the two decisions (H) and (P) of Definition 1.3 on the same block, and each completeness predicate carrying with it the condition (S) that its declared-order sum round nowhere. This is where Hermiticity of Alice's and Bob's emitted families enters the target conditions, and it is the only place it does;

D(iv) r maps into $\mathcal{Z} \sqcup \{\perp\}$;

D(v) *sifting is announcement-determined*: there is $\varsigma : \mathcal{K} \to \{\text{pass}, \text{fail}\}$ with $r(x, v) = \perp \iff \varsigma(a(x, v)) = \text{fail}$;

D(vi) in trusted-source mode $\bowtie_k \in \{\preceq, =, \succeq\}$ and each entry carries one of the two declared forms of Definition 2.5, homogeneous with $c_k \in \mathbb{R}$ or general with Θ_k Hermitian on A; in untrusted-source mode $\bowtie_k \in \{\leq, =, \geq\}$ with $\Theta_k = c_k \in \mathbb{R}$. The relation symbol and the type of the right-hand side, which is what fixes the tag (Definition 2.5), are combinatorial and are read exactly; the Hermiticity of a general Θ_k is the exact predicate (H) on one matrix;

D(vii) the feasible set of Definition 2.1 is non-empty, $\mathcal{W} \neq \varnothing$;

D(viii) *party computability*: each party announces a function of its own readout (a_A is defined on $\mathcal{O}_\mathsf{A}$ and a_B on $\mathcal{O}_\mathsf{B}$, jointly (1.2)) and the declared holder computes the raw key from its own readout and the announcements: there is a total $r_\mathsf{h} : \mathcal{O}_\mathsf{h} \times \mathcal{K} \to \mathcal{Z}_\perp$ with

$$r(x, v) = r_\mathsf{h}\big(\pi_\mathsf{h}(x, v),\, a(x, v)\big) \qquad \text{for all } (x, v) \in \mathcal{X} \times \mathcal{V}. \tag{1.10}$$

The *default party partition* is $\pi_\mathsf{A}(x, v) = x$, $\pi_\mathsf{B}(x, v) = v$, with A labelled A and every Q_s labelled B.

The general format. For a declaration in the general format of Definition 1.1 (DX1)–(DX6) the conditions read as follows, and in the basic format — one Alice sector, one measured factor, POVM form, two messages neither reading the other and no party-local entry — they are the eight above. D(i): the sector map is one factorised map (at most one factor tagged A, and then the first, at least one tagged Q, one block kind), every declared family on a measured factor and every entry's operator is a block list against the map of its factor, Alice's data against the factor tagged A, $d_Q = \prod_f \sum_{s_f} d_{f,s_f}$, and the restriction of idx to each factor's sectors is the order that factor's map fixes, the product order being derived and not declared. D(ii): τ_A is a block list along $\mathbb{T}$ with every block Hermitian and positive semidefinite and total trace 1, summed without rounding; in ensemble form additionally every block is non-zero. D(iii): the typing and the predicates of Definition 1.3 in their general-format reading, per measured factor and per Alice sector, the ensemble-form conditions in place of those on $\{M_x\}$ where that form is declared; ensemble form is refused in untrusted-source mode. D(vi): an entry carries its scope, which names a measured factor of the map, and one of the forms of Definition 2.5, the party-local form included, in either mode, with a real c_k; a general Θ_k is a block list along $\mathbb{T}$, Hermitian block by block. *And the relation symbol is read by form and not by mode alone*: an *untagged* entry carries $\bowtie_k \in \{\preceq, =, \succeq\}$ in trusted-source mode and $\bowtie_k \in \{\leq, =, \geq\}$ in untrusted-source mode, exactly as stated for D(vi), while an entry declared party-local carries $\bowtie_k \in \{\preceq, =, \succeq\}$ *in either mode*, its reading being the operator relation of the last line of (2.3) (Definition 2.5, the relation-symbol clause of the party-local form); where every row of that entry is 1×1 (every Alice block, and every sector block of every measured factor other than the

entry's scope, of dimension one, $d_A = 1$ at one measured factor included) that relation is between 1×1 matrices and the three operator symbols are the three scalar symbols, which is the degenerate case. Where no entry is party-local, as in every declaration of the basic format, this clause is D(vi) as stated. D(viii): the announcement slot is a transcript protocol (DX5), every table a_m being total on $\mathcal{O}_{\pi_m} \times \mathcal{C}_1 \times \cdots \times \mathcal{C}_{m-1}$, so that a message reads its sender's readout and earlier messages *by its signature*; the joint function a is the composition (1.8), and (1.10) holds with it. D(iv), D(v) and D(vii) keep their wording, with a the derived function and $\mathcal{W}$ the set of (2.3).

Uses Def. 1.1, Def. 1.3 · *Used by* Rem. 1.6, Lem. 2.2, Def. 3.3, Thm. 3.4, Thm. 3.7, Thm. 5.1, Rem. 5.3, Def. 6.1, Thm. 6.2, Thm. 7.5 · newly derived · accepted

The consuming engine decides the same five exact conditions as this module, and decides them alike. The declarations admitted here are a subset of those the engine admits: D(i) and D(iii) are stronger here and check V7 is one-sided.

Remark 1.6 (Relation to the consuming engine's admission conditions). The five conditions of Definition 1.3 are also the consuming engine's admission conditions, which that engine decides exactly, allowing no nonzero defect (Requirement "Exact decision by every consumer" (foundations/req:consumer-decides-exactly)), so the two modules decide *those five conditions* alike. *The agreement is on the five conditions and not on the declarations they are conditions of*: this module additionally imposes D(i)'s clause on $\mathsf{idx}|_{\mathbb{S}}$ (Definition 1.5), for which that engine's definition of an admissible declaration (Definition "Admissible declaration" (meat-core/def:admissible-declaration)) has no counterpart, and D(iii)'s Hermiticity clause, which that definition leaves to the typing of the tuple; and its feasibility check V7 (Definition 6.1) is one-sided: it may refuse a declaration whose feasible set is non-empty, for want of a verified witness. So the declarations admitted here are a *subset* of those that engine admits, and where the two differ this module refuses declarations that engine would admit, never the reverse.

Uses Def. 1.3, Def. 1.5 · newly derived · accepted

2 Feasible states and licensed constraints

The feasible set holds the normalised positive block-list states that meet every constraint entry and, in trusted-source mode, the declared marginal, each condition read exactly; Hermiticity of the constraint operators and of the state, with that of the declared right-hand sides, is what makes the two sides of each constraint condition comparable (Lemma 2.2). Restricting the states to sector-block-diagonal ones is a restriction in the unsafe direction that the operator type carries and the consuming engine proves sound (Remark 2.4). A constraint list is licensed for its source mode when every entry is declared in the form the mode admits or carries the party-local tag, and licensing reads the declared tag and compares no numbers. A party-local entry, whose condition holds against the state's marginal on the systems

other than its own factor, is sound only under a certificate owed by the producer of the reduction chain, which the attack-set licence of the list attests (Remark 2.6). The example writes out a declaration for an untrusted source serving two measuring parties, with a party-local entry.

The feasible set holds the normalised positive block-list states that meet the declared marginal and every constraint entry, each read exactly. The source mode decides whether an entry is an operator or a scalar condition, and the set carries no parameter and admits no window.

Definition 2.1 (Feasible set)**.** For a declaration $\mathfrak{D}$, the *feasible set* $\mathcal{W}$ is the set of ω presented as a sector-block list on AQ — $\omega = \bigoplus_s \omega^{(s)}$ with $\omega^{(s)} \in \mathcal{L}(AQ_s)$, so that $(\mathbb{1}_A \otimes \Pi_s)\omega(\mathbb{1}_A \otimes \Pi_{s'}) = 0$ for $s \neq s'$ holds by the presentation and is not a condition — such that

$$\omega^{(s)} \succeq 0 \;\; \forall s, \qquad \operatorname{tr}\omega = 1, \qquad \omega \models \mathfrak{D}, \qquad \omega \models \mathfrak{L}, \tag{2.1}$$

each read *exactly*, with $\operatorname{tr}\omega = \sum_s \operatorname{tr}\omega^{(s)}$ and with $\omega^{(s)} \succeq 0$ carrying, as in Definition 1.3, Hermiticity of the block. Here $\omega \models \mathfrak{D}$ is the marginal condition, $\omega_A = \tau_A$ in trusted-source mode, with $\omega_A = \sum_s \operatorname{tr}_{Q_s} \omega^{(s)}$, and no condition beyond $\operatorname{tr}\omega = 1$ in untrusted-source mode; and $\omega \models \mathfrak{L}$ reads

$$\begin{aligned} \text{trusted:} \quad & \operatorname{tr}_Q\big[(\mathbb{1}_A \otimes \hat{O}_k)\,\omega\big] = \sum_s \operatorname{tr}_{Q_s}\big[(\mathbb{1}_A \otimes \hat{O}_k^{(s)})\,\omega^{(s)}\big] \;\bowtie_k\; \Theta_k \quad \forall k; \\ \text{untrusted:} \quad & \operatorname{tr}\big[(\mathbb{1}_A \otimes \hat{O}_k)\,\omega\big] = \sum_s \operatorname{tr}\big[(\mathbb{1}_A \otimes \hat{O}_k^{(s)})\,\omega^{(s)}\big] \;\bowtie_k\; c_k \quad \forall k, \end{aligned} \tag{2.2}$$

where in the trusted line Θ_k is the *constructed* right-hand side $c_k\tau_A$ for an entry declared homogeneous and the *declared* Hermitian matrix for one declared general (Definition 2.5). The set carries no parameter, admits no tolerance window and relaxes no conjunct; in particular positivity is never relaxed.

The general format. For a declaration in the general format of Definition 1.1 (DX1)–(DX6), $\mathcal{W}$ is the set of ω presented as a list over the joint sectors, $\omega = \bigoplus_{(t,s)} \omega^{(t,s)}$ with $\omega^{(t,s)} \in \mathcal{L}(A_tQ_s)$ and every block positive semidefinite, so that block diagonality along $\mathbb{T}$ and along $\mathbb{S}$ both hold by the presentation, subject to the same conditions $\operatorname{tr}\omega = 1$, $\omega \models \mathfrak{D}$ and $\omega \models \mathfrak{L}$. Write $\hat{O}_k^{\uparrow(s)} := \mathbb{1} \otimes \hat{O}_k^{(s_{f_k})} \otimes \mathbb{1}$ for the s-block of the lifted operator of an entry scoped to f_k (DX6), and for a party-local entry scoped to f split $s = (s_{f^c}, s_f)$, with $Q_{f^c,s_{f^c}}$ the product of the other factors' blocks. The

conditions then read, block by block,

$$
\begin{aligned}
&\textstyle\sum_{(t,s)} \operatorname{tr} \omega^{(t,s)} = 1 && \text{(normalization);}\\
&\textstyle\sum_{s} \operatorname{tr}_{Q_s} \omega^{(t,s)} = \tau_A^{(t)} \quad \forall t && \text{(marginal, trusted-source mode);}\\
&\textstyle\sum_{s} \operatorname{tr}_{Q_s}\left[(\mathbb{1}_{A_t} \otimes \hat{O}_k^{\uparrow(s)})\, \omega^{(t,s)}\right] \bowtie_k \Theta_k^{(t)} \quad \forall t && \text{(trusted-source mode);}\\
&\textstyle\sum_{(t,s)} \operatorname{tr}\left[(\mathbb{1}_{A_t} \otimes \hat{O}_k^{\uparrow(s)})\, \omega^{(t,s)}\right] \bowtie_k c_k && \text{(untrusted-source mode);}\\
&\textstyle\sum_{s_f \in \mathbb{S}^f} \operatorname{tr}_{Q_{f,s_f}}\left[\left(\mathbb{1} \otimes (\hat{O}_k^{(s_f)} - c_k \mathbb{1})\right) \omega^{(t,(s_{f^c},s_f))}\right] \bowtie_k 0 \quad \forall (t, s_{f^c}) && \text{(party-local, either mode),}
\end{aligned}
\tag{2.3}
$$

the third line with $\Theta_k^{(t)} = c_k \tau_A^{(t)}$ for an entry declared homogeneous, the fourth for a scalar entry, and the last as an operator on $\mathcal{H}(A_t) \otimes \mathcal{H}(Q_{f^c,s_{f^c}})$. Each line is the corresponding operator condition on AQ: the marginal ω_A, the entry value $\operatorname{tr}_Q[(\mathbb{1}_A \otimes \hat{O}_k)\omega]$, τ_A and every Θ_k are block diagonal along $\mathbb{T}$, the party-local value $\operatorname{tr}_{Q_f}[(\mathbb{1} \otimes (\hat{O}_k - c_k \mathbb{1}_{Q_f}))\,\omega]$ is block diagonal along the pairs (t, s_{f^c}), and an equality or a semidefinite inequality between operators block diagonal for one decomposition holds exactly when it holds block by block. With $|\mathbb{T}| = 1$, one measured factor and no party-local entry, (2.3) is (2.1) with (2.2), line for line. With one measured factor the party-local line reads $\sum_s \operatorname{tr}_{Q_s}[(\mathbb{1}_{A_t} \otimes (\hat{O}_k^{(s)} - c_k \mathbb{1}))\omega^{(t,s)}] \bowtie_k 0$ on $\mathcal{H}(A_t)$, that is $\operatorname{tr}_Q[(\mathbb{1}_A \otimes \hat{O}_k)\omega] \bowtie_k c_k\, \omega_A$. The set is convex and compact: every line is an affine equality or an affine semidefinite or scalar inequality, and $\omega \succeq 0$ with $\operatorname{tr} \omega = 1$.

Uses Def. 1.1, Def. 1.3 · *Used by* Lem. 2.2, Rem. 2.3, Rem. 2.4, Def. 2.5, Def. 3.6, Thm. 3.7, Thm. 5.5, Thm. 6.2, Prop. 9.6, meat-core/def:feasible · newly derived · implemented (effective: accepted) · *Code* certiqs_key.protocol_compiler.feasibility:in_feasible_set · *Oracles* registry_protocol_compiler.py

For a Hermitian constraint operator and a Hermitian ω, the partial trace of their product over the measured system is Hermitian and its trace real; under D(ii) and D(vi) every trusted right-hand side is Hermitian. So both sides of each constraint line can be compared.

The trusted line of (2.2) compares two operators on A in the Löwner order, which is defined on Hermitian operators, and the untrusted line compares a trace with a real number. That the left-hand side is Hermitian is *not* automatic from $\omega = \omega^\dagger$ alone, a product of two Hermitian operators need not be Hermitian; it follows from the Hermiticity of $\hat{O}_k$ that D(iii) requires.

Lemma 2.2 (Hermiticity of the constraint values). Let $\hat{O}_k$ be an operator on Q with $\hat{O}_k = \hat{O}_k^\dagger$, as D(iii) of Definition 1.5 requires of every constraint operator, and let ω be an operator on AQ with $\omega = \omega^\dagger$. Then:

(i) $\operatorname{tr}_Q[(\mathbb{1}_A \otimes \hat{O}_k)\,\omega]$ is Hermitian;

(ii) $\operatorname{tr}\left[(\mathbb{1}_A \otimes \hat{O}_k)\,\omega\right]$ is real;

(iii) where D(ii) and D(vi) hold, the right-hand side of the trusted line of (2.2) is Hermitian: $c_k\tau_A$ is, for c_k real and τ_A Hermitian by D(ii), and a general Θ_k is, by D(vi).

Uses Def. 1.1, Def. 1.5, Def. 2.1 · *Used by* Rem. 2.3, Thm. 3.7, Prop. 9.6 · newly derived · accepted

Proof. Write $X \coloneqq \mathbb{1}_A \otimes \hat{O}_k$, which is Hermitian because $\hat{O}_k$ is. For any operator Y on AQ,

$$\operatorname{tr}_Q\big[(\mathbb{1}_A \otimes \hat{O}_k)\, Y\big] = \operatorname{tr}_Q\big[Y\,(\mathbb{1}_A \otimes \hat{O}_k)\big], \tag{2.4}$$

an identity of *linear* maps, holding for every Y whatsoever: in any product bases $\{|i\rangle\}$ of $\mathcal{H}(A)$ and $\{|q\rangle\}$ of $\mathcal{H}(Q)$, the (i,j) entry of either side equals $\sum_{q,q'} \langle q|\,\hat{O}_k\,|q'\rangle\ \langle i\,q'|\,Y\,|j\,q\rangle$, for the left-hand side directly, and for the right-hand side after exchanging the names of the two summation indices.

(i) Using $\omega = \omega^\dagger$, $X = X^\dagger$ and (2.4), $\big(\operatorname{tr}_Q[X\omega]\big)^\dagger = \operatorname{tr}_Q[(X\omega)^\dagger] = \operatorname{tr}_Q[\omega X] = \operatorname{tr}_Q[X\omega]$.

(ii) $\operatorname{tr}[X\omega]$ is the trace of the Hermitian operator $\operatorname{tr}_Q[X\omega]$ of (i), and so is real.

(iii) $c_k\tau_A$ is Hermitian, c_k being real and τ_A Hermitian by D(ii); a general Θ_k is Hermitian by D(vi). □

The left-hand sides of both constraint lines are Hermitian, respectively real, because the constraint operators and ω are Hermitian (Lemma 2.2). Without Hermiticity the lines lose their meaning, which is why it is a condition of the declaration, decided exactly.

Remark 2.3 (Hermiticity as a declared condition). By Lemma 2.2 both sides of the trusted line of (2.2) are Hermitian, so that their Löwner comparison is defined, and the left-hand side of the untrusted line is real, so that it can be compared with $c_k \in \mathbb{R}$. Drop Hermiticity and both lines lose their meaning rather than merely their soundness, which is why it is a condition of Definition 1.1 and of D(iii), and not only a check.

Where each hypothesis of that lemma comes from. $X = X^\dagger$, for $X = \mathbb{1}_A \otimes \hat{O}_k$, is D(iii)'s Hermiticity clause on $\hat{O}_k$, decided by check V3 of the validator, which decides the operators block by block (Definition 6.1). $\omega = \omega^\dagger$ is the Hermiticity half of the positivity condition of (2.1), decided by the feasibility check V7 on the candidate it verifies: a condition on ω and not an assumption about it. Both are decided exactly.

Uses Def. 1.1, Def. 2.1, Lem. 2.2 · newly derived · accepted

The feasible set ranges over sector-block-diagonal states alone, a restriction in the unsafe direction that the operator type carries. The consuming engine's master reduction proves it sound by sector dephasing.

Remark 2.4 (Restriction of the feasible set to block-diagonal states). *The restriction.* The consuming engine's argument is sound when its minimization ranges over the *sector-block-diagonal* ω of (2.1) rather than over all of $\mathcal{L}(AQ)$: every state that argument ranges over is carried into sector-block-diagonal form at no increase of the bound. This module states the restriction and does not discharge it. Its direction is the *unsafe* one — a smaller domain gives a larger infimum, so an unlicensed restriction overstates a key length. And it is carried by the operator *type*: an ω *is* a list of blocks and no expressible object has an off-sector entry (Definition 2.1), so nothing verifies it and nothing can. The argument that the sectors are a symmetry of every state the security proof must admit belongs to the consuming engine and to the reductions module; this module could not make it, since it never sees the reachable set. Definition 2.1 *defines* the feasible set, and it is not unsound if the restriction is unlicensed.

Where it is proved. The consuming engine's master reduction (Theorem "Reduction of the rate to the convex program" (meat-core/thm:reduction), step 1 of its proof, the sector dephasing, and Corollary "The reduction for arbitrary constraint lists" (meat-core/cor:reduction-constrained) for a non-empty list) proves the restriction: an arbitrary feasible state is dephased, feasibility and every constraint value survive because every F_v and every $\hat{O}_k$ is sector-block-diagonal, and the rest of the proof bounds the entropy by the objective at the dephased state.

In the general format (Definition 2.1, the set of (2.3)) the restriction is along the *joint* sectors: along the sectors of the measured system as before and along Alice's declared sectors. It is proved by the same step of the same theorem, which dephases by the joint pinching, under preconditions that hold by *type*: Alice's measurement slot, τ_A and every right-hand side are block lists against a *declared* factor tagged A, as $\{F_v\}$ and every $\hat{O}_k$ are against the measured factors (Theorem "Reduction of the rate to the convex program" (meat-core/thm:reduction), Step 1, with Proposition "Value neutrality along Alice's sectors" (meat-core/prop:a-neutral) for the statement that the restriction along Alice's sectors changes no value). The direction is unchanged and so is the warning: the licence is that theorem under a declared factor. A producer that emits a block-diagonal τ_A or $\{M_x\}$ without a factor tagged A has declared one Alice sector, and nothing downstream may read a restriction off the shape of its data.

Uses Def. 2.1 · newly derived · accepted

A constraint list is licensed when every entry is declared in the form its source mode admits, homogeneous scalar data in trusted mode and scalars in untrusted mode, or carries the party-local tag. Licensing reads the declared tag and compares no numbers.

Reading the tag of an entry off the type of its datum is a decision on the data handed over, and infers nothing from a property of an object the entry was built from (Requirement "Exact decision by every consumer" (foundations/req:consumer-decides-exactly)). The reductions module, which defines the reduction chain this module consumes, declares every entry's right-hand side as a real constant (Definition "Licensed entries and the party-local

form" (reductions/def:licensed), Corollary "The emitted constraint entries" (reductions/cor:entries)). On a one-factor emission of that module the form beside it is the tag value homogeneous in trusted-source mode, so every entry is homogeneous, the licensed form in trusted-source mode, and an arriving tag field agrees with the type read off the datum; where the measured system is a declared product of party-scoped factors, an entry may instead arrive tagged party-local, with a scope (Requirement "The party-local tag and its scope" (reductions/req:party-local-tag)). That licensing is decided exactly and at no window is part of Requirement 11.6.

Definition 2.5 (Licensed constraint list)**.** Each entry of $\mathfrak{L}$ carries its right-hand side in a declared form, and the form is part of the datum rather than a property measured of it (Definition 1.1). There are two *untagged* forms, called so because their tag is the type of their datum and needs no field of its own: a *homogeneous* entry declares a real scalar c_k, a *general* entry declares a Hermitian Θ_k on A; in untrusted-source mode only the scalar form exists. For these two forms the tag *is* the type of the right-hand side an entry carries, and it is read off that type: an entry whose right-hand-side slot holds a real scalar is homogeneous, and one whose slot holds a Hermitian matrix on A is general. *That reading is a type decision*: it inspects which of two objects a slot holds, compares no numbers, reads no matrix entry and measures nothing. The third form, party-local, is declared by its tag and is not read off the type of the datum. $\mathfrak{L}$ is *licensed for the declared source mode* when every entry is in the form that mode admits:

(a) *trusted-source mode:* every entry is declared homogeneous, carrying $c_k \in \mathbb{R}$; the right-hand side it denotes is then *constructed* as $\Theta_k := c_k \tau_A$ from the declared c_k and the declared τ_A, and the entry of (2.2) reads $\mathrm{tr}_Q[(\mathbb{1}_A \otimes \hat{O}_k)\,\omega] \bowtie_k c_k \tau_A$;

(b) *untrusted-source mode:* every right-hand side is a real number $\Theta_k = c_k$ and the entry reads $\mathrm{tr}\Big[(\mathbb{1}_A \otimes \hat{O}_k)\,\omega\Big] \bowtie_k c_k$.

Licensing is therefore a *type* check on the tag, and it compares no numbers: what is inspected is which of the two forms an entry declares; no operator identity is tested against any number, and nothing here is decided at a window. *Only in cases (a) and (b) are the entries written* $(\hat{O}_k, \bowtie_k, c_k)$; a general entry is written $(\hat{O}_k, \bowtie_k, \Theta_k)$ and is the form the notation table records under Θ_k. A general entry is admissible input, every structural claim of this contract being proved for it, but it is not licensed, so a list containing one carries no key-length claim (Theorem 9.4).

The party-local form, a third licensed form. In the general format of Definition 1.1 (DX6) an entry may be declared *party-local*: it carries its scope f, a block list $\hat{O}_k$ against the sector map of the measured factor f and a real c_k, it is written $(\hat{O}_k, \bowtie_k, c_k; f)$, and its state-level reading, in either

source mode, is

$$\mathrm{tr}_{Q_f}\Big[\big(\mathbb{1}\otimes(\hat{O}_k - c_k\,\mathbb{1}_{Q_f})\big)\,\omega\Big] \;\bowtie_k\; 0 \qquad \text{as an operator on the systems of } \omega \text{ other than } Q_f, \tag{2.5}$$

those systems being A and the measured factors $Q_{f'}$, $f' \neq f$; equivalently $\mathrm{tr}_{Q_f}[(\mathbb{1}\otimes\hat{O}_k)\,\omega] \bowtie_k c_k\,\mathrm{tr}_{Q_f}\,\omega$, the entry being homogeneous with respect to the marginal that is actually there. Block by block it is the last line of (2.3). The list is then *licensed* when every entry is either in the form the source mode admits, (a) or (b), read for its lifted operator, or is declared party-local. This is still a type check and compares no numbers: what is inspected is the declared tag. A party-local entry's right-hand-side slot holds a real scalar, as a homogeneous entry's does, so the type of the datum does not separate the two: an entry is party-local exactly when it carries that tag.

The soundness condition of a party-local entry. A party-local entry $(\hat{O}_k, \bowtie_k, c_k; f)$ meets the *soundness condition of a party-local entry* when every attack reaches Q_f through a channel Λ_f acting on that factor alone which carries the spectral certificate $\Lambda_f^\dagger(\hat{O}_k) \bowtie_k c_k\mathbb{1}$, that is, the adjoint of the channel applied to the entry's operator stands in the entry's relation to c_k times the identity.

What the tag changes, case by case. With one measured factor and trusted-source mode, (2.5) reads $\mathrm{tr}_Q[(\mathbb{1}_A\otimes\hat{O}_k)\,\omega] \bowtie_k c_k\,\omega_A$, which under the marginal condition $\omega_A = \tau_A$ is the form (a) of the same $(\hat{O}_k, \bowtie_k, c_k)$. With one measured factor and $d_A = 1$, in either mode, it is the comparison of real numbers $\mathrm{tr}\big[\hat{O}_k\omega\big] \bowtie_k c_k\,\mathrm{tr}\,\omega = c_k$, the form (b). *With one measured factor, untrusted-source mode and $d_A > 1$ it is neither*: it is an operator inequality on A against the marginal ω_A that is actually there, and the scalar entry (b) of the same $(\hat{O}_k, \bowtie_k, c_k)$ is its trace, implied by it and not equivalent to it. With more than one measured factor it implies, in either mode, the untagged reading of the same entry, which is obtained from it by the partial trace over the other measured factors (and over A in untrusted-source mode), and it is in general not implied by it. So the tag never weakens an entry, and of the four one-factor cases, by source mode and by whether $d_A = 1$, it changes something only in untrusted-source mode at $d_A > 1$.

Which relation symbols a tagged entry carries, in either mode. An entry declared party-local carries $\bowtie_k \in \{\preceq, =, \succeq\}$ *in either source mode*, because (2.5) is a relation between *operators* in either mode: its left-hand side is an operator on the systems of ω other than Q_{f_k}, which are A and the other measured factors. The scalar case is the degenerate one: where $d_{A_t}\prod_{f\neq f_k} d_{f,s_f} = 1$ for every row — every Alice block, and every sector block of every measured factor other than f_k, of dimension one, which with one measured factor means every Alice block of dimension one, whether $d_A = 1$ or $d_A = |\mathbb{T}| > 1$ — each row is a comparison of 1×1 matrices, that is of real numbers, and the three operator symbols are then the three

scalar symbols $\leq$, $=$, $\geq$. With one measured factor and $d_A = 1$ there is one row, and the reading is the form (b) of the same $(\hat{O}_k, \bowtie_k, c_k)$, as the case-by-case paragraph states; otherwise there is one such comparison per row, and the reading relates to the untagged forms as that paragraph states for its case. *Nothing about the untagged forms changes*: an untagged entry carries the symbols its mode admits, and the operator symbols beside a tag are a property of the third form and of no other. No entry of a declaration in the basic format is tagged.

Uses Def. 1.1, Def. 2.1 · *Used by* Rem. 2.6, Def. 3.1, Def. 3.3, Thm. 3.4, Thm. 5.5, Def. 6.1, Thm. 6.2, Def. 9.1, Thm. 9.4, Prop. 9.6, Ass. 10.7, Req. 11.1, Req. 11.5, meat-core/def:homog, meat-core/rem:tagged-entries-and-accumulation · newly derived · implemented (effective: accepted) · *Code* certiqs_key.protocol_compiler.gates:licensed_for_the_emitted_mode

A party-local entry shrinks the feasible set more than its untagged reading and is sound only if every attack reaches its factor through a channel on that factor alone with a spectral certificate. The reduction chain's producer owes it and the attack-set licence attests it; nothing here decides it.

Remark 2.6 (Soundness of a party-local entry). A party-local entry of Definition 2.5 shrinks the feasible set further than its untagged reading does, and it is sound only where it meets the soundness condition of a party-local entry stated in that definition, the spectral certificate on a channel acting on Q_f alone through which every attack reaches that factor. That is a certificate of the module that produces the chain, stated by the reductions module (Theorem "The attack-set licence" (reductions/thm:licence) (v), with Lemma "Party-local transport" (reductions/lem:party-local) and the attack set of Definition "Admissible attack sets in the consumer's format" (reductions/def:attset)); it is what the attack-set licence of a list carrying the tag attests, and nothing in this module decides it. A general entry whose right-hand side is $\Theta_k = c_k \mathbb{1}_A$ stays unlicensed in every case: the right-hand side of (2.5) is c_k times a marginal of ω, never c_k times an identity.

Uses Def. 2.5 · newly derived · accepted

An untrusted source serves two measuring parties; the measured system is a product of two factors with their own sector maps and families, and no product matrix is formed. A party-local entry bounds Bob's flag weight by 1/8 conditionally on every state of Alice's measured factor.

The example writes out the general format of Definition 1.1 for a protocol in which an untrusted source serves two measuring parties. It illustrates the format and asserts nothing about any physical protocol.

Example 2.7 (Declaration of a two-party protocol with an untrusted source). *Systems and sector map.* Untrusted-source mode, $d_A = 1$: the marginal slot holds none, $\mathcal{X} = \{x_0\}$ and $M_{x_0} = [1]$ (Remark "The one-dimensional retained system" (transmitter/rem:da1)). The sector map has no factor tagged A and two factors tagged Q, in this order: $f = \mathsf{a}$ with party label A and $f = \mathsf{b}$ with party label B (Definition 1.1 (DX1)). Each factor has a data sector of dimension 2 and a flag sector of dimension 1, $\mathbb{S}^f = (\mathrm{d}, \mathrm{fl})$, $d_{f,\mathrm{d}} = 2$, $d_{f,\mathrm{fl}} = 1$. The product map has the four sectors $(\mathrm{d},\mathrm{d}), (\mathrm{d},\mathrm{fl}), (\mathrm{fl},\mathrm{d}), (\mathrm{fl},\mathrm{fl})$ in that order, the last factor fastest, of dimensions

$4, 2, 2, 1$; $|\mathbb{T}| = 1$, so these are also the joint sectors, and a state ω is a list of four blocks of those sizes.

Family. Each factor declares its own family over $\mathcal{V}^f = (0, 1, \mathrm{fl})$, in this order: $F_0^{f,(\mathrm{d})} = \mathrm{diag}(1, 0)$, $F_1^{f,(\mathrm{d})} = \mathrm{diag}(0, 1)$, $F_{\mathrm{fl}}^{f,(\mathrm{d})} = 0$, and $F_0^{f,(\mathrm{fl})} = F_1^{f,(\mathrm{fl})} = [0]$, $F_{\mathrm{fl}}^{f,(\mathrm{fl})} = [1]$. Each is exactly admissible for its factor's map, its two completeness sums rounding nowhere (Definition 1.3, per measured factor). The outcome alphabet is $\mathcal{V} = \mathcal{V}^{\mathsf{a}} \times \mathcal{V}^{\mathsf{b}}$ with nine letters, and $F_{(v_{\mathsf{a}}, v_{\mathsf{b}})} = F_{v_{\mathsf{a}}}^{\mathsf{a}} \otimes F_{v_{\mathsf{b}}}^{\mathsf{b}}$ by the rule (1.5); its block on $(\mathrm{d}, \mathrm{fl})$, for instance, is $F_{v_{\mathsf{a}}}^{\mathsf{a},(\mathrm{d})} \otimes F_{v_{\mathsf{b}}}^{\mathsf{b},(\mathrm{fl})}$, a 2×2 matrix that is written down by nobody. Nine 9×9 arrays are never formed.

Party partition and announcements. $\pi_{\mathsf{A}}(x_0, v) = v_{\mathsf{a}}$ and $\pi_{\mathsf{B}}(x_0, v) = v_{\mathsf{b}}$, which is *not* the default party partition: under the default one both parties' data would sit in π_{B} and party computability would be vacuous. A transcript with $L = 2$: $\pi = (\mathsf{A}, \mathsf{B})$, each party announcing whether its outcome is a flag, $a_1(v_{\mathsf{a}}) = a_2(v_{\mathsf{b}}, \kappa_1) = \mathrm{fl}$ on the flag letter and ok otherwise. Sifting passes on $\kappa = (\mathrm{ok}, \mathrm{ok})$; $\mathsf{h} = \mathsf{A}$, $d_Z = 2$, and the key rule r_{A}^0 of the specification's public part (Definition 3.1 (d)), into $\mathcal{Z} = \{0, 1\}$, is $r_{\mathsf{A}}^0(v_{\mathsf{a}}, \kappa) = v_{\mathsf{a}}$ for $v_{\mathsf{a}} \in \{0, 1\}$ and $r_{\mathsf{A}}^0(\mathrm{fl}, \kappa) = 0$, a value no passing round reads, since $v_{\mathsf{a}} = \mathrm{fl}$ announces $\kappa_1 = \mathrm{fl}$. Party computability, (1.10), requires the raw key to be a function of Alice's own readout and the transcript, as it is.

A party-local entry. Let $\hat{O}$ be the block list on factor b with $\hat{O}^{(\mathrm{d})} = 0$ and $\hat{O}^{(\mathrm{fl})} = [1]$, the projector onto Bob's flag sector, and declare $(\hat{O}, \preceq, \frac{1}{8}; \mathsf{b})$ with the tag party-local. By the last line of (2.3), with $s_{f^c} = s_{\mathsf{a}}$, it reads, for each of Alice's two sectors s_{a},

$$\mathrm{tr}_{Q_{\mathsf{b},\mathrm{fl}}}\, \omega^{(s_{\mathsf{a}},\mathrm{fl})} - \tfrac{1}{8}\Big(\mathrm{tr}_{Q_{\mathsf{b},\mathrm{d}}}\, \omega^{(s_{\mathsf{a}},\mathrm{d})} + \mathrm{tr}_{Q_{\mathsf{b},\mathrm{fl}}}\, \omega^{(s_{\mathsf{a}},\mathrm{fl})} \Big) \preceq 0 \qquad \text{on } \mathcal{H}(Q_{\mathsf{a},s_{\mathsf{a}}}),$$

a 2×2 and a 1×1 condition: Bob's flag weight is at most $\frac{1}{8}$ *conditionally on every state of Alice's measured factor* a. The same entry without the tag reads $\mathrm{tr}\Big[(\mathbb{1} \otimes \hat{O})\omega\Big] \leq \frac{1}{8}$, one real inequality, which is the trace of the sum of the two conditions above and does not imply them: it allows all of Bob's flag weight to sit where Alice's measured factor is in one fixed state. Which reading the producer of the entries may emit is decided by its certificate (Definition 2.5, Remark 2.6), not here.

newly derived · accepted

3 Specification, compilation map and typing

A specification supplies the pieces, and the compilation map assembles the declaration from them, passing declared data through unchanged and constructing only the raw-key map, the index map and the weighted families. A specification is well-formed when its alphabets and systems are finite, its tables total, every declared measurement family, the presented physical one included, exactly admissible, its weights normalised and its constraint blocks Hermitian and tagged. Every declaration the map emits from a well-formed specification meets the typing conditions, and its index map is a bijection on every alphabet it indexes (Theorem 3.4). Its declared measurement families are POVMs block by block as identities of the declared numbers, per factor

and, in POVM form, per Alice sector in the general format, where the product family and an ensemble form's denoted family are POVMs by theorem; the statistics functional is a probability distribution on every normalised positive block-list state (Theorem 3.7).

Imports transmitter/def:source-segment, transmitter/def:source-spec, transmitter/def:srcert

A protocol specification is the compiler's input: the transmitter artefact, the receiver part, a two-segment reduction chain, the public part and the constraint entries. In the general format the receiver part and the chain come per measured factor, with a transcript and scoped entries.

The compiler's input comes in five parts. Parts (a)–(e) describe the specification of a declaration in the basic format; the paragraphs on the general format read the same five parts more widely. The descriptors a link carries are, for a source link, the transmitter's source descriptor and target descriptor; for a receiver link they are receiver-side. How the compiler treats the slots it reads is Requirement 11.3. Which objects check V1 compares, and why a supplied witness is compared at check V7 instead, is Remark 7.4.

In the general format, how (DX1) restricts the party partition is Remark 1.2; that the product rule (1.5) of (b′) is formed by no one is Requirement 11.4; and the ordered list of the factors' artefacts in (c′) is the one Requirement "Emission of several measuring parties as factors" (reductions/req:factorised-emission) emits. A certificate declared in block form is refused at gate (G2) of Definition 9.1. The ground is not that this module demands purity of every certificate: at several Alice sectors global purity fails for a presentation too. What separates the two forms is stated in Remark 9.2, and the operative ground for the refusal is the scope rule: admitting a block form would widen an unsafe-direction gate at the degenerate values.

Definition 3.1 (Protocol specification)**.** A *protocol specification* is a tuple

$$\mathfrak{S} = \big(\mathfrak{S}_{\mathrm{src}},\ \mathfrak{S}_{\mathrm{rec}},\ \mathfrak{R},\ \mathfrak{S}_{\mathrm{pub}},\ \mathfrak{S}_{\mathrm{con}}\big)$$

whose parts are:

(a) Source part $\mathfrak{S}_{\mathrm{src}}$. A source specification in the sense of Definition "Source specification" (transmitter/def:source-spec), presented to this module as the *transmitter artefact* in its declared mode: a finite non-empty alphabet $\mathcal{X}$ with its declared order, a finite-dimensional system A with declared computational basis, a POVM $\{M_x\}_{x\in\mathcal{X}}$ on A — itself possibly presented as a choice-weighted family (Definition 4.1) — and the specified signal ensemble $\{p_x, \rho_x\}_{x\in\mathcal{X}}$ on the transmitted system A' (Assumption "Emission of the specified ensemble" (transmitter/ass:ens)). The mode is read off the artefact's marginal slot and is never a second datum. In trusted-source mode the slot holds $\tau_A \in S_=(A)$ — at $d_A = 1$ the slot the transmitter filled with [1] included — and at $d_A \geq 2$ the artefact additionally carries a source-replacement certificate $(\tau_A, \Psi, \{p_x, \rho_x^{\mathrm{v}}\}_{x\in\mathcal{X}})$ on AA'' in the sense of Definition

"Source-replacement certificate" (transmitter/def:srcert), written against the virtual ensemble the source segment *starts from*, which lives on the virtual transmitted system A'', the source descriptor of Ξ_1 (Definition "Source segment and its interface conditions" (transmitter/def:source-segment) (S1), (S3)). When $L_{\mathrm{s}} = 0$ that system and that ensemble are the specified ones: $A'' = A'$ and $\rho_x^{\mathrm{v}} = \rho_x$. In untrusted-source mode the slot holds none and there is no certificate.

(b) Receiver part $\mathfrak{S}_{\mathrm{rec}}$. A finite-dimensional physical receiver system with a POVM $\{F_w^{\mathrm{ph}}\}_{w \in \mathcal{V}^{\mathrm{ph}}}$, possibly presented as a choice-weighted family; and, optionally, a public choice whose weight depends on the private setting of the measuring apparatus — a stochastic matrix $P(f \,|\, t)$ with the setting map $t : \mathcal{V} \to \mathcal{T}$ it is read through — to be applied by this module *after* the reduction chain, in the setting-dependent form of Definition 4.1.

(c) Reduction chain $\mathfrak{R} = (\mathfrak{R}_{\mathrm{src}}, \mathfrak{R}_{\mathrm{rec}})$. A reduction chain in the sense of Definition "Reduction chain" (reductions/def:chain), presented to this module as a record: an ordered *pair* of ordered lists of typed links, the *source segment* $\mathfrak{R}_{\mathrm{src}} = (\Xi_1, \ldots, \Xi_{L_{\mathrm{s}}})$ with $L_{\mathrm{s}} \geq 0$, which is a source segment in the sense of Definition "Source segment and its interface conditions" (transmitter/def:source-segment) — its links act on the transmitted system A' and replace Alice's signal ensemble — and the *receiver segment* $\mathfrak{R}_{\mathrm{rec}} = (\Lambda_1, \ldots, \Lambda_L)$ with $L \geq 1$, whose links act on the receiver system and replace Bob's measurement. What this module reads of a link, stated in its own words, is this: each link of either segment carries a type tag, source and target system descriptors, a status tag, its channel as data where the link type has one, the constraint entries it transports, and a licence slot. A receiver-stage link in the sense of Definition "Typed link" (reductions/def:link) carries more — its reachable-set statement and its carried operator pair — and this module reads none of it. The last link of $\mathfrak{R}_{\mathrm{rec}}$ has target descriptor $(Q, \mathbb{S}, (d_s)_s)$ with the sector projectors Π_s, and target POVM $\{F_v\}_{v \in \mathcal{V}}$. The two segments are separate lists because they have different types; the pair is composed source segment first. The interface conditions on a chain that this module decides on the record are those of Definition 7.1.

(d) Public part $\mathfrak{S}_{\mathrm{pub}}$. A party partition $\mathfrak{P}$ (readouts $\pi_{\mathsf{A}}, \pi_{\mathsf{B}}$ onto $\mathcal{O}_{\mathsf{A}}, \mathcal{O}_{\mathsf{B}}$, and the party labels), announcement maps $a_A : \mathcal{O}_{\mathsf{A}} \to \mathcal{C}_A$ and $a_B : \mathcal{O}_{\mathsf{B}} \to \mathcal{C}_B$, a sifting predicate $\varsigma : \mathcal{K} \to \{\mathrm{pass}, \mathrm{fail}\}$, a key rule $r_{\mathsf{h}}^0 : \mathcal{O}_{\mathsf{h}} \times \mathcal{K} \to \mathcal{Z}$, a holder $\mathsf{h} \in \{\mathsf{A}, \mathsf{B}\}$, a finite acceptance alphabet $\widehat{\mathcal{C}}$ and a map $\phi : \mathcal{K} \to \widehat{\mathcal{C}}$. All maps are total.

(e) Constraint part $\mathfrak{S}_{\mathrm{con}}$. A finite, possibly empty, list of entries. Each $\hat{O}_k$ is presented as a block list on Q together with the sector map it is written against, which is carried with it as for every other supplied Q-operator family, and is Hermitian block by block. Each entry carries a right-hand-side datum of one of two types, and the type fixes the entry's tag: homogeneous with a real c_k, or general with a Hermitian Θ_k on A, in trusted-source mode;

a real c_k in untrusted-source mode (Definition 2.5). The constraint part also carries the optional attack-set licence record and an optional feasibility witness $\omega^\star$, likewise a block list with its own map. The map each $\hat{O}_k$ carries is compared with the emitted $\mathfrak{s}$ by check V1, an entry declared here being on the same footing as one transported by a link. The witness's map is not compared there: $\omega^\star$ is a block list on AQ and not a Q-operator, it is no slot of (1.1), and branch (c) of check V7 is where it is required to be a block list against $\mathfrak{s}$ (Definition 6.1).

The general format. A specification to be compiled into the general format of Definition 1.1 (DX1)–(DX6) has the same five parts, read as follows. At one Alice sector, one measured factor, POVM form, two messages neither of which reads the other and no party-local entry these readings are (a)–(e) above, part for part on every component a derivation reads: at those values every such object named below is the object of the same name there and nothing else is presented. On the party labels of the tensor factors of each $\mathcal{H}(Q_s)$, which no derivation of this module reads and which the consuming engine's laboratory premise reads as naming the parties' laboratories, (DX1) restricts the party partition of (d). The party set is unchanged: it is $\{\mathsf{A}, \mathsf{B}\}$, with the two readout maps of $\mathfrak{P}$. What is new is that several measured factors may carry a party label and that the factor tagged A carries one too (DX1).

(a′) Source part. The artefact's marginal slot may hold a block list $(\tau_A^{(t)})_{t\in\mathbb{T}}$ along the Alice sectors the artefact declares, and Alice's measurement arrives in one of the two forms of (DX4): the POVM slot, or, in trusted-source mode only, the ensemble slot holding the block lists $\{N_x\}_{x\in\mathcal{X}}$ of (1.6). The source-replacement certificate may accordingly be declared in pure form, in block form, or as an ensemble presentation, all three in the sense of Definition "Source-replacement certificate" (transmitter/def:srcert). Which form a certificate is in is read off the artefact and is never a second datum, exactly as the mode is. That a form may be *presented* is the transmitter's rule and is stated here. Which forms this module *admits*, and what gate (G2) decides of each form, is Definition 9.1's and is stated once there.

(b′) Receiver part. $\mathfrak{S}_{\mathrm{rec}}$ is an ordered list, one entry per measured factor $f \in \mathbb{F}$ and in the order of $\mathbb{F}$: a party label; a finite-dimensional physical receiver system with a POVM $\{F_{w_f}^{\mathrm{ph},f}\}_{w_f\in\mathcal{V}^{\mathrm{ph},f}}$, possibly presented as a choice-weighted family; and, optionally, a setting-dependent public choice at that factor — a stochastic matrix $P_f(\,\cdot\,|\,\cdot\,)$ with the setting map on that factor's alphabet it is read through — to be applied after that factor's segment, in the setting-dependent form of Definition 4.1. No entry of the list carries an operator on another factor's system, and no product of physical families is presented: the product is the rule (1.5), formed by no one.

(c′) Reduction chain. $\mathfrak{R} = (\mathfrak{R}_{\mathrm{src}}, \mathfrak{R}_{\mathrm{rec}})$ with $\mathfrak{R}_{\mathrm{src}}$ as in (c) and $\mathfrak{R}_{\mathrm{rec}} = (\mathfrak{R}_{\mathrm{rec}}^f)_{f\in\mathbb{F}}$ one receiver segment per measured factor, each $\mathfrak{R}_{\mathrm{rec}}^f = (\Lambda_1^f, \ldots, \Lambda_{L_f}^f)$,

$L_f \geq 1$, a list of receiver-stage links whose source descriptor is factor f's physical receiver system and whose last target descriptor is $(Q_f, \mathbb{S}^f, (d_{f,s_f})_{s_f})$ with target POVM $\{F^f_{v_f}\}_{v_f \in \mathcal{V}^f}$. What this module reads of a link is what (c) lists and nothing more, of each segment separately: the segments are separate lists because their links act on different systems, and no link of one is composable with a link of another. Where the factors' data are the ordered list of the factors' artefacts that the reductions module emits, factor f's receiver segment and target family are that factor's own, and the product rule of Theorem "The emission of several measuring parties" (reductions/thm:emitted-product) is emitted in place of a joint family; that list is what the factor list of (DX1) is built from, in its order. The interface conditions on a chain that this module decides on the record are those of Definition 7.1, per factor for the receiver segments.

(d′) Public part. In place of a_A and a_B the public part declares the transcript of (DX5): the number $L \geq 1$ of messages and, for each m, its sender π_m, its finite alphabet $\mathcal{C}_m$ and its total table $a_m : \mathcal{O}_{\pi_m} \times \mathcal{C}_1 \times \cdots \times \mathcal{C}_{m-1} \to \mathcal{C}_m$. The sifting predicate ς, the key rule r^0_{h}, the holder h, the acceptance alphabet $\widehat{\mathcal{C}}$ and ϕ are declared on the derived $\mathcal{K} = \prod_m \mathcal{C}_m$ of (1.8), which is a set determined by the tables and declared by no one. All tables are total. The party partition $\mathfrak{P}$ is as in (d), with a party label per measured factor and, where there is one, for the factor tagged A.

(e′) Constraint part. Each entry presents, in addition to what (e) lists, its scope: the label of one measured factor of the chain; and $\hat{O}_k$ is a block list on Q_{f_k} together with the sector map of that factor, Hermitian block by block. Its right-hand-side datum is of one of the three forms of Definition 2.5: a real c_k tagged homogeneous, a Hermitian Θ_k tagged general and then itself a block list along $\mathbb{T}$, or a real c_k tagged party-local. The arriving tag is compared with the type of that datum at check V6, which also decides that the scope names a measured factor of the emitted map. An optional witness $\omega^\star$ is a block list over the joint sectors with its own map, compared at branch (c) of V7 and nowhere else, for the reason (e) gives.

Uses Def. 1.1, Def. 2.5, transmitter/def:source-segment, transmitter/def:source-spec, transmitter/def:srcert · *Used by* Def. 3.2, Thm. 3.4, Thm. 4.4, Rem. 4.9, Thm. 5.1, Rem. 5.2, Def. 6.1, Thm. 7.5, Thm. 9.4, Ass. 10.1, Ass. 10.2, Req. 11.8, Req. 11.9, Req. 13.7 · newly derived · implemented (effective: accepted) · *Code* certiqs_key.protocol_compiler.spec:ProtocolSpecification

A specification is well-formed when its declared data are as declared: finite alphabets and systems, total tables, exactly admissible measurement families, the presented physical one included, normalised weights and Hermitian, tagged constraint blocks. The compiler decides every clause.

Every clause below is a condition the compiler decides on the declared numbers and refuses when it fails, so a specification that is not well-formed yields a refusal and not a declaration. Clauses (i)–(iii) are decided by check V1

of Definition 6.1 on the emitted data and by the input types on the consumed tables, and the sector-map half of (vii) by check V1; the measurement families of (iv) by check V3 and condition (C3) of Definition 7.1, the conditional families of a choice-weighted presentation by Requirement 12.1, and the physical receiver family as presented by Requirement 13.5; (v) by Requirement 12.5; (vi) by Requirement 12.6; the Hermiticity in (vii) by check V3 and the tag by check V6 (Requirement 11.5); a failing check is refused (Requirement 13.1). The physical receiver space carries no sector map this module holds, so the physical family is read on it as one block, as Alice's data are read on A.

Definition 3.2 (Well-formed specification)**.** A protocol specification $\mathfrak{S}$ (Definition 3.1) is a *well-formed specification* when its declared data are as declared: (i) all alphabets are finite and non-empty; (ii) all systems are finite-dimensional with the declared dimensions; (iii) all tabulated maps are total on their stated domains; (iv) all declared POVM families are exactly admissible: every family other than the physical receiver family $\{F_w^{\mathrm{ph}}\}_{w\in\mathcal{V}^{\mathrm{ph}}}$ as presented, block by block in the sense of Definition 1.3; and the physical receiver family as presented (in the general format, each measured factor's, on that factor's physical receiver system), on its physical receiver space read as one block, condition (T) being empty for it as Definition 1.3 reads Alice's data: where it is presented directly, (S), (H), (P) and (C) of that definition hold of it in the declared order of its alphabet; where it is presented as a choice-weighted family (Definition 4.1), its conditional families are exactly admissible, block by block against the map they carry, and condition (S) holds of its own declared-order sum. A choice-weighted family whose conditional families carry a sector map is a block list against that map, and reading it as that block list or as one block decides the same (S), (H), (P) and (C), its off-block entries being exact zeros; (v) all public-choice weights are strictly positive — a domain condition on a declared scalar (Definition 4.1) — and sum to one; (vi) every setting-dependent weighting, at whichever measured system or measured factor it is declared, is a stochastic matrix, its entries non-negative and its rows summing to one exactly (the setting-dependent form of Definition 4.1); (vii) every constraint operator is a block list against the upstream module's sector map and Hermitian block by block, and every entry carries its declared right-hand-side tag.

Uses Def. 1.3, Def. 3.1, Def. 4.1 · *Used by* Thm. 3.4, Thm. 3.7, Thm. 5.1, Req. 13.5 · newly derived · accepted

The compilation map fills the declaration from the specification, passing declared data through and constructing only the raw-key map, the index map and the weighted families. In the general format it also assembles the factor list.

The compilation map assembles the declaration from the specification. It passes the declared data through, weights a measurement family where a public choice is presented, and constructs only the raw-key map, the index map and the weighted families. The list below fills the basic format; the paragraphs on the general format extend it slot for slot. That the engine passes the declared data through unchanged, constructs no right-hand side, and forms no product of two factors' families, no lifted operator and no other derived object of the general format as data is Requirements 11.3 and 11.4; the refusals the validator issues on the factor list and on Alice's slots are Requirement 13.1's.

At $d_A = 1$ a trusted-source request may leave the marginal slot empty; the transmitter then fills it with [1] and records the fill (Requirement "The unique marginal served at $d_A = 1$" (transmitter/req:marginal-fill)); why the two source modes impose the same condition there is Remark "The one-dimensional retained system" (transmitter/rem:da1). A certificate that is present and fails any of the five conditions of gate (G2) is refused at every d_A, row 3 of the gate outcome table of Definition 9.3. The order in which the binding rule below has condition (S) decided on a supplied family is the order that family was declared in and also the order the consuming engine re-accumulates in (Requirement "Summation order declared with the data" (foundations/req:declared-order)): one order at all three places.

Definition 3.3 (The compilation map)**.** The *compilation map* is the map $\mathfrak{S} \mapsto \mathfrak{D}$ that fills (1.1) as follows.

- *Alice's system and measurement.* A, d_A and $\{M_x\}$ from $\mathfrak{S}_{\mathrm{src}}$, weighted by Definition 4.1 where a choice family is presented.
- *Alice's marginal.* τ_A is what the transmitter artefact's marginal slot holds: in trusted-source mode the matrix in the slot — at $d_A = 1$ the value [1] the transmitter filled the slot with, the unique element of $S_{=}(A)$ — and $\tau_A = \mathsf{none}$ where the slot is empty. The mode is read off that slot and off nothing else. A certificate the artefact carries is consumed by gate (G2) of Definition 9.1.
- *The measured system and its family.* $(Q, \mathbb{S}, (d_s)_s)$ and $\{F_v\}$ from the last link of $\mathfrak{R}_{\mathrm{rec}}$. Where $\mathfrak{S}_{\mathrm{rec}}$ declares a setting-dependent public choice, the family is weighted after the chain by the setting-dependent form of Definition 4.1: the emitted family is then $N_{v,f} = P(f \mid t(v))\, F_v$ over the support of P, the emitted outcome alphabet is that support, and the setting map t through which P is read is the class map of the joint alphabet of the chain's producer where the chain was emitted by the reductions module (Definition "Joint alphabet, classes and the joint family of a

private setting" (reductions/def:joint-family) (a)), and is declared with the family otherwise.

- *The public maps.* $\mathfrak{P}$, a_A, a_B, ϕ, h verbatim from $\mathfrak{S}_{\mathrm{pub}}$, declared on the emitted alphabets.

- *The constraint list.* $\mathfrak{L}$ is the concatenation of $\mathfrak{S}_{\mathrm{con}}$'s entries with the entries transported by the links of $\mathfrak{R}_{\mathrm{src}}$ and then those of $\mathfrak{R}_{\mathrm{rec}}$, each segment in its own order, each entry with the datum its right-hand-side slot holds and with the tag that datum's type determines (Definition 2.5).

- *The raw-key map.*

$$r(x,v) \coloneqq \begin{cases} r^0_{\mathsf{h}}\big(\pi_{\mathsf{h}}(x,v),\, a(x,v)\big), & \varsigma(a(x,v)) = \text{pass}, \\ \perp, & \varsigma(a(x,v)) = \text{fail}, \end{cases} \qquad \text{with } d_Z \coloneqq |\mathcal{Z}|. \tag{3.1}$$

The index map. Alongside $\mathfrak{D}$ the compiler emits the *index map* idx, the label-to-index map: a choice, for each of $\mathcal{X}$, $\mathcal{V}$, $\mathbb{S}$, $\mathcal{O}_{\mathsf{A}}$, $\mathcal{O}_{\mathsf{B}}$, $\mathcal{C}_A$, $\mathcal{C}_B$, $\widehat{\mathcal{C}}$ and $\mathcal{Z}_\perp$, of a bijection onto an initial segment of $\mathbb{Z}_{\geq 0}$; every emitted list, matrix and function table is indexed through idx, and every semantic label is carried as metadata only. Its component on $\mathbb{S}$ is the total order the sector map $\mathfrak{s}$ already fixes, and not a second choice: one order on the sectors, required by the last clause of D(i) (Definition 1.5) and decided by V1. And idx is what fixes the declared summation order of Definition 1.3 on every alphabet it indexes; it indexes no basis of a matrix, a trace being accumulated in the row order of the emitted array.

The binding rule. On the remaining alphabets the choice is the compiler's, under one binding rule. Where a supplied family arrives with a declared order on the alphabet that indexes it — as the target POVM of the last link of $\mathfrak{R}_{\mathrm{rec}}$ does, its declarer having established its own completeness sum in that order — the compiler takes *that* order as idx's component there. So the order in which condition (S) of Definition 1.3 is decided on a supplied family here is the order that family was declared in, and no family is refused here for an ordering its declarer never chose. Where no order arrives with an alphabet the choice is free, and (S) is decided in the order the compiler chose and emits. This rule prescribes nothing to the declarer of a family: it constrains only which bijection this module picks.

Where a choice-weighted presentation is resolved, this same rule settles the order the decision on each conditional family is read in, and adds nothing to it. The family the weighting produces is indexed by one alphabet — $\mathcal{X}$ where the presentation is Alice's, the physical receiver alphabet where it is a measuring party's, which in Band A is the emitted $\mathcal{V}$ — and where a declared order arrives with that alphabet the rule above takes *it*, here as everywhere: $\mathcal{X}$ arrives ordered with the source part (Definition 3.1 (a)), and

the physical alphabet with the order the fibre sum of Definition 7.1 (C4) is accumulated in. Condition (S) on the conditional family of a choice value is then read in the *restriction of that order* to the elements that choice value contributes, which is a total order on $\mathcal{U}_t$ however the record listed it; the two conditions are independent, only the order restricting. Where no order arrives with that alphabet the free choice is made as Definition 4.1 says, by the record's own listing, and it is emitted. No declarer's order is overridden and no presentation is refused for an order. Where the setting-dependent weighting is applied, the emitted alphabet is the support of P and its order is the product order of Definition 4.1: v in its declarer's declared order and, within each v, f in the declared order of the choice alphabet, the pairs off the support omitted; that is the order (S) is decided in on the emitted family and the order idx emits on it.

The general format. For a specification presented as Definition 3.1's (a′)–(e′) the map fills the general format of Definition 1.1 (DX1)–(DX6) as follows. In the basic format — one Alice sector, one measured factor, POVM form, two messages neither reading the other and no party-local entry — every clause below is the clause of the same subject above, slot for slot, and the map emits what those clauses emit.

The factor list (DX1) is assembled and not declared as a whole: the factor tagged A is the Alice sector map the transmitter artefact declares, where it declares one, and it is the first of the list; where the artefact declares none, $\mathbb{T} = \{t_0\}$ and no A factor is emitted. The factors tagged Q are the entries of $\mathfrak{S}_{\mathrm{rec}}$ in their declared order, each carrying that entry's party label and the sector map of the last target descriptor of that entry's receiver segment, which Definition 7.1 (C1) decides per factor. The one block kind is the kind those maps carry, and the factors' agreement in it is decided by the same four-component comparison V1 makes, taken on the factor list (Definition 6.1, V1). The product sector map, its order, the product alphabet and the joint sectors are derived from that list by (DX1) and (DX3) and are declared, formed and emitted by no one: what is emitted is the factor list and the rules.

Alice's slots. τ_A is what the artefact's marginal slot holds, the block list along $\mathbb{T}$ included; Alice's measurement slot is the artefact's, in the form the artefact declares — the POVM slot or the ensemble slot of (DX4) — weighted by Definition 4.1 in that form where a choice family is presented. The mode is read off the marginal slot as above.

Per measured factor. $(Q_f, \mathbb{S}^f, (d_{f,s_f})_{s_f})$ and $\{F^f_{v_f}\}_{v_f \in \mathcal{V}^f}$ from the last link of that factor's receiver segment and, where that factor's entry of $\mathfrak{S}_{\mathrm{rec}}$ declares a setting-dependent public choice, weighted after that segment by the setting-dependent form of Definition 4.1, so that factor f's emitted family is $P_f(\cdot \mid \cdot)$ times that factor's delivered family over the support of P_f, and factor f's emitted alphabet $\mathcal{V}^f$ is that support in its product order. Each factor's weighting is applied to its own family and to nothing else. No

product of two factors' families is formed by this map, whether emitted or handed on; so the subject of the representability decision of Definition 4.1 is, as in the basic format, the products *that definition's* two constructions form, and no product block (1.5) is a product this map forms.

The public part and the raw key. $\mathfrak{P}$, ϕ, h verbatim from $\mathfrak{S}_{\text{pub}}$ and the transcript verbatim: the sender, the alphabet and the table of each message, declared on the emitted alphabets. The joint announcement function a is not a slot: it is the composition (1.8) of those tables, and r is (3.1) with *that* a, which is what Theorem 5.1 (iv), (vi) is stated of. $\mathfrak{L}$ is the concatenation above, each entry carrying in addition its scope, and each with the datum its right-hand-side slot holds and the tag that datum's type determines; for a party-local entry that is the tag field it arrives with beside a real scalar (Definition 2.5).

The index map in the general format. idx has one component per factor alphabet $\mathcal{V}^f$ and one per factor sector set $\mathbb{S}^f$, and, where an A factor is emitted, one on $\mathbb{T}$, which is the total order that factor's own map fixes. Its components on $\mathcal{V}$ and on $\mathbb{S}$ are the product orders (DX1) and (DX3) derive from the factors' orders and are not further choices, as its component on $\mathbb{S}$ above is not; the joint sectors are ordered with t slowest by (DX1). The binding rule above is applied per factor: where a factor's target family arrives with a declared order on its alphabet, the compiler takes that order there. So condition (S) of Definition 1.3 is decided per factor, in that factor's arriving order, and per Alice sector on Alice's data; no sum over a product alphabet is accumulated by this map or by its consumer.

Uses Def. 1.1, Def. 1.3, Def. 1.5, Def. 2.5, transmitter/def:source-spec · *Used by* Thm. 3.4, Rem. 3.5, Thm. 3.7, Def. 4.1, Thm. 4.4, Rem. 4.6, Rem. 4.8, Thm. 5.1, Thm. 5.5, Def. 6.1, Rem. 7.4, Thm. 7.5, Thm. 8.2, Thm. 9.4, Req. 11.1, Req. 11.3, Req. 11.8, Req. 12.1, Req. 13.11, meat-core/cor:reference-companion-factors, meat-core/prop:reference-companion · newly derived · implemented (effective: accepted) · *Code* certiqs_key.protocol_compiler.compile: compile_specification · *Oracles* registry_protocol_compiler.py

A declaration the compilation map emits satisfies the typing conditions D(i), D(ii), D(iv) and D(vi), and its index map is a bijection on every alphabet it indexes. Each follows from the construction or from a check decided on the emitted data, never from a source certificate.

A declaration is emitted only when no check refused it (Requirement 14.2, row 1). The typing conditions then follow either from how the compilation map builds the declaration or from a check that decides them on the emitted data.

Theorem 3.4 (Typing of an emitted declaration)**.** Let $\mathfrak{S}$ be a well-formed specification (Definition 3.2) satisfying Assumption 10.1, and let $\mathfrak{D}$ be a declaration that the compilation map of Definition 3.3 *emits*, that is, one for which no check of Definition 6.1 refused. Then $\mathfrak{D}$, together with the index map idx emitted alongside it, satisfies the target conditions D(i), D(ii),

D(iv) and D(vi) of Definition 1.5, and idx is a bijection on each alphabet it indexes, so that every emitted operator list is totally ordered and its index is recoverable from its semantic label.

Uses Def. 1.5, Def. 2.5, Def. 3.1, Def. 3.2, Def. 3.3, Def. 6.1 · *Assumes* A-SPEC · *Used by* Thm. 3.7 · newly derived · accepted · *Code* certiqs_key.protocol_compiler.compile: compile_specification · *Oracles* registry_protocol_compiler.py

Proof. D(i): each system of $\mathfrak{S}$ is declared finite-dimensional and each alphabet finite and non-empty (Definition 3.2 (i), (ii)); Q is finite-dimensional because $d_Q = \sum_{s\in\mathbb{S}} d_s$ is a finite sum of finite terms. The emitted maps are a_A, a_B, ϕ, π_{A}, π_{B} — total by Definition 3.1 (d) — and r, which is total because (3.1) is defined by a case distinction on the value of the total map $\varsigma \circ a$, both branches of which are defined on all of $\mathcal{X} \times \mathcal{V}$. The sector-map clause: the emitted map $\mathfrak{s}$ is the target descriptor of the last link of $\mathfrak{R}_{\mathrm{rec}}$ (Definition 3.3), and it is emitted once; every emitted Q-operator is a block list against *that* object, because Definition 3.3 fills $\{F_v\}$ from that same link's target POVM and $\mathfrak{L}$ from lists each carried against the map its declarer wrote it against. What is not automatic, and is where the hypothesis of the theorem is used, is that each declarer's map *is* $\mathfrak{s}$: a family presented against a different block kind, a different identifier list, a different order or a different dimension is a well-typed object of the wrong type, and check V1 rejects exactly that, so a declaration that is emitted at all is one for which V1 did not refuse. $d_Q = \sum_s d_s$ is then the identity that defines d_Q from the emitted map. The last clause of D(i), that the restriction of idx to $\mathbb{S}$ is the order $\mathfrak{s}$ fixes, holds *by construction*: Definition 3.3 takes that component of idx to be the sector map's own order and not a second order. Check V1 of Definition 6.1 re-decides it on the emitted data, a condition on an emitted object being decided on that object and never inferred from how it was built, and a declaration that is emitted at all is one for which V1 did not refuse.

D(ii): the marginal slot holds either a matrix or none, and no third value is emitted (Definition 3.3). The three numerical conjuncts are decided on the emitted matrix and on nothing else: check V2 of Definition 6.1 decides $\tau_A = \tau_A^{\dagger}$, $\tau_A \succeq 0$ and $\operatorname{tr}\tau_A = 1$, each exactly, and decides that the trace sum rounds nowhere; and a declaration that is emitted at all is one for which V2 did not refuse, so D(ii) holds of the declared numbers. This is the only route, a condition on the emitted matrix being decided exactly on that matrix and never inferred from the certificate (Requirement "Exact admissibility at every module boundary" (foundations/req:boundary-admission) (ii); for the trace sum Requirement "Exact verification before emission" (foundations/req:exact-emission) (v)): an emitted declaration satisfies D(ii) even where a certificate is malformed. The certificate supplies a *modelling* statement about the same matrix, not a substitute for that decision: in trusted-source mode τ_A is the certificate's first component, and (SR1), decided on the emitted data by gate (G2), relates it to $\operatorname{tr}_{A''}\Psi$, which is positive semidefinite with trace $\operatorname{tr}\Psi = 1$.

That is what makes the emitted marginal *the certificate's* marginal rather than an independently declared number; it is not what makes D(ii) hold.

D(iv): the range of (3.1) is contained in $\operatorname{ran} r^0_{\mathsf{h}} \cup \{\perp\} \subseteq \mathcal{Z} \sqcup \{\perp\}$, because r^0_{h} maps into $\mathcal{Z}$ by Definition 3.1 (d).

D(vi): the emitted list is the concatenation of the specification's entries with the entries transported by the links of both segments; each entry is emitted with its relation symbol and the datum its right-hand-side slot holds unchanged, and with the tag that datum's type fixes (Definition 2.5). Nothing in the hypotheses on $\mathfrak{S}$ prevents a link from transporting an entry whose declared form does not match the emitted source mode — an entry carrying a matrix where the mode admits only a scalar, or one whose datum is of neither admitted type — and that is a property of the declared data, not of the compilation map; so this clause is where the hypothesis of the theorem is used: check V6 of Definition 6.1 rejects exactly such an entry, and a declaration that is emitted at all is one for which V6 did not refuse.

idx: each component is chosen as a bijection onto an initial segment of $\mathbb{Z}_{\geq 0}$, which exists because the alphabet is finite and is injective and surjective by construction; the inverse table is emitted with it, so the semantic label of an index is recoverable.

The general format. For a declaration the general-format clauses of Definition 3.3 emit, the four conditions are read as in the general-format paragraph of Definition 1.5, and the argument is the one above with the checks read as in the general-format paragraph of Definition 6.1; a declaration that is emitted at all is one for which none of them refused. D(i): finiteness and totality as above, every table a_m of the transcript included; the factor list with at most one factor tagged A and then first, at least one tagged Q and one block kind, the dimension identities, the agreement of the map carried by each per-factor family, by each entry's operator and by Alice's declared data with the map of its factor, and the restriction of idx to each factor's sectors and to $\mathbb{T}$ are exactly the clauses V1 decides in its general-format reading, the product orders being derived by (DX1) and (DX3) and not further conditions. D(ii): V2 in its general-format reading decides every Alice block of τ_A Hermitian and positive semidefinite and their total trace equal to 1 with every addition rounding nowhere, and, where Alice's slot is in ensemble form, every block non-zero. D(iv): as above, r^0_{h} being typed into $\mathcal{Z}$ by Definition 3.1 (d), read on the derived $\mathcal{K}$ by (d′). D(vi): each entry is emitted with its scope, its relation symbol and its datum, and with the tag that datum's type fixes or, for the party-local form, the tag field it arrives with (Definition 2.5); V6 in its general-format reading decides the relation symbol by form, the datum against its tag, a general Θ_k Hermitian block by block along $\mathbb{T}$, and that the scope names a measured factor of the map, and refuses an entry failing any of them. The index map: each component — one per factor alphabet, one per factor sector set and, where an A factor is emitted, one on $\mathbb{T}$ — is chosen as above, and its components on $\mathcal{V}$ and on

$\mathbb{S}$ are the product orders, bijections because each factor's component is. □

Built from the sifting predicate and the holder's key rule, the raw-key map meets D(v) and D(viii) by construction. The only residual obligation is that the key rule map into the key values, a typing condition on one function table that A-SPEC carries.

Remark 3.5 (The raw-key map built from sifting and the key rule). A specification that supplied r directly would have to be *checked* for D(v) and D(viii). Built as (3.1), r satisfies both by construction (Theorem 5.1), and the only residual obligation is the side condition that r^0_{h} maps into $\mathcal{Z}$ rather than into $\mathcal{Z}_\perp$, which is a typing condition on the specification, decidable by inspection of one function table and carried as a clause of Assumption 10.1 (Remark 5.2).

Uses Def. 3.3 · newly derived · accepted

The joint outcome distribution of a declaration is given by measurement traces, and the acceptance statistic groups it through the public maps. Both functionals are affine on all operators and are probability distributions on every feasible state.

The two functionals below are defined on every operator on AQ, and not only on positive ones, because affinity is what Theorem 3.7 (iv) and Lemma 4.2 (iii) use and neither needs positivity. The consuming engine minimises over the same set: its own definition of the feasible set uses Definition 2.1.

Definition 3.6 (Declared statistics functional)**.** For a declaration $\mathfrak{D}$ and $\omega \in \mathcal{L}(AQ)$ put

$$P_{\mathfrak{D}}[\omega](x,v) := \operatorname{tr}\big[(M_x \otimes F_v)\,\omega\big], \qquad \mathbf{\Phi}[\omega](\hat{c}) := \sum_{(x,v):\,\phi(a(x,v))=\hat{c}} P_{\mathfrak{D}}[\omega](x,v). \tag{3.2}$$

Both are defined by trace formulas and are therefore defined, and affine, on all of $\mathcal{L}(AQ)$. The feasible set $\mathcal{W}$ of Definition 2.1 is contained in $\mathrm{Pos}(AQ)$, and on it the values are a probability distribution (Theorem 3.7 (iv), and (iv′) in the general format).

Uses Def. 2.1 · *Used by* Thm. 3.7, Thm. 4.4, Rem. 4.9 · newly derived · implemented (effective: accepted) · *Code* certiqs_key.protocol_compiler.statistics: phi_distribution · *Oracles* registry_protocol_compiler.py

Declared measurement families are exact POVMs block by block, per factor and, in POVM form, per Alice sector in the general format; the product and denoted ensemble families are POVMs by theorem. So D(iii) holds, and the statistics are a distribution on every normalised positive block-list state.

Theorem 3.7 (Exact measurement families and the statistics functional)**.** Let $\mathfrak{S}$ be a well-formed specification (Definition 3.2) satisfying Assumption 10.1, and let $\mathfrak{D}$ be emitted by Definition 3.3 in the basic format. Then:

(i) $\{M_x\}_{x\in\mathcal{X}}$ is a POVM on A and, for each $s \in \mathbb{S}$, the block family $\{F_v^{(s)}\}_{v\in\mathcal{V}}$ is a POVM on Q_s — each predicate holding *of the declared*

numbers, decided exactly — and each of the completeness sums $\sum_x M_x$ and $\sum_v F_v^{(s)}$, formed in the declared order, rounds nowhere, so that each computed sum *is* the sum it is written as (Definition 1.3);

(ii) every F_v and every $\hat{O}_k$ is sector-block-diagonal *by construction*: it is emitted as the list $\big(F_v^{(s)}\big)_s$, respectively $\big(\hat{O}_k^{(s)}\big)_s$, and the operator it denotes is $\bigoplus_s F_v^{(s)}$. The two formulations of block-diagonality remain equivalent — for an operator F on Q, $\Pi_s F \Pi_{s'} = 0$ for all $s \neq s'$ if and only if $F = \bigoplus_s \Pi_s F \Pi_s$ — and that equivalence is what licenses the *type*: a list of blocks denotes exactly the operators the vanishing condition picks out, and no other. It is not checked, because the condition has no data to be checked on;

(iii) hence D(iii) holds: its typing half by (ii) and check V1; its first four predicates — those on $\{M_x\}$ and on each block family $\{F_v^{(s)}\}_v$ — by (i); and its fifth, Hermiticity of $\hat{O}_k^{(s)}$ for each (k, s), by the last clause of check V3 (Definition 6.1), which decides it exactly on the emitted constraint entries and which (i) does not state; and

(iv) $P_{\mathfrak{D}}[\omega]$ of (3.2) and $\mathbf{\Phi}[\omega]$ are affine in ω, $\mathbf{\Phi}[\omega]$ depends on ω only through $P_{\mathfrak{D}}[\omega]$, and the fibres of $\phi \circ a$ partition $\mathcal{X} \times \mathcal{V}$, so $\mathbf{\Phi}[\omega]$ is the pushforward of $P_{\mathfrak{D}}[\omega]$ and inherits its total mass. For every block list ω with $\omega^{(s)} \succeq 0$ for each s and $\operatorname{tr} \omega = 1$ — in particular for every $\omega \in \mathcal{W}$ — $P_{\mathfrak{D}}[\omega]$ is a probability distribution on $\mathcal{X} \times \mathcal{V}$ and $\mathbf{\Phi}[\omega]$ one on $\widehat{\mathcal{C}}$.

The general format. Let $\mathfrak{D}$ instead be emitted by the general-format clauses of Definition 3.3, in the general format of Definition 1.1 (DX1)–(DX6). Then (i)–(iv) hold with the following readings, of which the four above are the case of the basic format.

(i′) for every measured factor f and every $s_f \in \mathbb{S}^f$ the block family $\{F_{v_f}^{f,(s_f)}\}_{v_f \in \mathcal{V}^f}$ is a POVM on Q_{f,s_f}; and for every Alice sector t, in POVM form $\{M_x^{(t)}\}_{x \in \mathcal{X}}$ is a POVM on A_t, and in ensemble form every $N_x^{(t)}$ is Hermitian and positive semidefinite, with $\sum_x N_x^{(t)} = \tau_A^{(t)}$ and $\tau_A^{(t)} \neq 0$ — each predicate holding of the declared numbers, decided exactly, and each of these sums, formed in its declared order, rounding nowhere. The product family (1.5) is a POVM on Q and, in ensemble form, the family (1.7) is a POVM on A, both as identities of real numbers, and neither is formed, decided or summed;

(ii′) every $F_{v_f}^f$ is a block list against factor f's sector map and every $\hat{O}_k$ one against the sector map of its scope, and τ_A, every operator of Alice's measurement slot and every general Θ_k is a block list along $\mathbb{T}$, each block diagonal by construction as in (ii); and the product family is

block diagonal for the product sector map, $\Pi_s F_v \Pi_{s'} = 0$ for $s \neq s'$, with the s-block of (1.5);

(iii′) hence D(iii) holds in its general-format reading (Definition 1.5); and

(iv′) (iv) holds for every list ω over the joint sectors with every block positive semidefinite and $\operatorname{tr}\omega = 1$ — in particular for every $\omega \in \mathcal{W}$ — with F_v the product (1.5) and, in ensemble form, M_x the operator (1.7) of the declaration $\mathfrak{D}$ denotes: $P_{\mathfrak{D}}[\omega]$ is a probability distribution on $\mathcal{X} \times \mathcal{V}$ and $\mathbf{\Phi}[\omega]$ one on $\widehat{\mathcal{C}}$.

Uses Def. 1.1, Def. 1.3, Def. 1.5, Def. 2.1, Lem. 2.2, Def. 3.2, Def. 3.3, Thm. 3.4, Def. 3.6, Def. 4.1, Thm. 5.1, Def. 6.1 · *Assumes* A-SPEC · *Used by* Thm. 4.4, Rem. 4.5, Thm. 6.2, Thm. 7.5 · newly derived · accepted · *Code* certiqs_key.protocol_compiler.validation:check_v3 · *Oracles* registry_protocol_compiler.py

Proof. (i) The predicates are decided on the emitted objects, not inferred (Requirement "Exact verification before emission" (foundations/req:exact-emission)): check V3 of Definition 6.1 decides, exactly, $M_x = M_x^\dagger$ and $M_x \succeq 0$ for each x, $\sum_x M_x = \mathbb{1}_A$ with the sum rounding nowhere, and, per block, $F_v^{(s)} = F_v^{(s)\dagger}$, $F_v^{(s)} \succeq 0$ and $\sum_v F_v^{(s)} = \mathbb{1}_{Q_s}$ with that sum rounding nowhere; a declaration that is emitted at all is one for which V3 did not refuse. Two constructions in this contract would have licensed an inference instead, and neither is used as one. Where the specification presents a family as choice-weighted, Lemma 4.2 (i) shows the weighted family is a POVM whenever the conditional families are; the emitted family's predicates are nonetheless decided, because the emitted matrices are the products $q_t N_{u|t}$ as represented, which the compiler forms exactly and then decides to be binary64 data before emitting them at all (Definition 4.1), and because the lemma says nothing about condition (S) on the emitted family's own sum. Where $\{F_v\}$ arrives as the target POVM of the last link of $\mathfrak{R}$, completeness is likewise decided by V3 on the emitted blocks and never inferred from the physical family; Theorem 7.5 records what the chain is and is not asked to guarantee. No floating-point budget is therefore needed anywhere below: nothing here depends on how much rounding a construction introduced, because the result is decided.

(ii) By Definition 3.3 the emitted $\{F_v\}$ and $\{\hat{O}_k\}$ are block lists against $\mathfrak{s}$, so the first sentence is the emission rule. For the equivalence: any $F \in \mathcal{L}(Q)$ satisfies $F = \sum_{s,s'} \Pi_s F \Pi_{s'}$, since $\sum_s \Pi_s = \mathbb{1}_Q$ and the Π_s are mutually orthogonal projectors; hence $\Pi_s F \Pi_{s'} = 0$ for all $s \neq s'$ holds if and only if $F = \sum_s \Pi_s F \Pi_s = \bigoplus_s \Pi_s F \Pi_s$, i.e. if and only if F is the direct sum of its diagonal blocks. So the block lists denote exactly the block-diagonal operators, which is what makes the type faithful: nothing expressible as a list violates the condition, and nothing satisfying the condition fails to be expressible as a list. This is a statement about the representation, not a

check on data. Where the specification presents $\{F_v\}$ as a choice-weighted family, the weighting is applied blockwise, $N^{(s)}_{u,t} = q_t N^{(s)}_{u|t}$, so the emitted object is a block list whatever the presentation (Lemma 4.2 (ii)).

(iii) is (i) and (ii) together with D(iii)'s statement. Its typing half is (ii), and the sector-map agreement that half rests on is check V1 (Theorem 3.4, D(i)). Of D(iii)'s five predicates, the four on $\{M_x\}$ and on the block families $\{F^{(s)}_v\}_v$ are (i). The fifth is not among them and is cited from where it is decided: D(iii) requires in addition that $\hat{O}^{(s)}_k$ be Hermitian for each (k, s), which (i) does not state, and which the last clause of check V3 (Definition 6.1) decides exactly on the emitted constraint entries — a declaration that is emitted at all being one for which V3 did not refuse, which is the route (i)'s own four take (Theorem 3.4). It is among the decisions (6.1) counts, as that equation's $m\,|\mathbb{S}|$ term, and it is the hypothesis of Lemma 2.2, which makes (2.2) well posed.

(iv) Affinity of both maps is immediate from linearity of the trace in ω; that $\mathbf{\Phi}[\omega]$ depends on ω only through $P_{\mathfrak{D}}[\omega]$ is the definition (3.2). Since $\phi \circ a$ is a total map $\mathcal{X} \times \mathcal{V} \to \widehat{\mathcal{C}}$ (Theorem 3.4, D(i)), the fibres $\{(x, v) : \phi(a(x, v)) = \hat{c}\}$ partition $\mathcal{X} \times \mathcal{V}$, so $\mathbf{\Phi}[\omega]$ is the pushforward of $P_{\mathfrak{D}}[\omega]$ and has the same total mass.

For the distribution statement, let ω be a block list with $\omega^{(s)} \succeq 0$ for each s and $\operatorname{tr}\omega = 1$. By (i) each M_x and each $F^{(s)}_v$ is Hermitian with non-negative spectrum, hence $M_x \succeq 0$ and $F_v = \bigoplus_s F^{(s)}_v \succeq 0$, positivity of a block-diagonal operator being positivity of each of its blocks; and $\omega = \bigoplus_s \omega^{(s)} \succeq 0$ for the same reason. Then non-negativity and realness together, since $M_x \otimes F_v \succeq 0$ and $\omega \succeq 0$ give $\operatorname{tr}[(M_x \otimes F_v)\omega] \geq 0$ — the trace of a product of two positive semidefinite operators is a non-negative *real* number. And normalization:

$$\sum_{x,v} P_{\mathfrak{D}}[\omega](x, v) = \operatorname{tr}\Big[\Big(\sum_x M_x \otimes \sum_v F_v\Big)\omega\Big] = \operatorname{tr}[(\mathbb{1}_A \otimes \mathbb{1}_Q)\,\omega] = \operatorname{tr}\omega = 1,$$

by bilinearity of the tensor product and the two completeness identities of (i). This step is where condition (S) is used: what (i) delivers is $\sum_x M_x = \mathbb{1}_A$ and $\sum_v F^{(s)}_v = \mathbb{1}_{Q_s}$ as identities of the declared numbers, and it delivers them only because the sums forming them round nowhere — a computed sum equal to the identity would not license this display. Hence $P_{\mathfrak{D}}[\omega]$ is a probability distribution, and so is its pushforward $\mathbf{\Phi}[\omega]$.

The general format. (i′) Check V3 in its general-format reading (Definition 6.1) decides, exactly: per measured factor, $F^{f,(s_f)}_{v_f}$ Hermitian and positive semidefinite for every (v_f, s_f) and, for each s_f, $\sum_{v_f} F^{f,(s_f)}_{v_f} = \mathbb{1}_{Q_{f,s_f}}$ with its declared-order sum rounding nowhere; on Alice's data, block by block along $\mathbb{T}$, in POVM form $M^{(t)}_x$ Hermitian and positive semidefinite for every (x, t) and $\sum_x M^{(t)}_x = \mathbb{1}_{A_t}$ for every t with that sum rounding

nowhere, and in ensemble form $N_x^{(t)}$ Hermitian and positive semidefinite for every (x,t) and $\sum_x N_x^{(t)} = \tau_A^{(t)}$ entry by entry with that sum rounding nowhere; and check V2 in its general-format reading decides $\tau_A^{(t)} \neq 0$ for every t where the slot is in ensemble form. A declaration that is emitted at all is one for which neither check refused, so each of these holds of the declared numbers. The product family: every $F_{v_f}^f = \bigoplus_{s_f} F_{v_f}^{f,(s_f)}$ is then Hermitian and positive semidefinite and $\sum_{v_f} F_{v_f}^f = \mathbb{1}_{Q_f}$, an identity of the declared numbers because each of its blocks' sums rounds nowhere; a tensor product of Hermitian positive semidefinite operators is Hermitian and positive semidefinite, and $\sum_v F_v = \sum_v \bigotimes_f F_{v_f}^f = \bigotimes_f \sum_{v_f} F_{v_f}^f = \bigotimes_f \mathbb{1}_{Q_f} = \mathbb{1}_Q$ by multilinearity, the sum running over $\mathcal{V} = \prod_f \mathcal{V}^f$ (Definition 1.1 (DX3)). The denoted family: Theorem 5.1 (v), whose hypothesis is the conditions Definition 1.3 states for the ensemble form, which are those just decided. Neither family is a datum (Requirement 11.4): nothing of either is formed, decided or summed, so no summation condition arises for either, and both statements are identities of the real numbers the declared data denote.

(ii′) By the general-format clauses of Definition 3.3 each $F_{v_f}^f$ is emitted as a block list against factor f's map and each $\hat{O}_k$ against the map of its scope, and by Definition 1.1 (DX2) Alice's data are block lists along $\mathbb{T}$; for each such list the argument of (ii), with the projectors $\Pi_{s_f}^f$, respectively Π_t^A, in place of Π_s, gives the equivalence. For the product family, $\Pi_s = \bigotimes_f \Pi_{s_f}^f$ by (1.4), so $\Pi_s F_v \Pi_{s'} = \bigotimes_f \Pi_{s_f}^f F_{v_f}^f \Pi_{s'_f}^f$; for $s \neq s'$ some factor has $s_f \neq s'_f$, and its tensor factor vanishes because $F_{v_f}^f$ is block diagonal for factor f's map; and $\Pi_s F_v \Pi_s = \bigotimes_f F_{v_f}^{f,(s_f)}$ is the block of (1.5).

(iii′) D(iii) in its general-format reading (Definition 1.5) asks for the typing and for the predicates of Definition 1.3 in their general-format reading, per measured factor and per Alice sector, with the ensemble-form conditions in place of those on $\{M_x\}$ where that form is declared, and it refuses ensemble form in untrusted-source mode. The typing is (ii′), the agreement of each list's map with the map of its factor being decided by check V1 in its general-format reading (Theorem 3.4, D(i)); the predicates and summation conditions on the per-factor families and on Alice's data are (i′); Hermiticity of $\hat{O}_k^{(s_{f_k})}$ for every entry and every block of its scope is the last clause of check V3 in its general-format reading; and an ensemble-form slot beside a marginal slot holding none is refused by that check's clause on which slot is filled. A declaration that is emitted at all is one for which V1 and V3 did not refuse.

(iv′) Affinity and the pushforward are as in (iv), with a the function (1.8) derives, total into $\mathcal{K}$ (Theorem 5.1 (iv)), and ϕ total on $\mathcal{K}$ (Theorem 3.4, D(i)). For the distribution statement let ω be such a list. $M_x \succeq 0$: in POVM form $M_x = \bigoplus_t M_x^{(t)}$ with every block positive semidefinite

by (i′), and in ensemble form by Theorem 5.1 (v); $F_v \succeq 0$ by (i′); and $\omega = \bigoplus_{(t,s)} \omega^{(t,s)} \succeq 0$, positivity of a block-diagonal operator being positivity of each block. So $\mathrm{tr}[(M_x \otimes F_v)\omega]$ is a non-negative real number. And $\sum_x M_x = \bigoplus_t \sum_x M_x^{(t)} = \bigoplus_t \mathbb{1}_{A_t} = \mathbb{1}_A$, in POVM form an identity of the declared numbers by (i′) and in ensemble form an identity of real numbers by Theorem 5.1 (v), while $\sum_v F_v = \mathbb{1}_Q$ by (i′); so the normalization display of (iv) holds with these operators and gives $\sum_{x,v} P_{\mathfrak{D}}[\omega](x,v) = \mathrm{tr}\,\omega = 1$. Condition (S) is used there per measured factor and, in POVM form, per Alice sector, which is what (i′) delivers; no sum over the product alphabet and no sum of the denoted family is computed, so none needs a summation condition. □

Were the acceptance-statistic map partial, its fibres would not partition the outcome pairs and the acceptance statistic would lose mass. Totality is therefore decided by check V1, not assumed.

Remark 3.8 (Totality of the acceptance-statistic map). If ϕ were partial, the fibres would not partition $\mathcal{X} \times \mathcal{V}$: the total mass of $\mathbf{\Phi}[\omega]$ would fall short of that of $P_{\mathfrak{D}}[\omega]$ by the mass of the unhandled classes, for some ω — a lost probability, and an acceptance statistic that does not sum to one. Totality is therefore decided by check V1 of Definition 6.1, not assumed, and a specification with an unhandled announcement class is refused (Requirement 13.1).

newly derived · accepted

4 Public choices as measurement weights

A public choice of known bias enters the declaration as a weight multiplying a complete conditional measurement, either independently of every setting or, at a measured factor of either party after its chain, through a stochastic matrix read off the private setting of the station that measures it. The main result, Theorem 4.4, shows that the encoding keeps the family a measurement, that the setting-independent form reproduces the declared weights exactly wherever it is the emitted family, that the setting-dependent relabelling commutes with the reduction chain, and that no separate fixed fraction is ever emitted. Each declared conditional family is decided before any product is formed; the remarks and the example say why that decision cannot be left to the validator, where the relabelling is applied, and why the outcome alphabet is chosen upstream. A factor enclosure of a delivered block survives the setting-dependent weighting, the weight going into its core (Lemma 4.3).

Imports foundations/def:binary64, foundations/def:factor-enclosure

A public choice enters as a weight on a complete conditional measurement: one declared weight per choice value, or, at a measured factor after its chain, a stochastic weight read through its station's setting. Weights and families are binary64 numbers read exactly; products are formed exactly.

A per-round public choice of known bias — a basis choice, a choice between test and generation rounds, a probe setting — is encoded as a weight inside the measurement operators and never as a separate fraction. There are two constructions: one declared weight per choice value, independent of every setting, and, at a measured factor after the reduction chain, a weight that depends on the private setting of the station that measures it.

Assumption 10.4 and Remark 4.8 are why a setting-dependent weight is read through the station's setting and through nothing else, and Remark 4.8 and Theorem 4.4 why that form is sound after the chain and is not a link of it. Which conditions of the weighted family do and do not follow from the decision on the conditional families, and why that decision is taken here rather than among the target conditions, is Remark 4.6; that the weighted family can satisfy all five while none of the conditional families it was built from is a POVM is Example 4.7. The order in which the weights are summed is declared with the public-choice record, as Requirement "Summation order declared with the data" (foundations/req:declared-order) requires of every summed predicate, and a presentation indexed by a product alphabet is refused under Requirement 11.4.

Definition 4.1 (Choice-weighted families)**.** Let S be a finite-dimensional system, $\mathcal{T}_{\mathrm{pub}}$ a finite non-empty *public-choice alphabet* with declared weights $q_t > 0$, $\sum_{t\in\mathcal{T}_{\mathrm{pub}}} q_t = 1$, and for each $t \in \mathcal{T}_{\mathrm{pub}}$ a POVM $\{N_{u|t}\}_{u\in\mathcal{U}_t}$ on S. The *choice-weighted family* is indexed by the disjoint union $\mathcal{V} \coloneqq \bigsqcup_{t\in\mathcal{T}_{\mathrm{pub}}} \mathcal{U}_t \times \{t\}$ and defined by

$$N_{u,t} \coloneqq q_t\, N_{u|t}, \qquad (u,t) \in \mathcal{V}. \tag{4.1}$$

Several independent choices compose by iterating (4.1); the resulting weight is the product of the individual weights. *What is declared, and therefore what is decided, is the innermost family and each weight list*: an iterated presentation declares $\{N_{u|t}\}_u$ for each value of each choice together with one weight list per choice, the decision on the conditional families is taken on those, and the intermediate families are this module's own products, which inherit admissibility by Lemma 4.2 (i) at each step and carry no second declared condition; the representability decision is taken on every product the map forms, the intermediate ones included. This is the *setting-independent form*: the weight of a choice value is one declared number, the same in every round and whatever else either party's apparatus did.

A weighting that depends on a station's setting. Let $\{F_v\}_{v\in\mathcal{V}}$ be a POVM on S over a finite alphabet $\mathcal{V}$ with a declared order — the target family of the last link of $\mathfrak{R}_{\mathrm{rec}}$, as Definition 3.3 takes it — and let $t : \mathcal{V} \to \mathcal{T}$ be a declared total map onto a finite non-empty *setting alphabet*, the *setting*

component of the outcome: the value of the private per-round setting of the measuring apparatus that the outcome v records. Where the chain was emitted by the reductions module, t is the class map of that module's joint alphabet (Definition "Joint alphabet, classes and the joint family of a private setting" (reductions/def:joint-family) (a): $\mathcal{V} = \mathcal{Y} \times \mathcal{T}$ with classes $\mathcal{V}_t$); where the family is declared directly, t is declared with it, the singleton $\mathcal{T} = \{*\}$ being the case of no setting. Let $\mathcal{F}$ be a finite non-empty *choice alphabet* with a declared order and let $P(f \mid t)$, $(t, f) \in \mathcal{T} \times \mathcal{F}$, be a *stochastic matrix*: $P(f \mid t) \geq 0$ and $\sum_{f \in \mathcal{F}} P(f \mid t) = 1$ for every $t \in \mathcal{T}$. The *support of the weighting* is

$$\mathcal{V}_P \coloneqq \big\{(v, f) \in \mathcal{V} \times \mathcal{F} : \ P(f \mid t(v)) > 0\big\}, \tag{4.2}$$

ordered by the product order — v in the declared order of $\mathcal{V}$ and, within each v, f in the declared order of $\mathcal{F}$, the pairs off the support omitted — and the choice-weighted family in its *setting-dependent form* is

$$N_{v,f} \coloneqq P(f \mid t(v))\, F_v, \qquad (v, f) \in \mathcal{V}_P. \tag{4.3}$$

The setting-independent form is the case $P(f \mid t) = q_f$ for every t, in which (4.3) is (4.1) with $\mathcal{T}_{\text{pub}} = \mathcal{F}$ and $N_{v|f} = F_v$ for every f — one conditional family, weighted by a choice that does not see the setting; conversely, (4.1) with conditional families that differ with the choice is not of the form (4.3), and the two forms are two constructions overlapping in that case. *In the overlap the two constructions do not agree on an order, and the order is a property of the presentation and not of the family*: the alphabet the first form produces is $\bigsqcup_f \mathcal{U}_f \times \{f\}$, listed choice-major, while the support (4.2) of the second is listed outcome-major, so the two are transposes of one another and one family declared through the two records has its condition (S) decided in two different orders. (S) is order-dependent by construction (Definition 1.3), and the two orders are not reconciled: what fixes the order is the record that was declared, and a declarer that declares the same family both ways must expect the decision its own record's order gives. *P is read through the setting and through nothing else.* It is declared on $\mathcal{T} \times \mathcal{F}$ and reaches an outcome only through $t(v)$. Zero entries of P are admitted, and the support is what they are for: a pair (v, f) with $P(f \mid t(v)) = 0$ is not an outcome, is not emitted and indexes nothing, so every emitted element of (4.3) carries a strictly positive weight, exactly as $q_t > 0$ requires of every element of (4.1). Which choice values a given setting admits is therefore declared by the zero pattern of P and not by a zero-weight outcome. The form is applied by the compiler *after* the reduction chain, to the family the last link delivered (Definition 3.3).

The reading of each declared condition. The weights are binary64 numbers and the conditional families binary64 matrices. So $\sum_t q_t = 1$ is decided *exactly*, with the sum accumulated in the declared order of $\mathcal{T}_{\text{pub}}$ — the order in which the declared public-choice record lists $\mathcal{T}_{\text{pub}}$, fixed there because

idx does not index $\mathcal{T}_{\text{pub}}$ (Definition 3.3) — and required to round nowhere; and "$\{N_{u|t}\}_u$ is a POVM on S" is exact admissibility in the sense of Definition 1.3, per block where S carries sectors: (H), (P) and (C) per block; (S) on the completeness sum $\sum_{u\in\mathcal{U}_t} N_{u|t}$, accumulated in the order stated below and required to round nowhere; and (T) as the agreement — in block kind, in block identifiers, in their order and in their dimensions — of the maps *the conditional families of one presentation themselves carry*: there is one such map for the presentation, and the family the weighting produces is a block list against *it*. On a system carrying no sector decomposition, Alice's A, (T) is empty. This condition is decided of each $t \in \mathcal{T}_{\text{pub}}$ separately, on the declared numbers.

In which order (S) is read on a conditional family. The family the weighting produces is indexed by one alphabet, and that alphabet carries one order: $\mathcal{X}$ where the presentation is Alice's, which arrives with its declared order (Definition 3.1 (a)); the physical receiver alphabet where it is a measuring party's, which is the order the fibre sum of Definition 7.1 (C4) is accumulated in and, in Band A, is the emitted alphabet's own. Condition (S) on the conditional family $\{N_{u|t}\}_u$ is decided in the *restriction of that order* to the elements the choice value t contributes — a total order on $\mathcal{U}_t$ whatever the presentation listed, so no compatibility of the record's listing with it is required and none is decided. Where a declared order arrives with that alphabet the compiler takes it, by the binding rule of Definition 3.3, to which this definition adds nothing; where none arrives, the choice the compiler makes and emits is the record's own listing, t in the order it lists $\mathcal{T}_{\text{pub}}$ in and, within each t, u in the order it lists $\mathcal{U}_t$ in. *No presentation is refused for an order*: what is fixed here is which order a decision is read in, not what a declarer may declare.

The remaining declared conditions, at the same reading. The strict inequality $q_t > 0$ is a domain condition on a declared real scalar, decided by its sign, and Definition 1.3 carries no strict-inequality row because none is wanted — $q_t \neq 0$ is what Lemma 4.2 (ii) needs, and a negative q_t would reverse the sign of an element of the emitted family, which no public choice of known bias can mean. For the setting-dependent form the same reading applies to P: each entry is a binary64 number, $P(f\,|\,t) \geq 0$ is decided by sign, each row sum $\sum_f P(f\,|\,t) = 1$ is decided exactly in the declared order of $\mathcal{F}$ and required to round nowhere, the support (4.2) is decided by the sign of each entry, and "$\{F_v\}$ is a POVM on S" is the exact admissibility the chain's (C3) already decided of the family the last link delivered. *This form has no second conditional family to decide*: the choice value is drawn after the measurement, $\{F_v\}$ is the whole of what the decision on the conditional families has as its subject here, and (C3) is where it was taken.

What this module constructs, and the two emission decisions. The products $q_t N_{u|t}$ of (4.1) and $P(f\,|\,t(v))F_v$ of (4.3) are formed *exactly* on the declared dyadic numbers. A product of two binary64 numbers need not be

one, so the compiler decides, before emitting anything, that every entry of every $N_{u,t}$, respectively of every $N_{v,f}$, is a binary64 number — on every product this map forms, whether the product is emitted or handed on — and that the emitted family satisfies Definition 1.3 — (H), (P), (C) and, on its own declared-order sum over $\mathcal{V}$, respectively over $\mathcal{V}_P$ in its product order, (S). These are the two *emission decisions*.

The lattice of the products. Let the arriving entries be integer multiples of 2^{-k} of modulus at most 1, and let each q_t be an integer multiple of 2^{-k_q}; then every entry of $q_t N_{u|t}$ is an integer multiple of $2^{-(k+k_q)}$ of modulus at most 1, so the constructed family lies on a lattice k_q steps finer than the one it was handed and is exactly binary64 whenever $k + k_q \leq 53$. Where a q_t is not a power of two the finer lattice can leave binary64 altogether: the decimal literal 0.9 is an integer multiple of 2^{-53} and of no coarser one, so $k + k_q > 53$ for every $k \geq 1$ and the exact product of a declared entry by it need not be a binary64 number. The same arithmetic governs $P(f \,|\, t(v))F_v$ with the exponent k_P of the entries of P in place of k_q: a P whose non-zero entries are dyadic with small exponent — $\frac{1}{2}$, $\frac{1}{4}$, 1 — composes with any family the chain delivers at $k \leq 53 - k_P$, and a decimal literal in P can leave binary64 exactly as a decimal q_t can.

The general format. In the general format of Definition 1.1 (DX1)–(DX6) both forms above are read *on the system of the party that declares them*, and nothing else of this definition changes. At one measured factor, in POVM form and with no factor tagged A, the readings below are the text above, sentence for sentence.

Which system S is. For a presentation of Alice's it is A, and (T) is *not empty where a factor is tagged A*: the conditional families are then block lists along $\mathbb{T}$ and (T) is the four-component agreement of the maps they carry with that factor's map, as on any other sectored system. For a presentation of a measuring party's it is that party's *own* measured factor: one presentation per factor, decided against that factor's own map; *a presentation indexed by a product alphabet has no declared system to be a family on* — the product is the rule (1.5) and is no datum (Definition 3.3, the general format). So the decision "$\{N_{u|t}\}_u$ is a POVM on S" and the products this module forms are per factor, and the representability decision keeps exactly the subject it has: the products *these two constructions* form, and no product block of (1.5).

The ensemble form of Alice's slot. Where Alice's measurement slot is declared in ensemble form (DX4), a choice-weighted presentation of it declares, for each $t \in \mathcal{T}_{\mathrm{pub}}$, a family $\{N_{x|t}\}_{x \in \mathcal{X}}$ of block lists along $\mathbb{T}$ with every block Hermitian and positive semidefinite, and *in place of the POVM condition* the ensemble condition of (1.6): for every Alice sector t', $\sum_x N_{x|t}^{(t')} = \tau_A^{(t')}$, the sum accumulated in the order stated above and rounding nowhere. That condition is decided *of each t separately and on the declared numbers*, with

(H), (P) per block, that identity in place of (C), and (T) as above. *Why the construction is the same construction.* With $N_{x,t} \coloneqq q_t N_{x|t}$ of (4.1), the exact real numbers the data denote satisfy

$$\sum_{(x,t)\in\mathcal{V}} N_{x,t}^{(t')} = \sum_{t\in\mathcal{T}_{\mathrm{pub}}} q_t \sum_{x\in\mathcal{X}} N_{x|t}^{(t')} = \Big(\sum_{t\in\mathcal{T}_{\mathrm{pub}}} q_t\Big)\tau_A^{(t')} = \tau_A^{(t')} \tag{4.4}$$

for every t', by the decision just taken and by $\sum_t q_t = 1$; so a choice-weighted presentation in ensemble form whose conditional ensembles are admitted denotes a weighted family with the ensemble identity of (1.6). *That is a consistency statement and not a substitute for the decision on the emitted family*: the emitted family's own (S), (H), (P) and its identity with the declared τ_A are decided at check V3, on the emitted blocks and in the emitted order, and neither condition implies the other: (4.4) is an identity of real numbers and the emitted decision is about the additions that compute it. In untrusted-source mode there is no ensemble form and this paragraph has no subject.

The setting-dependent form, per measured factor. It is applied by the compiler after *that factor's* receiver segment, to the family that factor's last link delivered, with that factor's own setting map, support (4.2) and product order; the setting component read is a component of *that factor's* outcome, the private per-round setting of the station that measures that factor, whichever party's label the factor carries. This form has no second conditional family here either, per factor: (C3) of Definition 7.1, read for that factor, is where the delivered family was decided.

Uses Def. 1.1, Def. 1.3, Def. 3.3 · *Used by* Def. 3.2, Thm. 3.7, Lem. 4.2, Lem. 4.3, Thm. 4.4, Rem. 4.5, Rem. 4.6, Ex. 4.7, Rem. 4.8, Rem. 4.9, Thm. 8.2, Ass. 10.4, Rem. 10.5, Req. 11.9, Req. 12.1, Req. 12.2, Req. 12.3, Req. 12.4, Req. 12.5, Req. 12.6, Req. 12.7, Req. 12.8, Req. 13.5, Req. 13.7, Req. 13.8, Req. 13.11, meat-core/cor:reference-companion-factors, meat-core/prop:reference-companion · newly derived · implemented (effective: accepted) · *Code* certiqs_key.protocol_compiler.weights: decide_conditional_families · *Oracles* registry_protocol_compiler.py

Weighting keeps a measurement a measurement and acts block by block. The independent form has the declared state-independent choice marginal; the setting-dependent form returns the delivered family when the choice is forgotten, and its choice marginal may depend on the state through the setting.

Both constructions multiply declared measurement elements by positive numbers (a zero entry of P indexes no element), so what survives is what a positive scalar preserves; the choice marginal is where the two forms part. Part (i) is about the mathematical products; what is decided of the emitted binary64 family, and where, is Remark 4.5. No identity of the form (4.5) is wanted for the setting-dependent form: that the marginal of a public choice may depend on the state through the setting is the output-state test-statistics model (Assumption "Output-state test statistics" (reductions/ass:tsm)), which this lemma neither uses nor contradicts

(Theorem 4.4). That the emitted declaration carries that model is Requirement 12.9.

Lemma 4.2 (Properties of the choice-weighted families)**.** With the notation of Definition 4.1, and reading its conditions exactly:

(i) $\{N_{u,t}\}_{(u,t)\in\mathcal{V}}$ is a POVM on S; and so is $\{N_{v,f}\}_{(v,f)\in\mathcal{V}_P}$ of the setting-dependent form, over the support of P;

(ii) if $S = Q$ carries the sector decomposition and every $N_{u|t}$ is sector-block-diagonal, then so is every $N_{u,t}$, and conversely (for $q_t > 0$); equivalently, on block lists the weighting acts blockwise, $N^{(s)}_{u,t} = q_t\, N^{(s)}_{u|t}$, so a weighted family is a block list against the same sector map as the conditional families it was built from. The same holds of the setting-dependent form: every $N_{v,f}$ is a scalar multiple of the F_v it was built from, $N^{(s)}_{v,f} = P(f \mid t(v))\, F^{(s)}_v$ block by block, and $N_{v,f}$ is sector-block-diagonal iff F_v is, the weight being strictly positive on the support;

(iii) *(the choice marginal, for the setting-independent form)* for every $\omega \in \mathcal{L}(S)$ with $\operatorname{tr}\omega = 1$ and every $t \in \mathcal{T}_{\mathrm{pub}}$,

$$\sum_{u\in\mathcal{U}_t} \operatorname{tr}[N_{u,t}\,\omega] = q_t, \tag{4.5}$$

independently of ω; and for any complementary system R and any $\omega \in \mathcal{L}(RS)$ of unit trace likewise $\sum_u \operatorname{tr}[(\mathbb{1}_R \otimes N_{u,t})\,\omega] = q_t$, the operator being written with its factors in the declared order of the tensor factors. For a choice declared on Bob's side, $S = Q$ and $R = A$, this reads $\sum_u \operatorname{tr}[(\mathbb{1}_A \otimes N_{u,t})\omega] = q_t$; for a choice declared on Alice's side, $S = A$ and $R = Q$, it reads $\sum_u \operatorname{tr}[(N_{u,t} \otimes \mathbb{1}_Q)\omega] = q_t$. *This part is asserted of the setting-independent form and of it alone*;

(iv) *(what is asserted of the setting-dependent form instead)* forgetting the choice value returns the family the chain delivered,

$$\sum_{f:\,(v,f)\in\mathcal{V}_P} N_{v,f} = F_v \qquad \text{for every } v \in \mathcal{V}, \tag{4.6}$$

an exact operator identity on S; and the marginal of the choice value is the declared matrix applied to the *setting marginal* of the outcome statistics,

$$\sum_{v:\,(v,f)\in\mathcal{V}_P} \operatorname{tr}[N_{v,f}\,\omega] = \sum_{t\in\mathcal{T}} P(f \mid t) \sum_{v:\,t(v)=t} \operatorname{tr}[F_v\,\omega] \qquad \text{for every } \omega \in \mathcal{L}(S), \tag{4.7}$$

which depends on ω only through the *setting marginal* $m_t(\omega) \coloneqq \operatorname{tr}[\Pi_t\omega]$, $\Pi_t \coloneqq \sum_{v:\,t(v)=t} F_v$, with $\Pi_t \succeq 0$ and $\sum_t \Pi_t = \mathbb{1}_S$. If $P(f \mid t) = q_f$ for every t it equals q_f for every unit-trace ω — the setting-independent case,

and (4.5) in the form the first construction gives it. If every class sum Π_t is a multiple $\lambda_t \mathbb{1}_S$ of the identity — as for the identity link, where the emitted family is the physical joint family — it is state-independent as well, with value $\sum_t P(f \mid t)\lambda_t$. Otherwise it is, in general, a function of the state, and *no identity of the form* (4.5) *is asserted for this form.*

Uses Def. 4.1 · *Used by* Lem. 4.3, Thm. 4.4, Rem. 4.5, Rem. 4.6, Ex. 4.7, Req. 13.11, meat-core/cor:reference-companion-factors, meat-core/prop:reference-companion · newly derived · implemented (effective: accepted) · *Code* certiqs_key.protocol_compiler.weights: choice_mass · *Oracles* registry_protocol_compiler.py

Proof. (i) Each $N_{u,t} = q_t N_{u|t} \succeq 0$ since $q_t > 0$ and $N_{u|t} \succeq 0$, and $\sum_{(u,t)\in\mathcal{V}} N_{u,t} = \sum_t q_t \sum_{u\in\mathcal{U}_t} N_{u|t} = \sum_t q_t \mathbb{1}_S = \mathbb{1}_S$, using completeness of each conditional POVM — an identity of the declared numbers by condition (S) on each conditional family — and $\sum_t q_t = 1$, likewise an identity by (S) on the weights. Hermiticity passes through the real positive multiple. For the setting-dependent form, each $N_{v,f} = P(f \mid t(v))F_v$ is Hermitian and positive semidefinite as a positive multiple of F_v, and $\sum_{(v,f)\in\mathcal{V}_P} N_{v,f} = \sum_{v\in\mathcal{V}} \Big(\sum_{f:\,P(f|t(v))>0} P(f \mid t(v))\Big) F_v = \sum_{v\in\mathcal{V}} F_v = \mathbb{1}_S$, because the omitted terms of each inner sum are exactly the zero entries of the row $P(\,\cdot \mid t(v))$, so the inner sum is the row sum, which is 1 — an identity of the declared numbers by the exact row-sum decision of Definition 4.1 — and $\sum_v F_v = \mathbb{1}_S$ is a hypothesis on the family the lemma is given (it is the completeness that condition (C3) of the interface conditions on a chain (Definition 7.1) decides).

(ii) $\Pi_s N_{u,t} \Pi_{s'} = q_t \Pi_s N_{u|t} \Pi_{s'}$, which vanishes iff $\Pi_s N_{u|t} \Pi_{s'}$ does, because $q_t \neq 0$ — and $q_t \neq 0$ is needed for the converse direction only. Likewise $\Pi_s N_{v,f} \Pi_{s'} = P(f \mid t(v)) \Pi_s F_v \Pi_{s'}$ with $P(f \mid t(v)) > 0$ on the support, and on block lists the scalar multiplies each block.

(iii) $\sum_u \operatorname{tr}\big[q_t N_{u|t} \omega\big] = q_t \operatorname{tr}\big[(\sum_u N_{u|t})\omega\big] = q_t \operatorname{tr}[\mathbb{1}_S \omega] = q_t \operatorname{tr}\omega = q_t$; the bipartite statement is the same computation with the identity on R adjoined and $\operatorname{tr}\omega = 1$, and is insensitive to which tensor slot S occupies, since the trace is.

(iv) (4.6) is the inner sum of (i): for fixed v, $\sum_{f:\,(v,f)\in\mathcal{V}_P} P(f \mid t(v)) F_v = \big(\sum_f P(f \mid t(v))\big) F_v = F_v$, the omitted terms being zero. For (4.7), group the outcomes by their setting: $\sum_{v:\,(v,f)\in\mathcal{V}_P} \operatorname{tr}[N_{v,f}\omega] = \sum_t \sum_{v:\,t(v)=t,\ P(f|t)>0} P(f \mid t) \operatorname{tr}[F_v \omega] = \sum_t P(f \mid t) \sum_{v:\,t(v)=t} \operatorname{tr}[F_v \omega]$, the restriction $P(f \mid t) > 0$ being immaterial since the excluded terms vanish; the inner sum is $\operatorname{tr}[\Pi_t \omega] = m_t(\omega)$ by linearity of the trace, Π_t is a sum of positive semidefinite operators, and $\sum_t \Pi_t = \sum_v F_v = \mathbb{1}_S$ since the classes partition $\mathcal{V}$. If $P(f \mid t) = q_f$ for every t, the right-hand side is $q_f \sum_t m_t(\omega) = q_f \operatorname{tr}\omega = q_f$ for every unit-trace ω. If $\Pi_t = \lambda_t \mathbb{1}_S$ for every t, then $m_t(\omega) = \lambda_t \operatorname{tr}\omega = \lambda_t$ for every *unit-trace* ω and the right-hand side is $\sum_t P(f \mid t)\lambda_t$, whatever that state is. Outside these two cases the right-hand side varies with the state for some families and does not for others, and "in general" is asserted in that weak sense and in

no stronger one. At $|\mathcal{T}| = 2$ it is exactly decided: writing $m_{t_2} = \operatorname{tr}\omega - m_{t_1}$, the right-hand side is $P(f\,|\,t_2) + \big(P(f\,|\,t_1) - P(f\,|\,t_2)\big)m_{t_1}(\omega)$ on unit-trace states, which varies with ω precisely when Π_{t_1} is not a multiple of the identity — so that m_{t_1} takes at least two values — *and* $P(f\,|\,t_1) \neq P(f\,|\,t_2)$; this witnesses that the case occurs. Beyond two settings those two hypotheses are no longer enough: on $S = \mathbb{C}^2$ take $\mathcal{V} = \{v_1, v_2, v_3\}$ with $F_{v_1} = \frac{1}{2}\,|0\rangle\langle 0|$, $F_{v_2} = \frac{1}{2}\,|1\rangle\langle 1|$, $F_{v_3} = \frac{1}{2}\mathbb{1}_S$, a POVM since the three sum to $\mathbb{1}_S$, and $t(v_i) = t_i$, so that $\Pi_{t_i} = F_{v_i}$ and Π_{t_1} is not a multiple of the identity; with $P(f\,|\,t_1) = P(f\,|\,t_2) = x \neq y = P(f\,|\,t_3)$ the right-hand side is $x\,\operatorname{tr}[(\Pi_{t_1} + \Pi_{t_2})\omega] + y\,m_{t_3}(\omega) = \frac{1}{2}(x + y)$ for every unit-trace ω, because $\Pi_{t_1} + \Pi_{t_2} = \frac{1}{2}\mathbb{1}_S = \Pi_{t_3}$. What makes the marginal state-independent is a condition on the *sums* Π_t weighted by the column $P(f\,|\,\cdot)$ — that $\sum_t P(f\,|\,t)\Pi_t$ be a multiple of the identity — and this lemma asserts no such condition, because nothing downstream needs one. □

Weighting a delivered element by its positive stochastic weight keeps a factor enclosure of each of its blocks: the same box, the core multiplied by the weight. A pair outside the support of the weighting is no outcome and carries no block.

The compiler weights the family a chain delivers by a stochastic weight read through the setting (Definition 4.1). Where a block of a delivered element is known through a factor enclosure, as on a reference companion (Requirement 13.11), the weight is a positive rational and goes into the core, and the columns are untouched.

Lemma 4.3 (A factor enclosure through a setting-dependent weighting). With the notation of the setting-dependent form of Definition 4.1, let $(v, f) \in \mathcal{V}_P$, let s be a sector of S, and let (L_c, ϱ, W) be a factor enclosure (Definition "Factor enclosure of a block" (foundations/def:factor-enclosure)) of the block $F_v^{(s)}$. Then $(L_c, \varrho, P(f\,|\,t(v))\,W)$ is a factor enclosure of the block $N_{v,f}^{(s)} = P(f\,|\,t(v))\,F_v^{(s)}$, with the same matrix box and the same number of columns. A pair $(v, f) \in \mathcal{V} \times \mathcal{F}$ outside $\mathcal{V}_P$ indexes no element of the weighted family, and so no block.

Uses Def. 4.1, Lem. 4.2, foundations/def:binary64, foundations/def:factor-enclosure · *Used by* Req. 13.11, meat-core/cor:reference-companion-factors · newly derived · implemented (effective: accepted) · *Code* certiqs_key.protocol_compiler.companion:_weighted_factor

Proof. A pair (v, f) lies in $\mathcal{V}_P$ exactly when $P(f\,|\,t(v)) > 0$ (4.2), and $P(f\,|\,t(v))$ is a binary64 number, so a positive dyadic rational (Definition "Binary64 numbers, declared values and dyadic lattices" (foundations/def:binary64) (i)); hence $P(f\,|\,t(v))\,W$ is diagonal with positive rational diagonal entries. By Lemma 4.2 (ii), $N_{v,f}^{(s)} = P(f\,|\,t(v))\,F_v^{(s)}$. The triple (L_c, ϱ, W) being a factor enclosure of $F_v^{(s)}$, there is an L that the box (L_c, ϱ) contains with $F_v^{(s)} = LWL^\dagger$, so $N_{v,f}^{(s)} = L\big(P(f\,|\,t(v))\,W\big)L^\dagger$ with the same L: the box and the new core make a factor enclosure of $N_{v,f}^{(s)}$, with the columns of the

old one. The last sentence is the reading of the support in Definition 4.1: a pair with $P(f \mid t(v)) = 0$ is not an outcome and indexes nothing. □

A public choice of known bias enters as weights inside the measurement operators: the family stays a POVM, fixed weights are reproduced exactly as choice probabilities where the weighted family is emitted, and setting-dependent weights commute with the chain. A fraction outside them is refused.

The theorem has two halves: (i)–(iv) for a choice of known bias declared on either party's side, (v)–(viii) for a choice whose weight depends on the private setting of the apparatus that measures a system, whichever party draws it, applied after the chain. By (ii) the weight inside the operators reproduces the declared choice statistics exactly wherever the weighted family is the one emitted. A measuring party's presentation in front of a receiver segment that squashes is the other case: there the weighted family is the physical one handed to the chain, the emitted family is the chain's, and the first half says what still holds. The assumption of the output-state test-statistics model that (vii) declares is Assumption "Output-state test statistics" (reductions/ass:tsm); it is named here, and the theorem does not use it. Remark 4.8 explains why the relabelling of (vi) is not a link of the chain; it is the relabelling of Remark "Classical maps before a squasher, and relabelling after one" (reductions/rem:intertwining), which leaves it to the consumer to apply.

Theorem 4.4 (Encoding of public choices in the measurement operators). Let a per-round public choice of known bias — a basis choice, a choice between test and generation rounds, a probe setting — be declared on either party's side with alphabet $\mathcal{T}_{\mathrm{pub}}$, weights (q_t) and conditional POVMs $\{N_{u|t}\}$, under Assumption 10.4. Then the compiler forms the choice-weighted family (4.1), and (iv) below holds. Where that family is the emitted one — the presentation is Alice's, or it is a measuring party's and the last link of that party's receiver segment delivers the weighted family itself, on its own alphabet, as its target POVM, as on the Band A route (Definition 8.1 (b)) — the compiler emits it, and (i)–(iii) hold as well:

(i) the emitted family is a POVM and preserves sector block-diagonality (Lemma 4.2 (i),(ii)), so the encoding cannot break D(iii) — and it is emitted only after the two decisions of Definition 4.1, so that what is emitted is binary64 data whose own predicates D(iii) is then decided on (Theorem 3.7 (i), and (i′) in the general format);

(ii) the encoding is *exact*: for every state, the marginal probability of the choice value t computed from the emitted declaration equals the declared weight q_t (Lemma 4.2 (iii));

(iii) the choice value is a component of the joint outcome, hence enters every downstream quantity only through $P_{\mathfrak{D}}[\omega]$ and $\mathbf{\Phi}[\omega]$ of Definition 3.6; no factor q_t is emitted anywhere outside a POVM element; and

(iv) the alternative encoding — declaring the conditional families $\{N_{u|t}\}$ together with a separate state-independent fraction q_t to be applied outside the operators — is never emitted, and a specification requesting it is refused.

Where the presentation is a measuring party's and the last link of that party's receiver segment delivers any other family, the weighted family is the physical receiver family handed to the chain (Definition 3.1 (b)) and is not emitted: it is a POVM on the physical receiver space, and for every unit-trace state there the marginal probability of t computed from it is q_t (Lemma 4.2 (i), (iii)); no factor q_t is emitted anywhere outside a POVM element, and (iv) holds. The emitted family is that link's target POVM, whose own conditions are (C3)'s and check V3's (Definition 7.1), and nothing here asserts (ii) or (iii) of it; the declared test-statistics model is the output-state model, as (vii) says of the setting-dependent form. Let, further, a public choice $f \in \mathcal{F}$ be declared whose weight depends on the private setting of the apparatus that measures the measured system, as a stochastic matrix $P(f \,|\, t)$ read through the setting map $t : \mathcal{V} \to \mathcal{T}$ of the family $\{F_v\}_{v\in\mathcal{V}}$ the last link of $\mathfrak{R}_{\mathrm{rec}}$ delivered — the setting-dependent form of Definition 4.1; in the general format, at a measured factor, read through that factor's setting map on the family the last link of that factor's receiver segment delivered, with S that factor's system — under Assumption 10.4. Then the compiler applies it *after* the reduction chain, emits the family (4.3), $N_{v,f} = P(f \,|\, t(v))F_v$ over the support $\mathcal{V}_P$, and:

(v) the emitted family is a POVM and preserves sector block-diagonality (Lemma 4.2 (i),(ii)), every element being a scalar multiple of an element the chain delivered, so the encoding cannot break D(iii); and it is emitted only after the two decisions of Definition 4.1 on the products $P(f \,|\, t(v))F_v$, so that what is emitted is binary64 data whose own predicates D(iii) is then decided on (Theorem 3.7 (i), and (i′) in the general format);

(vi) *(soundness: the relabelling commutes with the chain)* for every channel Λ into S and every family $\{G_v\}_{v\in\mathcal{V}}$ on its input space with $\Lambda^\dagger(F_v) = G_v$ for every v,

$$\Lambda^\dagger\big(N_{v,f}\big) = P(f \,|\, t(v))\, G_v \qquad \text{for every } (v,f) \in \mathcal{V}_P : \tag{4.8}$$

the weighted target and the identically weighted input family stand in the same relation under the *same* channel, with the same target space, the same sector map, the same constraint entries transported along the chain and every pull-back unchanged. In this module's setting G is the γ-coarse-grained physical family of Definition 7.1 (C4), $G_v = \sum_{w\in\gamma^{-1}(v)} F_w^{\mathrm{ph}}$, so the emitted family and the physical family weighted

by $P(f \mid t(\gamma(w)))$ satisfy the pullback identity (7.1) with the outcome correspondence $(w, f) \mapsto (\gamma(w), f)$ restricted to the support, wherever (C4) is decidable, and are related as (C4)'s families are wherever it is not. This is Lemma "Linearity: post-processing, private settings and mixtures" (reductions/lem:linear) (i) for the stochastic matrix from $\mathcal{V}$ to $\mathcal{V} \times \mathcal{F}$ with entries $\delta_{vv'} P(f \mid t(v'))$;

(vii) *(what is not asserted)* the exactness clause (ii) is asserted for the setting-independent form and for it alone. For this form the marginal of f is the quantity (4.7) of Lemma 4.2 (iv), a function of the state through the setting marginal, equal to the declared q_f for every state when $P(f \mid t) = q_f$ and state-dependent in general otherwise; nothing here asserts otherwise, no state-independent fraction is emitted, and the declared test-statistics model is the output-state model. What (iv) says of the fixed-fraction encoding holds of this form unchanged;

(viii) *(refusals)* a weighting presented through anything but the setting map — indexed by $\mathcal{V}$, by the device outcome, or by any component of v other than $t(v)$ — is refused; a P with a negative entry or a row whose declared-order sum is not exactly 1 is refused; and the two emission decisions of Definition 4.1 refuse, for a product that is not representable or with the predicate that failed. Each refusal emits no declaration.

Uses Def. 3.1, Def. 3.3, Def. 3.6, Thm. 3.7, Def. 4.1, Lem. 4.2, Def. 7.1, Def. 8.1 · *Assumes* A-COIN, A-LINK · *Used by* Rem. 4.8, Req. 12.4, Req. 12.6, Req. 12.7, Req. 12.9 · newly derived · implemented (effective: accepted) · *Code* certiqs_key.protocol_compiler.weights: resolve_povm · *Oracles* registry_protocol_compiler.py

Proof. Under the hypothesis of (i)–(iii) the emitted family is the weighted family itself, on its own alphabet: for Alice's presentation by Definition 3.3, which emits her measurement slot from the presentation; for a measuring party's because the emitted family is the target POVM of the last link (Definition 3.3), which by hypothesis is the weighted family on $\mathcal{V} = \bigsqcup_t \mathcal{U}_t \times \{t\}$ — on the Band A route by Definition 8.1 (b), with γ the identity on $\mathcal{V} = \mathcal{V}^{\mathrm{ph}}$ and $F_v = F_v^{\mathrm{ph}}$ entry for entry. So (i) and (ii) are Lemma 4.2, read of the emitted family and against D(iii) and the emission decisions of Definition 4.1. For (iii): the emitted alphabet is $\bigsqcup_t \mathcal{U}_t \times \{t\}$ and the emitted matrices are the $N_{u,t}$; the choice value t is recoverable from the outcome index alone, and no other emitted slot carries q_t. Since $P_{\mathfrak{D}}[\omega]$ and $\mathbf{\Phi}[\omega]$ are functions of ω and the emitted operators only (Definition 3.6), the claim follows. (iv) is a rule of the compilation map, not a derived statement, and is stated here because it is the rule the encoding exists to enforce: the emitted declaration carries the output-state model, and no path of the compiler produces an artefact carrying a fixed-fraction model. Requirement 12.9 binds its refusal.

For a measuring party's presentation whose last link delivers any other family, Definition 3.1 (b) makes the weighted family the physical receiver family, the object (C2) and (C4) of Definition 7.1 read, and Definition 3.3 fills the emitted family from the last link's target POVM, so the weighted family is not emitted. Lemma 4.2 (i) and (iii) are stated on any finite-dimensional S and apply with S the physical receiver space: the weighted family is a POVM there and its choice marginal is q_t for every unit-trace state there. No slot of the declaration is filled from the weights: the emitted family is the target POVM and the remaining slots are the other parts' (Definition 3.3), so no factor q_t is emitted outside a POVM element, and (iv) is the rule of the compilation map as before. Nothing is asserted of the emitted family beyond (C3) and check V3, and nothing more could be in general: (C2) admits any tabulated total surjection γ, so t need not be recoverable from an emitted outcome; and where γ keeps the choice value, every emitted outcome carrying the t of each physical outcome it receives, so that (C4) summed over the emitted outcomes carrying t reads $\Lambda^\dagger(\Pi_t) = q_t\mathbb{1}$ on the physical receiver space with Π_t the sum of the emitted elements carrying t, that fixes Π_t only through $\Lambda^\dagger$ and does not make it $q_t\mathbb{1}_Q$ off the image of the channel. The sentence on the test-statistics model is, as in (vii), a statement of what the emitted declaration carries.

(v) is Lemma 4.2 (i), (ii) for the setting-dependent form, read against D(iii) and the emission decisions exactly as (i) is.

(vi) By linearity of $\Lambda^\dagger$, $\Lambda^\dagger(P(f\,|\,t(v))F_v) = P(f\,|\,t(v))\Lambda^\dagger(F_v) = P(f\,|\,t(v))G_v$, which is (4.8); nothing else is used, and in particular nothing about Λ beyond linearity of its adjoint. That the two weighted families are POVMs is (v) and its mirror on the input side (the same computation with G for F, $\sum_v G_v = \Lambda^\dagger(\sum_v F_v)$ being the identity where $\Lambda^\dagger$ is unital, which is the trace-preservation clause of the obligation of the chain's producer and not a step of this proof). The identity (4.8) is the instance of the post-processing part (i) of Lemma "Linearity: post-processing, private settings and mixtures" (reductions/lem:linear) for the relation $G \preceq_\Lambda F$ and the stochastic matrix $\Pi\big((v',f)\,|\,v\big) := \delta_{vv'}P(f\,|\,t(v'))$ from $\mathcal{V}$ to $\mathcal{V}\times\mathcal{F}$, every column of which sums to $\sum_f P(f\,|\,t(v)) = 1$: $(\Pi F)_{(v,f)} = P(f\,|\,t(v))F_v$ and $(\Pi G)_{(v,f)} = P(f\,|\,t(v))G_v$, and the computation above is that lemma's argument for Π, written out. The target space, its sector map and the transported entries are untouched because a relabelling changes the index set of the two families and nothing else: every constraint operator $\hat{O}_k$ is an operator on Q, not a function of v, and its pull-back $\Lambda^\dagger(\hat{O}_k)$ is computed through the same Λ. For the compiler's instance of it, substitute (C4)'s $G_v = \sum_{w\in\gamma^{-1}(v)} F^{\mathrm{ph}}_w$: then $\Lambda^\dagger(N_{v,f}) = \sum_{w\in\gamma^{-1}(v)} P(f\,|\,t(\gamma(w)))F^{\mathrm{ph}}_w$, which is (7.1) for the family $\{P(f\,|\,t(\gamma(w)))F^{\mathrm{ph}}_w\}_{(w,f)}$ under the correspondence $(w,f)\mapsto(\gamma(w),f)$, a total surjection onto $\mathcal{V}_P$ because γ is one onto $\mathcal{V}$ and the weight depends on w only through $\gamma(w)$; where (C4) is not de-

cidable from the data, (4.8) is still an identity of linear maps, and it says of the relation the chain's producer asserts and nobody here decides (Definition 7.1 (C4), second sentence) exactly what it says of a decided one.

(vii) is Lemma 4.2 (iv): (4.7) is the marginal of f computed from the emitted declaration, it equals q_f for every state in the setting-independent case, and it is in general a function of the state otherwise. The theorem asserts no value for it in that case; the sentence about the declared model is a statement of what the emitted declaration carries, as (iv) is.

(viii) is the rule of the compilation map that Definition 4.1 states and Requirements 12.4, 12.6 and 12.7 bind, stated beside (iv) because it is the rule the encoding exists to enforce: a weighting that reaches an outcome through anything but its setting would make f a function of the device outcome, which is the inference Assumption 10.4 excludes; a row of P with non-negative entries whose sum is not 1 would make (4.6) false at every v of that setting with $F_v \neq 0$, the inner sum over the support (4.2) being that row sum, and a negative entry, which the support never keeps and which therefore reaches no emitted element, is refused because Assumption 10.4 reads $P(\,\cdot \mid t)$ as the distribution the choice f is drawn from, and a negative number is no probability; and the two emission decisions are those of the setting-independent form, taken on the products of this one. □

The exact choice marginal fixes the sum over outcomes, not the individual probabilities, so no state-independent weight can be factored out of a security argument. Nor does it establish D(iii) of the emitted binary64 family, which check V3 decides after the emission decisions.

Remark 4.5 (Scope of the exact encoding). Equation (4.5) says the choice *marginal* reproduces the declared weight for every state. What it fixes is that marginal alone: it does *not* license treating q_t as a fraction that may be factored out of a security argument, since q_t appears in (4.5) only after summing $\mathrm{tr}[N_{u,t}\omega]$ over u, and the terms $\mathrm{tr}[N_{u,t}\omega]$ that carry it depend on ω. That is what separates the emitted encoding from a fixed fraction applied outside the operators. The exactness statement matters because it removes the only respectable motive for that unsound encoding: the weight carried inside the operators already reproduces the declared choice statistics exactly. Assumption 10.4 is where the physical content sits — a choice correlated with the adversary is not a choice of known bias, and no encoding repairs that.

Nor does it say that the emitted family passes V3. Lemma 4.2 (i) is a statement about the mathematical products $q_t N_{u|t}$, respectively $P(f \mid t(v))F_v$; what is emitted is those products as represented, as binary64 data, and two things can go wrong between the two. A product of binary64 numbers need not be binary64, and a family whose entries are all binary64 may still have a declared-order sum that rounds. Both are decided before emission, by the emission decisions of Definition 4.1, and D(iii) is decided on the emitted blocks afterwards, by check V3 (Theorem 3.7 (i), and (i′) in the general format), and never inferred from the lemma, no part of which rests on that theorem. A weighting that fails either decision is refused, which is the direction that costs a good declaration rather than admitting a bad one. Both paragraphs are about the products of Definition 4.1 in either of its forms; what differs between

the forms is only the first: for the setting-dependent form the exactness statement, Theorem 4.4 (ii), is *not asserted at all*, the marginal of the choice value being state-dependent through the setting marginal of the outcome statistics (Lemma 4.2 (iv)), and no one is tempted to factor it out.

Uses Thm. 3.7, Def. 4.1, Lem. 4.2 · *Assumes* A-COIN · newly derived · accepted

Each conditional family is decided by the compilation map, not the validator, since the emitted family's conditions do not imply it; the converse gives (T), (H), (P) and, with (S), (C). For a choice-weighted physical family, only its own (S) is left to decide on the family itself.

Remark 4.6 (Deciding the conditional families). A choice-weighted presentation is two declared things and one constructed one: the weights, the conditional families $\{N_{u|t}\}_{u\in\mathcal{U}_t}$, and the products the compiler forms from them — the *weighted family*, which is the emitted one where the presentation is Alice's or where the receiver segment is the identity, and is the *physical* family handed to the chain where it is Bob's at a non-trivial one. The compiler decides all three (Definition 4.1, Requirement 12.1) — the weights by sign and by an exact declared-order sum, each conditional family for exact admissibility in the sense of Definition 1.3, and every product it forms for representability, with the five conditions decided once more on the weighted family wherever that family is the emitted one. This remark records why the middle decision cannot be dropped in favour of the last, what it buys where the weighted family is *not* emitted, and what it does not disturb.

It is a decision of the compilation map and not a check of the validator. The conditional families are declared input: they are no slot of the emitted declaration, their index (u, t) is not an emitted index, and the family the consumer receives is the one product family. So the decision is taken where the presentation is read, before any product is formed (Requirement 12.1), beside the decisions on the weights and not among V1–V8. Nothing in Definition 6.1 gains or loses a conjunct by it, and Theorem 6.2 (i) and (ii) pair the same checks with the same target conditions whether or not the decision is taken: the target conditions D(i)–D(viii) quantify over the emitted declaration, which this decision does not change, and a candidate that meets them is refused at no check because of it. What the decision affects is which *specifications* reach a candidate at all, which is the compilation map's business and not the validator's.

The emitted family's conditions do not imply the conditional families'. Example 4.7 is two weights and two one-element conditional families, neither of them a POVM, whose weighted family satisfies (T), (H), (P), (C) and (S) and passes V1's map comparison and check V3 with every product exactly representable. The reason it can happen is visible in the arithmetic: (C) on the emitted family is one identity, $\sum_t q_t \sum_u N_{u|t} = \mathbb{1}_S$, and the inner sums enter it only through that one combination, so a deficit under one choice value is paid for by a surplus under another. Deciding the outer identity therefore says nothing about the inner ones, and (4.9) is what "nothing" looks like.

The converse does hold, in four of the five, and it is stated of the weighted family — whether or not that family is the emitted one. Suppose the conditional families of one presentation all carry one map and are each exactly admissible for it, the weights satisfy $q_t > 0$ and $\sum_t q_t = 1$ with that sum rounding nowhere, and the representability decision has passed on the products, so that every entry of the weighted family *is* the real product $q_t N_{u|t}$. Then, of that family: (T) holds *against*

that same map — the map the conditional families carry, which for a physical presentation is that of the module presenting it and not $\mathfrak{s}$ — the weighting acting blockwise (Lemma 4.2 (ii)); (H) holds, a real multiple of a Hermitian block being Hermitian; (P) holds, $q_t > 0$ and $N^{(s)}_{u|t} \succeq 0$ giving $q_t N^{(s)}_{u|t} \succeq 0$, the product being exact; and the *real* sum over the alphabet that indexes it is the identity on each block, which is the computation of Lemma 4.2 (i) read on the declared numbers — it is available as an identity of those numbers precisely because (S) and (C) were decided of each conditional family and of the weights. What does *not* follow is (S) on the weighted family's own declared-order sum: that condition is on binary64 additions — of weighted operands, accumulated across the boundaries between choice values — and no hypothesis on the operands controls them (Remark 1.4). *Only the order restricts, never the condition*: the order the conditional family's (S) is read in is the restriction of the order on the alphabet that indexes the weighted family (Definition 4.1), and the two conditions are independent in both directions. Given (S), (C) follows, the computed sum then being the real one. So (S) is the one of the five that still needs its own decision, and the emission decision of Definition 4.1 is not made redundant by this one.

And here the two cases part company. Where the weighted family *is* the emitted one — a presentation on Alice's side, or one on Bob's where the receiver segment is the identity — (H), (P), (C) and (S) are decided on it again by check V3 and its map (T) again by check V1 (Definition 6.1), because those are the conditions the consumer's admission rule reads and this module decides what it emits rather than inferring it (Requirement "Exact verification before emission" (foundations/req:exact-emission)); what the converse buys there is not a check saved but the knowledge that the surviving refusals are (S)'s and representability's (Remark 4.5). Where it is *not* — a physical receiver family presented choice-weighted at a non-trivial receiver segment, which lives on the physical receiver space and is no Q-operator family — the converse is what the decision on it rests on (Requirement 13.5): (T), (H) and (P) hold of it by the decision on its conditional families, (S) on its own declared-order sum is the one condition decided of the family itself, and (C) of it follows from that (S) together with the conditional (C), (S) and the weight sum. The emitted family at such a segment is a different object, the target POVM of the last link, and its own five are (C3)'s and then V1's and V3's (Definition 7.1).

The setting-dependent form has no second subject. There the family that is weighted is the single family $\{F_v\}_{v\in\mathcal{V}}$ the last link of $\mathfrak{R}_{\mathrm{rec}}$ delivered, and the choice value is drawn *after* the measurement, with probability $P(f \,|\, t(v))$; there is no family of measurements indexed by f, and $\{N_{v,f}\}_v$ at fixed f is not one and is not meant to be (Lemma 4.2 (iv), Theorem 4.4 (vii)). Read through the overlap of the two forms — $P(f \,|\, t) = q_f$ for every t, where the one conditional family is $\{F_v\}$ itself — the decision above applies with that family as its subject, and it was taken already: condition (C3) of Definition 7.1 decides the exact admissibility of the family the last link delivered, per block. So no conditional family of the setting-dependent form goes undecided and no second decision is added there; what the compiler decides of P is what Definition 4.1 states of it.

What stays assumed. A specification whose conditional families are not measurements is refused: what is lost is a declaration whose per-choice reading is false, and Example 4.7 shows that such a declaration implements a probability of the choice value other than the one the declared public-choice record states and As-

sumption 10.4 reads. Assumption 10.1 keeps what no function of the declared data decides — that the declared dimensions, alphabets and tables describe the apparatus and the declared measurements it names are the ones performed. The physical receiver family as presented is decided on every route (Requirement 13.5); where it is presented in choice-weighted form, its conditional families are read here like any other and their five conditions are decided, so what is decided of the presented family itself shrinks to condition (S) on its own declared-order sum — from which its (C) then follows, by the converse above.

Uses Def. 1.3, Def. 3.3, Def. 4.1, Lem. 4.2, Def. 6.1, Thm. 6.2 · newly derived · accepted

Two weights of one half and two one-element conditional families that are not POVMs give a weighted family passing checks V1 and V3 exactly. The emitted conditions therefore do not imply the per-choice ones, and the declared weight is then not the probability the declaration implements.

Example 4.7 (An exact weighted family built from incomplete conditional families). Let S be two-dimensional carrying the single sector $\mathbb{S} = \{s_0\}$ with $d_{s_0} = 2$, so that a block list against that map is one 2×2 array and condition (T) of Definition 1.3 is the agreement of one identifier, one dimension and one block kind. Let $\mathcal{T}_{\mathrm{pub}} = \{t_0, t_1\}$ with $q_{t_0} = q_{t_1} = \frac{1}{2}$, let $\mathcal{U}_{t_0} = \mathcal{U}_{t_1} = \{u_0\}$ — one outcome per choice value — and let the conditional families be

$$N_{u_0|t_0} = \operatorname{diag}\left(\tfrac{1}{2}, \tfrac{3}{2}\right), \qquad N_{u_0|t_1} = \operatorname{diag}\left(\tfrac{3}{2}, \tfrac{1}{2}\right). \tag{4.9}$$

Every entry is an integer multiple of 2^{-1} and each weight is $\frac{1}{2}$, so every product of (4.1) is exactly a binary64 number and the representability decision of Definition 4.1 passes. The emitted family is

$$N_{u_0,t_0} = \operatorname{diag}\left(\tfrac{1}{4}, \tfrac{3}{4}\right), \qquad N_{u_0,t_1} = \operatorname{diag}\left(\tfrac{3}{4}, \tfrac{1}{4}\right).$$

All five conditions hold of it. (T): there is one block and one map. (H) and (P): both arrays are real, diagonal and entrywise non-negative. Its declared-order sum over $\mathcal{V} = \{(u_0, t_0), (u_0, t_1)\}$ is two binary64 additions per entry, $0 + \frac{1}{4}$ then $\frac{1}{4} + \frac{3}{4}$ and $0 + \frac{3}{4}$ then $\frac{3}{4} + \frac{1}{4}$, each exact, so (S) holds; and its value is $\operatorname{diag}(1, 1) = \mathbb{1}_S$, so (C) holds. So check V3 passes on the emitted family — it is V3 that decides (H), (P), (C) and (S) — and check V1's four-component comparison passes on its map, which is where (T) is decided (Definition 6.1); and so does every decision on the weights: $q_t > 0$ by sign, and $\frac{1}{2} + \frac{1}{2} = 1$ exactly in the declared order of $\mathcal{T}_{\mathrm{pub}}$.

Neither conditional family is a POVM. Their completeness sums are

$$\sum_{u \in \mathcal{U}_{t_0}} N_{u|t_0} = \operatorname{diag}\left(\tfrac{1}{2}, \tfrac{3}{2}\right), \qquad \sum_{u \in \mathcal{U}_{t_1}} N_{u|t_1} = \operatorname{diag}\left(\tfrac{3}{2}, \tfrac{1}{2}\right),$$

each a one-term sum, whose single addition $0 + N_{u_0|t}$ of (1.9) is exact, and neither is $\mathbb{1}_S$: condition (C) fails for both, of the declared numbers and through no rounding, while (T), (H), (P) and (S) hold of both. So there is no experiment "draw t with probability q_t, then measure $\{N_{u|t}\}_u$" for these numbers, and the hypothesis of Definition 4.1 — that each $\{N_{u|t}\}_u$ be a POVM on S — is violated by a specification whose weighted family passes every decision taken on the emitted data.

And the declaration then implements a choice probability the record does not declare. At $\omega = |0\rangle\langle 0|$ the choice marginal computed from the emitted declaration

is $\sum_u \mathrm{tr}[N_{u,t_0}\,\omega] = \frac{1}{4}$, while the record declares $q_{t_0} = \frac{1}{2}$ and Assumption 10.4 reads that number as the bias of the coin Bob drew: (4.5) of Lemma 4.2 (iii) is false of the pair, its hypothesis having failed, and with it the exactness Theorem 4.4 (ii) asserts. *No slot of the declaration carries the wrong number* — no public-choice weight is emitted outside a POVM element (Theorem 4.4 (iii)), and a consumer of the emitted operators sees a consistent POVM implementing $\frac{1}{4}$. The failure is therefore between the specification and the declaration the module built from it, which is where a decision on the specification is the only thing that can catch it, and is why the per-choice reading is needed.

The presentation is refused, naming (C) at t_0 — the first failure, the decision stopping there (Requirement 12.1); nothing here is rescaled, renormalised or repaired (Remark 4.6).

Uses Def. 1.3, Def. 4.1, Lem. 4.2 · newly derived · accepted

A choice whose bias depends on a station's setting is applied after the chain, to both sides of the last link's relation, by one stochastic matrix; by linearity the relation keeps the same channel. It is not a link, and it is read through the setting alone.

Remark 4.8 (The setting-dependent relabelling after the chain). A public choice whose bias depends on the private setting of the apparatus that measures a system, whichever party draws it — a test or generation round drawn against a probe setting, a probe light that fires only in test rounds — is the case the setting-dependent form of Definition 4.1 exists for, and this remark records how the compiler handles it. In the general format everything below holds per measured factor, with that factor's receiver segment, family and setting map in place of $\mathfrak{R}_{\mathrm{rec}}$, $\{F_v\}$ and t (Definition 3.3).

What is applied, and where. The compiler applies the stochastic matrix $P(f\,|\,t)$, read through the setting map $t : \mathcal{V} \to \mathcal{T}$, to the family $\{F_v\}$ the last link of $\mathfrak{R}_{\mathrm{rec}}$ delivered, *after* the reduction chain has been consumed and its interface conditions decided (Definition 7.1) and *before* the validator and the gates read the emitted declaration: the emitted family is $N_{v,f} = P(f\,|\,t(v))F_v$ over the support $\mathcal{V}_P$, the emitted alphabet is $\mathcal{V}_P$ in its product order, and the public part is declared on it (Definition 3.3). The same weighting is, in the same stroke, applied to the physical side of the last link's relation: the family the emitted one is a reduction of is $\{P(f\,|\,t(\gamma(w)))F_w^{\mathrm{ph}}\}_{(w,f)}$ under the correspondence $(w,f) \mapsto (\gamma(w),f)$ (Theorem 4.4 (vi)). Both sides are relabelled by one stochastic matrix, which is what makes the next statement true.

Why it is sound. The soundness is Theorem 4.4 (vi): by linearity of the adjoint, the relation the last link asserts survives the stochastic matrix $\delta_{vv'}P(f\,|\,t(v'))$ from $\mathcal{V}$ to $\mathcal{V}\times\mathcal{F}$, whose columns sum to one, with the *same* channel: Q, its sector map and its dimension are unchanged, every element $P(f\,|\,t(v))F_v$ is a scalar multiple of an existing one so that block diagonality holds by construction (Lemma 4.2 (ii)), every transported constraint operator has the same pull-back, and every entry's certificate is untouched. That theorem is this module's instance, with the one-line linearity argument written out, of the post-processing part (i) of Lemma "Linearity: post-processing, private settings and mixtures" (reductions/lem:linear) and of the relabelling that Remark "Classical maps before a squasher, and relabelling after one" (reductions/rem:intertwining) states and leaves to the consumer. What that theorem does *not* assert is the choice marginal: the probability of f is state-dependent through the setting marginal of the outcome statistics (Lemma 4.2 (iv)), which is

the output-state test-statistics model (Assumption "Output-state test statistics" (reductions/ass:tsm), Requirement 12.9), and no fraction is factored out.

Why it is not a link. A link in the sense of Definition "Typed link" (reductions/def:link) composes a *new* channel with the chain and carries a reachable-set statement for it; a relabelling composes no channel — it changes the alphabet at both ends of an existing relation and leaves the channel where it was. It therefore has no kind, no register and no position in a chain (Definition "Reduction chain" (reductions/def:chain)), and that module declares none: the chain record this module consumes and re-emits is unchanged by the weighting, the setting map it is read through is the class map of the joint alphabet of Definition "Joint alphabet, classes and the joint family of a private setting" (reductions/def:joint-family), and the weighting is recorded beside the chain as this module's own construction (Requirement 11.9). Putting f into the alphabet *before* the chain instead is sound as well but strictly weaker where the squasher is flag-shaped: every value of f then opens flag sectors of its own, and the squashed model gains adversarial freedom no physical state produces; applied after the chain, no sector is indexed by f and $N_{v,f} = 0$ as an operator wherever $P(f \mid t(v)) = 0$, so a zero-probability pair is unreachable for every state rather than merely of zero weight. That comparison is not a claim of this module; the claim is the soundness of the relabelling, Theorem 4.4 (vi).

Why the weighting is read through the setting only. P is declared on $\mathcal{T} \times \mathcal{F}$ and reaches an outcome through $t(v)$ alone. A weighting that reached it through the device outcome would make f an inference from that outcome, not a choice, and the relabelling would then be applied to a variable the adversary can influence through the state; that is what Assumption 10.4 excludes and what Requirement 12.4 enforces by type. The setting t itself is a variable the squasher's channel depends on — the conditional families differ with it — which is why it is in the alphabet before the first link and f is not.

With no setting-dependent choice declared, the emitted family is the chain's $\{F_v\}$ unchanged. What the emitted alphabet may be is the upstream modules' question, not this module's (Remark 4.9).

Uses Def. 3.3, Def. 4.1, Thm. 4.4, Def. 7.1 · *Assumes* A-COIN · *Used by* Req. 11.9 · newly derived · accepted

Bob's emitted outcome alphabet is chosen upstream, the receiver's declared coarse-graining deciding how much of the click pattern survives. Every object of this module indexed by it is stated over any finite alphabet, so a fine-grained instance changes declared data and no claim here.

Remark 4.9 (The outcome alphabet is chosen upstream). Nothing in this module fixes what Bob's outcome alphabet is. The emitted $\mathcal{V}$ is the alphabet of the family the last link of $\mathfrak{R}_{\mathrm{rec}}$ delivered (Definition 3.1 (c)), and that family's alphabet is the upstream modules': the raw outcome of a threshold-detector receiver is the *click pattern*, the set $S \subseteq [K]$ of ports that fired (Definition "Click-pattern alphabet" (receiver/def:clickalphabet)), and it is the receiver's declared coarse-graining — a surjection from $2^{[K]}$ onto the emitted alphabet (Definition "Declared coarse-graining and the emitted POVM" (receiver/def:coarse), Theorem "Coarse-graining of the click-pattern POVM" (receiver/thm:coarse-graining)) — that decides how much of the pattern survives, *before* the joint alphabet with Bob's setting is formed (Definition "Joint alphabet, classes and the joint family of a private setting"

(reductions/def:joint-family)) and before any reduction runs. A coarse-graining that keeps every pattern, one that keeps only click, no-click and multi-click, and one that keeps the single clicks and merges the rest are three declarations of that map, and this module receives the result of whichever was declared.

Every object of this module that is indexed by $\mathcal{V}$ is stated over an arbitrary finite non-empty alphabet and admits any finite disjoint union. The family slot $\{F_v\}_{v\in\mathcal{V}}$ of (1.1) is a family of block lists over a finite alphabet, and D(iii) is decided element by element whatever $|\mathcal{V}|$ is. The outcome correspondence γ of Definition 7.1 (C2) is a total surjection from any finite physical alphabet onto any finite emitted one, the direction being the only thing (C2) fixes (Remark 7.6), so a finer emitted alphabet is a γ with smaller fibres and nothing else. The readouts of the party partition and Bob's announcement map a_B (Definition 3.1 (d)) are declared on the alphabets the specification names, and ϕ (Definition 3.6) on the announcement classes; a finer $\mathcal{V}$ is a larger table for each and no new condition. And both forms of Definition 4.1 are constructions over a disjoint union — $\bigsqcup_t \mathcal{U}_t \times \{t\}$, and the support $\mathcal{V}_P \subseteq \mathcal{V} \times \mathcal{F}$ — so a disjoint union over settings and choice values is what they produce, not something they have to be extended to.

A *fine-grained instance* — one whose receiver keeps more of the click pattern than the coarsest sound choice, or one whose outcome carries a setting and a choice value beside the detection result — therefore changes the receiver's declared coarse-graining and the instance's public part, and changes no claim of this module: the compiler's claims are stated for the alphabet it is handed. What it does change downstream is not this module's either: the size of the declaration, the number of exact decisions V3 takes, and which coarse outcomes carry a bound (Theorem "Coarse-graining of the click-pattern POVM" (receiver/thm:coarse-graining) (iii): a bound on a coarse outcome does not descend to its fibres), which is the receiver's trade-off and the reason its coarse-graining is declared data rather than a convention. Which alphabets that coarse-graining can reach is the receiver's question; where a setting-dependent choice value enters the alphabet is Remark 4.8.

Uses Def. 1.1, Def. 3.1, Def. 3.6, Def. 4.1, Def. 7.1 · newly derived · accepted

5 Sifting, key computability and feasibility

Each announcement is a function of its sender's readout and of the announcements before it, sifting is determined by the announcements, and the holder computes the raw key locally, for a transcript of messages as for two announcement maps (Theorem 5.1). Feasibility is the one target condition that is not a typing or a finite syntactic check: an empty constraint list has an explicit product state, and otherwise the compiler verifies a default candidate and, if it fails, a supplied witness, and refuses when neither verifies (Theorem 5.5).

Each announcement is a function of its sender's readout and of the announcements before it, sifting is determined by the announcements, and the holder computes the raw key locally. An ensemble-form measurement slot of Alice's denotes a POVM.

Theorem 5.1 (Announcement locality, sifting and party computability)**.** Let $\mathfrak{S}$ be a well-formed specification (Definition 3.2) satisfying Assumption 10.1, so that in particular r^0_{h} maps into $\mathcal{Z}$ (Assumption 10.1 (ii)), and let $\mathfrak{D}$ be emitted by Definition 3.3. Then:

(i) *(announcement locality)* a_A is a function of $\pi_{\mathsf{A}}(x, v)$ alone and a_B of $\pi_{\mathsf{B}}(x, v)$ alone, and the joint announcement is (1.2);

(ii) *(D(v))* the specification's ς — consumed, and not itself a slot of (1.1) — witnesses announcement-determined sifting: $r(x, v) = \perp \iff \varsigma(a(x, v)) =$ fail;

(iii) *(D(viii))* r is party-computable: the total map

$$r_{\mathsf{h}}(o, \kappa) := \begin{cases} r^0_{\mathsf{h}}(o, \kappa), & \varsigma(\kappa) = \text{pass}, \\ \perp, & \varsigma(\kappa) = \text{fail}, \end{cases} \qquad (o, \kappa) \in \mathcal{O}_{\mathsf{h}} \times \mathcal{K}, \tag{5.1}$$

satisfies (1.10).

(iv) *(transcript protocols)* if the announcement slot holds a transcript protocol (Definition 1.1 (DX5)), then for every m the m-th component of $a(x, v)$ is a function of $\pi_{\pi_m}(x, v)$ and of the components before it, and of nothing else; a is total; and (ii) and (iii) hold with this a. For $L = 2$, $\pi = (\mathsf{A}, \mathsf{B})$ and a_2 independent of κ_1 this is (i);

(v) *(ensemble form)* if Alice's measurement slot is declared in ensemble form and the conditions of Definition 1.3 for that form hold, then the operators (1.7) satisfy $M^{(t)}_x \succeq 0$ and $\sum_x M^{(t)}_x = \mathbb{1}_{A_t}$ for every t, and $(\tau^{(t)}_A)^{1/2} M^{(t)}_x (\tau^{(t)}_A)^{1/2} = N^{(t)}_x$: the denoted declaration satisfies the POVM clause of D(iii) on A as an identity of real numbers, and $\operatorname{tr}[M_x \tau_A] = \sum_t \operatorname{tr} N^{(t)}_x$.

(vi) *(the compiled general format)* if $\mathfrak{S}$ is presented as Definition 3.1 (a′)–(e′) and $\mathfrak{D}$ is emitted by the general-format clauses of Definition 3.3, then: the announcement slot of $\mathfrak{D}$ holds the transcript $\mathfrak{S}_{\text{pub}}$ declares, message for message; the joint announcement function of $\mathfrak{D}$ is (1.8) evaluated at *those* tables and is a slot of nothing; and Alice's measurement slot holds the form the transmitter artefact declares. Consequently (iv) applies to $\mathfrak{D}$, hence (ii) and (iii) hold of it, and (v) applies to it exactly where that form is the ensemble form.

Moreover, under the default party partition these read: $a_A : \mathcal{X} \to \mathcal{C}_A$, $a_B : \mathcal{V} \to \mathcal{C}_B$, and $r(x, v) = r_{\mathsf{A}}(x, a(x, v))$ for $\mathsf{h} = \mathsf{A}$, respectively $r(x, v) = r_{\mathsf{B}}(v, a(x, v))$ for $\mathsf{h} = \mathsf{B}$.

Uses Def. 1.1, Def. 1.3, Def. 1.5, Def. 3.1, Def. 3.2, Def. 3.3 · *Assumes* A-SPEC · *Used by* Thm. 3.7 · newly derived · accepted · *Code* certiqs_key.protocol_compiler.spec: derived_holder_key_rule · *Oracles* registry_protocol_compiler.py

Proof. (i) is Definition 3.1 (d) together with (1.2): the announcement maps are declared on the readout alphabets, so no other signature is expressible.

(ii) Fix (x, v) and write $\kappa = a(x, v)$. If $\varsigma(\kappa) = \text{fail}$ then (3.1) gives $r(x, v) = \perp$. If $\varsigma(\kappa) = \text{pass}$ then $r(x, v) = r^0_{\mathsf{h}}(\pi_{\mathsf{h}}(x, v), \kappa) \in \mathcal{Z}$, and $\perp \notin \mathcal{Z}$ by $\mathcal{Z}_\perp = \mathcal{Z} \sqcup \{\perp\}$, so $r(x, v) \neq \perp$. The two implications are the two directions of the equivalence.

(iii) r_{h} of (5.1) is total on $\mathcal{O}_{\mathsf{h}} \times \mathcal{K}$ because ς is total and both branches are defined everywhere on their respective preimages. Fix (x, v) and put $o = \pi_{\mathsf{h}}(x, v)$, $\kappa = a(x, v)$. If $\varsigma(\kappa) = \text{pass}$, both sides of (1.10) equal $r^0_{\mathsf{h}}(o, \kappa)$ by (3.1) and (5.1); if $\varsigma(\kappa) = \text{fail}$, both equal $\perp$.

(iv) By (1.8) the m-th component is $a_m(\pi_{\pi_m}(x, v), \kappa_1(x, v), \dots, \kappa_{m-1}(x, v))$, and a_m has no other argument, its domain being declared as $\mathcal{O}_{\pi_m} \times \mathcal{C}_1 \times \cdots \times \mathcal{C}_{m-1}$; totality follows by induction on m from totality of the tables and of the readout maps. The proofs of (ii) and (iii) use of a only that it is a total map into $\mathcal{K}$ on which ς and r^0_{h} are defined, so they apply unchanged. With $L = 2$ and $a_2(o, \kappa_1) = a_B(o)$, (1.8) is (1.2).

(v) Fix t and drop it. $\tau_A \neq 0$ is positive semidefinite, so $\operatorname{tr} \tau_A > 0$ and $w_x := \operatorname{tr} N_x / \operatorname{tr} \tau_A \geq 0$ with $\sum_x w_x = 1$ by $\sum_x N_x = \tau_A$. With $S := \tau_A^{-1/2}$ on the support, zero off it, and P the support projector: $SN_xS \succeq 0$ and $w_x(\mathbb{1} - P) \succeq 0$, so $M_x \succeq 0$, and $\sum_x M_x = S\tau_A S + (\mathbb{1} - P) = \mathbb{1}$. From $0 \preceq N_x \preceq \tau_A$, every vector in the kernel of τ_A is in the kernel of N_x, so $PN_xP = N_x$; then $\tau_A^{1/2} M_x \tau_A^{1/2} = PN_xP + w_x \tau_A^{1/2}(\mathbb{1} - P)\tau_A^{1/2} = N_x$, using $\tau_A^{1/2} S = P$ and $\tau_A^{1/2}(\mathbb{1} - P) = 0$. The trace statement follows by taking traces and summing over t.

(vi) The three clauses are what the general-format clauses of Definition 3.3 do, and they are read off that definition and not inferred: the transcript is passed through verbatim, *a is not a slot* but the composition (1.8) of the tables passed through, and Alice's measurement slot is passed through in the form the artefact declares, that definition converting neither form into the other. For the consequences: Assumption 10.1 (ii) gives r^0_{h} into $\mathcal{Z}$ and Definition 3.2 (iii) the totality of the declared public tables, which is all (iv) consumes besides the signatures, and the signatures are the ones (d′) of Definition 3.1 declares; so (iv) applies, and (ii), (iii) with it, by (iv)'s last two sentences. Where the slot is the ensemble slot, the conditions (v) is stated under are exactly the general-format conditions Definition 1.3 states for that form — (S), (H), (P) of $\{N_x^{(t)}\}_x$ per Alice sector, the identity of its declared-order sum with $\tau_A^{(t)}$, and $\tau_A^{(t)} \neq 0$ — so (v) applies to it. *Nothing of (i) is claimed of such a declaration*: (i) is the two-map case, which is $L = 2$ with a_2 independent of κ_1, and (iv)'s last sentence is where the two meet.

The default-partition statements follow by substituting $\pi_{\mathsf{A}}(x, v) = x$ and $\pi_{\mathsf{B}}(x, v) = v$. □

Without the condition that the key rule map into the key values, the specification's sifting predicate stops witnessing announcement-determined sifting. Check V5 catches only a key rule that withholds a key on part of a passing class, so the condition is a clause of A-SPEC.

Remark 5.2 (Necessity of the key-rule side condition). Drop the requirement that r^0_{h} map into $\mathcal{Z}$, and Theorem 5.1 (ii) fails: a key rule that returns $\perp$ on a *passing* class makes $r(x, v) = \perp$ for pairs whose announcement class ς calls pass, so the specification's ς no longer witnesses announcement-determined sifting. The two ways this can happen differ in what is still decided. Where the key rule returns $\perp$ on part of a passing class, no predicate witnesses D(v): whether a round is sifted is not determined by the announcements, a sifting event becomes invisible to the public record, and check V5 of Definition 6.1 refuses the candidate. Where it returns $\perp$ on the whole of a passing class, D(v) still holds, witnessed by a predicate that calls that class fail, and V5 passes; what is lost is that the specification's ς is that witness, and (1.3) of Definition 1.1, which reads the specification's ς, may then count as key-live an acceptance value that no key-producing class reaches. This is why the condition is stated in the specification's type (Definition 3.1 (d)) and carried as a clause of Assumption 10.1, V5 catching only its first failure.

Uses Def. 3.1 · newly derived · accepted

Read against the joint outcome pair, party computability would hold for every raw-key map; read against the holder's readout it is a genuine restriction. It matters most where both parties' data share one outcome symbol, as when no source is trusted.

Remark 5.3 (Party computability read against the readouts). Read against the joint pair (x, v) instead of against the readouts, (1.10) would be satisfied by *every* r whatsoever, since r is by definition a function of (x, v); the condition would be vacuous exactly for the declaration shapes that need it most — those which place both parties' data in one outcome symbol, as the $d_A = 1$ embedding of Remark "The one-dimensional retained system" (transmitter/rem:da1) does. Checked against π_{h}, it is a genuine restriction: it fails as soon as two pairs with the same holder readout and the same announcements carry different key values (check V8 of Definition 6.1). That embedding — Alice's measured components folded into the receiver-side outcome, the party partition naming which components she holds — is the shape in which a protocol with no trusted source is declared, and the argument above is why party computability is checked against the partition in that shape.

Uses Def. 1.5 · newly derived · accepted

Alice announces the kind of round, Bob discloses his bit only on test rounds, and Alice then discloses hers only on detected test rounds. Party computability holds by the signatures of the tables, and two independent announcement maps cannot express the protocol.

Example 5.4 (A three-message transcript). Default readouts, $\pi_{\mathsf{A}}(x, v) = x$ and $\pi_{\mathsf{B}}(x, v) = v$. Alice's letter is $x = (z, b)$, with $z \in \{\text{test}, \text{gen}\}$ saying whether the round is a test or a generation round and a bit $b \in \{0, 1\}$; Bob's letter is

$v \in \{0, 1, \varnothing\}$, a bit or no detection. The transcript (Definition 1.1 (DX5)) has $L = 3$ messages:

(m1) $\pi_1 = \mathsf{A}$, $\mathcal{C}_1 = \{\text{test}, \text{gen}\}$, $a_1(z, b) = z$: Alice announces the kind of round;

(m2) $\pi_2 = \mathsf{B}$, $\mathcal{C}_2 = \{0, 1, \text{click}, \varnothing\}$, and $a_2(v, \kappa_1) = v$ if $\kappa_1 = \text{test}$ or $v = \varnothing$, and $a_2(v, \text{gen}) = \text{click}$ for $v \in \{0, 1\}$: Bob discloses his bit on test rounds and only the detection on generation rounds;

(m3) $\pi_3 = \mathsf{A}$, $\mathcal{C}_3 = \{0, 1, *\}$, and $a_3((z, b), \kappa_1, \kappa_2) = b$ if $\kappa_1 = \text{test}$ and $\kappa_2 \neq \varnothing$, and $*$ otherwise: Alice discloses her bit on detected test rounds.

By (1.8), $a((\text{test}, b), v) = (\text{test}, v, b)$ for $v \in \{0, 1\}$, $a((\text{test}, b), \varnothing) = (\text{test}, \varnothing, *)$, $a((\text{gen}, b), v) = (\text{gen}, \text{click}, *)$ for $v \in \{0, 1\}$ and $a((\text{gen}, b), \varnothing) = (\text{gen}, \varnothing, *)$: seven classes occur out of the twenty-four of $\mathcal{K} = \mathcal{C}_1 \times \mathcal{C}_2 \times \mathcal{C}_3$, and the tables are total on all of their domains as D(viii) asks, their values on combinations that never occur being immaterial. Sifting passes on $(\text{gen}, \text{click}, *)$ alone; with $\mathsf{h} = \mathsf{A}$ the key rule is $r^0_{\mathsf{A}}((z, b), \kappa) = b$, a function of Alice's readout. The acceptance map may send each test class (test, v, b) to the pair (v, b), so that the test statistics see both bits and the generation rounds disclose neither.

What two announcement maps cannot express. In the basic format, with two maps a_A on $\mathcal{O}_{\mathsf{A}}$ and a_B on $\mathcal{O}_{\mathsf{B}}$, Bob's announcement is a function of v alone and cannot depend on the kind of round, which he learns only from (m1); the only way to write this protocol there is to put z into Bob's readout, which misstates what Bob measures. Here party computability holds *by the signatures*: a_2 has no argument through which x could enter except the message κ_1 that Alice has made public (Theorem 5.1 (iv)).

What it is read under. Every message of a round follows every measurement of that round (Assumption "Sequential round structure" (transmitter/ass:seq)): (m1) is sent after Bob has measured, so his measurement cannot depend on it, and the declaration has no slot in which such a dependence could be written, $\{F_v\}$ being one fixed family. A protocol in which Bob chose his measurement after hearing z is outside this format and outside the fixed-measurement round model.

newly derived · accepted

An empty constraint list has an explicit product state in the feasible set. Otherwise the compiler verifies, exactly and block by block, the default candidate and, if it fails, a supplied witness, and refuses when neither verifies.

D(vii) is the one target condition that is not a typing or a finite syntactic check: whether $\mathcal{W} \neq \varnothing$ depends on the interaction of the marginal with the constraint entries. It is therefore treated in two cases, and the second one ends in a refusal rule rather than a theorem; the engine's order of the two candidates and its refusal are Requirement 13.3.

Theorem 5.5 (Feasibility by an explicit or a verified candidate)**.** Let $\mathfrak{D}$ be a candidate declaration satisfying D(i)–D(iii), and let $\mathcal{W}$ be as in Definition 2.1. (The hypothesis is on the conditions alone: nothing below uses that $\mathfrak{D}$ was emitted by Definition 3.3, and Theorem 6.2 applies this theorem to candidates.)

(i) If $\mathfrak{L} = \varnothing$ then $\mathcal{W} \neq \varnothing$. Explicitly, the block list

$$\omega^{\star(s)} = \begin{cases} \tau_A \otimes \dfrac{\mathbb{1}_{Q_s}}{d_Q}, & \text{trusted-source mode,} \\ \dfrac{\mathbb{1}_A}{d_A} \otimes \dfrac{\mathbb{1}_{Q_s}}{d_Q}, & \text{untrusted-source mode,} \end{cases} \qquad s \in \mathbb{S}, \tag{5.2}$$

which denotes $\tau_A \otimes \mathbb{1}_Q/d_Q$, respectively $\mathbb{1}_A/d_A \otimes \mathbb{1}_Q/d_Q$, lies in $\mathcal{W}$.

(ii) If $\mathfrak{L} \neq \varnothing$ and $\mathfrak{D}$ satisfies D(vi) as well, which makes each constraint leg of (2.1) a well-posed comparison, then $\mathcal{W} \neq \varnothing$ holds if and only if some block list ω satisfies the conditions of (2.1). The compiler decides those conditions *exactly*, by finitely many arithmetic operations and per block, on at most two candidates and in a fixed order: first the *default candidate* (5.2), which satisfies every conjunct of (2.1) except possibly the constraint legs (by the proof of (i)) and which the compiler forms in order to test and never emits; and, if that candidate fails a constraint leg, a *supplied* witness $\omega^\star$, consumed and verified and never emitted. It refuses exactly when the default candidate fails and either no witness is supplied or the supplied one fails.

The general format. For a candidate declaration in the general format of Definition 1.1 (DX1)–(DX6) satisfying D(i)–D(iii), and D(vi) for part (ii), with $\mathcal{W}$ the set of (2.3), both parts hold with the joint sectors in place of the sectors. In (i) the default candidate is the block list over the joint sectors

$$\omega^{\star(t,s)} = \begin{cases} \tau_A^{(t)} \otimes \dfrac{\mathbb{1}_{Q_s}}{d_Q}, & \text{trusted-source mode,} \\ \dfrac{\mathbb{1}_{A_t}}{d_A} \otimes \dfrac{\mathbb{1}_{Q_s}}{d_Q}, & \text{untrusted-source mode,} \end{cases} \qquad (t,s) \in \mathbb{T} \times \mathbb{S}, \tag{5.3}$$

and in (ii) the verification is the per-joint-block one listed in the proof, which forms no array larger than one joint block. At $|\mathbb{T}| = 1$ and one measured factor (5.3) is (5.2) and the two lists of decisions coincide item for item.

Uses Def. 1.1, Def. 1.3, Def. 2.1, Def. 2.5, Def. 3.3 · *Used by* Def. 6.1, Thm. 6.2, Rem. 6.3, Rem. 7.4, Req. 13.3 · newly derived · implemented (effective: accepted) · *Code* certiqs_key.protocol_compiler.feasibility: check_feasibility · *Oracles* registry_protocol_compiler.py

Proof. Idea. The default candidate is $\tau_A \otimes \mathbb{1}_Q/d_Q$, or $\mathbb{1}_A/d_A \otimes \mathbb{1}_Q/d_Q$ where there is no marginal, presented block by block; every conjunct of the feasible set except the constraint legs holds of it by direct computation, and at an empty list there are no constraint legs. It is formed and decided in exact rational arithmetic and never emitted, so its entries need not be binary64. Verifying any candidate is a finite list of exact decisions, each on one block or on one $d_A \times d_A$ matrix. The general format repeats both arguments over the joint sectors.

Step 1 (the default candidate lies in $\mathcal{W}$ at an empty list, (i)). Write $\chi_s := \mathbb{1}_{Q_s}/d_Q$, so that $\sum_s \operatorname{tr}\chi_s = \sum_s d_s/d_Q = 1$. The sector decomposition is not a condition here: $\omega^\star$ is presented as a list, one array on AQ_s per s, so the off-block conjunct holds by the presentation and there is nothing to verify.

Positivity, per block. In trusted-source mode $\omega^{\star(s)} = \tau_A \otimes \chi_s$ is Hermitian, τ_A being Hermitian by D(ii) and χ_s a positive real multiple of an identity; and it is positive semidefinite, being the tensor product of the positive semidefinite τ_A (D(ii)) with the positive semidefinite χ_s. In untrusted-source mode $\omega^{\star(s)} = \mathbb{1}_A/d_A \otimes \chi_s \succ 0$ outright. *Trace.* $\operatorname{tr}\omega^\star = \sum_s \operatorname{tr}\omega^{\star(s)} = \operatorname{tr}\tau_A \sum_s \operatorname{tr}\chi_s = \operatorname{tr}\tau_A = 1$ in trusted-source mode, by D(ii); in untrusted-source mode $\operatorname{tr}\omega^\star = 1$ directly. *Marginal.* In trusted-source mode $\omega^\star_A = \sum_s \operatorname{tr}_{Q_s}\omega^{\star(s)} = \tau_A \sum_s \operatorname{tr}\chi_s = \tau_A$; in untrusted-source mode there is no marginal condition beyond the trace, already checked. *Constraints.* With $\mathfrak{L} = \varnothing$ the conjunction of (2.2) is empty and holds vacuously. Hence $\omega^\star \in \mathcal{W}$.

Step 2 (the number model of Step 1). Two points of that argument concern the number model. First, every step used D(ii) *as an identity of the declared numbers*, which is what Definition 1.3 delivers — $\operatorname{tr}\tau_A = 1$ with its sum rounding nowhere, in particular. Second, *the entries of $\omega^\star$ need not be binary64 numbers, and nothing requires them to be*: $1/d_Q$ is a rational, dyadic only when d_Q is a power of two, and $\omega^\star$ is *never emitted*. It is formed and decided in exact rational arithmetic on the declared dyadic entries — the exact arithmetic in which the exact island (Definition "The exact island" (foundations/def:exact-island) (i)) takes its decisions on declared data, applied here to a candidate the compiler forms itself and never emits — so every predicate above is decided, not approximated, and no representability question arises for an object no consumer ever sees. A *supplied* witness is a different matter: it is declared data, hence dyadic, and its predicates are decided exactly on those dyadic numbers.

Step 3 (the verification of a candidate, (ii)). The stated equivalence is Definition 2.1 read literally. The verification of a candidate is finite and *per block*. Writing $n_s := d_A d_s$ for the size of the s-block, no array larger than one $n_s \times n_s$ block is formed at any point; where $|\mathbb{S}| = 1$ that one block is $d_A d_Q$ square, and is a block and not an exception (Theorem 8.2 (i)). It consists of

- $|\mathbb{S}|$ positivity decisions, each on an $n_s \times n_s$ matrix — the Hermiticity decision $\omega^{(s)} = \omega^{(s)\dagger}$, entry by entry, and the semidefiniteness of the resulting Hermitian matrix by exact symmetric elimination, of which the largest is on $\max_s n_s$ and not on $d_A d_Q$;
- one trace decision, $\operatorname{tr}\omega = 1$ on the scalar $\sum_s \operatorname{tr}\omega^{(s)}$, a sum of $|\mathbb{S}|$ block traces accumulated in the declared order;
- in trusted-source mode one marginal decision: form $\omega_A = \sum_s \operatorname{tr}_{Q_s}\omega^{(s)}$,

which is $|\mathbb{S}|$ partial traces onto A and $|\mathbb{S}|-1$ additions of d_A^2 entries, then $\omega_A = \tau_A$ over d_A^2 entries;

- and, per entry k, one constraint leg: in trusted-source mode $\sum_s \mathrm{tr}_{Q_s}[(\mathbb{1}_A \otimes \hat{O}_k^{(s)})\omega^{(s)}]$, a matrix of size d_A assembled from $|\mathbb{S}|$ block terms, compared with Θ_k in the ordering $\bowtie_k$ — an exact Löwner comparison, i.e. the semidefiniteness of one $d_A \times d_A$ difference for an inequality, or one exact entrywise equality when $\bowtie_k$ is $=$; in untrusted-source mode the scalar $\sum_s \mathrm{tr}\left[(\mathbb{1}_A \otimes \hat{O}_k^{(s)})\omega^{(s)}\right]$ compared with c_k as real numbers, according to the emitted source mode (D(vi)). The right-hand side of a trusted-source-mode leg is $c_k\tau_A$ for an entry tagged homogeneous and the declared Θ_k for one tagged general (Definition 2.5); the compiler forms $c_k\tau_A$ here in order to *test* the candidate, and does not emit it — the consuming engine constructs its own from the same declared c_k and τ_A, so the two form the same matrix.

Step 4 (one decision for an equality). The equality decision is equivalent to the two inequality conditions. For Hermitian operands X, Y, set $D = X - Y$. If $D \succeq 0$ and $-D \succeq 0$, each real eigenvalue of D is both nonnegative and nonpositive, hence zero; Hermitian diagonalization gives $D = 0$. Conversely $D = 0$ implies both inequalities. Thus entrywise $X = Y$ decides their conjunction exactly, in one predicate. The argument applies to every operator row of the general format below as well.

All of these are finitely many arithmetic operations on the declared data, and the largest object any of them forms is one $n_s \times n_s$ block or one $d_A \times d_A$ matrix.

Step 5 (the general format, (i)). Write $\chi_s := \mathbb{1}_{Q_s}/d_Q$ as above, so $\sum_s \mathrm{tr}\,\chi_s = 1$. The sector decomposition, now along the joint sectors, is again the presentation and not a condition: (5.3) is one array per joint sector. *Positivity, per joint block*: in trusted-source mode $\tau_A^{(t)} \otimes \chi_s$ is Hermitian and positive semidefinite, each $\tau_A^{(t)}$ being so by D(ii) in its general-format reading; in untrusted-source mode the block is positive definite outright. *Trace*: $\sum_{(t,s)} \mathrm{tr}\,\omega^{\star(t,s)} = \left(\sum_t \mathrm{tr}\,\tau_A^{(t)}\right)\left(\sum_s \mathrm{tr}\,\chi_s\right) = 1$ in trusted-source mode by D(ii), and $\sum_t (d_{A_t}/d_A) \sum_s \mathrm{tr}\,\chi_s = 1$ in untrusted-source mode by $d_A = \sum_t d_{A_t}$ (D(i)). *Marginal*, per Alice sector and in trusted-source mode: $\sum_s \mathrm{tr}_{Q_s}\,\omega^{\star(t,s)} = \tau_A^{(t)} \sum_s \mathrm{tr}\,\chi_s = \tau_A^{(t)}$, which is the second line of (2.3); in untrusted-source mode there is no marginal condition beyond the trace. *Constraints*: with $\mathfrak{L} = \varnothing$ the three constraint lines of (2.3) are empty conjunctions. So $\omega^\star \in \mathcal{W}$, and the two points of Step 2 apply unchanged: every step reads D(ii) as an identity of the declared numbers, and the entries of (5.3) need not be binary64 because the candidate is formed and decided in exact rational arithmetic and is never emitted.

Step 6 (the general format, (ii)). The equivalence is Definition 2.1 in its

general-format reading, read literally. Writing $n_{t,s} \coloneqq d_{A_t} d_s$ for the size of the joint block, the verification of a candidate consists of

- $|\mathbb{T}|\,|\mathbb{S}|$ positivity decisions, one Hermiticity and one exact symmetric elimination per joint block, the largest on $\max_{(t,s)} n_{t,s}$;
- one trace decision on $\sum_{(t,s)} \operatorname{tr} \omega^{(t,s)}$, accumulated in the declared order of the joint sectors;
- in trusted-source mode $|\mathbb{T}|$ marginal decisions, one per Alice sector: $|\mathbb{S}|$ partial traces onto A_t, $|\mathbb{S}| - 1$ additions and one comparison, each over $d_{A_t}^2$ entries;
- and, per entry k, its own leg. For an entry read through its lifted operator, in trusted-source mode one exact Löwner comparison per Alice sector on a $d_{A_t} \times d_{A_t}$ difference for an inequality, or one exact entrywise equality when $\bowtie_k$ is $=$, the right-hand side $c_k \tau_A^{(t)}$ for a homogeneous entry and the declared $\Theta_k^{(t)}$ for a general one; in untrusted-source mode one comparison of real numbers against c_k. For an entry declared party-local and scoped to f, one exact Löwner comparison per pair (t, s_{f^c}) for an inequality, or one exact entrywise equality per pair when $\bowtie_k$ is $=$, on a difference of size $d_{A_t}\, d_{f^c,s_{f^c}} = n_{t,s}/d_{f,s_f}$, assembled from the $|\mathbb{S}^f|$ joint blocks of that row by partial traces over Q_{f,s_f} — which is the last line of (2.3), and is *never larger than one joint block*. The same decision rule applies in either source mode; a 1×1 row uses one real-number comparison as its scalar specialization.

No lifted operator is formed at any of these steps. The value $\operatorname{tr}_{Q_s}[(\mathbb{1}_{A_t} \otimes \hat{O}_k^{\uparrow(s)})\, \omega^{(t,s)}]$ is computed by contracting the joint block against the declared block $\hat{O}_k^{(s_{f_k})}$ on the tensor factor $Q_{f_k,s_{f_k}}$ of that block and tracing out the others, which reads one joint block and one block of the scope and forms no array on Q; the identity of that contraction with the displayed trace is the definition of the lifted operator in (DX6). *So the largest object any decision of this verification forms is one $n_{t,s} \times n_{t,s}$ joint block, one $d_{A_t} \times d_{A_t}$ matrix, or one party-local difference, and each of the last two is at most one joint block*; at $|\mathbb{T}| = 1$ and one measured factor the list is the list of Step 3, item for item, with $n_{t_0,s} = n_s$. □

Deciding non-emptiness of the feasible set for a general list is a semidefinite feasibility problem, while whoever produced the entries can exhibit a reachable state. A supplied witness turns D(vii) into a check, and refusing when no candidate verifies avoids an infimum over the empty set.

Remark 5.6 (Feasibility by witness rather than by search). Deciding $\mathcal{W} \neq \varnothing$ for a general list is a semidefinite feasibility problem. Whoever produced the entries produced them from a reachable-state argument and can exhibit a reachable state,

and a supplied witness turns D(vii) into a check. Refusing when no candidate verifies is the safe direction: a declaration whose feasible set is empty makes the consuming engine's infimum an infimum over the empty set, which is exactly the pathology the refusal prevents.

newly derived · accepted

6 The emitted validator

The validator's eight checks are decided exactly on the candidate declaration. The main result, Theorem 6.2, shows that passing every check establishes all eight target conditions of the declared numbers, that every check except the feasibility check is complete, so that a well-formed declaration can be refused only for want of a verified witness, and that every decision is finite, exact and taken block by block. Such a refusal means that no verified feasible state was shown, not that the protocol is defective.

Eight exact checks on a candidate declaration: sector maps, the marginal, the operators, the key alphabet, sifting, constraint typing, feasibility and party computability. Feasibility passes only on an empty list, a verified default candidate or a verified supplied witness.

The validator is the list of checks the compiler runs on a candidate declaration before it emits one. A candidate failing a check is refused with a reason naming the check (Requirement 13.1); what passing all eight establishes is Theorem 6.2, and which conditions are decided on the chain rather than here, and which objects check V1 does not compare, is Remark 7.4. That check V7 decides on quantities formed in exact arithmetic is Requirement "Exact arithmetic on the island, unbudgeted rounding elsewhere" (foundations/req:exact-island) at work, and that no product block, lifted operator or denoted family of an ensemble form is a datum is Requirement 11.4.

Definition 6.1 (The emitted validator)**.** Given a candidate declaration (1.1) with its emitted index map, the checks are as follows. *Every operator condition below is decided exactly*, in the sense of Definition 1.3, and every combinatorial condition exactly as well.

V1 (finiteness, totality, and the sector map)
every declared alphabet is finite and non-empty and every declared map is tabulated on all of its stated domain; every declared system carries a finite dimension; $d_Q = \sum_{s \in \mathbb{S}} d_s$; the restriction of idx to $\mathbb{S}$ is the total order the sector map fixes; *and the sector maps agree*: there is one emitted map $\mathfrak{s}$, and the map carried by each supplied Q-operator family is compared against it in *block kind*, in the *list of block identifiers*, in their *order* and in their *dimensions*. A disagreement on any of the four is refused, naming the first position at which the lists

differ. The families compared are exactly the Q-operator families of (1.1), which is the list D(i) quantifies over and no wider one: the emitted family $\{F_v\}$, which the last link of $\mathfrak{R}_{\mathrm{rec}}$ delivers as its target POVM against the map the chain's producer wrote it against, and each constraint entry of $\mathfrak{L}$, declared in $\mathfrak{S}_{\mathrm{con}}$ or transported by a link of either segment. Those two slots are the whole of (1.1) that carries a Q-operator (Definition 1.1), so the list is exhaustive by construction rather than by enumeration; the objects outside it are compared where they belong. The entries of $\mathfrak{S}_{\mathrm{con}}$ are compared as well as the transported ones: Definition 3.3 fills $\mathfrak{L}$ from both sources, Definition 3.1 (e) has each declared $\hat{O}_k$ carry its own sector map, and an entry written against a different map is the error this check exists for, whichever source it arrived from. The comparison is combinatorial — it compares labels and integers, never matrix entries — and it is the whole of the structural verification the emitted declaration receives: block-diagonality is carried by the type (Theorem 3.7 (ii)), so the one structural error that can still arise is two parties disagreeing about what the sectors are, and V1 is the check for it.

V2 (marginal slot)
the slot holds none, or a matrix τ_A on A — one block, A carrying no sector decomposition — satisfying $\tau_A = \tau_A^\dagger$, $\tau_A \succeq 0$ and $\operatorname{tr}\tau_A = 1$, each decided exactly, and with the trace accumulated over the row index verified to round nowhere; a failure of the last is refused.

V3 (operators, block by block)
$M_x = M_x^\dagger$ and $M_x \succeq 0$ for each $x \in \mathcal{X}$, and $\sum_x M_x = \mathbb{1}_A$ with the sum, in the emitted index order, verified to round nowhere, a failure of which is refused; $F_v^{(s)} = F_v^{(s)\dagger}$ and $F_v^{(s)} \succeq 0$ for each $(v, s) \in \mathcal{V} \times \mathbb{S}$, and, for each s, $\sum_v F_v^{(s)} = \mathbb{1}_{Q_s}$ with that sum verified to round nowhere, a failure being refused with the sector and the outcome index at which the first inexact addition occurred; and $\hat{O}_k^{(s)} = \hat{O}_k^{(s)\dagger}$ for each (k, s). Every one of these is decided exactly. There is no off-block condition: an emitted Q-operator is a block list, so there is no $\Pi_s F_v \Pi_{s'}$ or $\Pi_s \hat{O}_k \Pi_{s'}$ to evaluate and nothing that could be non-zero.

V4 (key alphabet)
every tabulated value of r lies in $\mathcal{Z} \sqcup \{\perp\}$.

V5 (sifting consistency)
for all (x, v) and (x', v') with $a(x, v) = a(x', v')$: $r(x, v) = \perp \iff r(x', v') = \perp$.

V6 (constraint typing, including the tag)
when the marginal slot is filled: $\bowtie_k \in \{\preceq, =, \succeq\}$, and the entry's

right-hand side is of one of the two admitted types, which is what fixes its tag (Definition 2.5): homogeneous, whose datum is of real scalar type, or general, whose datum is a matrix on A with $\Theta_k = \Theta_k^\dagger$ decided exactly. When the slot holds none: $\bowtie_k \in \{\leq, =, \geq\}$ and the right-hand side is of real scalar type. A real scalar right-hand side is accepted in trusted-source mode at every d_A, and is the licensed form there (Definition 2.5). The relation symbol, the type of the right-hand side and the tag that type fixes are compared exactly; an entry whose right-hand-side slot holds neither admitted object, or a complex-typed right-hand side in untrusted-source mode, is refused by type and not measured. The check also reads an object (1.1) does not carry, the tag field of an *arriving* entry, in $\mathfrak{S}_{\mathrm{con}}$ or transported by a link: where an arriving entry carries an explicit tag field, V6 compares that field with the type of the datum it names, and a disagreement is refused. An entry of (1.1) in either of the two forms above is the triple $(\hat{O}_k, \bowtie_k, \mathrm{rhs}_k)$ and holds no tag field independent of the type of its datum, the emitted tag being read off that type (Definition 2.5); so on a candidate declaration this clause has no subject, refuses nothing, and is not among the conjuncts D(vi) is paired with (Theorem 6.2, proof). An entry carrying no tag field is not refused: its tag is the type of its datum and is read off it, which is what lets an entry declared as a real constant, carrying no tag field, be read here.

V7 (feasibility; one-sided, three branches)
V7 passes in exactly three cases:

(a) $\mathfrak{L} = \varnothing$; or

(b) $\mathfrak{L} \neq \varnothing$ and the default candidate (5.2), which the compiler forms and does not emit, passes the per-block verification of Theorem 5.5 (ii); or

(c) $\mathfrak{L} \neq \varnothing$, the default candidate fails, and a witness $\omega^\star$ is supplied *as a block list against* $\mathfrak{s}$ — the map it carries compared with $\mathfrak{s}$ in block kind, in identifiers, in their order and in their dimensions, exactly as V1 compares the map of a declared Q-operator family — and passes that same verification. This is the one place a witness's sector map is compared, and it is here rather than at V1 because $\omega^\star$ is a block list on AQ, is no slot of (1.1), and is quantified over by no target condition (Definition 1.5).

Otherwise V7 fails and the candidate is refused, the refusal naming, where a witness was supplied and rejected on its map, the first position at which the two lists differ. The verification in (b) and (c) is the $|\mathbb{S}|$ block positivity decisions — Hermiticity and semidefiniteness per block — the trace, the marginal equality in trusted-source

mode, and one leg per constraint entry, all decided exactly, on quantities formed in exact arithmetic (Theorem 5.5 (ii)). It is taken on a candidate satisfying D(i)–D(iii) and D(vi), which V1–V3 and V6 decide; a candidate failing one of them is refused there. V7 is the one check of the list that is *not* equivalent to its target condition: it implies D(vii) and is implied by nothing weaker than a candidate that verifies (Theorem 6.2 (iii)).

V8 (party computability)
a_A and a_B are tabulated on $\mathcal{O}_{\mathsf{A}}$, $\mathcal{O}_{\mathsf{B}}$; and for all (x, v), (x', v') with $\pi_{\mathsf{h}}(x, v) = \pi_{\mathsf{h}}(x', v')$ and $a(x, v) = a(x', v')$: $r(x, v) = r(x', v')$.

The general format. On a candidate declaration in the general format of Definition 1.1 (DX1)–(DX6) the eight checks read as follows, every operator condition again being decided exactly. In the basic format — one Alice sector, one measured factor, POVM form, two messages neither reading the other and no party-local entry — each reading is the check of the same name above, conjunct for conjunct, so on such a candidate the two readings admit and refuse alike.

V1 in the general format. Finiteness and totality as above, every table of the transcript included: each a_m is tabulated on all of $\mathcal{O}_{\pi_m} \times \mathcal{C}_1 \times \cdots \times \mathcal{C}_{m-1}$, which is also where the signature clause of D(viii) lives: a table whose stated domain names a later message's alphabet is not of that type and is refused here. *The sector map is one factorised map*: at most one factor tagged A and, where there is one, first in the list; at least one factor tagged Q; *one block kind across the factors*, a disagreement being refused, naming the first factor at which the kind differs; $d_{Q_f} = \sum_{s_f} d_{f,s_f}$ for every factor and $d_Q = \prod_f d_{Q_f}$; and, where an A factor is declared, $d_A = \sum_t d_{A_t}$. The restriction of idx to each factor's sector set is the total order that factor's own map fixes, and its restriction to $\mathbb{T}$ is the order the A factor's map fixes; the product orders on $\mathbb{S}$ and $\mathcal{V}$ and the order on the joint sectors are derived by (DX1) and (DX3) and are not compared, there being no second datum to compare them with. *And the maps agree, factor by factor*: the map carried by each declared family $\{F^f_{v_f}\}_{v_f}$ is compared against the map of factor f, and the map carried by each entry's operator $\hat{O}_k$ against the map of that entry's scope, in the same four components, with the same refusal naming the first differing position. *Alice's declared data are compared too, against the map of the factor tagged A*: the map carried by τ_A, the map carried by each operator of Alice's measurement slot — M_x in POVM form, N_x in ensemble form — and the map carried by each Θ_k of an entry declared general are each compared against that factor's map in the same four components (block kind, block identifiers, their order, dimensions), with the same refusal naming the first differing position. That comparison is what makes condition (T) of Definition 1.3 decided for Alice's data, which

its general-format paragraph calls non-empty for them; it is decided here and nowhere else, and without it a candidate whose Alice block list carried a different block kind, a different identifier list or a different order would pass every check and violate D(i). Where no factor is tagged A there is one Alice block and no map to compare, as in V1 above. *The list of compared objects is again exhaustive by construction*: the objects D(i) quantifies over are the per-factor families, the entries' operators and Alice's declared data. The product family is no datum, and the lifted operator of a scoped entry is no datum; the descriptors that the last links of a chain produce, decided per factor at Definition 7.1 (C1), and a supplied witness, compared at branch (c) of V7, stay outside the list; and the physical families stay outside it too, one per factor.

V2 in the general format. The slot holds none, or a block list $(\tau_A^{(t)})_t$ against the A factor's map with $\tau_A^{(t)} = \tau_A^{(t)\dagger}$ and $\tau_A^{(t)} \succeq 0$ for every t and $\sum_t \operatorname{tr} \tau_A^{(t)} = 1$, each trace accumulated over the row index and the block traces then added in the declared order of $\mathbb{T}$, with *every* one of those additions verified to round nowhere; a failure of the last is refused. *Where Alice's measurement slot is in ensemble form, additionally* $\tau_A^{(t)} \neq 0$ *for every* t, a comparison of declared entries with the datum 0, a failure being refused naming the sector. At $|\mathbb{T}| = 1$ this is V2 above, the one block being the whole of A.

V3 in the general format. Per measured factor: for every $f \in \mathbb{F}$, every $v_f \in \mathcal{V}^f$ and every $s_f \in \mathbb{S}^f$, $F_{v_f}^{f,(s_f)}$ Hermitian and positive semidefinite, and for each s_f the sum $\sum_{v_f} F_{v_f}^{f,(s_f)} = \mathbb{1}_{Q_{f,s_f}}$ with its declared-order sum verified to round nowhere, a failure being refused with the *factor*, the sector and the outcome index at which the first inexact addition occurred. *On Alice's data, block by block along* $\mathbb{T}$: in POVM form $M_x^{(t)}$ Hermitian and positive semidefinite for every (x,t) and $\sum_x M_x^{(t)} = \mathbb{1}_{A_t}$ for every t, the sum in the emitted index order verified to round nowhere, a failure being refused with the Alice sector; in ensemble form $N_x^{(t)}$ Hermitian and positive semidefinite for every (x,t) and, *in place of the completeness identity*, the identity $\sum_x N_x^{(t)} = \tau_A^{(t)}$ of (1.6) decided entry by entry against the declared block of τ_A, a failure being refused with the sector and the first differing entry, with that declared-order sum verified to round nowhere, a failure being refused with the sector and the index of the first inexact addition. *An ensemble-form slot beside a marginal slot holding* none *is refused*; this is D(iii)'s general-format clause, and it is a decision on which of two slots is filled, not on a number. And $\hat{O}_k^{(s_{f_k})}$ is Hermitian for every entry and every block of its scope. *Nothing is decided on a product block* (1.5)*, on a lifted operator or on the denoted family* (1.7): none of the three is a datum, and that the first is a POVM on Q and the third a POVM on A are theorems (Definition 1.1 (DX3), Theorem 5.1 (v)).

V4, V5, V7 in the general format. V4 and V5 keep their wording, with a the function (1.8) derives from the transcript's tables; V5's pairs are compared on the values of that function and on nothing about how it was built. V7 keeps its three branches and its refusal, and what its branches (b) and (c) verify is the per-*joint*-sector verification of Theorem 5.5 (ii) in the general format, on a candidate satisfying D(i)–D(iii) and D(vi), in their general-format reading, which V1–V3 and V6 decide (a candidate failing one of them is refused there): the default candidate is (5.2) read per joint sector, and a supplied $\omega^\star$ is required to be a block list over the joint sectors, its map compared in the same four components at branch (c) and there alone.

V6 in the general format. When the marginal slot is filled: $\bowtie_k \in \{\preceq, =, \succeq\}$, and the entry's right-hand side is of one of the *three* admitted forms, which with the arriving tag field is what fixes its tag (Definition 2.5): homogeneous with a real scalar, general with a block list $(\Theta_k^{(t)})_t$ along $\mathbb{T}$ decided Hermitian block by block, or party-local with a real scalar. When the slot holds none: the right-hand side is of real scalar type, either untagged or tagged party-local. *The relation symbol is decided by form and not by mode alone*: for an *untagged* entry $\bowtie_k \in \{\preceq, =, \succeq\}$ when the slot is filled and $\bowtie_k \in \{\leq, =, \geq\}$ when it holds none, exactly as in V6 above; for an entry declared party-local, $\bowtie_k \in \{\preceq, =, \succeq\}$ *in either case*, its reading being an operator relation (Definition 2.5, the relation-symbol clause of the party-local form; Definition 1.5, D(vi) in the general format), with the scalar symbols admitted as well where every row of that entry is 1×1 — every Alice block, and every sector block of every measured factor other than the entry's scope, of dimension one, $d_A = 1$ at one measured factor included — the three operator symbols being the three scalar symbols there. This is a decision on a declared symbol beside a declared tag and measures nothing, and at an entry not declared party-local it is the clause of V6 above word for word, so nothing that clause admits or refuses moves. *Every entry's scope names a measured factor of the emitted sector map*, a comparison of a label against the factor list, a failure being refused naming the entry; this is the general-format half of D(vi) and is decided and never measured, as the relation symbol is. *A party-local entry is the one that arrives carrying that tag value*, and it carries a tag field the type of its datum does not determine, that type not separating it from a homogeneous one. On a candidate carrying one, the arriving-tag clause of V6 above therefore has a subject, and it is this comparison: the datum is of the type *that* tag admits, a real scalar, and the scope names a measured factor; a party-local tag beside a matrix right-hand side is refused. The sentence of V6 above that an entry holds no tag field independent of the type of its datum is true of the two untagged forms, homogeneous and general, and of them alone. *Nothing is inferred*: an entry arriving without the party-local tag is never read party-locally. So the arriving-tag clause refuses no party-local entry beside a real scalar, which a

literal reading of it would otherwise do, and wherever no entry is tagged the reading is that of V6 above (Definition 2.5).

V8 in the general format. The announcement clause reads the transcript: every a_m is tabulated on all of its stated domain, which is V1's clause restated where D(viii) pairs with it; the key clause is unchanged, with a the derived function.

Uses Def. 1.1, Def. 1.3, Def. 1.5, Def. 2.5, Def. 3.1, Def. 3.3, Thm. 5.5 · *Used by* Thm. 3.4, Thm. 3.7, Rem. 4.6, Thm. 6.2, Def. 7.1, Rem. 7.4, Thm. 7.5, Req. 11.5, Req. 11.6, Req. 13.1, Req. 13.3 · newly derived · implemented (effective: accepted) · *Code* certiqs_key.protocol_compiler.validation: validate · *Oracles* registry_protocol_compiler.py

Passing the validator's eight checks establishes every target condition of a well-formed declaration, and every check but the feasibility check is complete, so a well-formed declaration is refused only when no default or supplied candidate is verified feasible. Every decision is finite and exact.

The consuming engine takes D(i)–D(viii) as hypotheses without re-deriving them, so what a pass of the validator buys has to be exactly those conditions. The theorem pairs each check of Definition 6.1 with its target condition of Definition 1.5, names the one check that is not equivalent to its condition, and counts what the checks compute. Remark 5.6 records why an empty feasible set is the dangerous error and a spurious refusal is not. For the clause of (iv) that the validator assembles no array larger than one matrix of size d_A, d_s or $d_A d_s$ for a single sector s, see also Requirement 11.2, by which the engine holds every operator on Q as its block list.

Theorem 6.2 (Soundness, completeness and decidability of the validator). For a candidate declaration with finite tabulated data:

(i) *(soundness, on all eight conditions)* if V1–V8 all pass then D(i)–D(viii) all hold, of the declared numbers;

(ii) *(completeness, on seven of the eight)* if D(i)–D(vi) and D(viii) hold then V1–V6 and V8 all pass;

(iii) *(V7 is one-sided, and the gap is exactly its third branch)* the converse of (i) fails, and it fails only at V7: D(vii) does *not* imply V7. For a witness, take untrusted-source mode with Q a qubit carrying the single sector $\mathbb{S} = \{s_0\}$, every other slot well-formed, and the one-entry list $\mathfrak{L} = \{(|0\rangle\langle 0|_Q\,,\ \geq,\ 1)\}$, whose entry reads $\mathrm{tr}[(\mathbb{1}_A \otimes |0\rangle\langle 0|)\,\omega] \geq 1$. Then $\hat{O}_1$ is a one-block list, Hermitian, the typing is that of D(vi), and $\omega = \eta \otimes |0\rangle\langle 0|$ is feasible for any state $\eta \in S_=(A)$ — it is Hermitian and positive semidefinite outright, has unit trace, and its constraint value is exactly 1 — so $\mathcal{W} \neq \varnothing$ and D(i)–D(viii) all hold. But $\mathfrak{L} \neq \varnothing$, and the default candidate (5.2) that branch (b) of V7 tries first gives $\mathrm{tr}[(\mathbb{1}_A \otimes |0\rangle\langle 0|)(\mathbb{1}_A/d_A \otimes \mathbb{1}_Q/2)] = \frac{1}{2}$, which fails ≥ 1; so with no witness supplied branch (c) is unavailable, V7 fails and the well-formed

declaration is refused. Exactly the same happens for every declaration with $\mathfrak{L} \neq \varnothing$ presented without a witness whose feasible set is non-empty but does not contain (5.2). The refusal errs on the safe side — a refused declaration is never consumed — and the incompleteness is deliberate: deciding D(vii) in general is a semidefinite feasibility problem this module does not solve;

(iv) *(decidability, and what is computed)* each of V1–V6 and V8 is decided by finite enumeration over the declared alphabets together with finitely many arithmetic operations on the declared matrices; V7 is decided in the same sense on the candidate of its branch (b) or (c), and is otherwise a refusal. The arithmetic part consists of *exact decisions*: an identity, a semidefiniteness, or the exactness of an addition. The following counts give the complete applicable predicate inventory for well-typed data, not the number an early-terminated execution necessarily performs. Write $I_{\mathrm{tr}} = 1$ in trusted-source mode and $I_{\mathrm{tr}} = 0$ when the marginal slot holds none. The complete arithmetic inventory of V2, V3 and V6 is

$$\underbrace{3I_{\mathrm{tr}}}_{\text{V2}} + \underbrace{2|\mathcal{X}| + 1}_{\text{V3 on } A} + \underbrace{|\mathbb{S}|\,\big(2|\mathcal{V}| + 1\big)}_{\text{V3 on } Q} + \underbrace{m\,|\mathbb{S}|}_{\text{V3 on } \mathfrak{L}} + \underbrace{m_{\mathrm{gen}}}_{\text{V6}} \tag{6.1}$$

predicate decisions, where $m_{\mathrm{gen}} \leq m$ counts the entries declared general — the only ones carrying a matrix whose Hermiticity is decided; V6's remaining content is m comparisons of a relation symbol, a tag and a type, which are decided and not computed. *Beside these stand the summation decisions of condition (S)*: one on $\operatorname{tr} \tau_A$ in trusted mode and none on the absent marginal in untrusted mode; one on $\sum_x M_x$; and one per sector on $\sum_v F_v^{(s)}$, so $|\mathbb{S}| + 1 + I_{\mathrm{tr}}$ of them, each a pass over the additions of one declared-order sum. And V7, on the candidate it verifies, takes $2|\mathbb{S}| + 1$ decisions — Hermiticity and semidefiniteness per block, and the trace — one further in trusted mode for the marginal equality, and one decision per constraint entry: semidefiniteness for an inequality, exact entrywise equality for the relation $=$ (Theorem 5.5 (ii)); in untrusted mode a constraint leg is one comparison of real numbers instead. Every one of them is taken on a matrix of size d_A, d_s or $d_A d_s$ for a single s; *the validator assembles no array larger than one such matrix*, which where $|\mathbb{S}| = 1$ is d_Q or $d_A d_Q$ square, the one block being the whole space there (Theorem 8.2 (i)). In trusted mode the V2 term is 3 and not 4: D(ii)'s named Hermiticity condition and the Hermiticity half of its positivity condition are the same decision, taken once. In untrusted mode V2 decides only the marginal slot's none branch, with no marginal arithmetic or marginal summation predicate.

The general format. All four parts hold for a candidate declaration in the general format of Definition 1.1 (DX1)–(DX6), with the checks read as

in the general-format paragraph of Definition 6.1 and the conditions as in that of Definition 1.5: (i) soundness on all eight and (ii) completeness on D(i)–D(vi) and D(viii); (iii) with the *same* witness, which is a general-format declaration at $|\mathbb{T}| = 1$, one measured factor and no party-local entry; and (iv) with the predicate decisions of V2, V3 and V6 numbering

$$\underbrace{I_{\mathrm{tr}}\big(2|\mathbb{T}|+1+|\mathbb{T}|\,\nVdash[\text{ensemble form}]\big)}_{\text{V2}} + \underbrace{|\mathbb{T}|\,\big(2|\mathcal{X}|+1\big)}_{\text{V3 on } A} + \underbrace{\sum_{f\in\mathbb{F}} |\mathbb{S}^f|\big(2|\mathcal{V}^f|+1\big)}_{\text{V3 per measured factor}} + \underbrace{\sum_{k=1}^{m} |\mathbb{S}^{f_k}|}_{\text{V3 on } \mathfrak{L}} + \underbrace{m_{\mathrm{gen}}\,|\mathbb{T}|}_{\text{V6}}, \tag{6.2}$$

beside them the summation decisions of condition (S) — one on $\operatorname{tr}\tau_A$ in trusted mode only, one per Alice sector on Alice's family, and one per factor and sector on that factor's family, so $|\mathbb{T}| + \sum_f |\mathbb{S}^f| + I_{\mathrm{tr}}$ of them — and V7, on the candidate it verifies, $2|\mathbb{T}|\,|\mathbb{S}|+1$ decisions, $|\mathbb{T}|$ further in trusted mode for the marginal equalities, and per entry k either $|\mathbb{T}|$ operator comparisons, one comparison of real numbers in untrusted mode, or, for a party-local entry, $|\mathbb{T}|\,|\mathbb{S}|/|\mathbb{S}^{f_k}|$ operator comparisons. Each operator comparison is one semidefiniteness decision for an inequality or one exact entrywise equality decision for $=$ (Theorem 5.5 (ii), general format). *Every one of them is taken on a matrix of size d_{A_t}, d_s or $d_{A_t}d_s$ for a single joint sector, or of size $d_{A_t}d_s/d_{f,s_f}$ for a single party-local row; the validator assembles no array larger than one joint block.* In the basic format — $|\mathbb{T}| = 1$, one measured factor, POVM form, two messages neither reading the other and no party-local entry — (6.2) is (6.1) term for term, and every other count of this paragraph is the corresponding count of (iv).

Uses Def. 1.1, Def. 1.3, Def. 1.5, Def. 2.1, Def. 2.5, Thm. 3.7, Thm. 5.5, Def. 6.1 · *Used by* Rem. 4.6, Rem. 6.3, Thm. 7.5, Def. 9.1, Thm. 9.4, Req. 13.2, Req. 13.4 · newly derived · implemented (effective: accepted) · *Code* certiqs_key.protocol_compiler.validation:decidability_counts · *Oracles* registry_protocol_compiler.py

Proof. Idea. Parts (i) and (ii) are proved together, by pairing each check with its target condition. Three kinds of conjunct arise: combinatorial ones, which both sides state as the same comparison of labels, integers or tabulated values; numerical ones, which both sides state as the same exact predicate on the same declared numbers; and structural ones, which the block type makes true and neither side states. Two pairs, V5/D(v) and V8/D(viii), are combinatorial but need an argument. V7 implies D(vii), which completes (i), and (iii) is the counterexample stated with it. Part (iv) counts the decisions. The general format takes the same steps, the point being that on each side the quantifiers still run over one list.

Step 1 (combinatorial conjuncts: the restatement is literal). V1's finiteness, totality, dimension-sum, index-order and sector-map clauses; V4's key-alphabet clause; V6's relation-symbol, tag and type clauses. Each compares labels, integers or tabulated values, so V1/D(i), V4/D(iv) and the combina-

torial half of V6/D(vi) are equivalences in both directions. For V1's sector-map clause we *name* both lists rather than identify them: D(i) asks that there be one map and that every declared Q-operator be a block list against *it*, and the declared Q-operators of (1.1) are exactly the family $\{F_v\}_{v\in\mathcal{V}}$ together with the m entries $\hat{O}_1, \ldots, \hat{O}_m$ of $\mathfrak{L}$ (Definition 1.1); V1 compares the four components of the map carried by *each of those same objects* against the emitted one, and compares the map of nothing else (Definition 6.1). The two quantifiers therefore run over one list, and two finite lists of labels and integers agree or they do not. *Neither the descriptor that the chain's last link produces nor a supplied feasibility witness is on either side*: the first is a component of the chain record, decided at Definition 7.1 (C1), and the second is a block list on AQ tested at branch (c) of V7 — so a candidate meeting D(i)–D(vi) and D(viii) cannot be refused at V1 on account of either, which is what (ii) asserts and what a wider V1 would have made false. *V1's index-order clause is the last clause of D(i)*, word for word: both ask that the restriction of idx to $\mathbb{S}$ be the total order $\mathfrak{s}$ fixes, and that comparison is again of two finite lists of labels. Both directions therefore hold of it as well, which is what makes (ii) true of a *candidate*: a candidate meeting D(i)–D(vi) and D(viii) meets that clause too, so V1 cannot refuse it there.

V6's combinatorial clauses pair with D(vi) on the same m objects. D(vi) asks of each entry of $\mathfrak{L}$ that its relation symbol lie in the set its emitted source mode admits and that its right-hand-side slot hold one of the two admitted data, whose type *is* the tag (Definition 2.5); V6 decides those two things, of those same m entries, keyed on whether the marginal slot is filled — which is the emitted source mode (Definition 1.1). V6's remaining clause, that an explicit tag field on an *arriving* entry agree with the type of the datum it names, is not a further conjunct on (1.1): no entry of a declaration carries such a field, so on a candidate that clause has no subject and cannot refuse one (Definition 6.1, V6). The pairing is therefore between equal lists here as well.

Step 2 (numerical conjuncts: the two sides are the same exact predicate). V2's three, V3's five and V6's Hermiticity of a Θ_k declared general are, by construction, the same conditions on the same declared numbers as the corresponding conjuncts of D(ii), D(iii) and D(vi): both Definition 1.5 and Definition 6.1 state them through Definition 1.3, exactly, with no window on either side. So V2/D(ii), the five-predicate half of V3/D(iii) and the Hermiticity half of V6/D(vi) are equivalences in both directions, a literal restatement. Condition (S) pairs the same way: it is a clause of D(ii) and of D(iii) (Definition 1.5) and a clause of V2 and V3, stated once in Definition 1.3 and read by both.

Step 3 (structural conjuncts: there is nothing left to pair). D(iii)'s last two conjuncts, $\Pi_s F_v \Pi_{s'} = 0$ and $\Pi_s \hat{O}_k \Pi_{s'} = 0$, hold by the emitted type (Theorem 3.7 (ii)) and have no counterpart in V3. They are therefore vacuous in the soundness direction — V3 need not establish what cannot fail —

and vacuous in the completeness direction — a declaration satisfying them is not thereby subjected to any check. The pairing is not broken by their absence; it is one conjunct shorter on each side. One direction of (ii) does fail for a reason that is not numerical, and it is a *typing* failure: an operator presented outside the block type, as a full $d_Q \times d_Q$ array, is refused at V1 whatever its off-block entries are, exactly zero included, because it is not a declaration in the sense of Definition 1.1. That is why (ii) is stated for declarations rather than for operators.

Step 4 (V5 ⇔ D(v) and V8 ⇔ D(viii)). These two pairs are combinatorial but need an argument.

V5 ⇔ D(v). (⇐) If ς exists as in D(v) and $a(x,v) = a(x',v') = \kappa$, then $r(x,v) = \perp$ and $r(x',v') = \perp$ are both equivalent to $\varsigma(\kappa) = \text{fail}$, so V5 holds. (⇒) Assume V5 and define $\varsigma(\kappa) \coloneqq \text{fail}$ if there exists $(x,v) \in a^{-1}(\kappa)$ with $r(x,v) = \perp$, and $\varsigma(\kappa) \coloneqq \text{pass}$ otherwise (in particular on $\kappa \notin \operatorname{ran} a$, where the choice is immaterial because the condition of D(v) quantifies over pairs (x,v)). This ς is total. For (x,v) with $\kappa = a(x,v)$: if $r(x,v) = \perp$ then $\varsigma(\kappa) = \text{fail}$ by construction; if $\varsigma(\kappa) = \text{fail}$ then some $(x',v') \in a^{-1}(\kappa)$ has $r(x',v') = \perp$, and V5 gives $r(x,v) = \perp$. Hence D(v).

V8 ⇔ D(viii). (⇐) If (1.10) holds with a total r_{h}, then equal holder readouts and equal announcements give equal key values, which is V8; the announcement clause of D(viii) is the first clause of V8. (⇒) Assume V8 and define $r_{\mathsf{h}}(o,\kappa) \coloneqq r(x,v)$ for any (x,v) with $\pi_{\mathsf{h}}(x,v) = o$ and $a(x,v) = \kappa$ if such a pair exists — well defined precisely by V8's implication — and $r_{\mathsf{h}}(o,\kappa) \coloneqq \perp$ on pairs (o,κ) that are not realized. This r_{h} is total on $\mathcal{O}_{\mathsf{h}} \times \mathcal{K}$ and satisfies (1.10) for every (x,v), since every such pair realizes $(\pi_{\mathsf{h}}(x,v), a(x,v))$. Hence D(viii). The value chosen on unrealized pairs is immaterial: (1.10) quantifies over (x,v) only.

Steps 1 to 4 account for (ii) in full, and for (i) except at V7.

Step 5 (V7 ⇒ D(vii), which completes (i); and (iii)). Assume V1–V8 all pass. By the equivalences of Steps 1 to 3, D(i)–D(iii) hold. If V7 passed by branch (a), $\mathfrak{L} = \varnothing$ and Theorem 5.5 (i) applies — its hypotheses are exactly D(i)–D(iii) — and exhibits $\omega^\star \in \mathcal{W}$. If V7 passed by branch (b) or (c), the candidate it verified satisfies every condition of Definition 2.1, decided exactly (Theorem 5.5 (ii), whose further hypothesis D(vi) holds because V6 passed, by Steps 1 and 2), so that candidate lies in $\mathcal{W}$. Either way D(vii) holds. Statement (iii) is proved by the counterexample given with it, which is checked there against Definition 1.5 condition by condition.

Step 6 (decidability, (iv)). V1 enumerates the declared tables and compares finitely many declared integers and labels, the sector-map comparison included: it is $|\mathbb{S}|$ identifier comparisons, $|\mathbb{S}|$ dimension comparisons, one block-kind comparison and one length comparison, per supplied family, plus the comparison of idx's sector order with $\mathfrak{s}$'s. V2, V3 and V6 take the decisions (6.1) counts, on matrices of the declared finite dimensions: in trusted

mode 3 for V2 — $\tau_A = \tau_A^\dagger$, $\tau_A \succeq 0$, $\operatorname{tr}\tau_A = 1$, the first serving both the Hermiticity condition of D(ii) and the Hermiticity half of its positivity condition; in untrusted mode none, because V2 takes its none branch; $2|\mathcal{X}|+1$ for V3's Alice half, one Hermiticity and one semidefiniteness per x and one completeness identity on A; $|\mathbb{S}|(2|\mathcal{V}|+1)$ for its Bob half, the same per (v,s) and one completeness identity per s; $m\,|\mathbb{S}|$ Hermiticity decisions for its constraint half; and m_{gen} for V6. Each is finite: an identity is a comparison of entries, and a semidefiniteness on a Hermitian matrix of size n is decided by exact symmetric elimination with the zero-pivot row test, $O(n^3)$ exact rational operations. The summation decisions of (S) are one pass over the additions of each applicable declared-order sum; the marginal trace sum exists only in trusted mode. Early termination can omit later decisions within a check, which is why this complete inventory need not equal the length of a failed execution's transcript. V4, V5 and V8 enumerate $|\mathcal{X}||\mathcal{V}|$ pairs, respectively $(|\mathcal{X}||\mathcal{V}|)^2$ ordered pairs, and compare tabulated values. V7 is Theorem 5.5 (ii), which is likewise per block, and *on both of its branches the decision is taken in exact arithmetic*: on branch (b) the candidate is formed by the compiler and decided in exact rational arithmetic, and on branch (c) the supplied witness's membership is decided exactly on the dyadic numbers it declares — not because those numbers are exactly representable, which by Definition 1.3 warrants nothing about the additions and products a decision forms, but because every decision this module takes on a computed quantity is taken exactly (Requirement "Exact arithmetic on the island, unbudgeted rounding elsewhere" (foundations/req:exact-island)).

The general format. The three kinds of conjunct are the same three, and *no fourth kind arises*; what has to be checked is that on each side the quantifiers still run over one list. The steps are those above.

Step 1 in the general format (combinatorial conjuncts). V1's finiteness, totality, factor-list, dimension, index-order and map-comparison clauses; V4's; V6's relation-symbol, tag, type and *scope* clauses. Each compares labels, integers or tabulated values, so each is an equivalence in both directions once the two quantifiers are seen to run over one list, which needs saying in four places. *First, the factor list.* D(i) asks that the sector map be one factorised map — at most one factor tagged A and then first, at least one tagged Q, one block kind, the dimension identities — and V1 decides those same clauses on the same list (Definition 6.1, the general format), so they pair literally. *Second, the objects whose maps are compared.* In the general format the declared Q-operators are the per-factor families $\{F^f_{v_f}\}_{v_f}$, one per $f \in \mathbb{F}$, together with the m operators $\hat{O}_1, \ldots, \hat{O}_m$, each against the map of its scope (Definition 1.1 (DX1), (DX3), (DX6)); *and Alice's declared data are on the list beside them* — τ_A, every operator of her measurement slot, and every Θ_k of an entry declared general, each against the map of the factor tagged A, which is where Definition 1.3 states condition (T) non-

empty for them. D(i) asks that there be one factorised map and that *each* of those objects be a block list against the map of its own factor; V1 compares the four components of the map carried by each of *those same* objects and of nothing else. *So the Alice-side conjunct is on both sides and pairs like the others*, which is what makes (i) sound and (ii) complete for it: a candidate whose Alice block list carries a different block kind, a different identifier list or a different order is refused at V1, and a candidate meeting D(i) is not refused there, both sides comparing two finite lists of labels and integers. Where no factor is tagged A, Alice's data are single arrays carrying no map and both sides are empty of them, as in Step 1 above. *A product block* (1.5) *and the lifted operator of a scoped entry are on neither side*: neither is a declared datum, so D(i) does not quantify over either and V1 compares neither — which is what keeps (ii) true, a wider V1 refusing candidates D(i)–D(vi) and D(viii) admit. The descriptors that the chain's last links produce, one per factor, and a supplied witness stay outside both sides for the reasons of Step 1, now read per factor and over the joint sectors. *Third, the index order.* D(i) asks that the restriction of idx to each factor's sectors be the order that factor's map fixes, and records that the product order is *derived*; V1 decides exactly that and compares nothing about the derived orders, there being no second datum to compare them against. Both directions hold, so a candidate meeting D(i)–D(vi) and D(viii) cannot be refused there. *Fourth, the transcript.* D(viii) asks that the announcement slot be a transcript protocol with every table a_m total on $\mathcal{O}_{\pi_m} \times \mathcal{C}_1 \times \cdots \times \mathcal{C}_{m-1}$ — so that a message reads its sender's readout and the earlier messages *by its signature* — and V1's totality clause together with V8's announcement clause decide that same thing of those same L tables. A table whose stated domain names a later message's alphabet is not of the declared type and is refused at V1; so on a candidate the signature clause is not a numerical conjunct and needs no pairing beyond this one.

And V6 pairs with D(vi) on the same m entries, now with three forms and a scope. D(vi) asks of each entry that its relation symbol lie in the set *its form and its emitted mode* admit — the mode's set for an untagged entry, $\{\preceq, =, \succeq\}$ in either mode for an entry declared party-local, with the scalar symbols also admitted where every row of that entry is 1×1 — that its right-hand-side slot hold one of the three admitted data — whose type, together with the arriving tag field for the party-local form, *is* the tag (Definition 2.5) — and that its scope name a measured factor of the map; V6 decides those three things of those same m entries, keyed on whether the marginal slot is filled and on the declared tag, and its symbol clause is D(vi)'s symbol clause set for set, so that pair is an equivalence in both directions as before. The clause of V6 about an arriving tag field is again not a further conjunct on (1.1) for a homogeneous or general entry, whose tag is the type; *for a party-local entry it is the tag, because the type of the datum does not separate that form from the homogeneous one* (Definition 2.5), and

D(vi) in its general reading asks for that declared form, so here too the two sides ask the same thing of the same objects.

Step 2 in the general format (numerical conjuncts). V2's, V3's and V6's Hermiticity of a $\Theta_k^{(t)}$ declared general are, by construction, the same exact predicates on the same declared numbers as the corresponding conjuncts of D(ii), D(iii) and D(vi), both sides stating them through Definition 1.3 in its general-format reading, exactly and with no window: per measured factor and per factor block on the measured families, and per Alice sector on Alice's data. Condition (S) pairs the same way, per factor and per Alice sector. *Two pairs are new and both are equivalences.* The ensemble form: V3's identity $\sum_x N_x^{(t)} = \tau_A^{(t)}$ with its summation condition, and V2's $\tau_A^{(t)} \neq 0$, are exactly the conditions Definition 1.3 states for that form and D(ii), D(iii) ask; and V3's refusal of an ensemble-form slot beside a marginal slot holding none pairs with D(iii)'s clause that ensemble form is refused in untrusted-source mode — a decision on *which of two slots is filled*, hence combinatorial, hence an equivalence in both directions.

Step 3 in the general format (structural conjuncts: again nothing is left to pair, and the list of them is longer). D(iii)'s typing conjuncts hold by the emitted type along $\mathbb{T}$ as well as along $\mathbb{S}$ and have no counterpart in V3; and *three further statements that a naive reading would put on the data are theorems and are conjuncts of neither side*: that the product family $\{F_v\}_{v\in\mathcal{V}}$ is a POVM on Q, block diagonal for the product map (Definition 1.1 (DX3)); that the family (1.7) an ensemble form denotes is a POVM on A — proved at Theorem 5.1 (v), *named here as a pointer and used by no step below*, what this step needs being only that neither D(iii) nor V3 states it; and that a lifted operator is Hermitian and block diagonal for the product map, which follows from the Hermiticity of $\hat{O}_k^{(s_{f_k})}$ that V3 does decide. So each is vacuous in the soundness direction — V3 need not establish what follows — and vacuous in the completeness direction, a declaration satisfying it being subjected to no check for it. The typing failure that makes (ii) a statement about declarations rather than operators is unchanged, and has two more instances: a family presented as one array over the product alphabet, and an Alice operator presented as one array on A where an A factor is declared, are refused at V1 whatever their entries are, because neither is a declaration in the sense of Definition 1.1.

Steps 4 and 5 in the general format (V5, V8 and V7; and (iii)). The proofs of V5 $\Leftrightarrow$ D(v) and V8 $\Leftrightarrow$ D(viii) in Step 4 use of a only that it is a total map into $\mathcal{K}$, and construct ς and r_{h} on that alphabet; both hold verbatim with $\mathcal{K} = \prod_m \mathcal{C}_m$ and a the function (1.8) derives, *whose totality is derived here and not imported*: $\kappa_1 = a_1 \circ \pi_{\pi_1}$ is total because a_1 and π_{π_1} are, and if $\kappa_1, \ldots, \kappa_{m-1}$ are total on $\mathcal{X} \times \mathcal{V}$ then so is κ_m, a_m being tabulated on all of $\mathcal{O}_{\pi_m} \times \mathcal{C}_1 \times \cdots \times \mathcal{C}_{m-1}$ (V1) and π_{π_m} total; induction on m gives a total into $\mathcal{K}$, which is all the two proofs consume of it. For V7 $\Rightarrow$ D(vii): if V7 passed

by branch (a) then $\mathfrak{L} = \varnothing$ and Theorem 5.5 (i) in its general-format reading exhibits (5.3) in $\mathcal{W}$, its hypotheses being exactly D(i)–D(iii); if by branch (b) or (c), the candidate it verified satisfies every line of (2.3), decided exactly (Theorem 5.5 (ii), general format, D(vi) holding because V6 passed), so it lies in $\mathcal{W}$. Either way D(vii) holds, and (i) is complete. *And (iii) holds in the general format with the witness already given*: that declaration has one Alice sector, one measured factor, POVM form, two messages and no party-local entry, at which values D(i)–D(viii) in their general reading are the eight conditions read of it in (iii) (Definition 1.5, the general format) and V7's three branches are the three branches read of it there, so it is a general-format declaration satisfying D(i)–D(viii) that V7 refuses. The incompleteness is therefore at V7 and, by the equivalences just listed, only there.

Step 6 in the general format ((iv)). V1 again enumerates declared tables and compares declared integers and labels: per factor, $|\mathbb{S}^f|$ identifier and $|\mathbb{S}^f|$ dimension comparisons, one block-kind and one length comparison per supplied family against that factor's map — *Alice's τ_A, her measurement operators and each Θ_k of an entry declared general against the A factor's map among them*, the same four comparisons each — plus the comparisons of idx's components with the factors' orders and with $\mathbb{T}$'s. V2, V3 and V6 take the decisions (6.2) counts: 2 per Alice sector for V2 — Hermiticity, which serves both D(ii)'s named condition and the Hermiticity half of its positivity condition, and semidefiniteness — and one trace decision, with one non-zero comparison per sector in ensemble form, all of these V2 decisions and the marginal trace-summation decision being present only in trusted mode; when the slot holds none, V2 returns without taking any of them. V3 has $2|\mathcal{X}| + 1$ per Alice sector for its Alice half, the $+1$ being the completeness identity in POVM form and the marginal identity in ensemble form; $|\mathbb{S}^f|(2|\mathcal{V}^f| + 1)$ per measured factor; $|\mathbb{S}^{f_k}|$ Hermiticity decisions per entry; and $m_{\text{gen}}|\mathbb{T}|$ for V6. Each is finite and each is taken on a matrix of one of the four sizes named in the statement, a semidefiniteness on size n costing $O(n^3)$ exact rational operations as before. V4, V5 and V8 enumerate $|\mathcal{X}||\mathcal{V}|$ pairs, respectively $(|\mathcal{X}||\mathcal{V}|)^2$ ordered pairs, of *tabulated* values, and a's table is finite because each a_m's is. V7 is Theorem 5.5 (ii) in its general-format reading, which is per joint block and in exact arithmetic on both of its branches, for the reasons given there. □

Soundness keeps an ill-formed declaration from being emitted; completeness, which holds for every target condition but feasibility, makes a refusal name a real defect. A refusal at V7 means only that no verified feasible state was shown.

Remark 6.3 (The two directions of the validator). The two directions do different jobs and it matters which one is available where. *Soundness* — Theorem 6.2 (i), and it holds for all eight conditions — is the safety direction: without it the compiler could emit a declaration that passes every check and is nonetheless ill-formed, which is the failure this module exists to prevent, since a downstream engine consumes

D(i)–D(viii) as hypotheses without re-deriving them. *Completeness* is what makes a *refusal* informative: where it holds, a well-formed declaration is never rejected, so a refusal names a real defect and not a limitation of the checker. It holds for D(i)–D(vi) and D(viii) (Theorem 6.2 (ii)); both directions of each of those pairs are proved, not just the one that keeps bad declarations out. Both directions are read on the declared numbers: a pass buys D(i)–D(viii) of the declared numbers, which is what the consuming engine requires of them, and a declaration well-formed of those numbers passes every check but V7.

Completeness does *not* hold for D(vii), and that is a limitation of this checker (Theorem 6.2 (iii)). *Its scope is the third branch of V7 and no wider*: a declaration with an empty constraint list passes on branch (a), and one whose feasible set contains the default candidate (5.2) passes on branch (b), with nothing supplied in either case. What is refused is a well-formed declaration with a non-empty constraint list whose feasible set does not contain the default candidate and for which no witness was supplied. Three consequences follow. A refusal at V7 means "I was not shown a feasible state, and the one I tried is not one", never "there is none" (Requirement 13.4). Whoever presents a declaration refused for that reason has a remedy inside this interface — supply $\omega^\star$ (Requirement 13.3) — and no other check has that character. And the asymmetry is the deliberate one: the direction that is missing costs a rejected good declaration, while the direction that is present is the one whose absence would cost a consumed bad one.

Uses Thm. 5.5, Thm. 6.2 · *Used by* Req. 13.4 · newly derived · accepted

7 Consuming the reduction chain

The chain arrives as a record of two segments, source links and receiver links, and the compiler decides what the record makes decidable: descriptor agreement, the outcome correspondence, the exact admissibility of the target measurement, and the pullback and source identities where channels are supplied as data. A non-empty source segment adds registry checks and two admission rules. Theorem 7.5 states what a verified identity buys and names what is never discharged here: that each link is a sound reduction and that any restricted attack set is licensed. Where the reductions module discharges them is recorded, and no claim here relies on that discharge (Remark 7.7).

Imports transmitter/def:source-segment

A presented chain must match descriptors, map physical outcomes onto emitted ones, deliver an exactly admissible target measurement and, where channels are data, satisfy the pullback and source identities exactly. A non-empty source segment adds registry checks and two admission rules.

The chain is the compiler's third input. A *squashing* link replaces Bob's measurement and therefore acts on the receiver space; a source link replaces Alice's signal ensemble and therefore acts on the transmitted system A', not on the receiver space at all. Every link carries two descriptors, of what it starts from and of what it produces: for a source link these are the transmitter's source descriptor and target descriptor; for a receiver link they are

receiver-side, and the last receiver link produces the emitted system with its sector map. One list of links cannot carry both kinds, and a single descriptor rule cannot type both, so the record is a pair of segments with a condition set each, and the composition-order rule is a property of the pair rather than a tag. The warrant for that order is the published one, [1, Rem. 9.2], and on the module side Remark "Recorded stage order, immaterial to soundness for licensed lists" (reductions/rem:ordering), which states why a source link after a receiver link is not free; this module types the record and states neither.

That the identity decided at (C4) and (S3) is the identity written is Requirement "Exact arithmetic on the island, unbudgeted rounding elsewhere" (foundations/req:exact-island) at work, and a producer repairs its own emission under Requirement "Snapping by the declarer only" (foundations/req:who-snaps). (C1)–(C5), (S1)–(S4), the licensed-only rule and the source-mode rule are decided where the chain is consumed, before a declaration exists, so a failure produces a refusal and no declaration (Requirement 13.6), and the gate outcome table of Definition 9.3 needs no row for them. The grounds of the licensed-only rule and of the source-mode rule are Remark 7.3.

Definition 7.1 (Interface conditions on a chain)**.** A reduction chain $\mathfrak{R} = (\mathfrak{R}_{\mathrm{src}}, \mathfrak{R}_{\mathrm{rec}})$ presented to the compiler, with $\mathfrak{R}_{\mathrm{src}} = (\Xi_1, \dots, \Xi_{L_{\mathrm{s}}})$ and $\mathfrak{R}_{\mathrm{rec}} = (\Lambda_1, \dots, \Lambda_L)$, satisfies the *interface conditions on a chain* when the following hold.

Receiver segment.

(C1) *(descriptor match)* the target descriptor of Λ_i equals the source descriptor of Λ_{i+1} for $1 \le i < L$; the source descriptor of Λ_1 is the physical receiver space of $\mathfrak{S}_{\mathrm{rec}}$; and the target descriptor of Λ_L is $(Q, \mathfrak{s})$ — that is, the last link's target descriptor *is* the sector map. That equality is decided by the same four-component comparison V1 makes on the map of a declared Q-operator family (Definition 6.1);

(C2) *(outcome correspondence)* the declared outcome correspondence is a tabulated total surjection $\gamma : \mathcal{V}^{\mathrm{ph}} \to \mathcal{V}$ from the *physical* alphabet onto the *emitted* one — the coarse-graining direction, of which a bijection is the special case;

(C3) *(target POVM)* $\{F_v\}_{v \in \mathcal{V}}$ is a family of block lists against $\mathfrak{s}$ — *typing*, held by the emitted form and not checked — and for each s the block family $\{F_v^{(s)}\}_v$ is a POVM on Q_s, decided *exactly* on the emitted blocks themselves: $F_v^{(s)}$ Hermitian and positive semidefinite for each v, $\sum_v F_v^{(s)} = \mathbb{1}_{Q_s}$, and the declared-order sum forming it rounding nowhere (Definition 1.3). This condition is check V3 restricted to $\{F_v\}$ and states the same list of conjuncts (Theorem 7.5 (ii));

(C4) *(pullback, where checkable)* if every link of $\mathfrak{R}_{\mathrm{rec}}$ supplies its channel

as data then, with $\Lambda \coloneqq \Lambda_L \circ \cdots \circ \Lambda_1$,

$$\Lambda^{\dagger}(F_v) = \sum_{w \in \gamma^{-1}(v)} F_w^{\mathrm{ph}} \qquad \text{for every } v \in \mathcal{V}, \tag{7.1}$$

an *exact* matrix identity, decided entry by entry on the emitted matrices and on the supplied channel data, with the fibre sum accumulated in the declared order of $\mathcal{V}^{\mathrm{ph}}$. Where some link supplies no channel as data, only a theorem that constructs one, the identity is not decidable from the emitted data and is carried instead as an obligation of the chain's producer, Assumption 10.6;

(C5) *(slots)* each link carries its constraint entries and its licence slot, both possibly empty, and its status tag.

Source segment. The source segment satisfies the interface conditions (S1)–(S4) of Definition "Source segment and its interface conditions" (transmitter/def:source-segment) — descriptor match with the virtual transmitted system A'' as the source descriptor of Ξ_1, on which the certificate of Definition "Source-replacement certificate" (transmitter/def:srcert) is written, and every descriptor Alice-side; index preservation; the source identity $\Xi(\rho_x^{\mathrm{v}}) = \rho_x$ for every x, that definition's equation (S3), where every link supplies its channel as data; and the slots — *decided by this module on the presented data*, exactly as that definition states them. When $L_{\mathrm{s}} = 0$ the segment is empty, Ξ is the identity, $A'' = A'$ and $\rho_x^{\mathrm{v}} = \rho_x$, so (S3) holds by inspection.

Between the segments.

(X1) *(order)* the chain is composed source segment first and receiver segment second. This is not a tag and is not checked: it is the shape of the record. A source-map link has no slot in $\mathfrak{R}_{\mathrm{rec}}$ to be written into, and a link presented in the wrong segment fails that segment's descriptor rule — (C1) for a source map, since A' is not the physical receiver space, and (S1) for a squashing map, since Q is not an Alice-side descriptor.

At which reading (C4) and (S3) are decided. Both are *matrix equalities on declared data* — a composite channel supplied as binary64 data, applied to binary64 POVM elements on one side and to binary64 signal states on the other — and both are decided *exactly*, entry by entry, on the numbers the compiler holds. *The composite, the adjoint and the fibre sum are formed in exact arithmetic as well*, so the identity decided is the identity written; that the operands are binary64 is not what makes the decision exact, a product of two binary64 numbers need not be one. Exactness here is an obligation of the chain's producer (Assumption 10.6): a producer whose composite lands off the identity by a rounding has not delivered the reduction the interface asks for, and repairs its own emission; no window is opened here. Everything

else in this definition — descriptors, alphabets, fibres, slots, segments — is combinatorial and is read exactly, as D(vi)'s relation symbols are.

The general format: one receiver segment per measured factor. For a chain presented as Definition 3.1 (c′), with $\mathfrak{R}_{\mathrm{rec}} = (\mathfrak{R}^f_{\mathrm{rec}})_{f\in\mathbb{F}}$, the conditions (C1)–(C5) are decided *of each segment separately, and of nothing across two segments.* (C1): within segment f the target descriptor of a link is the source descriptor of the next, the source descriptor of Λ^f_1 is factor f's physical receiver system, and the target descriptor of $\Lambda^f_{L_f}$ is $(Q_f, \mathbb{S}^f, (d_{f,s_f})_{s_f})$ — *that factor's own sector map, and not the product map*, which is derived by Definition 1.1 (DX1) and is the target descriptor of nothing; the same four-component comparison decides it. (C2): one tabulated total surjection γ_f from factor f's physical alphabet onto the alphabet factor f's last link delivers. (C3): the target family $\{F^f_{v_f}\}$ is a family of block lists against factor f's map and, for each s_f, a POVM on Q_{f,s_f} decided exactly on that factor's blocks — which is check V3 restricted to that factor, conjunct for conjunct, in its general-format reading. (C4): where every link of segment f supplies its channel as data, with $\Lambda_f \coloneqq \Lambda^f_{L_f} \circ \cdots \circ \Lambda^f_1$,

$$\Lambda_f^{\dagger}\big(F^f_{v_f}\big) = \sum_{w_f \in \gamma_f^{-1}(v_f)} F^{\mathrm{ph},f}_{w_f} \qquad \text{for every } v_f, \tag{7.2}$$

an exact matrix identity on that factor's matrices, the fibre sum accumulated in the declared order of that factor's physical alphabet. *The product composite $\bigotimes_f \Lambda_f$ and the product identity on $\{F_v\}$ are formed and decided by no one*: the identity $\big(\bigotimes_f \Lambda_f\big)^{\dagger}(F_v) = \bigotimes_f \Lambda_f^{\dagger}(F^f_{v_f})$ is a consequence of (7.2) and of the product rule (1.5) — it is Theorem "The emission of several measuring parties" (reductions/thm:emitted-product) (ii), named here and decided here by nothing — and (C4) is the per-factor identity and nothing wider. (C5) is per link, as above. Where some link of segment f supplies no channel as data, only a theorem that constructs one, (7.2) for *that* segment is not decidable from the emitted data and is carried as Assumption 10.6, exactly as above and per factor.

A non-empty source segment, and the conditions it brings. A presented chain may carry one. The rule by which a non-empty source segment composes with a receiver segment is a theorem of the reductions module (Lemma "Licensed admissibility survives precomposition with a source channel" (reductions/lem:precomposition), Theorem "The two-sided reduction in trusted-source mode" (reductions/thm:two-sided), with Definition "Reduction chain" (reductions/def:chain)), in trusted-source mode; and the module that admits source-link kinds may emit an artefact carrying a link of an admitted kind. What follows is what this module decides of such a segment, on the presented data and exactly; whether a presented chain carries one is data of the record and is asserted here of no artefact.

- *(S1)–(S4) as Definition "Source segment and its interface conditions" (transmitter/def:source-segment) states them*, which is what the source-segment paragraph above already says; and within (S3), *the split by kind of witness*: for a link supplying its channel as data the per-link identity is decided here, entry by entry, with the composite formed in exact arithmetic; for a link supplying a theorem constructor it is *not decided here at all* but is the conclusion of the claim that link's obligation record names, and the composite identity is the composition of the per-link ones. So what this module decides of (S3) is the data links' share of it, and the constructor links' share travels as the obligation Assumption 10.6 names — Theorem 7.5 (iv), (v) — and is discharged nowhere in this contract.

- *(A1)–(A3) of that definition, which are decided on data and are not part of (S1)–(S4)*: that the triple a link's obligation record carries — its kind, the claim id it names, and the assumption short ids it names — appears *entry by entry* in the soundness registry the artefact carrying the segment emits, both components of that registry entry's assumption list compared, so that a record naming a claim's closure and omitting the source model's attestation fails as one naming neither does; that every assumption of that entry is in the artefact's assumption ledger; and that the segment is presented inside an artefact of the module that admits its links. These are comparisons of finitely many declared strings and of a finite list against a finite list, and *they are decided here because they are decidable on the presented record*, which is why they are named beside (S1)–(S4) rather than left to Assumption 10.6. *What they do not decide* is that the named claim's conclusion applies to these descriptors — family membership is the emitting module's assertion and no consumer re-proves a theorem — and a link supplying its channel as data carries no obligation record and is decided by (S3) instead.

- *the licensed-only rule.* Beside a non-empty source segment *every* entry of the emitted $\mathfrak{L}$ is required to carry a real scalar as its right-hand side — homogeneous, party-local, or an untrusted-mode scalar (Definition 2.5).

- *the declared source mode. A declaration carrying a non-empty source segment is in trusted-source mode*: the rule fails where the marginal slot of (1.1) holds none (Definition 1.1), the untrusted-source mode, while $L_{\mathrm{s}} \geq 1$.

(X1) in the general format. The order rule is unchanged and is still the type of the record: a source-map link has no slot in any receiver segment to be written into, and a link presented in the wrong list fails that list's

descriptor rule — (C1) of the factor whose segment it was written into, or (S1). In the reductions module the composite of a chain is a pair, and the reason a source link may not follow a receiver link is a theorem of that module (Lemma "Licensed admissibility survives precomposition with a source channel" (reductions/lem:precomposition), Theorem "The two-sided reduction in trusted-source mode" (reductions/thm:two-sided)) beside the published warrant [1, Rem. 9.2]; this module types the record, decides the descriptors, and states neither.

Uses Def. 1.1, Def. 1.3, Def. 6.1, transmitter/def:source-segment · *Used by* Thm. 4.4, Rem. 4.8, Rem. 4.9, Rem. 7.2, Rem. 7.3, Rem. 7.4, Thm. 7.5, Rem. 7.6, Thm. 8.2, Ass. 10.6, Req. 11.2, Req. 13.6, Req. 13.8 · newly derived · implemented (effective: accepted) · *Code* certiqs_key.protocol_compiler.chain: check_chain · *Oracles* registry_protocol_compiler.py

The compiler's (C1)–(C3) are conditions on the record's fields and differ from the reductions module's composability, tuple chaining and stage order, whose numbers they share. That module proves the chain composes; this one decides on the record what can be decided there and names the rest.

Remark 7.2 (Chain conditions here and in the reductions module). The two modules that meet at the chain record describe it from two sides: the reductions module *defines* the chain and *proves* that it composes (Definition "Reduction chain" (reductions/def:chain), Definition "Typed link" (reductions/def:link), Theorem "Composition of a chain" (reductions/thm:chain)); this module *decides* what it can decide on the presented record and *names* what it cannot. Nothing is asserted twice: the numbered conditions of Definition 7.1 are this module's, are stated on the record's fields alone, and are not the composability (C1), tuple chaining (C2) and stage order (C3) of that module's definition, whose numbering they happen to share — "(C1) of Definition 7.1" and "reductions' composability (C1)" are different conditions on the same object.

Uses Def. 7.1 · newly derived · accepted

The licensed-only rule and the source-mode rule are the two halves of the reductions module's admission rule for a non-empty source segment. Both refuse where the chain is consumed; only the first refuses a case an artefact of this repository can present.

Definition 7.1 states two rules for a non-empty source segment, the licensed-only rule and the rule on the declared source mode, and Requirement 13.6 names their refusals. This remark gives their grounds and compares what each of them refuses.

Remark 7.3 (Grounds of the source-segment rules). *Where the two rules come from.* The licensed-only rule is the consuming side of the admission rule of Definition "Reduction chain" (reductions/def:chain), whose reason is Lemma "Licensed admissibility survives precomposition with a source channel" (reductions/lem:precomposition) and its counterexample (Remark "Recorded stage order, immaterial to soundness for licensed lists" (reductions/rem:ordering)); this module states neither and decides the type of the entry. The rule on the declared source mode is the other half of

that admission rule, which admits a non-empty source segment beside a constraint list only where every entry is licensed *and only in trusted-source mode*: the entry half is the licensed-only rule and the mode half is this one, and both are decided in Definition 7.1 on the presented data, the entry half by reading the declared tag of each entry and the mode half by reading one declared slot.

The licensed-only rule refuses a case the basic format admits. A non-empty source segment is declarable there — the source-segment paragraph of Definition 7.1 admits $L_s \geq 1$ — and, *were this rule absent*, an entry declared general beside one would, where no other condition or gate refused it, reach the gates and be emitted by row 5 of the gate outcome table of Definition 9.3 as a declaration that is not key-length-eligible (Requirement 14.2). Under the rule the case is refused where the chain is consumed, before any declaration exists, so row 5 is never reached for it. The two outcomes are not the same, and the second is strictly the more conservative. *The case is reachable*, and the rule is then simply a refusal, which is the direction a consumer may always take.

Why the source-mode rule refuses rather than weakening the outcome: nothing licenses a source map in untrusted-source mode. The statements that make a non-empty segment sound — Theorem "The two-sided reduction in trusted-source mode" (reductions/thm:two-sided) and Lemma "Exactness of the source map in trusted-source mode" (reductions/lem:source-transfer) — are stated in trusted-source mode alone, because a source map replaces *Alice's preparation* and only a trusted specification has one: an untrusted specification carries no certificate and no marginal, its Alice-side system is an output of the attack, and A-SRC is asserted in trusted-source mode alone (Assumption "Retention of the system A in trusted-source mode" (transmitter/ass:src)). So a declaration of that shape would carry a replaced ensemble with no statement transferring anything to it, which is the one thing a consumer may not emit. *No artefact of this repository can present the pairing, and the rule is stated so that it is decided rather than left to provenance*: a non-empty segment is accepted only inside an artefact of the module that admits its links, by (A3) of Definition "Source segment and its interface conditions" (transmitter/def:source-segment) as Definition 7.1 decides it, and the one submodule that admits a kind declares its data in trusted-source mode (Corollary "The emitted decoy declaration" (transmitter/decoy/cor:emitted-decoy) (i)). Unlike the licensed-only rule, which refuses a case the basic format declares and which, were the licensed-only rule absent and nothing else refused it, row 5 would emit as not key-length-eligible, the source-mode rule refuses nothing that an artefact of this repository can present; what it removes is the possibility that a later kind emitted in another mode would pass every condition stated in Definition 7.1.

Uses Def. 7.1 · newly derived · accepted

The last clause of (C1), (C4) and the source identity are decided on the chain alone; V3 re-runs (C3) on the emitted family, weighted where a weighting follows the chain. The chain's last target descriptor, a supplied witness and the physical family stay outside V1's list, keeping V1 complete.

Some conditions on what the compiler emits are decided where the chain is consumed and some by the validator. This remark says which, and which

objects each of them compares.

Remark 7.4 (Chain conditions and validator checks). *What is decided on the chain.* The last clause of Definition 7.1 (C1), that the target descriptor of Λ_L is $(Q, \mathfrak{s})$, is decided at (C1) and nowhere else: it is a condition on the chain record and not on a slot of (1.1), so no target condition quantifies over it and check V1 does not read it (Definition 6.1). Nothing is lost by keeping it there alone: the emitted $\mathfrak{s}$ *is* that descriptor by construction (Definition 3.3), and what V1 decides on the emitted data is that the map carried by this same link's target POVM $\{F_v\}$ is $\mathfrak{s}$. (C4) and (S3) are conditions on the chain and not checks of the validator either: they are decided where the chain is consumed, and they are *not* among the decisions counted by (6.1) of Theorem 6.2, a count of V2, V3 and V6 alone; V7's decisions are counted in the prose that follows that equation and are not terms of it. What V3 re-runs on the emitted data is (C3), or, where the Bob-side weighting of Definition 4.1 follows the chain, (C3)'s predicates on the weighted family the chain's family yields (Theorem 7.5 (ii)); (C1)'s last clause is re-run by no check, V1 reading the maps of the declared Q-operator families and no others; and nothing re-runs (C4) or (S3).

Which objects V1 does not compare. At a non-trivial chain two families answer to the receiver's measurement, the emitted family $\{F_v\}$ and the physical family $\{F_w^{\mathrm{ph}}\}$, and V1 compares the first only. *Two objects are outside V1's list, and each is compared where it belongs*, because carrying either in V1 would make V1 refuse a candidate that D(i)–D(viii) admit, and so falsify Theorem 6.2 (ii). *The chain's last target descriptor* is required to be $(Q, \mathfrak{s})$ by (C1), which decides that equality by the same four-component comparison and is where a chain is refused for it, as above; what V1 decides instead, that the map carried by that link's target POVM $\{F_v\}$ is $\mathfrak{s}$, is a condition on a declared Q-operator and is D(i)'s. *A supplied feasibility witness* is not a Q-operator at all: $\omega^\star$ is a block list on AQ, an input consumed and never emitted (Theorem 5.5 (ii)), and branch (c) of V7 already requires it to be a block list against $\mathfrak{s}$, which is the one place its map is compared. *The physical family* $\{F_w^{\mathrm{ph}}\}$ *is not in the list either*: at a non-trivial receiver segment it lives on the physical receiver space, which is not Q, so it is not a Q-operator family and has no map to compare against $\mathfrak{s}$; its descriptor is matched instead by (C1), against the source descriptor of Λ_1. The two coincide exactly when the receiver segment does not change the space — in Band A, where $\Lambda_1 = \mathrm{id}$ and $F_v = F_v^{\mathrm{ph}}$ — and there the comparison is the same one either way. Including the physical family at a non-trivial chain would not be a useful refusal: it would refuse every such chain, and V1 is claimed complete for D(i), which asks only that every declared Q-*operator* be a block list against $\mathfrak{s}$.

Uses Def. 3.3, Thm. 5.5, Def. 6.1, Def. 7.1 · newly derived · accepted

Chain conditions are decided by finite computation; a verified pullback identity reproduces the coarse-grained physical statistics and a verified source identity maps the virtual source onto the specified one. Link soundness and attack-set licences stay obligations this theorem does not discharge.

Consuming a chain buys only what its record makes decidable. The theorem states what each decided identity implies and lists what stays an obligation of the chain's producer; where those obligations are discharged for the chains the reductions module emits is Remark 7.7. The attack-set licence that (v) names is Assumption 10.7. It occurs in no hypothesis of this theorem: it is

spent at the gate that tests for its record, Theorem 9.4 (iii), and is named here and not declared.

How the compiler applies the Bob-side weighting of (ii) after the chain is Remark 4.8. Remark 7.6 reads the clause of (iii) on a coarse-graining in the opposite direction as the reason (C2) runs from the physical alphabet onto the emitted one. That (A1)–(A3) do not decide whether the claim an obligation record names applies to the descriptors of the link carrying that record, as (v′) says, is Remark "Scope of the source-segment conditions" (transmitter/rem:segment-scope).

Theorem 7.5 (Decided identities and named obligations of a consumed chain)**.** Let a chain be presented to the compiler, with the interface conditions of Definition 7.1. Then:

(i) (C1)–(C5) and (S1)–(S4) are decided by finite enumeration and finitely many arithmetic operations on the emitted data, and a chain failing any of them is refused. (X1) is not decided, because it is not a condition on data: it is the type of the record;

(ii) (C3) is exactly the part of D(iii) concerning $\{F_v\}$, in both halves — typing against $\mathfrak{s}$, and the per-block predicates and summation condition of Definition 1.3 — hence a chain passing the interface conditions yields an emitted $\{F_v\}$ for which Theorem 3.7 applies. Where the Bob-side weighting of Definition 4.1 is applied after the chain (Definition 3.3), (C3) is decided on the family the chain delivered, the emitted family is its weighting $N_{v,f} = P(f \,|\, t(v))F_v$, check V3 decides the emitted family itself (Theorem 3.7 (i)), and Lemma 4.2 (i), (ii) is what says the weighting cannot break what (C3) established; the identity of conjunct lists asserted here is between (C3) and D(iii) on the chain's family;

(iii) *(what a verified (C4) buys)* when the composite receiver channel Λ is given as data, (C4) is *equivalent* to the statement that measuring the emitted family $\{F_v\}$ after Λ reproduces the γ-*coarse-grained* physical measurement statistics:

$$\operatorname{tr}[F_v\,\Lambda(\rho)] = \sum_{w\in\gamma^{-1}(v)} \operatorname{tr}\Big[F^{\mathrm{ph}}_w\,\rho\Big] \tag{7.3}$$

for every $v \in \mathcal{V}$ and every operator ρ on the physical receiver space. Consistency with completeness holds *under one hypothesis this contract names and does not verify*: if the supplied channel data are trace-preserving, so that $\Lambda^\dagger$ is unital, then summing (7.1) over v returns completeness of the physical family and nothing more, whereas the same identity read with a coarse-graining in the opposite direction — a total surjection $\gamma' : \mathcal{V} \to \mathcal{V}^{\mathrm{ph}}$, with $\Lambda^\dagger(F_v) = F^{\mathrm{ph}}_{\gamma'(v)}$ — does not in general, forcing every physical element covered more than once by γ'

to vanish. That trace preservation is Assumption 10.6, not a check: nothing here evaluates it;

(iv) *(what a verified (S3) buys)* when the composite source channel Ξ is given as data, (S3) of Definition "Source segment and its interface conditions" (transmitter/def:source-segment) — the source identity, that definition's equation, decided by this module on the presented data (Definition 7.1) — says that the virtual source, pushed through Ξ, is the specified source: $\Xi(\rho_x^{\mathrm{v}}) = \rho_x$ for every x, hence

$$(\mathrm{id}_X \otimes \Xi)\Big(\sum_{x\in\mathcal{X}} p_x\,|x\rangle\langle x|_X \otimes \rho_x^{\mathrm{v}}\Big) = \sum_{x\in\mathcal{X}} p_x\,|x\rangle\langle x|_X \otimes \rho_x, \tag{7.4}$$

and, averaging, $\sum_x p_x \Xi(\rho_x^{\mathrm{v}}) = \sum_x p_x \rho_x$: the prepare-and-measure states of the emitted virtual description map onto the specified ones, index by index, with the weights unchanged by (S2). The compiler decides the source identity (S3) and nothing beyond it; and

(v) the following are *named and never discharged here*: that each link of either segment is a sound reduction — that its channel exists as claimed for a link supplying only a theorem constructor, that a link supplying its channel as data supplies a completely positive and trace-preserving map, and that every reachable state after the reduction satisfies every transported constraint entry — and that any restricted attack set claimed by a link is licensed. These are obligations of the chain's producer, the soundness of each link being Assumption 10.6. No claim of this contract discharges them for a presented chain in general — for the Band A identity link, which Definition 8.1 (b) fixes with its channel supplied as data, its outcome correspondence and its target POVM, Theorem 8.2 (v) discharges every clause of Assumption 10.6 by exhibition — and no proof below invokes them.

The general format. For a chain presented as Definition 3.1 (c′), with the interface conditions read as in the general-format paragraph of Definition 7.1, (i)–(v) hold with the following readings, of which the five above are the case of one measured factor and an empty source segment.

(i′) (C1)–(C5) are decided per measured factor and (S1)–(S4) as before, all by finite enumeration and finitely many arithmetic operations, and a chain failing any of them is refused. *And (A1)–(A3), the licensed-only rule and the declared-source-mode rule are decided as well*: the first three by comparing finitely many declared strings and one finite list against another, the fourth by reading m declared tags, and the fifth by reading one declared slot. (X1) is again not decided, being the type of the record.

(ii′) (C3) is exactly the part of D(iii) concerning the declared measured families, *factor by factor*: for each f, (C3) read for f and D(iii)'s clauses on $\{F^f_{v_f}\}$ are the same list of conjuncts, so a chain passing the interface conditions yields per-factor families for which Theorem 3.7 (i′) applies. *That the product family is a POVM on Q is on neither side of that pairing*, being a consequence of the factors' conditions and no condition on data (Definition 1.1 (DX3), Theorem 6.2, general format). The Bob-side weighting sentence applies per factor, to that factor's delivered family.

(iii′) *(what a verified (C4) buys, per factor and jointly)* when every link of segment f supplies its channel as data, (7.2) is equivalent to $\mathrm{tr}\left[F^f_{v_f}\Lambda_f(\rho_f)\right] = \sum_{w_f\in\gamma_f^{-1}(v_f)}\mathrm{tr}\left[F^{\mathrm{ph},f}_{w_f}\rho_f\right]$ for every v_f and every operator ρ_f on factor f's physical receiver space. *And the joint statement follows for every operator, product or not*: if every segment supplies its channels as data then, with $\Lambda \coloneqq \bigotimes_f \Lambda_f$, $\gamma \coloneqq \prod_f \gamma_f$ and $F^{\mathrm{ph}}_w \coloneqq \bigotimes_f F^{\mathrm{ph},f}_{w_f}$,

$$\Lambda^\dagger(F_v) = \sum_{w\in\gamma^{-1}(v)} F^{\mathrm{ph}}_w, \qquad \text{hence} \qquad \mathrm{tr}[F_v\,\Lambda(\rho)] = \sum_{w\in\gamma^{-1}(v)} \mathrm{tr}\left[F^{\mathrm{ph}}_w\,\rho\right] \tag{7.5}$$

for every $v\in\mathcal{V}$ and every operator ρ on the product physical space. *Neither side of (7.5) is formed or decided by anything*: it is a consequence of the per-factor identities, and the objects in it — the product channel, the product physical family, the product target family — are data of no one. The consistency reading holds under the same hypothesis read per factor: if every factor's supplied channel data are trace preserving, so that each $\Lambda^\dagger_f$ is unital, then so is $\Lambda^\dagger$ and summing (7.5) over v returns completeness of the product physical family and nothing more. That trace preservation is Assumption 10.6, per factor, and is evaluated nowhere.

(iv′) (S3) is as in (iv), with the split by kind of witness: what a *decided* (S3) buys is (7.4) and its averaged form, for the data links' share of the composite; for a link supplying a theorem constructor this module decides nothing and what its share buys is what the claim its obligation record names concludes, which this contract neither states nor asserts.

(v′) the list of obligations named and never discharged here gains three items, each per the general format: that each link of *each* receiver segment is a sound reduction, factor by factor; that a non-empty source segment composes with the receiver segments as a reduction, which is Theorem "The two-sided reduction in trusted-source mode" (reductions/thm:two-sided), used by no step here; and that the claim an obligation record names applies to that link's descriptors, which

(A1)–(A3) do *not* decide. These are again the obligations of (v), Assumption 10.6 and the attack-set licence, and the emitting module's own assertion; no claim of this contract discharges them, save as (v) says of the Band A identity link, and no proof below invokes them.

Uses Def. 1.1, Def. 1.3, Def. 1.5, Def. 3.1, Def. 3.3, Thm. 3.7, Def. 6.1, Thm. 6.2, Def. 7.1, transmitter/def:source-segment · *Assumes* A-LINK · *Used by* Rem. 7.6, Req. 13.6 · newly derived · accepted · *Code* certiqs_key.protocol_compiler.chain: check_chain · *Oracles* registry_protocol_compiler.py

Proof. Idea. Items (i) and (i′) count what each condition decides. Items (ii) and (ii′) compare two lists of conjuncts. Items (iii) and (iii′) are the Hilbert–Schmidt duality between an adjoint identity and a statistics identity, both exact, with trace preservation of the supplied channels as the one hypothesis of the consistency reading. Items (iv) and (iv′) are the linearity of a channel, and (v) and (v′) are statements about what this contract asserts.

Step 1 ((i), decidability). (C1) and (S1) compare finitely many declared descriptors — for (C1)'s last clause, the four components of a sector map, the same combinatorial comparison V1 makes on the map of a declared Q-operator family and taken here on the descriptor, at (C1); (C2) enumerates a finite table and checks totality and surjectivity by counting preimages; (C3) is check V3 of Definition 6.1 restricted to $\{F_v\}$, namely $|\mathbb{S}|(2|\mathcal{V}|+1)$ exact decisions on blocks of size d_s together with $|\mathbb{S}|$ summation decisions, finite by Theorem 6.2 (iv); (C4) decides $|\mathcal{V}|$ matrix identities on matrices of the declared dimensions — the adjoint of a channel given as data (by Kraus operators or as a matrix acting on operators) is computed by finitely many arithmetic operations, and each right-hand side of (7.1) is a finite sum over the tabulated fibre $\gamma^{-1}(v)$; (S2) compares two finite index sets and $|\mathcal{X}|$ declared weights; (S3) decides $|\mathcal{X}|$ matrix identities on matrices of the declared dimensions; (C5) and (S4) check the presence of fields. (X1) is a statement about which of the two lists a link was written into, which the record answers by construction.

Step 2 ((ii), the conjunct lists). (ii) is the statement of (C3) against D(iii), conjunct by conjunct. D(iii) asks two things of the emitted $\{F_v\}$: that it be a family of block lists against $\mathfrak{s}$, and that each block family $\{F_v^{(s)}\}_v$ satisfy the predicates and the summation condition of Definition 1.3. (C3) states the first as the type of the object the last link delivers and the second as those same conditions, on the emitted blocks. The two lists of conjuncts are identical, so (C3) is D(iii) restricted to $\{F_v\}$ and Theorem 3.7 applies to what the chain delivered. Nothing in (ii) is inferred from the physical family, so a chain that merely *claims* a POVM property cannot pass one off as checked: the predicates are decided on the matrices the compiler emits. The block-diagonality half, by contrast, is a property the type carries and no check confirms. An entry that the chain's producer has *mis-assigned* to a block is a well-typed object no predicate can catch. V1 refuses a mis-

assignment that also disagrees about the sectors; one that does not is a defect of the chain's producer under Assumption 10.6, named there and not discharged here.

Step 3 ((iii), what a verified (C4) buys). The measurement channel of $\{F_v\}$ sends σ to $\sum_v \operatorname{tr}[F_v\sigma]\,|v\rangle\langle v|$; the γ-coarse-graining of the physical measurement channel sends ρ to $\sum_v \big(\sum_{w\in\gamma^{-1}(v)} \operatorname{tr}\big[F^{\mathrm{ph}}_w\rho\big]\big)\,|v\rangle\langle v|$, the fibres $\gamma^{-1}(v)$ partitioning $\mathcal{V}^{\mathrm{ph}}$ because γ is a total surjection (C2). Two maps into the same classical register agree on all inputs iff their diagonal entries agree, i.e. iff (7.3) holds; and by the definition of the adjoint with respect to the Hilbert–Schmidt inner product, $\operatorname{tr}[F_v\Lambda(\rho)] = \operatorname{tr}\big[\Lambda^\dagger(F_v)\rho\big]$, so (7.3) holds for all ρ iff (7.1) holds for all v — the second direction because an operator whose trace against every ρ vanishes is the zero operator, the trace pairing being non-degenerate. Both directions are available here because both sides are exact identities. For the consistency claim, summing (7.1) over v gives $\Lambda^\dagger\big(\sum_v F_v\big) = \sum_{w\in\mathcal{V}^{\mathrm{ph}}} F^{\mathrm{ph}}_w$; with $\sum_v F_v = \mathbb{1}_Q$ from (C3) and $\Lambda^\dagger$ unital — which is where trace preservation of the supplied data is used, and it is Assumption 10.6 and not a check — the left-hand side is $\mathbb{1}$, so what (C4) returns is $\sum_w F^{\mathrm{ph}}_w = \mathbb{1}$: completeness of the physical family, a condition that family already satisfies, so (C4) adds no restriction beyond the two completeness conditions. The opposite direction, under the same hypothesis, is the identity $\Lambda^\dagger(F_v) = F^{\mathrm{ph}}_{\gamma'(v)}$ for every $v \in \mathcal{V}$, with $\gamma' : \mathcal{V} \to \mathcal{V}^{\mathrm{ph}}$ a total surjection. Summing it over v, the left-hand side is again $\Lambda^\dagger\big(\sum_v F_v\big) = \mathbb{1}$, by linearity of $\Lambda^\dagger$, (C3) and unitality. On the right, the fibres $\gamma'^{-1}(w)$ partition $\mathcal{V}$ because γ' is total, so grouping the terms by the value of γ' gives $\sum_{w\in\mathcal{V}^{\mathrm{ph}}} |\gamma'^{-1}(w)|\,F^{\mathrm{ph}}_w = \mathbb{1}$. The physical family is a POVM (Definition 3.1 (b)): $\sum_w F^{\mathrm{ph}}_w = \mathbb{1}$ and every $F^{\mathrm{ph}}_w \succeq 0$. Subtracting, $\sum_{w\in\mathcal{V}^{\mathrm{ph}}} \big(|\gamma'^{-1}(w)| - 1\big)F^{\mathrm{ph}}_w = 0$, and every coefficient is a non-negative integer because γ' is surjective, so every term is positive semidefinite. A vanishing sum of positive semidefinite operators has every term zero: its trace is a sum of non-negative traces, so each term has trace zero, and a positive semidefinite operator of trace zero is zero. Hence $F^{\mathrm{ph}}_w = 0$ for every w with $|\gamma'^{-1}(w)| \geq 2$. That is a restriction the physical family need not satisfy, so this reading does not in general return completeness and nothing more. It does exactly when every physical element covered more than once already vanishes, in particular when γ' is a bijection: then every non-zero term of $\sum_w |\gamma'^{-1}(w)|\,F^{\mathrm{ph}}_w$ has coefficient one, and the summed identity is $\sum_w F^{\mathrm{ph}}_w = \mathbb{1}$. In the bijective case γ is a bijection, (7.1) reads $\Lambda^\dagger(F_v) = F^{\mathrm{ph}}_{\gamma^{-1}(v)}$, and that is the published form of the condition, [1, Lemma 9.3, Eq. (146)], read at the level of operators — there the two families are indexed by one alphabet.

Step 4 ((iv), what a verified (S3) buys). By linearity, $\mathrm{id}_X \otimes \Xi$ sends $\sum_x p_x\,|x\rangle\langle x|_X \otimes \rho^{\mathrm{v}}_x$ to $\sum_x p_x\,|x\rangle\langle x|_X \otimes \Xi(\rho^{\mathrm{v}}_x)$; substituting $\Xi(\rho^{\mathrm{v}}_x) = \rho_x$ from

the source identity (S3) gives (7.4), and tracing out X gives its averaged form. The step is linear in each ρ_x^{v} and uses no property of the declared weights. That the converse implication — security of the virtual protocol transfers to the real one — requires more than the source identity is exactly the content of Assumption 10.6: the published template is [1, Lemma 9.1, Eqs. (138)–(139)], whose hypothesis is the existence of such a Ξ and whose conclusion this module does not assert.

Step 5 ((v), what is asserted). (v) is a statement about what this contract asserts. Its content is that Assumption 10.6 and the attack-set licence are named as obligations of the chain's producer, and that no proof above invokes them: no step invokes Assumption 10.6 as an established obligation, its trace-preserving clause entering (iii) only as the stated hypothesis of the consistency reading; no step invokes the attack-set licence; and no step uses a theorem of the reductions module. The discharge (v) names is Theorem 8.2's, for the Band-A identity link alone, and no step above rests on it. The trace-preserving clause of Assumption 10.6 is the hypothesis under which (iii)'s consistency reading holds, so this theorem's statement is conditional on it and declares it. The order rule (X1) is not an exception — it is a *typing* property of the emitted record, and its warrant is the published one: [1, Rem. 9.2] resolves the tension between source maps, which are formulated for unrestricted attack sets, and squashing maps, which restrict them, by applying the source-map lemma first, so that restrictions arise only afterwards. The compiler enforces the order; it does not claim that either lemma applies to a given device.

The general format. The steps are those above, read per factor.

Step 1 in the general format ((i′)). Each clause of (i) is the clause of (i) applied to one segment, the data being per factor and finite; the refusal naming the factor is the record's own field. For (A1): the registry an artefact emits is a finite list of triples of declared strings, the obligation record a triple of declared strings, and the comparison is entry by entry, including the two components of the assumption list; the ledger membership test is a finite list against a finite list. For (A3): the provenance test reads which module emitted the artefact the segment is presented in, a declared field. The licensed-only rule reads m tags, terminating and exact. The declared-source-mode rule reads one declared slot, whether the marginal slot holds none, beside whether the source segment is empty, which the record answers by its shape. None of the five is an operator predicate and none is decided at a window.

Step 2 in the general format ((ii′)). For each f, D(iii) asks of $\{F_{v_f}^f\}$ that it be a family of block lists against factor f's map and that each block family satisfy the predicates and the summation condition of Definition 1.3, and (C3) read for f states the type and those same conditions on that factor's emitted blocks; the two lists of conjuncts are identical, factor by factor. What Step 2 says of the physical family and of the block-diagonality half

applies per factor unchanged, the mis-assignment exposure now being one per factor and V1's per-factor map comparison what catches an assignment that also disagrees about that factor's sectors.

Step 3 in the general format ((iii′)). The per-factor equivalence is (iii)'s argument with Q_f, Λ_f, γ_f and factor f's two families in place of Q, Λ, γ and the two families: the fibres $\gamma_f^{-1}(v_f)$ partition factor f's physical alphabet because γ_f is a total surjection (C2), the adjoint identity is the Hilbert–Schmidt pairing, and both directions are available because both sides are exact identities. For (7.5): the adjoint of a tensor product of maps is the tensor product of their adjoints, so $\Lambda^\dagger(F_v) = \bigotimes_f \Lambda_f^\dagger(F_{v_f}^f)$ by (1.5), this is $\bigotimes_f \sum_{w_f \in \gamma_f^{-1}(v_f)} F_{w_f}^{\mathrm{ph},f}$ by (7.2), and expanding the product of sums by multilinearity gives $\sum_{w \in \gamma^{-1}(v)} \bigotimes_f F_{w_f}^{\mathrm{ph},f}$, the index set being exactly $\gamma^{-1}(v) = \prod_f \gamma_f^{-1}(v_f)$. The statistics form follows for *every* operator ρ on the product physical space, product or not, because $\mathrm{tr}[F_v \Lambda(\rho)] = \mathrm{tr}\big[\Lambda^\dagger(F_v)\rho\big]$ is linear in ρ and (7.5) is an identity of operators. For the consistency reading, a tensor product of unital maps is unital, so summing over v and using $\sum_{v_f} F_{v_f}^f = \mathbb{1}_{Q_f}$ per factor (C3) gives $\sum_w F_w^{\mathrm{ph}} = \mathbb{1}$, which the product physical family already satisfies; nothing is added and nothing is evaluated.

Steps 4 and 5 in the general format ((iv′) and (v′)). The first is (iv)'s computation, which is linear in each ρ_x^{v} and reads only the identity, together with the split by kind of witness made in (S3) of Definition "Source segment and its interface conditions" (transmitter/def:source-segment), restated here and not re-derived. The second is again a statement about what this contract asserts: no step above invokes Assumption 10.6 as an established obligation, per factor or otherwise, or the attack-set licence, and none uses a theorem of the reductions module, and the three items named are named and not used — in particular no step above composes a source segment with a receiver segment, which is why the composition rule appears in the list rather than in a hypothesis. What is declared here is unchanged: the trace-preserving clause of Assumption 10.6 occurs in (iii′)'s consistency reading as in (iii)'s, and the attack-set licence is named here and spent at the gate (Theorem 9.4 (iii)). □

With a trace-preserving chain, a correspondence from the emitted alphabet onto the physical one forces every physical element covered more than once to vanish, so it admits no coarse-graining. The correspondence therefore runs from the physical alphabet onto the emitted one.

Remark 7.6 (Direction of the outcome correspondence). The direction in (C2) is forced, and the argument that forces it is a *typing* argument: (C2) fixes the direction of a tabulated map, it is combinatorial, and it is read exactly.

Suppose instead a total surjection $\gamma' : \mathcal{V} \to \mathcal{V}^{\mathrm{ph}}$ from the emitted alphabet onto the physical one, with the pullback read as $\Lambda^\dagger(F_v) = F_{\gamma'(v)}^{\mathrm{ph}}$. With trace-preserving channel data, Theorem 7.5 (iii) says that this reading forces $F_w^{\mathrm{ph}} = 0$ for every outcome w covered more than once; the computation is Step 3 of its proof. The

only non-injective γ' that survives is one whose extra outcomes are null, i.e. no coarse-graining at all. Coarse-graining is a real operation — a squashed family is typically *coarser* than the physical one — so the correspondence must run from the finer alphabet to the coarser, which is (C2), and the identity it feeds must sum the fibre, which is (7.1). Emitting the pair $(\gamma, (7.1))$ rather than a bare correspondence of indices is what keeps the statistics identity (7.3) true outcome by outcome.

Uses Def. 7.1, Thm. 7.5 · newly derived · accepted

For a chain the reductions module emits, its licence theorem discharges A-ATK for the entries it certified and, at status exact, its chain, transfer and emission theorems discharge A-LINK, also on a physical side presented by restriction or weighted after the link. Only Requirement 13.10 uses them.

Theorem 7.5 (v) names the soundness of each link and the attack-set licence as obligations of the chain's producer and discharges neither. This remark records where they are discharged for the chains the reductions module emits, also where a factor's family is weighted after its link, and for the Band-A identity link.

Remark 7.7 (Discharge of the chain obligations by the reductions module). *Where the two obligations are discharged.* For a chain emitted by the reductions module with status *exact*, Assumption 10.6 is discharged by Theorem "Composition of a chain" (reductions/thm:chain) (i)–(iii), Theorem "Security transfer from a constrained squashed model" (reductions/thm:transfer) and Theorem "Exact emission of a reduction instance" (reductions/thm:emit) (iii), its link-soundness clause being Theorem "Composition of a chain" (reductions/thm:chain) (i)–(iii) and Theorem "Exact emission of a reduction instance" (reductions/thm:emit) (iii). For a chain emitted by the reductions module, the attack-set licence of Assumption 10.7 is discharged by Theorem "The attack-set licence" (reductions/thm:licence), the restricted attack set being that of Definition "Admissible attack sets in the consumer's format" (reductions/def:attset), for the entries the chain transports, each carrying the spectral pull-back certificate that module discharged for it (Corollary "The emitted constraint entries" (reductions/cor:entries)); an entry of $\mathfrak{S}_{\mathrm{con}}$ carrying no such certificate is covered by no theorem of that module, and for a list containing one the licence stays the obligation Assumption 10.7 names. For the identity link of Band A, supplied as data, with the identity outcome correspondence and the declared family as its target POVM (Definition 8.1 (b)), Assumption 10.6 is discharged by exhibition (Theorem 8.2 (v)).

Several measured factors. Where each measured factor's receiver segment is the chain of that factor's artefact, emitted by the reductions module with status *exact*, and each such segment is that factor's instance link alone, Assumption 10.6 is discharged factor by factor as above, and jointly: the product of the factors' channels squashes the product of their declared models to the product target exactly (Theorem "The emission of several measuring parties" (reductions/thm:emitted-product) (ii)), and the physical protocol measures that product by A-MEAS (Assumption "Each measuring party's device as one POVM per round" (reductions/ass:meas)) (iv), so Theorem "Security transfer from a constrained squashed model" (reductions/thm:transfer) applies to the joint protocol, and every entry whose scope is a factor is sound for the joint reachable states (Theorem "The emission of several measuring parties" (reductions/thm:emitted-product) (iii), with Theorem "The

attack-set licence" (reductions/thm:licence) (v)). Where each factor's physical side is presented by restriction, as in the next paragraph, the product of the presented links is the presented link of the product, its inclusion being $\bigotimes_f \iota_f$ and its channel $\Lambda \circ \bigotimes_f \mathcal{I}_f$, since $\bigotimes_f (\Lambda_f \circ \mathcal{I}_f) = \big(\bigotimes_f \Lambda_f\big) \circ \bigotimes_f \mathcal{I}_f$; the next paragraph applies to it with $\iota = \bigotimes_f \iota_f$, ι_f being the identity on a factor that presents its declared model itself. Where a further link follows a factor's instance link, that theorem states nothing about the product, and the joint transfer stays the obligation Assumption 10.6 names. The engine omits the row at several measured factors only where every factor's segment is discharged in this way (Requirement 13.10).

A physical side presented by restriction. Where a receiver segment is its instance link alone, the link these theorems discharge has as its source family the declared model M'^{ph} on the physical system B, with channel Λ and $\Lambda^\dagger(F_v) = M'^{\mathrm{ph}}_v$ (Theorem "Exact emission of a reduction instance" (reductions/thm:emit) (iii)). The conclusion about the device is that link's: security transfers to the protocol on B for every input there. Let a specification present in its place a subspace V of $\mathcal{H}(B)$, with isometric inclusion ι, as the physical receiver space, the compressions $F^{\mathrm{ph}}_v = \iota^\dagger M'^{\mathrm{ph}}_v \iota$ as the physical receiver family, and the identity as the outcome correspondence. The discharge carries over to the link so presented. Its channel is $\Lambda \circ \mathcal{I}$ with $\mathcal{I}(X) = \iota X \iota^\dagger$, a composition of channels since $\iota^\dagger \iota = \mathbb{1}_V$; its adjoint is $Y \mapsto \iota^\dagger \Lambda^\dagger(Y) \iota$, so $(\Lambda \circ \mathcal{I})^\dagger(F_v) = F^{\mathrm{ph}}_v$, which is (7.1) for it, and every carried operator's pull-back is the compression of its pull-back under Λ. The presented link is therefore again a squashing relation (Definition "Squashing relation" (reductions/def:squashing)), to which Theorem "Security transfer from a constrained squashed model" (reductions/thm:transfer) and Theorem "Composition of a chain" (reductions/thm:chain) (iii) apply as to any; and an entry certificate $\Lambda^\dagger(\hat{O}_k) \bowtie_k c_k \mathbb{1}_B$ compresses to $(\Lambda \circ \mathcal{I})^\dagger(\hat{O}_k) \bowtie_k c_k \mathbb{1}_V$, compression by ι being positive and unital. The presented link inherits its conclusions from the link on B and adds none: read alone it would restrict the inputs to V, which the link on B does not. That the presented family is this compression is a fact of the instance and not of this paragraph. For the flat instance with $V = \mathcal{H}(C_{\le N})$ it holds because its declared model is block diagonal at the cutoff with the declared datum below it, and the target's data sectors carry that datum (Proposition "The flat instance of the post-processed receiver" (reductions/wpfss/prop:flat-instance), (I1)–(I2) and its declared model), so the block of F_v on the data sectors is $\iota^\dagger M'^{\mathrm{ph}}_v \iota$. No other instance has a claim stating that its docked block is such a compression, so on the routes of every other instance the row stays (Requirement 13.10).

A factor weighted after its link. Where a setting-dependent weighting $P(f \mid t)$ is applied after a factor's segment (Definition 4.1), the chain record is the unweighted one (Remark 4.8), and the discharge above carries over to the weighted protocol. With $\Pi\big((v', f) \mid v\big) \coloneqq \delta_{vv'} P(f \mid t(v'))$, a stochastic matrix from $\mathcal{V}$ to $\mathcal{V} \times \mathcal{F}$, Lemma "Linearity: post-processing, private settings and mixtures" (reductions/lem:linear) (i) turns the squashing relation of the source family and $\{F_v\}$ into the squashing relation, under the same channel, of $\{P(f \mid t(v))\, M'^{\mathrm{ph}}_v\}$, respectively of the identically weighted presented family, and the emitted $\{P(f \mid t(v))\, F_v\}$, so Theorem "Security transfer from a constrained squashed model" (reductions/thm:transfer) applies to the weighted protocol as to the unweighted one, the transported entries being untouched by a weighting. On the identity link of Band A the same holds at the identity channel: the delivered family is the declared one entry for entry (Definition 8.1 (b)), so the emitted weighted family is the identically weighted declared

family, the same matrices, and Theorem 8.2 (v) covers the weighted protocol.

The producer's statement of the same model. The reductions module states the same round model, typed by source mode, and the same restricted set as the round model of Definition "Round model and licensed attack sets" (meat-core/def:roundmodel), in Definition "Round model and attack sequences, typed by source mode" (reductions/def:roundmodel) and Definition "Admissible attack sets in the consumer's format" (reductions/def:attset): the latter reproduces the attack sets of Definition "Round model and licensed attack sets" (meat-core/def:roundmodel) in both source modes, and the former types the physical attack sequence, its squashed sequence being the one on Q.

Used by a requirement only. These theorems are used by no theorem, definition or remark of this contract, so they are in the closure of none of its results; Requirement 13.10 uses the ones on which the omission of the row of Assumption 10.6 rests, so that what the omission rests on is computed. What a run inherits from the reductions module reaches its assumption manifest as the ledger rows of each reductions artefact it consumed, and not through this module's graph; the manifest's claim closure is that of the root claims the run's key-rate result names, read from the generated claim-closure table.

Used by Req. 13.10 · newly derived · accepted

8 The Band-A route

On the direct route a finite measurement is declared on a single sector and the chain is one identity link, given as data, that transports no constraint and delivers the declared measurement unchanged, so every chain condition but the exact admissibility of the declared receiver family holds by inspection and the link is sound by exhibition (Theorem 8.2). That admissibility and the other measurement predicates are still decided exactly, and the residual physical content of the receiver is the ideal-receiver assumption. A specification whose declared data already contradict that assumption is refused (Requirement 13.7), and so is one whose link is not decided on the record to be that identity link (Requirement 13.8).

Imports transmitter/def:source-segment

A Band-A specification declares a finite measurement directly on one sector, with an empty source segment and one identity receiver link, given as data, that carries no constraints and delivers the declared measurement. It is the direct route, with no optical characterisation or photon-number step.

The one receiver link of a Band-A chain is the identity link of Example "The identity link" (reductions/ex:identity); the definition fixes that link in its own words. A link record carries its channel, its outcome correspondence and its target POVM as three separate data, so all three are fixed here: that the channel is the identity says nothing of the measurement the link delivers.

Definition 8.1 (Band-A specification)**.** A specification is *Band A* when (a) its receiver part declares a POVM $\{F_w^{\mathrm{ph}}\}_{w\in\mathcal{V}^{\mathrm{ph}}}$ directly on a finite-

dimensional system, with no characterization step and no photon-number structure, and the emitted sector set is the singleton $\mathbb{S} = \{s_0\}$ with $\Pi_{s_0} = \mathbb{1}_Q$; and (b) its reduction chain has an *empty source segment*, $L_s = 0$ — the source segment of Definition "Source segment and its interface conditions" (transmitter/def:source-segment) at length zero, required here of a Band-A chain and of no other — and a receiver segment of one link, $L = 1$, $\Lambda_1 = \mathrm{id}_Q$ supplied as data, with status *exact*, empty constraint slot and empty licence slot, whose outcome correspondence is the identity map $\gamma = \mathrm{id}$ on $\mathcal{V} = \mathcal{V}^{\mathrm{ph}}$ and whose target POVM is the declared family itself: $F_v = F_v^{\mathrm{ph}}$, entry for entry, for every $v \in \mathcal{V}$.

Uses transmitter/def:source-segment · *Used by* Thm. 4.4, Thm. 8.2, Req. 13.7, Req. 13.8 · newly derived · implemented · *Code* certiqs_key.protocol_compiler.declared_premises:is_band_a · *Oracles* registry_instances_bb84.py, registry_protocol_compiler.py

On the direct one-sector route every chain condition but the receiver family's exact admissibility holds by inspection, and the identity link, which delivers the declared measurement, is sound by exhibition. The receiver's residual physical content is the ideal-receiver assumption.

On the direct route no reduction runs: the chain is one identity link, given as data, which transports no constraint entry and delivers the declared measurement unchanged. The theorem records what that makes trivial and what it does not. The one link of the receiver segment is the identity link of Example "The identity link" (reductions/ex:identity). That the link delivers the declared measurement on the identity outcome correspondence is part of the hypothesis, Definition 8.1 (b), and not a consequence of its channel being the identity: a link record carries its target POVM and its outcome correspondence as data of their own, and the engine decides both on the record (Requirement 13.8). The attack-set licence (Assumption 10.7) is asserted only for a non-empty constraint list; on this route that list is the specification's own (iii), so where the specification declares no entry the licence is not invoked and nothing proved here rests on it.

Theorem 8.2 (The Band-A route)**.** For a Band-A specification:

(i) the block structure of a Band-A declaration is the one-block structure: $\mathfrak{s}$ is the singleton map $\big((s_0, d_{s_0})\big)$, $d_Q = d_{s_0}$, and every Q-operator is a one-element block list, so that its single block *is* the whole operator. Two consequences follow, and they differ. The typing half of D(iii) is not vacuous but trivial to satisfy: an operator is a one-element list, and V1's sector-map comparison is one identifier and one dimension. The five predicates of D(iii) are *not* vacuous at all — with $|\mathbb{S}| = 1$ the per-block predicates are the full-space ones, decided once each on $d_Q \times d_Q$ matrices, and a Band-A declaration is decided exactly like any other, condition (S) included;

(ii) the interface conditions of Definition 7.1 hold by inspection, save the predicates and summation condition of (C3), which are decided: descriptors match because source and target of id coincide; the outcome correspondence is the identity bijection $\gamma = \mathrm{id}$ on $\mathcal{V} = \mathcal{V}^{\mathrm{ph}}$, as Definition 8.1 (b) requires; (C3)'s typing half holds because the declared receiver family is already a one-element block list against the singleton map, and its predicates and summation condition are the declared receiver POVM's own, so (C3) holds exactly when that family is exactly admissible, which is decided on it and is not given by inspection; the pullback identity (C4) reads $\mathrm{id}^{\dagger}(F_v) = F_v = F_v^{\mathrm{ph}}$ and holds *identically* — entry for entry, since $\mathrm{id}^{\dagger} = \mathrm{id}$, every fibre $\gamma^{-1}(v)$ is a singleton, and the delivered F_v *is* the declared F_v^{ph}, the same matrix, by Definition 8.1 (b); the source conditions (S1)–(S4) of Definition "Source segment and its interface conditions" (transmitter/def:source-segment) are vacuous, quantifying over an empty list of links — the empty source segment being that definition's at $L_{\mathrm{s}} = 0$, which Definition 8.1 (b) requires of a Band-A chain and which is therefore a hypothesis of this theorem — and the virtual ensemble of (S3) is the specified one, so the source identity reads $\rho_x = \rho_x$; the ordering (X1) holds because the source segment is empty; and the slots of (C5) are present and empty. The one link of the receiver segment is the identity link. *So both conditions that a general chain has to decide on computed data are met here by exhibition*, and what a verified pullback identity and a verified source identity buy on a general chain holds on this path without appeal to any obligation of the chain's producer: measuring the family the link delivers after id reproduces the physical statistics, $\mathrm{tr}[F_v\,\mathrm{id}(\rho)] = \mathrm{tr}\left[F_v^{\mathrm{ph}}\,\rho\right]$ for every v and every operator ρ, and id is exactly trace-preserving *by exhibition*, the supplied channel data being exactly that map rather than binary64 data trusted to be trace-preserving, so its adjoint is unital and summing the pullback identity over v returns $\sum_v F_v^{\mathrm{ph}} = \mathbb{1}$, which on this path is the same statement as (C3)'s own completeness, $\{F_v^{\mathrm{ph}}\}$ being $\{F_v\}$: the identity holds as a triviality and carries no content this path did not already have;

(iii) the link transports no constraint entry, so $\mathfrak{L}$ is the specification's own list $\mathfrak{S}_{\mathrm{con}}$ and the constraint-soundness obligation of the link (the transported-constraint clause of Assumption 10.6) is vacuous; where $\mathfrak{S}_{\mathrm{con}}$ is empty, $\mathfrak{L} = \varnothing$ and the two-part gate of Theorem 9.4 is not engaged;

(iv) consequently no emitted slot, and no claim of this contract used in the Band-A path, depends on an artefact of the receiver module or of the reductions module. The residual physical content of the receiver is Assumption 10.8, and it is an idealization of the receiver, not a reduction of it; an entry the specification declares itself engages the

gate, and with it the attack-set licence of Assumption 10.7, as on every route; and

(v) *(the identity link is sound)* the Band-A chain satisfies every condition of Assumption 10.6: (i)(a) the channel of its one link, id_Q, exists, the link supplying it as data; (i)(b) that link satisfies the pullback identity (7.1); (i)(c) the protocol using the measurement the link delivers is the protocol using the declared receiver family $\{F_v^{\mathrm{ph}}\}$, the delivered $\{F_v\}$ being that family, so security of the one is security of the other, and where a setting-dependent weighting $P(f\,|\,t)$ is applied after the link (Definition 4.1), the emitted family $\{P(f\,|\,t(v))\,F_v\}$ is the identically weighted declared family $\{P(f\,|\,t(v))\,F_v^{\mathrm{ph}}\}$, so the same holds of the weighted protocol; (ii) its source segment is empty; (iii) its link transports no constraint entry; and (iv) id_Q is completely positive and trace-preserving. No clause of Assumption 10.6 is therefore a premise on this path; that the declared family is the measurement the receiver performs is Assumption 10.8, and no condition on the link.

Uses Def. 1.3, Def. 3.3, Def. 4.1, Def. 7.1, Def. 8.1, transmitter/def:source-segment · *Assumes* A-IDEAL · *Used by* Req. 13.7, Req. 13.8 · newly derived · accepted · *Oracles* registry_instances_bb84.py, registry_protocol_compiler.py

Proof. (i) $\sum_s d_s = d_{s_0}$ is the one-term sum, so $d_Q = d_{s_0}$. The typing half: a block list against a singleton map is a one-element list, and $\bigoplus_{s\in\{s_0\}} F^{(s_0)} = F^{(s_0)}$, so the operator and its block coincide; V1's comparison is then a comparison of two one-element lists. The predicates: at $|\mathbb{S}| = 1$ the family $\{F_v^{(s_0)}\}_v$ is $\{F_v\}_v$ and $Q_{s_0} = Q$, so each per-block condition of Definition 1.3 — Hermiticity, positivity, completeness, and the summation condition (S) on the completeness sum — is literally the full-space one, and none of them is empty.

(ii) Each condition is evaluated at $L_{\mathrm{s}} = 0$ and $L = 1$, $\Lambda_1 = \mathrm{id}$: (C1) both descriptors of id_Q, at its source and at its target, are $(Q, \{s_0\}, (d_{s_0}))$ and the physical receiver space is Q by Definition 8.1 (a), and the two descriptors carry the same singleton sector map, so (C1) holds — and the emitted $\{F_v\}$ carries that same singleton map, so V1's comparison passes as well; (C2) the outcome correspondence is the identity map on $\mathcal{V} = \mathcal{V}^{\mathrm{ph}}$ (Definition 8.1 (b)), a tabulated bijection, in particular a total surjection with singleton fibres; (C3): its typing half is (i), the family being a one-element list against the singleton map, and by (i) its blocks are the declared receiver family itself, the same matrices, so its predicates and summation condition are that family's exact admissibility, which is decided exactly on those matrices and which nothing in this theorem supplies; (C4) for every operator Y and every ρ, $\mathrm{tr}[Y\,\mathrm{id}(\rho)] = \mathrm{tr}[Y\rho]$, so $\mathrm{id}^\dagger = \mathrm{id}$ and (7.1), whose fibre $\gamma^{-1}(v)$ is $\{v\}$ for $\gamma = \mathrm{id}$, reads $F_v = F_v^{\mathrm{ph}}$; this holds entry for entry because Definition 3.3

fills $\{F_v\}$ from the last link's target POVM, and that target POVM is the declared family itself, $F_v = F_v^{\mathrm{ph}}$ for every v, by Definition 8.1 (b), so the delivered matrix *is* the declared one — a clause of the hypothesis, and not a consequence of the link's channel being id, which fixes nothing of the target POVM; (C5) is Definition 8.1 (b), which gives the one link its status tag, *exact*, and its constraint and licence slots, both empty; (S1)–(S4) quantify over the links of $\mathfrak{R}_{\mathrm{src}}$ and that list is empty, and the source identity (S3) holds with Ξ the identity and $\rho_x^{\mathrm{v}} = \rho_x$, as Definition "Source segment and its interface conditions" (transmitter/def:source-segment) says of the empty segment; (X1) is the shape of the record, with nothing in the first component.

(iii) $\mathfrak{L}$ is the concatenation of the specification's entry list $\mathfrak{S}_{\mathrm{con}}$ with the entries transported by the links (Definition 3.3); the source segment has no links and the single receiver link has an empty constraint slot (Definition 8.1 (b)), so the links transport no entry and $\mathfrak{L}$ is $\mathfrak{S}_{\mathrm{con}}$'s list. The transported-constraint clause of Assumption 10.6 quantifies over the entries a link transports and is vacuous. Where $\mathfrak{S}_{\mathrm{con}}$ is empty, $m = 0$ and the gate's premise "$m \geq 1$" fails, so Theorem 9.4 (ii) is not engaged, and no step of this proof invokes that theorem's statement.

(iv) By (ii) and (iii) every emitted slot is filled from $\mathfrak{S}_{\mathrm{src}}$, $\mathfrak{S}_{\mathrm{rec}}$, $\mathfrak{S}_{\mathrm{pub}}$ and $\mathfrak{S}_{\mathrm{con}}$, and the only chain-derived slots (Q, $\mathbb{S}$, (d_s), $\{F_v\}$, $\mathfrak{L}$) take the values of the directly declared receiver data and of the specification's own entry list. The channel-existence clause of Assumption 10.6 — that the link's channel exists as claimed — is discharged by exhibition, since id is a channel; what remains of the receiver is Assumption 10.8. An entry of $\mathfrak{S}_{\mathrm{con}}$ is declared by the specification, as on any route, and the gate it engages is Theorem 9.4's, whose licence is Assumption 10.7; nothing in this proof rests on it.

(v) Each clause is evaluated at $L_{\mathrm{s}} = 0$ and $L = 1$, $\Lambda_1 = \mathrm{id}_Q$ (Definition 8.1 (b)). (i)(a) and (iv): id_Q is completely positive and trace-preserving, the identity on $\mathcal{L}(Q)$ tensored with any identity being an identity; it is a channel that exists, and the link supplies it as data, exactly that map (Definition 8.1 (b)). (i)(b) is the pullback identity of (ii), $\mathrm{id}^{\dagger}(F_v) = F_v = F_v^{\mathrm{ph}}$ entry for entry. (i)(c): by (ii), that is by the target-POVM clause of Definition 8.1 (b), the delivered $\{F_v\}$ is the declared $\{F_v^{\mathrm{ph}}\}$, the same matrices, and nothing else of the protocol depends on the link, so the protocol using the delivered measurement and the protocol using the declared one are one protocol, and a security statement about it implies itself. Where a setting-dependent weighting is applied after the link, the emitted family is $\{P(f \mid t(v))\, F_v\}_{(v,f)\in\mathcal{V}_P}$ (Definition 3.3, Definition 4.1), and with $F_v = F_v^{\mathrm{ph}}$ entry for entry it is the declared family weighted by the same P through the same setting map, the same matrices again; so the weighted protocol using the emitted measurement and the weighted protocol using the declared one are again one protocol. Clause (ii) of the assumption quantifies over the

links of the source segment, and there are none. Clause (iii) quantifies over the constraint entries the link transports, and its constraint slot is empty (Definition 8.1 (b)). □

9 The emission gates

A well-formed declaration may back a key-length claim only when every check passes, the transmitter's source-replacement licence record is present and re-decided wherever the marginal slot is filled and Alice's dimension is at least two, and a non-empty constraint list is licensed and carries an attack-set licence. The main result, Theorem 9.4, shows that every gate is decidable or a presence test and that every gate failure has one fixed outcome, a refusal or an emission that is not key-length-eligible; passing a gate asserts nothing about the apparatus or about attack-set inclusion. Dropping a trusted marginal, and reading homogeneous constraints as scalars, enlarges the feasible set and so can only lower a minimised quantity (Proposition 9.6); no reachable row of the gate outcome table invokes it.

Imports transmitter/def:source-spec, transmitter/def:srcert, transmitter/thm:certificate-sufficiency

An emitted declaration may back a key-length claim when every check passes, the transmitter's source-replacement licence record is present and re-decided wherever a marginal is declared at dimension two or more, and a non-empty constraint list is licensed and carries an attack-set licence.

A declaration that passes validation is well-formed; whether it may back a key-length claim is a further question, decided by three gates. The first is stated on the checks rather than on the target conditions, because the checks are what a program evaluates and the two are not interchangeable at V7 (Theorem 6.2 (iii)). The emitted flag records the outcome (Requirement 14.1), the licence record the second gate reads is passed on to the consuming engine (Requirement 11.7), and why that gate admits an ensemble presentation but refuses a block-form certificate is Remark 9.2.

Gate (G2) re-decides the transmitter's conditions because a consumer decides and never infers (Requirement "Exact decision by every consumer" (foundations/req:consumer-decides-exactly)). A trusted-mode specification at $d_A \geq 2$ without the record does not reach this gate at all, the transmitter refusing it before this module sees it; that, and that no record is demanded at $d_A = 1$, is Requirement "Certificate and attestation demanded only at $d_A \geq 2$" (transmitter/req:licence-scope) (the mode itself is defined in Definition "Source specification" (transmitter/def:source-spec)). What the attack-set licence record attests for a party-local entry, and why that entry needs it, is Remark 2.6.

Definition 9.1 (Key-length eligibility)**.** An emitted declaration is *key-length-eligible* when all of the following hold:

(G1) V1–V8 all pass — whereupon D(i)–D(viii) all hold of the declared numbers, by the soundness half of Theorem 6.2 (i);

(G2) *(a consumption gate on the transmitter artefact)* if the marginal slot is filled and $d_A \geq 2$: the transmitter artefact carries the source-replacement licence record of Definition "Source-replacement certificate" (transmitter/def:srcert) — the certificate $(\tau_A, \Psi, \{p_x, \rho_x^{\mathrm{v}}\})$ with its attestation (SR3) — whose *five decidable conditions* the transmitter decided exactly on the same data (Theorem "Checkability and sufficiency of the source-replacement certificate" (transmitter/thm:certificate-sufficiency) (i)) and which *this module re-decides on the emitted data*: $\Psi = \Psi^\dagger$; $\Psi^2 = \Psi$; $\operatorname{tr}\Psi = 1$, the trace accumulated over the row index and rounding nowhere; (SR1); and (SR2) — each *exactly*, as matrix and scalar identities. The transmitter's decision is not taken on its word. *No condition of (G2) is stated on a vector.* The certificate's declared datum is the operator Ψ; the unit vector $|\psi\rangle$ with $\Psi = |\psi\rangle\langle\psi|$ exists by $\Psi^2 = \Psi$ and $\operatorname{tr}\Psi = 1$ and is never itself declared (Definition "Source-replacement certificate" (transmitter/def:srcert)). Clause (SR3) carries no number and is not decidable (Theorem 9.4 (i)): (G2) asks for its presence only. At $d_A = 1$ no record is demanded, the licence being vacuous there;

(G3) if $m \geq 1$: (a) $\mathfrak{L}$ lies in the licensed class of the emitted source mode (Definition 2.5) — *a type check on each entry's right-hand-side tag, comparing no numbers — and* (b) the attack-set licence record, the attestation of Assumption 10.7, is present.

(G2) by form of the certificate. The five conditions above are those of a certificate declared in pure form: a pure-form certificate is re-decided exactly as stated, and refused when one of the five fails.

A certificate declared in block form is refused at this gate by its declared form, whatever its blocks, by row 3 of the gate outcome table of Definition 9.3. A block-form certificate (Definition "Source-replacement certificate" (transmitter/def:srcert)) is a positive semidefinite unit-trace block list on which that definition imposes no purity, so one with more than one non-zero block, or whose one non-zero block is not a rank-one projector, also fails $\Psi^2 = \Psi$; but the refusal does not rest on that, since one whose one non-zero block is a rank-one projector can meet all five conditions above.

An ensemble presentation declares no Ψ and no $\{M_x\}$ to re-decide. For it the gate decides, in place of the five conditions, what the presentation declares: (E1), (E2) and (E3) of Definition "Source-replacement certificate" (transmitter/def:srcert) on the declared weighted ensemble, and that the emitted declaration is in ensemble form with its Alice slots *the* ones the presentation defines — the sector structure of the factor tagged A is the matched structure, and for every t and x the block $N_x^{(t)}$ is entry for entry the transpose

of the declared block $\sigma_x^{\mathrm{v},(t)}$ and $\tau_A^{(t)}$ the transpose of their declared-order sum. *A presentation accompanying a POVM-form declaration is refused*, nothing tying the declared $\{M_x\}$ to it, which is the transmitter's own rule for a POVM-form consumer. In every form, (G2) asks for the presence of (SR3).

(G3)(a) with the party-local form. The licensed class of Definition 2.5 contains, in either source mode, the entries declared party-local; the decision remains a type check on each entry's tag and compares no numbers. A list carrying such an entry is key-length-eligible only with the attack-set licence record of (b), as every non-empty list is, and what that record attests for a tagged entry is a certificate of the module that produces the chain, on the scoped factor's own channel (Definition 2.5).

Uses Def. 1.1, Def. 1.3, Def. 2.5, Thm. 6.2, transmitter/def:srcert, transmitter/thm:certificate-sufficiency · *Used by* Rem. 9.2, Def. 9.3, Thm. 9.4, Req. 11.7, Req. 14.1, Req. 14.2, Req. 14.3, Req. 14.4 · newly derived · implemented (effective: accepted) · *Code* certiqs_key.protocol_compiler.gates: GateId · *Oracles* registry_protocol_compiler.py, registry_transmitter.py

Gate (G2) refuses every block-form certificate by its form, pure or not, and admits an ensemble presentation, whose blocks are rank one by theorem; the refusal rests on scope, not on that difference. At several Alice sectors a presentation is not globally pure, and the gate does not demand it.

Gate (G2) of Definition 9.1 re-decides a pure-form certificate, refuses a block-form one and decides an ensemble presentation by what it declares. What purity adds to a certificate, and why the certificate has a block form and an ensemble presentation at all, is Remark "Purity and exact declarability of the certificate forms" (transmitter/rem:certificate-forms); this remark says what separates the forms at the gate.

Remark 9.2 (The source gate on each certificate form). *The block form.* Gate (G2) decides $\Psi^2 = \Psi$ of a pure-form certificate and refuses a block-form certificate by its declared form, whatever its blocks (Definition 9.1). Most block-form certificates fail $\Psi^2 = \Psi$ as well — every one with two or more non-zero blocks, and every one whose one non-zero block is not a rank-one projector — but one whose one non-zero block is a rank-one projector is pure and is refused all the same. So (G2) is not the consumer that Remark "Purity and exact declarability of the certificate forms" (transmitter/rem:certificate-forms) describes, which demands purity of every certificate and refuses a block form for want of it: the ground of the refusal is the scope rule stated at the end of this remark.

What separates a general block form from an ensemble presentation is not the word "purity", which is waived on both sides once Alice carries several sectors. A general block form declares blocks that are positive semidefinite and *not known to be rank one*, so the "only if" half of Theorem "Checkability and sufficiency of the source-replacement certificate" (transmitter/thm:certificate-sufficiency) (ii) — proved for a block form exactly where every non-zero block is a positive multiple of a rank-one projector — is lost with it, and a source/POVM mismatch would then be excluded only by the POVM condition decided on its own. A presentation's blocks are rank one *by theorem*, so that half is not lost.

An ensemble presentation at one Alice sector. For an ensemble presentation there is no declared Ψ and no declared $\{M_x\}$ to re-decide: purity of the blocks, (SR1) and (SR2) are *theorems* of the presentation (Lemma "Certificate defined by an ensemble presentation" (transmitter/lem:ensemble-presentation) (i), (ii)), that lemma's (ii) giving every non-zero block as a positive multiple of a rank-one projector. At one Alice sector no purity requirement is waived: there the Ψ the presentation defines is that one block, of unit trace, hence a rank-one projector, so the five conditions of (G2) hold of it as theorems rather than as decisions, and the gate demands of such a presentation everything it demands of a pure-form certificate.

At two or more Alice sectors. There the defined operator is $\Psi = \bigoplus_t \Psi_t$ with $\operatorname{tr}\Psi_t = g_t$ and $\sum_t g_t = 1$, so $\Psi^2 = \bigoplus_t g_t\Psi_t \neq \Psi$ as soon as two blocks are non-zero, by the same arithmetic by which a block form with two non-zero blocks fails $\Psi^2 = \Psi$. Global purity is therefore not demanded of a presentation with several Alice sectors. What the gate rests on there instead is two statements of the transmitter's, both available for a presentation, of which a general block form has only the second.

(a) Every block is rank one by theorem (Lemma "Certificate defined by an ensemble presentation" (transmitter/lem:ensemble-presentation) (ii)), which is exactly the hypothesis under which Theorem "Checkability and sufficiency of the source-replacement certificate" (transmitter/thm:certificate-sufficiency) (ii) holds in *both* directions.

(b) Theorem "Checkability and sufficiency of the source-replacement certificate" (transmitter/thm:certificate-sufficiency) (iii), the sufficiency statement at the module boundary, assumes (SR1)–(SR3) and that module's sequential assumption and *reads purity nowhere.*

The asymmetry in (a) is not relied on. The "only if" direction is what a consumer that does *not* decide the POVM condition separately would need, and this module decides it at check V3 in either form — $\sum_x M_x^{(t)} = \mathbb{1}_{A_t}$ in POVM form, the ensemble identity of (1.6) in ensemble form (D(iii)) — which is stronger than the compressed condition that converse delivers; and for a presentation there is in any case no mismatch to detect, $\{M_x\}$ being *defined* by the presentation rather than declared and gate (G2) pinning the declaration's Alice slots to the transposed declared blocks entry for entry, so that direction holds vacuously. Clause (a) records what the two forms differ in; it does not carry the refusal of a general block form, which rests on scope: admitting one would widen (G2) in the unsafe direction, to certificates that can accompany a declaration in the basic format.

Where a presentation can occur. A presentation is admitted only beside an ensemble-form declaration, and no declaration in the basic format is in ensemble form.

No form's (SR1)–(SR2) is assumed. In none of its forms are the conditions (SR1)–(SR2) of a source-replacement certificate an assumption of this module: what gate (G2) decides of each form is Definition 9.1's and Theorem 9.4's, and (SR1)–(SR2) are named as a hypothesis at the one place a derivation uses them, Theorem "Checkability and sufficiency of the source-replacement certificate" (transmitter/thm:certificate-sufficiency).

Uses Def. 9.1 · newly derived · accepted

The gate outcome table gives each gate failure a refusal naming what failed or an emission that is not key-length-eligible; its two certificate-absent rows give the compiler no outcome. Where a declaration meets a refusal row and an emission row, the refusal wins.

Rows 2 and 4 describe a case that does not reach the compiler: no protocol specification meets their premise (Theorem 9.4 (ii)), and the transmitter refuses such a request before any artefact exists (Requirement 14.2). Their cell says so; the table gives the compiler no outcome there.

Definition 9.3 (The gate outcome table)**.** The *gate outcome table* is the following table of the failures of the gates of Definition 9.1 and their outcomes; where the failures of one declaration meet the premises of a refusal row and of an emission row both, the outcome is the refusal.

Gate failure to emitted outcome. No row substitutes a default value, drops an entry, changes the source mode or synthesizes an attestation.

What failed	*Outcome*
(G1): any of V1–V8	refusal, naming the check
(G2): trusted mode at $d_A \geq 2$, certificate *absent*, and $m = 0$	none (no outcome of the compiler)
(G2): trusted mode *at any* d_A, certificate *present* but one of the five conditions of Definition 9.1 (G2) fails — Ψ Hermitian, $\Psi^2 = \Psi$, $\operatorname{tr}\Psi = 1$ rounding nowhere, (SR1), (SR2)	refusal, naming the condition
(G2): trusted mode at $d_A \geq 2$, certificate absent, and $m \geq 1$	none (no outcome of the compiler)
(G3)(a): $\mathfrak{L}$ not in the licensed class — some entry declared general in trusted-source mode	emission, not key-length-eligible
(G3)(b): the attack-set licence record absent	emission, not key-length-eligible

The cell of rows 2 and 4 assigns the compiler no outcome.

Uses Def. 9.1 · *Used by* Thm. 9.4, Req. 14.2 · newly derived · implemented (effective: accepted) · *Code* certiqs_key.protocol_compiler.gates:resolve_gates

Each gate is decidable or a presence test, and each failure has one fixed outcome, a refusal or an emission flagged as backing no key length, never a default, dropped entry, changed source mode or licence the compiler writes. Passing asserts nothing about the apparatus or attack-set inclusion.

One principle fixes every outcome. A check that fails, so that no declaration is shown to be well-formed, or a certificate that does not verify is a refusal, and no declaration is emitted; a well-formed declaration on verified data whose key-length claim is unsupported is emitted, not key-length-eligible, and may be explored numerically. The refusal strings and the flag that carry

these outcomes are Requirement 14.2's.

Theorem 9.4 (The emission gates and their outcomes). (i) *(decidability)* (G1) is decidable by Theorem 6.2; (G2)'s conditions — the five of Definition 9.1 (G2), namely Ψ Hermitian, $\Psi^2 = \Psi$, $\operatorname{tr}\Psi = 1$ with the trace rounding nowhere, (SR1) and (SR2) — are finitely many exact identities on the emitted data (Theorem "Checkability and sufficiency of the source-replacement certificate" (transmitter/thm:certificate-sufficiency) (i)); (G3)(a) is decided by *reading the type of the datum each entry's right-hand-side slot holds, which is its tag*, and by nothing else — it compares no matrix against any other and constructs no right-hand side; (G3)(b) is a presence test. (G2)'s clause (SR3) and (G3)(b)'s record are attestations and are *not* decidable; they are tested for presence only;

(ii) *(outcome of a failure)* If any gate fails, the compiler emits no key-length-eligible declaration, and which of the *two* permitted outcomes it produces — refusal, or emission of a declaration that is not key-length-eligible — is fixed by the gate outcome table (Definition 9.3) and is not an implementer's choice; where the failures of one declaration meet the premises of a refusal row and of an emission row both, the outcome is the refusal. No protocol specification meets the premise of rows 2 and 4, the two rows of the gate outcome table that describe a certificate-absent case: by Definition 3.1 (a) a transmitter artefact in trusted-source mode at $d_A \geq 2$ carries a source-replacement certificate. The compiler never drops a constraint entry silently, never substitutes a default, never changes the source mode it was handed, and never synthesizes a licence.

(iii) *(no assertion)* Passing (G3)(b) records that an attestation was supplied. It asserts nothing about attack-set inclusion, and no claim of this contract does. Likewise passing (G2) records that the transmitter's licence record is present and that its five decided conditions hold of the emitted data: it asserts nothing about the apparatus, and what it buys for the run is bought under Assumption 10.3 — the certificate is decided once, for the one declaration that describes every round, which is where this module spends round homogeneity (Fact "Source replacement for prepare-and-measure protocols" (transmitter/fact:sr) drops the per-round superscript of the source state under exactly that assumption of the consumer's).

The general format. For a declaration in the general format of Definition 1.1 (DX1)–(DX6) the three parts hold with the following readings, of which the three above are the case of one Alice sector, one measured factor, POVM form, a pure-form certificate and no party-local entry.

(i′) *(decidability)* (G1) is decidable by Theorem 6.2 in its general-format reading. (G2) keeps its five conditions and is decidable in *one fur-*

ther declared form (Definition 9.1): the five conditions of a pure-form certificate, as above, a certificate declared in block form being refused at this gate; and, for an ensemble presentation, (E1), (E2), (E3) on the declared weighted ensemble together with the entry-for-entry identity of the declaration's Alice slots with the transposes of the declared blocks and of their declared-order sum — finitely many exact identities, comparisons of labels and integers, and finitely many summation decisions, the number fixed by $|\mathbb{T}|$, $|\mathcal{X}|$ and the declared dimensions. *Purity is decided by nothing there*: a presentation declares no Ψ, and purity of its blocks, (SR1) and (SR2) are theorems of the presentation (Lemma "Certificate defined by an ensemble presentation" (transmitter/lem:ensemble-presentation) (i), (ii)), the second giving every non-zero block as a positive multiple of a rank-one projector. *At one Alice sector it is not thereby waived*, the defined Ψ being that one block, of unit trace, hence a rank-one projector of which the five conditions hold. *At two or more it is waived*, and the gate rests there on two statements — rank-one blocks by theorem, which is the hypothesis under which Theorem "Checkability and sufficiency of the source-replacement certificate" (transmitter/thm:certificate-sufficiency) (ii) holds in both directions, and that theorem's (iii), which reads purity nowhere — the first of which a general block form does not supply. (G3)(a) is decided by reading the type of the datum each entry's right-hand-side slot holds and, for the third form, the tag field it arrives with — three forms being in the licensed class, and still *nothing computed and nothing compared numerically.* (G3)(b) is a presence test, and (SR3) is tested for presence in every form.

(ii′) *(no row is added)* The outcome of every failure of the general format is fixed by a *row the gate outcome table already has.* A general-format check of Definition 6.1 that fails is (G1) and is row 1, the refusal naming the check. A certificate that is *present* and fails a condition of (G2) is row 3: a certificate that does not verify is a defect in the supplied data, not a weaker claim. *Row 3's premise is read of the declared form*, which reads the cell and adds no row: for a pure-form certificate the five conditions the cell enumerates are the conditions, and the cell is literal; for a certificate declared in block form the condition it fails is the form itself, (G2) admitting no block-form certificate (Definition 9.1), and the cell's premise — a certificate present and failing a condition of (G2) — covers it, whether or not it also fails $\Psi^2 = \Psi$; for an ensemble presentation the conditions are (E1) to (E3) and the identity of the declaration's Alice slots with the transposed declared blocks, and the cell's premise — a certificate present and failing a condition of (G2) — covers them, the enumeration in it being the pure form's list and not a restriction of the premise. *An ensemble*

presentation accompanying a POVM-form declaration is a failure of that kind and is refused (row 3), because nothing ties the declared $\{M_x\}$ to the presentation (Definition 9.1). A list carrying an entry declared general in trusted-source mode is row 5 and is emitted, not key-length-eligible; *a list carrying a party-local entry is not that row*, that form being licensed in either mode.

(iii′) *(no assertion)* Item (iii) holds as stated, and passing (G3)(b) records, rather than asserts, one further item: for a list carrying a party-local entry, what the attack-set licence record attests is a certificate of the module that produces the chain, on the scoped factor's own channel (Definition 2.5), and *no function of the emitted matrices decides it*, exactly as for the untagged forms. Passing (G2) on an ensemble presentation likewise records that the transmitter's record is present and that the conditions of *that form* hold of the emitted data, and asserts nothing about the apparatus; what it buys for the run is bought under Assumption 10.3 as in (iii), the certificate being decided once for the one declaration that describes every round.

Uses Def. 1.1, Def. 1.3, Def. 2.5, Def. 3.1, Def. 3.3, Thm. 6.2, Def. 9.1, Def. 9.3, transmitter/def:source-spec, transmitter/thm:certificate-sufficiency · *Assumes* A-ATK, A-RHOM · *Used by* Rem. 9.7, Req. 14.2, Req. 14.3 · newly derived · implemented (effective: accepted) · *Code* certiqs_key.protocol_compiler.gates: resolve_gates · *Oracles* registry_protocol_compiler.py, registry_transmitter.py

Proof. Idea. Each gate condition is either a finite list of exact identities, a type read off each constraint entry, or a presence test; the outcome of a failure is the one the gate outcome table of Definition 9.3 fixes, no row of it changes the source mode or drops an entry, and by the definition of a protocol specification no specification meets its certificate-absent rows; and the general format adds decisions of the same kinds and failures that land on existing rows.

Step 1 (decidability, (i)). Each clause is as stated: Theorem 6.2 (iv) for (G1); Theorem "Checkability and sufficiency of the source-replacement certificate" (transmitter/thm:certificate-sufficiency) (i) for the (G2) identities. For (G3)(a): by Definition 2.5 the licensed class is defined by which of the two forms each entry *declares*, and the tag is a field of the datum (Definition 1.1), so deciding the class is reading m tags and checking that the datum each names has the type it names — m comparisons of a label and a type, terminating and exact. *Nothing is computed and nothing is compared numerically.* Presence tests are decidable trivially. That (SR3) and the licence record are attestations is Definition "Source-replacement certificate" (transmitter/def:srcert) (SR3) and Assumption 10.7: neither is a property of the emitted matrices, so no function of them decides it.

Step 2 (the marginal slot and the certificate-absent rows, for (ii)). τ_A reaches the emitted data only through the transmitter artefact's marginal

slot — Definition 3.3 passes that slot through, and no input of this module carries a marginal or a mode separately (Definition 3.1 (a): the mode is read off the artefact's marginal slot and is never a second datum). The premise of rows 2 and 4 is an artefact in trusted mode at $d_A \geq 2$ whose certificate is absent, and by Definition 3.1 (a) a transmitter artefact in trusted-source mode at $d_A \geq 2$ additionally carries a source-replacement certificate; so no protocol specification meets that premise. That the engine refuses such a request is Requirement 14.2's, and no step here uses it. At $d_A = 1$, where (G2) does not demand the certificate at all, the slot holds $\tau_A = [1]$, the value the transmitter filled it with (Definition 3.1 (a)), and this module passes the slot through — that is not a default substituted for a missing datum here, which item (ii) forbids, because this module fills nothing: $S_{=}(A) = \{[1]\}$ is a singleton, so D(ii) admits that value and no other, and at $d_A = 1$ the two source modes impose the same condition (Definition "Source specification" (transmitter/def:source-spec)), so the filled slot excludes nothing.

Step 3 ((ii) and (iii)). (ii) and (iii) are rules of the compilation map, stated here because they are the behaviour the gate exists to produce. (ii)'s precedence clause is part of that rule and is what makes the table fix *one* outcome for each declaration: one declaration can meet the premise of a refusal row and of an emission row at once — a certificate failing a condition of (G2), row 3, beside a list carrying an entry declared general, row 5 — and the clause takes the refusal, so no declaration whose supplied data failed a check or a certificate is emitted. (ii) is consistent: the one non-refusal outcome is exactly the one for which this contract proves the result sound — an ineligible declaration is not consumed by a key-length statement at all — and no row changes the source mode, re-reads a constraint entry or drops one. No row invokes Proposition 9.6: the rows on which it could bear, the certificate-absent ones, are met by no protocol specification (Step 2). (iii)'s second sentence is Assumption 10.3 read where it is spent: (G2) decides the certificate once and Theorem "Checkability and sufficiency of the source-replacement certificate" (transmitter/thm:certificate-sufficiency) (iii) is a statement about one round's source state, so the flag says of the run only what one declaration can say of every round.

Step 4 (the general format, (i′)). Theorem 6.2 (iv) in its general-format reading gives (G1). For (G2): each condition Definition 9.1 lists for a form is either an identity or an order relation between declared numbers, decided as Definition 1.3 decides such a predicate, or a comparison of labels, integers and structures, or one summation decision — and each list is finite, its length fixed by $|\mathbb{T}|$, $|\mathcal{X}|$ and the declared dimensions. That purity is decided by nothing *for a presentation* is read off Definition "Source-replacement certificate" (transmitter/def:srcert): a presentation declares no Ψ, and purity, (SR1) and (SR2) are conclusions of the transmitter's lemma, so no function of the emitted data is asked for them. Three separate statements describe what the gate then demands. *First*, the five conditions are decided of every

certificate declared in pure form, and at one Alice sector — the only case a declaration in the basic format can present — they *hold* of the Ψ a presentation defines, by Lemma "Certificate defined by an ensemble presentation" (transmitter/lem:ensemble-presentation) (ii) with unit total weight. *Second*, a presentation is admitted only beside an *ensemble-form* declaration, which no basic-format declaration is; and a certificate declared in *block form* is refused here (Definition 9.1). *Third*, at $|\mathbb{T}| \geq 2$ the defined $\Psi = \bigoplus_t \Psi_t$ has $\Psi^2 = \bigoplus_t g_t \Psi_t \neq \Psi$ once two blocks are non-zero, so global purity is not demanded of a presentation there, and what the gate rests on in its place is what Remark 9.2 names — rank-one blocks by theorem, giving both directions of Theorem "Checkability and sufficiency of the source-replacement certificate" (transmitter/thm:certificate-sufficiency) (ii), and that theorem's (iii), which reads purity nowhere. A general block form lacks the first and has the second, that theorem being stated in every form; the refusal of a block form rests on scope and not on that difference, and the distinction is not the word "purity" (Remark 9.2). For (G3)(a): by Definition 2.5 the licensed class is defined by which of the three forms an entry *declares*, the first two by the type of the datum and the third by the tag field beside a real scalar, so deciding the class is reading m tags and checking that the datum each names has the type it names — m comparisons of a label and a type, terminating and exact, and again nothing computed: no form of the general format introduces a numerical comparison.

Step 5 (the general format, (ii′)). (ii′) is a statement about where each failure lands, and it is proved by exhibiting the row: a failing check is (G1) by the definition of that gate, and (G1)'s row is a refusal naming the check, so every failing check of the general format lands on that row; a certificate present and failing a condition of (G2) is the premise of row 3, its outcome a refusal naming the condition. *A block-form certificate meets that premise by its form*: the record is present, and the condition of (G2) it fails is that (G2) admits no certificate declared in block form (Definition 9.1), which holds of every block-form certificate whatever its blocks — including one whose one non-zero block is a rank-one projector and which therefore meets $\Psi^2 = \Psi$ — so the outcome is a refusal naming the form. *An ensemble presentation beside a POVM-form declaration meets the premise too*: the record is present, and the condition of its form that it fails is the one requiring the declaration's Alice slots to be *the* slots the presentation defines, which a POVM-form declaration has not got — so the outcome is a refusal, and the refusal names which condition failed. No premise of any other row is met by a general-format failure, and each of the six rows keeps its outcome; in particular no general-format failure yields a key-length-eligible declaration, and none yields an emission on a row whose outcome is a refusal.

Step 6 (the general format, (iii′)). The added item is the soundness condition of a party-local entry (Definition 2.5; that the party-local form is sound only under it is Remark 2.6), read where the record is tested: every

attack reaches the scoped factor through a channel on that factor alone carrying the spectral certificate, which is a certificate of the module that produces the chain (Definition 9.1), and the gate tests for the presence of the record that attests it. *So admitting the third form at (G3)(a) moves no decision from an attestation to this module and none from this module to an attestation*: it adds one licensed tag and one clause to what the same record attests. The rest of (iii) holds as stated, including where Assumption 10.3 is spent. □

In untrusted-source mode gate (G3)(a) holds vacuously, so the constraint gate rests on the attack-set licence alone. At dimension one (G3)(a) is still a type check, so a general-tagged 1×1 entry is unlicensed, although it could always have been declared homogeneous.

Gate (G3) of Definition 9.1 reads as two checks, a type check on the list and a presence test for the licence. This remark says what the type check amounts to in untrusted-source mode and at $d_A = 1$; what the gate report records of a vacuous pass is Requirement 14.4.

Remark 9.5 (The licensed-class gate in untrusted-source mode and at $d_A = 1$). Gate (G3)(a) is a genuine restriction in trusted-source mode: an entry declared general fails it, and such an entry is exactly what an upstream module emits when it has an operator right-hand side rather than a scalar one. It is *vacuously true* in untrusted-source mode at any d_A, where every right-hand side is already a scalar by D(vi), so the gate degenerates there from a restriction into a type check that a well-formed declaration passes by construction, and rests entirely on (G3)(b), which no program can verify.

In trusted-source mode at $d_A = 1$ the gate is not vacuous. Every 1×1 Hermitian Θ_k is $\Theta_k \cdot [1]$, but under the declared tag an entry tagged general declares a matrix and fails a type check at every dimension: (G3)(a) is a type check at every d_A, so such an entry is unlicensed (Definition 2.5), and the consuming engine reads its gate the same way (Remark "Degeneracy of the licensed-class half of the gate" (meat-core/rem:cgate)). A declaration carrying one is emitted, not key-length-eligible. Its producer could have declared the entry homogeneous with the same condition, since at $d_A = 1$ the constructed $c_k \tau_A = c_k[1]$ is the scalar itself, so there the gate rejects nothing a producer cannot trivially avoid.

Used by Req. 14.4 · newly derived · accepted

Dropping a trusted marginal and reading homogeneous constraints as scalars enlarges the feasible set, for every admissible marginal, so it can only lower a minimised objective. It compares the two source modes and is no rule of emission.

A trusted marginal is one more condition on the states the adversary may hold. Dropping it, and reading each homogeneous entry as its scalar, can only admit more states; the proposition states this as an inclusion of feasible sets, and Remark 9.7 says what it licenses and what it does not.

Proposition 9.6 (Dropping a trusted marginal enlarges the feasible set)**.** Let a specification request trusted-source mode with marginal τ_A satisfying D(ii), and a licensed list $\{(\hat{O}_k, \bowtie_k, c_k)\}_{k=1}^{m}$, $m \geq 0$, satisfying D(iii) — every

$\hat{O}_k$ a block list against $\mathfrak{s}$ and Hermitian block by block, which the proof spends through Lemma 2.2 — with every entry declared homogeneous, so that the trusted-mode right-hand side it denotes is the constructed $c_k \tau_A$ (Definition 2.5). Let $\mathcal{W}^{\mathrm{tr}}$ be the feasible set of the trusted-mode declaration and $\mathcal{W}^{\mathrm{un}}$ that of the untrusted-mode declaration obtained by replacing the marginal slot with none and reading each entry's right-hand side as the scalar c_k, all other slots unchanged. Then

$$\mathcal{W}^{\mathrm{tr}} \subseteq \mathcal{W}^{\mathrm{un}}, \tag{9.1}$$

and consequently, for *any* functional J on $\mathcal{L}(AQ)$, $\inf_{\mathcal{W}^{\mathrm{un}}} J \leq \inf_{\mathcal{W}^{\mathrm{tr}}} J$. The inclusion is *uniform in* τ_A: it holds for every τ_A satisfying D(ii), and the untrusted set does not depend on which one the request carried.

Uses Def. 1.3, Def. 2.1, Lem. 2.2, Def. 2.5 · *Used by* Rem. 9.7 · newly derived · accepted

Proof. Let $\omega \in \mathcal{W}^{\mathrm{tr}}$. The block presentation and $\operatorname{tr}\omega = 1$ are shared by the two definitions, as are the per-block positivity conditions; the untrusted marginal condition is empty, and a set defined by a conjunction is contained in the set defined by any sub-conjunction, which settles $m = 0$ outright and for every τ_A.

For $m \geq 1$, fix k and consider first $\bowtie_k$ equal to $\succeq$, so that $\operatorname{tr}_Q[(\mathbb{1}_A \otimes \hat{O}_k)\omega] \succeq c_k \tau_A$. Both sides are Hermitian — the left by Lemma 2.2, using $\hat{O}_k$ Hermitian from D(iii) and ω Hermitian from (2.1), and the right because c_k is real and τ_A is Hermitian by D(ii) — so the Löwner comparison is well posed and the traces below are real. Taking traces of both sides and using that $Y \succeq Y'$ implies $\operatorname{tr} Y \geq \operatorname{tr} Y'$ (the trace of a positive semidefinite operator is non-negative, applied to $Y - Y'$),

$$\operatorname{tr}\big[(\mathbb{1}_A \otimes \hat{O}_k)\,\omega\big] = \operatorname{tr}\Big(\operatorname{tr}_Q\big[(\mathbb{1}_A \otimes \hat{O}_k)\,\omega\big]\Big) \;\geq\; \operatorname{tr}(c_k \tau_A) = c_k \operatorname{tr}\tau_A = c_k,$$

using $\operatorname{tr}\tau_A = 1$ from D(ii) — *an identity of the declared numbers, which is what Definition 1.3 delivers and what condition (S) on the trace is there to make true* — and the fact that a partial trace followed by a trace is the full trace. This is the untrusted entry with $\geq$. The case $\bowtie_k$ equal to $\preceq$ is identical with the inequality reversed, and the case $=$ follows by applying both. Hence $\omega \in \mathcal{W}^{\mathrm{un}}$, which is (9.1). The infimum statement is immediate: an infimum over a superset is not larger than an infimum over a subset. *Uniformity in* τ_A: the only property of τ_A used is $\operatorname{tr}\tau_A = 1$, which D(ii) gives for every admissible marginal, and no τ_A appears in $\mathcal{W}^{\mathrm{un}}$ at all.

At $d_A = 1$ the inclusion is an equality. There $\tau_A = [1]$ is the only marginal and $\omega_A = \tau_A$ is the condition $\operatorname{tr}\omega = 1$, which $\mathcal{W}^{\mathrm{un}}$ already imposes; and every licensed entry's constructed right-hand side $c_k[1]$ is the scalar c_k itself, so the two constraint conditions coincide entry by entry. Hence the

two feasible sets are equal and the two emissions are the same declaration (Remark "The one-dimensional retained system" (transmitter/rem:da1) states the device-side half of this). □

Proposition 9.6 licenses dropping a trusted marginal, which can only lower a minimised quantity, and never adding or weakening one; no reachable gate row uses it. At d_A ≥ 2 the maximally mixed marginal is strictly stronger than no marginal, and at d_A = 1 it is [1], which excludes nothing.

Remark 9.7 (Dropping a marginal, never replacing it). *The direction of the licence.* Proposition 9.6 licenses *dropping* a marginal, never adding or weakening one. Emitting the untrusted-mode declaration when the source-replacement attestation is absent is therefore a change in the safe direction for any downstream quantity obtained by minimizing over the feasible set: dropping the marginal *lowers* any minimized quantity, so it can only cost rate; it can never turn an unsupported declaration into a supported-looking one. A consumer that drops a declared marginal — the transmitter's untrusted-source mode, which is always available — is doing exactly what the proposition settles, in the direction it says.

Where the compiler uses it. The compiler exercises that licence on no row of the gate outcome table (Definition 9.3) that fixes the outcomes of Theorem 9.4 (ii): the certificate-absent rows are unreachable through the transmitter artefact, no protocol specification meeting their premise (Theorem 9.4 (ii)), the transmitter refusing a trusted-mode request at $d_A \geq 2$ without a certificate before this module sees it (Requirement 14.2), and the proposition stands as a statement about the feasible sets, not as a rule of emission. On the certificate-absent row with $m \geq 1$ the inclusion is *not* what is missing, since it holds at every m: what is missing is a referent for the attestation, the module that produces the chain having certified its entries against $c_k\tau_A$, and with no declared τ_A that attestation (Assumption 10.6, named and not used here) has none.

No neutral marginal. In particular, at $d_A \geq 2$, $\tau_A = \mathbb{1}_A/d_A$ is not a neutral way to say "nothing is known about the source": it fixes the marginal to maximally mixed and excludes every ω whose A-marginal differs from it at all, which is strictly stronger than none, which imposes no marginal condition. The compiler never substitutes it, filling no slot (Requirement 11.3), and at $d_A \geq 2$ neither does the transmitter (Assumption "Retention of the system A in trusted-source mode" (transmitter/ass:src), Remark "The two source modes" (transmitter/rem:source-mode)). At $d_A = 1$ the maximally mixed marginal is [1], the only state on A, and it excludes no normalized ω, so it imposes the same condition as none (Remark "The one-dimensional retained system" (transmitter/rem:da1)); it is the one value the transmitter fills into an empty marginal slot (Requirement "The unique marginal served at $d_A = 1$" (transmitter/req:marginal-fill)), and the compiler passes it through unchanged.

Uses Thm. 9.4, Prop. 9.6 · newly derived · accepted

10 Assumptions

The declared data, the declared measurements included, describe the apparatus: the physical receiver measurements, in every round and with no

memory of the others, and, where no source-replacement certificate is demanded, Alice's are the ones performed. The key rule yields a key value on every input; where no certificate is demanded Alice measures as declared in every round; one declaration describes every round; public choices are private coins of declared bias, and the Band-A receiver is an ideal qubit measurement. The soundness of each reduction link and the attack-set licence are obligations of the module that presents the chain. The compiler decides every condition of a well-formed specification, and its presence checks establish none of these assumptions.

Imports foundations/def:round-homogeneity, transmitter/def:source-segment

Declared dimensions, alphabets, tables and measurements describe the apparatus: the receiver's in every round whatever the others did and, where no source certificate is demanded, Alice's of two or more letters; every input yields a key. The compiler checks only the last, in part.

The compiler decides, on the declared numbers, every condition of a well-formed specification. What no function of the declared data decides is whether those data are the apparatus: a mis-declared dimension, or a measurement that is complete as written but is not the one performed, passes every check. Where a source-replacement certificate is demanded, in trusted-source mode at $d_A \geq 2$, Alice's measurement is tied to the source through that certificate, and clause (i) says nothing of it. On the receiver side clause (i) includes that the measurement keeps no memory from round to round, on every route, a directly declared family and a chain from elsewhere included: no declared datum describes more than one round. The key rule's codomain is decided only in part: check V5 refuses a key rule that withholds a key on part of a passing class, and not one that withholds it on the whole of such a class.

Assumption 10.1 [A-SPEC] (Specification faithful to the apparatus)**.** For a protocol specification $\mathfrak{S}$ (Definition 3.1): (i) the declared dimensions, alphabets and tables of $\mathfrak{S}$ describe the apparatus, and so do its declared measurements, namely: the physical receiver family $\{F_w^{\mathrm{ph}}\}_{w \in \mathcal{V}^{\mathrm{ph}}}$ (Definition 3.1 (b); in the general format, each measured factor's family of (b′)) is the measurement performed, in every round, on that round's physical receiver system alone, whatever happens in the other rounds, their outcomes and public messages included; and where no source-replacement certificate is demanded of $\mathfrak{S}$ (in untrusted-source mode, or at $d_A = 1$), $\{M_x\}$ (Definition 3.1 (a)) is the measurement Alice performs on A; (ii) the key rule r_{h}^0 is valued in $\mathcal{Z}$.

The engine does not verify this assumption; a run that rests on it records it as unverified.

Reading: Your declared numbers describe your apparatus: each system has its declared dimension, every alphabet and table (announcements, sifting, the raw key bit) is what your devices and software do, and the key rule yields a key value, never "no key", on every input. Each measuring station, of either party, performs exactly its declared measurement

in every round, on that round's input alone; afterpulsing or a dead time violates this. A declared receiver instrument's measurement is the model the engine emits from it; the emission step is recorded, not bounded. Where no source-replacement certificate is demanded (an untrusted source, or a one-dimensional A), Alice's declared measurement of her letter is the one she performs; A-ALICE-MEAS adds every round, alone. The engine checks that the declaration is well formed, not that it matches your hardware.

Uses Def. 3.1 · *Category* protocol structure · *Affects* secrecy · *Assumed by* Thm. 3.4, Thm. 3.7, Thm. 5.1, meat-core/thm:keylength · *Machine key* `specification_describes_apparatus` · newly derived · accepted

Where no source-replacement certificate is demanded, in untrusted-source mode or at d_A = 1, and at least two letters are declared, Alice's letter in every round is the outcome of her declared measurement on that round's system A alone, whatever happens in the other rounds.

The condition narrows Assumption 10.1 to Alice's measurement on A where no source-replacement certificate is demanded: clause (i) there makes $\{M_x\}$ the measurement Alice performs on A, and this condition makes it so in every round separately. It constrains the measurement and not the systems measured: in untrusted-source mode the systems A of different rounds may be correlated, the adversary supplying them.

Assumption 10.2 [A-ALICE-MEAS] (Alice's declared measurement where no source certificate is demanded)**.** (i) Where no source-replacement certificate is demanded of the protocol specification (in untrusted-source mode, or at $d_A = 1$), the letter $x \in \mathcal{X}$ Alice obtains in every round is the outcome of the declared measurement $\{M_x\}$ (Definition 3.1 (a)) applied to that round's system A alone, the measurement she performs on A being $\{M_x\}$ whatever happens in the other rounds, their outcomes and public messages included.

The engine does not verify this assumption; a run that rests on it records it as unverified.

Reading: Applies where no source-replacement certificate is demanded and at least two letters are declared. Where the system A is one-dimensional, the declared measurement is only the probabilities of Alice's letter, which is then her local randomness; with a trusted source the engine refuses letter probabilities that differ from the source's and a sent state that depends on the letter. With an untrusted source and A of dimension two or more, Alice measures at her station the system A she receives. In every round she obtains her letter by the same declared measurement on that round's system alone, whatever happens in other rounds, their outcomes and the public messages included. Systems from an untrusted source, being the adversary's, may be correlated across rounds. A device of Alice's declared as a measured factor is a receiver measurement of the specification, not covered here.

Uses Def. 3.1 · *Category* protocol structure · *Affects* secrecy · *Refines* A-SPEC · *Assumed by* meat-core/lem:honest-attacks, meat-core/thm:keylength · *Machine key* `alice_measurement_as_declared` · newly derived · accepted

The same specification, source mode and operators describe every round: the sameness clause of round homogeneity, not the absence of memory across rounds. One compiled declaration and one certificate decision then serve the whole run.

Rounds are not averaged and per-round variation is not modelled: one declaration stands for every round. Only sameness is asserted; that the devices have no memory across rounds, the second clause of round homogeneity, is not part of this assumption.

Assumption 10.3 [**A-RHOM**] (Round homogeneity of the declaration)**.** (i) Every round of the protocol is described by the same specification, hence by the same declaration, the same source mode and the same operators: the family of the specification, the declaration, the source mode and the operators satisfies clause (i), sameness, of round homogeneity (Definition "Round homogeneity of a per-round family" (foundations/def:round-homogeneity)).

The engine does not verify this assumption; a run that rests on it records it as unverified.

Reading: One protocol description covers the whole run. Every round uses the same measurements, the same setting choices and their probabilities, and the same classical processing, and, where the source is trusted, the same source, so one declaration and one certificate decision apply to all n rounds. A run in which you change settings, or your devices drift so that later rounds need different operators, violates this. Only parts of a run fixed before it starts, each processed as its own execution and composed under A-SEPARATE, which also asks that no device carry a state from one part into the next, can be analysed separately; a split chosen after seeing the data is not covered. That no device remembers earlier rounds is not part of this premise: for the measuring stations it is A-SPEC, for Alice's letter A-ALICE-MEAS, and where the source is trusted the source premise.

Uses foundations/def:round-homogeneity · *Category* protocol structure · *Affects* secrecy · *Assumed by* Thm. 9.4, meat-core/thm:accumulated-entropy · *Machine key* `round_homogeneity_declaration` · newly derived · accepted

Every public choice is drawn with its declared bias; those of fixed bias are jointly independent of each other, the quantum systems and the adversary, and one weighted by a station's setting depends on the round through that setting only. It lets a weight inside the measurement describe the round.

A public choice of known bias enters the declaration as a weight inside the measurement operators, and that weight describes the round only if the choice is drawn as declared. For a choice f weighted through the private setting of the station that measures a system, the conditional independence given t in clause (ii) is what makes the only dependence of f on the round its dependence through t: a value whose weight varied with the device outcome would be an inference from that outcome and not a choice, and such a weighting is refused by type.

Assumption 10.4 [**A-COIN**] (Public choices as private coins of declared bias)**.** (i) The declared public choices $t \in \mathcal{T}_{\mathrm{pub}}$ are drawn from private random sources with their declared weights q_t, mutually independent, whichever party draws each, and, taken together, independent of the quantum systems

and of the adversary; (ii) a public choice f declared with a setting-dependent weighting $P(f \mid t)$ — the setting-dependent form of Definition 4.1, which may be declared at any measured factor — is drawn in each round by a party whose readout records the setting component t of the outcome, from a private random source with the declared conditional distribution $P(\cdot \mid t)$ given that round's setting t; given t, f is independent of the quantum systems, of the letter and every outcome of the round (the device outcome y included), of every other declared public choice and of the adversary, taken together; (iii) every such choice is announced, or withheld, exactly as the public structure declares.

The engine does not verify this assumption; a run that rests on it records it as unverified.

Reading: Every public choice of your protocol (for example an actively chosen basis, or whether a round is a test round; a basis set passively by a beam splitter is part of the measurement, not a choice), whichever party draws it and at whichever of its stations, comes from a local random number generator with exactly the declared probabilities, and each is announced or kept secret exactly as declared. The choices with fixed probabilities are independent of each other and, all together, of the quantum signals and of anything an attacker does. A choice whose probabilities depend on a station's private setting is drawn by a party whose recorded outcome includes that setting, and may depend on that setting only, never otherwise on Alice's letter, another choice or a detection result.

Uses Def. 1.1, Def. 4.1 · *Category* local randomness · *Affects* secrecy · *Assumed by* Thm. 4.4, Rem. 4.5, Rem. 4.8 · *Machine key* `public_choice_weights` · newly derived · accepted

A state-dependent marginal of a setting-conditioned public choice is not a violation of A-COIN: it is the output-state model of the reductions module. A violation is repaired by no encoding, only by a different protocol description.

Remark 10.5 (Scope of the private-coin assumption). No encoding repairs a violation of Assumption 10.4; the remedy is a different protocol description. Under a violation the weighted POVM misdescribes the round: for the setting-independent form (4.5) no longer gives the physical choice statistics, and for the setting-dependent form the relabelled family of Remark 4.8 no longer describes how f was drawn; a choice correlated with the adversary, or with the device outcome beyond its setting, is exactly the attack the encoding assumes away. The state-dependent *marginal* of f that the setting-dependent form of Definition 4.1 produces is not a violation: it is the output-state model of Assumption "Output-state test statistics" (reductions/ass:tsm), and Lemma 4.2 (iii) is asserted for the setting-independent form only (Theorem 4.4).

Uses Def. 4.1 · newly derived · accepted

Every link of the reduction chain is sound: its channel exists and is a channel, the reduced protocol's security implies the physical one's, and every transported constraint holds on reachable states. A decided pullback or source identity is a necessary condition only, never this obligation.

The compiler decides the pullback identity and the source identity (S3) exactly where the data make them decidable; those are necessary conditions, and not this obligation. The obligation belongs to whoever presents the

chain, a channel supplied as data included. On the Band-A identity link, given as data with the identity outcome correspondence and the directly declared family as its target, every clause holds by exhibition, and on a single instance link the reductions module emits with status exact its own theorems prove every clause; there nothing is left to attest. The published templates are [1, Lemma 9.3, Eqs. (146)–(147)] for the squashing case and [1, Lemma 9.1, Eqs. (138)–(139)] for the source case, composed in the order of [1, Rem. 9.2].

Assumption 10.6 [A-LINK] (Soundness of every reduction link)**.** Each link of either segment of the reduction chain is sound: (i) for a *receiver* link, (a) where the link supplies no channel as data, only a theorem that constructs one, its channel exists as claimed; (b) it satisfies the pullback identity (7.1); and (c) security of the protocol using the emitted measurement implies security of the protocol using the physical one; (ii) for a *source* link, (a) the channel of the source identity (S3) of Definition "Source segment and its interface conditions" (transmitter/def:source-segment) exists as claimed; and (b) security of the protocol preparing the virtual ensemble implies security of the protocol preparing the specified one; (iii) for a link of either segment, every reachable state after the reduction satisfies every constraint entry the link transports; (iv) a link of either segment that supplies its channel *as data* supplies a completely positive and trace-preserving map.

The engine does not verify this assumption; a run that rests on it records it as unverified.

Reading: Every reduction step between your physical devices and the finite model the engine solves is sound, and so is their combination. Each step's channel exists, a channel supplied as numbers is a genuine quantum channel, and a receiver step maps the model's measurement back to the physical one. Security of the reduced protocol implies security of the physical one, and every constraint a step passes on holds for every state an attacker can reach. The run lists this premise wherever it cannot point to a proof: it is not listed on the idealised Band-A route, whose one step is checked to be the identity, nor where every measuring device's single reduction step was built in this run by the engine's reductions module with status exact, with no source step and outcomes kept as they are, where that module's theorems prove it. A chain or channel from elsewhere must come with the proof.

Uses Def. 7.1, transmitter/def:source-segment · *Category* supplied artefact · *Affects* secrecy · *Assumed by* Thm. 4.4, Thm. 7.5, meat-core/thm:keylength · *Machine key* `reduction_chain_sound` · newly derived · accepted

For a licensed non-empty constraint list that backs a key-length claim, its producer's reduction represents every physical attack with finite registers and every signal at its first channel by an admissible attack sequence. It lets constraints restrict the attacks considered.

The licence is the attack-set licence of the module that produces the chain, and it is that module's obligation. The published template for such a certificate is [1, Lemma 9.3, Eqs. (146)–(147)] (squashing maps: the squashed POVM pulls back to the physical one and every composed physical attack lands in the restricted set), applied in the order of [1, Rem. 9.2] (source

maps before squashing restrictions). No part of the condition is specific to trusted-source mode: the licensed class and the admissibility format follow the declaration's source mode.

Assumption 10.7 [A-ATK] (Attack-set licence)**.** For a non-empty constraint list $\mathfrak{L}$ that is licensed for the declared source mode (Definition 2.5) and backs a key-length claim: (i) the reduction chain that produced the declaration certifies the *licensed reduction*: security of the physical protocol reduces to security of the consuming engine's generated round model (Definition "Round model and licensed attack sets" (meat-core/def:roundmodel)) with attack sequences restricted to the $\mathfrak{L}$-admissible ones, that is, every attack available against the physical protocol whose registers are finite-dimensional and whose first channel receives every transmitted system is represented, after the reduction, by a sequence whose per-round channel A_j lies, for every round j, in the restricted set $\mathfrak{A}_{\mathfrak{L}}(E_{j-1}, QE'_j)$ of that definition, with no A output, in trusted-source mode, and in its untrusted-source-mode counterpart, with A supplied by the adversary, in untrusted-source mode.

The engine does not verify this assumption; a run that rests on it records it as unverified.

Reading: Applies only where constraints restrict the attacks the analysis considers, each in the form your declared source mode admits, which the engine checks. Whoever produced the reduction chain has proved that every attack on your physical system becomes, after the reduction, an attack that respects every listed constraint in every round. The proof is asked of attacks with a finite-dimensional memory, of any size, that receive every signal at the start; the engine extends the security statement to every other attack by a limit (Corollary "Security of the protocol and its net key" (meat-core/cor:composed-security)). For constraints the reductions module produces and certifies, that proof is its licence theorem. A constraint you add yourself, or one supplied without such a certificate, needs that proof from you, or must be removed; the analysis without constraints stays valid.

Uses Def. 2.5 · *Category* supplied artefact · *Affects* secrecy · *Assumed by* Thm. 9.4, meat-core/cor:constrained-key-length, meat-core/ex:fss, meat-core/lem:range, meat-core/rem:cgate, meat-core/rem:no-constraints, meat-core/rem:not-licensed, meat-core/thm:constrained-accumulation, meat-core/thm:keylength · *Machine key* `attack_set_license` · newly derived · accepted

On the Band-A route the declared receiver is an exactly projective qubit measurement with unit efficiency and no loss, dark counts, multi-photon events or misalignment, on a transmitted system that is exactly a qubit. The route is an explicit idealization, not a squashing argument.

Used to reproduce known comparison values, a Band-A declaration is sound, and its idealization is visible in this assumption. It is an idealization, not a reduction: no squashing argument is made and none is needed, because nothing infinite-dimensional is being reduced.

Assumption 10.8 [A-IDEAL] (Ideal qubit receiver on the Band-A route)**.** In a Band A specification: (i) the declared receiver is exactly an ideal projective qubit measurement; (ii) it has unit detection efficiency, no dark counts, no loss, no multi-photon events and no misalignment; (iii) the transmitted

system is exactly a qubit.

The engine does not verify this assumption; a run that rests on it records it as unverified.

Reading: Only for an idealised Band-A analysis: your receiver is a perfect qubit measurement, with unit detection efficiency, no loss, no dark counts, no multi-photon events and no misalignment, and what reaches it in each round is exactly one qubit. The engine refuses a Band-A declaration whose own numbers contradict this: more than one measuring station, a measured system (or, with a trusted source, a transmitted system) of dimension other than two, or a measurement not made of projectors (per basis, where a basis choice is declared). No real optical receiver meets this; use the route to reproduce textbook comparison values. Declare a real device through the receiver module and a squashing reduction.

Category receiver · *Affects* secrecy · *Assumed by* Thm. 8.2 · *Machine key* `ideal_qubit_receiver` · newly derived · accepted

Part II
Engine requirements

What an implementation of this module must do, record or refuse; each requirement names the result that motivates it.

11 Requirements: the emitted declaration

The engine emits the declaration under its machine keys, with the index map beside it, and holds every operator on the measured system as its block list. It passes the declared data it reads through unchanged, forms none of the derived objects of the general format as data, and decides every predicate per measured factor and per Alice sector on declared blocks. It reads each constraint tag off its datum or off the field it arrives with, infers none, and decides nothing at a window. With the declaration it passes on the transmitter's source-replacement licence record where one is demanded, and records the choice weighting beside the chain record. At a one-dimensional A in trusted-source mode it refuses a specification whose source emission depends on the letter or whose letter weights differ from Alice's measurement.

Imports transmitter/def:source-segment, transmitter/def:srcert

The sector map is emitted as three keys, its identifiers, their dimensions and its block kind; the index map goes beside the declaration and each constraint entry's tag with the entry. The sifting predicate and the key-live set are derived from the specification and are not emitted.

Definition 1.1 makes the sector map one object and derives the sifting predicate and the key-live set from the specification; this requirement says which of these the emitted declaration carries.

Requirement 11.1 (Machine keys of the emitted declaration). The engine emits the sector map $\mathfrak{s}$ of a protocol declaration as three machine keys, `sector_index_set` for its ordered block identifiers, `sector_dims` for their dimensions and `sector_block_kind` for its block kind, so that a consumer comparing sector maps can compare all three and not two of them. It emits the label-to-index map idx of Definition 3.3 alongside the declaration, under `label_index_map`, and with each constraint entry the tag of its right-hand side, under `constraint_rhs_tag`, so that the consuming engine's type check has a datum (Definition 2.5). The sifting predicate ς and the set $\widehat{\mathcal{C}}_{\mathrm{key}}$ of key-live values are not emitted.

Motivated by Def. 1.1, Def. 2.5, Def. 3.3 · accepted · *Code* certiqs_key.protocol_compiler.declaration:CandidateDeclaration

Every operator on the measured system is held and emitted as its block list. Where there are several sectors its direct sum is formed only as the argument of chain condition (C4), decided there and discarded.

By Definition 1.1 an operator on the measured system is its block list, so sector block-diagonality holds by type and the declaration holds no array larger than one sector block.

REQUIREMENT 11.2 (Operators on the measured system held as block lists). The engine holds and emits every operator on Q as its block list against the sector map $\mathfrak{s}$ (Definition 1.1). Where $|\mathbb{S}| > 1$ it does not materialize the direct sum $\bigoplus_s F^{(s)}$ of such a list, with one exception: condition (C4) of Definition 7.1 forms it as the argument of that one decision, decides it and discards it. The convention `operator_form` states this rule and its scope.

Motivated by Def. 1.1, Def. 7.1 · accepted

The engine passes the declared data it reads into a declaration through unchanged: the marginal slot, Alice's measurement slot in its declared form, the transcript, and each entry's scope and right-hand side. It fills no slot, constructs no right-hand side and converts no form.

The compilation map of Definition 3.3 reads declared data from the transmitter artefact, the chain and the specification, and constructs only the raw-key map, the index map and the weighted families. Data passed on unchanged keep their declarer (Definition "Declared data and the parties to a module boundary" (foundations/def:declared-data) (iii)), and a consumer repairs nothing it was handed (Requirement "Exact decision by every consumer" (foundations/req:consumer-decides-exactly)).

REQUIREMENT 11.3 (Declared data passed through unchanged). The engine passes through unchanged the declared data that the compilation map reads into a declaration: the transmitter artefact's marginal slot, the value [1] the transmitter filled it with at $d_A = 1$ included; Alice's measurement slot, in the form the artefact declares, which it changes only by weighting it in that form (Definition 4.1) where a choice family is presented; the transcript; and each constraint entry's right-hand-side datum and, where the entry carries one, its scope. It fills no slot and demands no certificate of its own. It constructs no right-hand side of an emitted constraint entry: it forms no $c_k\tau_A$ for one and reads no c_k off any matrix. It converts neither form of Alice's measurement slot into the other: it forms neither (1.7) from an ensemble slot nor $\tau_A^{1/2} M_x \tau_A^{1/2}$ from a POVM slot.

Motivated by Def. 1.1, Def. 3.3, Def. 4.1, foundations/def:declared-data, foundations/req:consumer-decides-exactly · accepted · *Code* certiqs_key.protocol_compiler.compile:compile_specification

The product family, the lifted operators and the family an ensemble form denotes are rules of the general format and never data: nothing forms, rounds or decides them. Every exact predicate is decided per measured factor and per Alice sector on declared blocks.

In the general format of Definition 1.1 the product family, the lifted operators and the POVM an ensemble form denotes are defined by rules from declared data; what is declared, and decided, is the data the rules read.

Requirement 11.4 (Per-factor decisions on the declared blocks). For a declaration in the general format of Definition 1.1: no entry of a product family (1.5) is formed as a datum, rounded or decided on, here or downstream; the lifted operator of a scoped entry is never formed as a datum; and the family (1.7) that an ensemble form denotes is formed by no one and decided by nothing. Every exact predicate is decided per measured factor, on that factor's blocks and in that factor's declared order, and on Alice's data per Alice sector (Definition 1.3). The engine refuses by type a presentation of a choice-weighted family indexed by a product alphabet (Definition 4.1).

Motivated by Def. 1.1, Def. 1.3, Def. 4.1 · accepted

The tag of an untagged entry is read off the type of its datum, and an arriving tag field is checked against that type; the party-local tag is read off the field it arrives with. No tag is inferred and none is taken on the upstream module's word.

For the two untagged forms of Definition 2.5 the tag is the type of the right-hand side; the party-local form is the one form the type does not separate from another.

Requirement 11.5 (Reading the tag of a constraint entry). The engine reads the tag of an entry of one of the two untagged forms of Definition 2.5 off the type of its right-hand-side datum. Where such an entry arrives carrying a tag field, check V6 of Definition 6.1 compares that field with the type of the datum it names and refuses a disagreement; the tag is never taken on the upstream module's word against the object the entry holds. An entry is read as party-local exactly when it arrives carrying that tag; check V6 then requires its right-hand-side slot to hold a real scalar and its scope to name a measured factor of the emitted sector map, and refuses otherwise. No tag is inferred: an entry arriving without the party-local tag is never read party-locally, whatever module emitted it.

Motivated by Def. 2.5, Def. 6.1 · implemented (effective: accepted) · *Code* certiqs_key.protocol_compiler.validation:check_v6

The engine declares no numerical tolerance, so no check of the validator reads a window and nothing in this contract is decided at one. Licensing and the admission predicates are decided exactly.

A consumer decides every admission predicate on declared data exactly and reads no tolerance (Requirement "Exact decision by every consumer" (foundations/req:consumer-decides-exactly), Requirement "Scope of the declared tolerance" (foundations/req:tolerance-scope)).

REQUIREMENT 11.6 (Exact decisions without a tolerance). The engine declares no numerical tolerance, and there is no window in this contract at which anything could be decided: no check of the validator (Definition 6.1) reads a window or is decided at one, and licensing (Definition 2.5) and the predicates of Definition 1.3 are decided exactly.

Motivated by Def. 1.3, Def. 2.5, Def. 6.1, foundations/req:consumer-decides-exactly, foundations/req:tolerance-scope · implemented (effective: accepted) · *Code* certiqs_key.protocol_compiler.validation:validate

A declaration carries the transmitter's source-replacement licence record, re-decided by gate (G2), to the consuming engine exactly when it is emitted in trusted-source mode with Alice's dimension at least two. Its certificate and attestation are the ones supplied.

The record gate (G2) consumes is what licenses a declared marginal at $d_A \geq 2$, and the consuming engine receives it with the declaration rather than from the transmitter.

REQUIREMENT 11.7 (The source-replacement licence passed on with the declaration). The engine passes the transmitter's source-replacement licence record `source_replacement_license` (Definition "Source-replacement certificate" (transmitter/def:srcert)) on to the consuming engine with a declaration exactly when it emits that declaration in trusted-source mode at $d_A \geq 2$, after gate (G2) of Definition 9.1 has re-decided its conditions on the emitted data; the certificate and the attestation (SR3) it carries are the ones supplied, and the engine asserts no attestation of its own (Requirement "Pass-through of the retention attestation" (transmitter/req:attestation-record)). At $d_A = 1$ the record is neither demanded nor emitted (Requirement "Certificate and attestation demanded only at $d_A \geq 2$" (transmitter/req:licence-scope)).

Motivated by Def. 9.1, transmitter/def:srcert, transmitter/req:attestation-record, transmitter/req:licence-scope · accepted · *Code* certiqs_key.protocol_compiler.compile:compile_specification

In trusted-source mode at d_A = 1 each weighted member of the source's ensemble must equal the letter's measurement weight times the summed ensemble, decided exactly, or the specification is refused. Alice's letter weights then equal the source's, and no emitted state depends on the letter.

At $d_A = 1$ no source-replacement certificate is demanded (Remark "The one-dimensional retained system" (transmitter/rem:da1)), and the consuming round model takes Alice's system to be trivial (Proposition "The two source modes coincide at a trivial Alice system" (meat-core/prop:trivial-alice-system), Definition "Round model and licensed attack sets" (meat-core/def:roundmodel)): her letter is the outcome of $\{[m_x]\}$ on a one-dimensional register, independent of every system the adversary holds. In trusted-source mode the source premise (Assumption "Emission of the specified ensemble" (transmitter/ass:ens), or the premise a source model declares in its place) has the source emit ρ_x given the letter, and those systems reach the adversary. The two descriptions agree only where the emission does not depend on the letter and the letter weights agree, which is condition (SR2) of Definition "Source-replacement certificate" (transmitter/def:srcert) read at $d_A = 1$: with A trivial, $\sigma_x = p_x \rho_x^{\mathrm{v}}$ and $\Psi = \sum_y \sigma_y$, the condition $\mathrm{tr}_A[(M_x \otimes \mathbb{1}_{A''})\Psi] = \sigma_x$ reads $m_x \Psi = \sigma_x$. Taking the trace gives $m_x = p_x$, the identity on which Assumption 10.2 and the source premise agree on the letter distribution; on a letter with $p_x > 0$ it gives $\rho_x^{\mathrm{v}} = \sum_y p_y \rho_y^{\mathrm{v}}$, one common state. Both sides are declared numbers, so the engine decides the identity. In untrusted-source mode no source premise is emitted and there is nothing to compare.

Requirement 11.8 (Letter-independent emission at a one-dimensional trusted A). For a specification in trusted-source mode at $d_A = 1$, the engine decides, for every $x \in \mathcal{X}$ in the declared order and exactly on the declared numbers, the identity

$$\sigma_x \;=\; m_x \sum_{y \in \mathcal{X}} \sigma_y ,$$

where m_x is the one entry of Alice's measurement element M_x, read as the declaration carries it (Definition 3.3: for a choice-weighted family the weighted element the engine forms; in the ensemble form, N_x, which equals M_x at $d_A = 1$ by (1.7)), and $\sigma_x = p_x \rho_x^{\mathrm{v}}$ is the weighted member for x of the virtual ensemble the transmitter artefact emits, which at an empty source segment is the specified one (Definition "Source segment and its interface conditions" (transmitter/def:source-segment)); where the artefact emits the ensemble by its weighted members, σ_x is the declared member. Products and sums are formed in exact arithmetic, entry by entry. A specification for which the identity fails at some letter is refused, before a declaration is emitted, with `da1_source_identity_failed`, naming the first such letter in the declared order and the first entry at which the two sides differ.

Reading: With a trusted source and a one-dimensional retained system A, Alice's letter is local randomness that the analysis treats as independent of everything the source sends

out. The engine therefore requires two things of the declaration and refuses it otherwise: Alice's letter probabilities equal the source's, number for number, and every letter of positive probability is sent as the same state. A source whose emitted state depends on the letter is declared with a retained system A of dimension two or more and a source-replacement certificate.

Motivated by Def. 1.1, Def. 3.1, Def. 3.3, Ass. 10.2, meat-core/def:roundmodel, meat-core/prop:trivial-alice-system, transmitter/ass:ens, transmitter/def:source-segment, transmitter/def:srcert, transmitter/rem:da1 · accepted · *Code* certiqs_key. protocol_compiler.declared_premises:decide_da1_source_identity

Both forms of the choice-weighted family are formed, and emitted wherever they are the emitted family. The setting-dependent weighting adds no link and no register to the chain record, which stays unchanged, and is recorded beside it, one entry per weighted factor.

A relabelling composes no channel and so has no place in a chain (Remark 4.8); what the compiler applied is recorded beside the chain instead.

Requirement 11.9 (Emission of the weighted families and the weighting record). The engine emits both forms of the choice-weighted family of Definition 4.1, each where it is the emitted family: the setting-independent form (4.1) on either party's side, and the setting-dependent form (4.3) at a measured system, of whichever party, after that system's receiver segment. A setting-independent presentation of a measuring party's physical receiver family is the emitted family only where that party's receiver segment delivers it as its target POVM; in front of any other segment it is handed to the chain as the physical receiver family of Definition 3.1 (b), and the emitted family is the chain's. For the setting-dependent relabelling it emits no link and no register for the choice value, and the chain record it consumes and re-emits is unchanged by the weighting; the weightings are recorded beside the chain in the emitted artefact, in `choice_weighting_record`, one entry per measured factor at which a weighting is applied, each entry naming that factor; like the setting-independent choice-weighted family, they are this module's own construction.

Motivated by Def. 3.1, Def. 4.1, Rem. 4.8 · accepted

12 Requirements: choice weighting

The engine decides each declared conditional family, per choice value, before it forms any product, and rejects a weight that is not a finite binary64 number as malformed input; decimal or rational user weights may be prepared as binary64 before the module boundary. It refuses a setting-dependent weighting read through anything but the station's setting, decides the weight list and the stochastic matrix exactly, and emits a weighted family only after deciding every product representable and the emitted family exactly admissible. It declares no lattice exponent, snaps nothing, and never emits a fixed fraction outside the measurement operators.

Each conditional family of a choice-weighted presentation is decided exactly admissible, per choice value, before any product is formed or any chain condition decided. A failure is refused with its own reason, naming the first failure.

A choice-weighted family can satisfy every condition of exact admissibility while none of the conditional families it was built from is a measurement (Example 4.7), so each conditional family is decided on its own, on the declared input (Remark 4.6).

Requirement 12.1 (Decision on each declared conditional family). Wherever a choice-weighted presentation is read, on either party's side, the engine decides, of each choice value $t \in \mathcal{T}_{\mathrm{pub}}$ separately and on the declared numbers, the condition Definition 4.1 places on the conditional family $\{N_{u|t}\}_u$ — in POVM form, its exact admissibility in the reading of that definition; in ensemble form, the condition that definition states in its place: (H) and (P) per block, the ensemble identity in place of (C), and (T) — before it forms any product and before any condition of Definition 7.1 is decided on the chain: wherever the presented physical family is the weighted one, that decision and the representability decision on the products it permits precede the consumption of the chain, so that no condition on the chain is decided on a product that was not decided first. A failure is refused with `conditional_family_not_admissible`, naming the declaring party, and the choice value, the predicate, the block and the index of the *first* failure — for (S), the index of the first inexact addition; in ensemble form the predicate named is the ensemble identity where that is the one that failed. The decision stops at the first failure and does not enumerate the rest.

Motivated by Def. 3.3, Def. 4.1, Rem. 4.6, Ex. 4.7, Def. 7.1 · implemented (effective: accepted) · *Code* certiqs_key.protocol_compiler.weights:decide_conditional_families

A declared weight in either form that is not exactly a finite binary64 number is rejected as malformed input before it enters any sum or product. This is the declared input type, not a failed predicate, and it has no refusal string of its own.

The conditions of Definition 4.1 are decided on declared numbers, so the declared weights have to be numbers of the type they are decided on.

Requirement 12.2 (Declared weights as binary64 input). The weights of either form of Definition 4.1 — each q_t of the setting-independent form and each entry of P of the setting-dependent form — are binary64 numbers. The engine rejects as malformed input a declared weight, in either form, that is not exactly representable as a finite binary64 number, before that weight is used in an accumulation or a product. This is the declared input type: the rejection is not a failed summation predicate, and it adds no row of the gate report and no refusal string.

Motivated by Def. 4.1 · implemented (effective: accepted) · *Code* certiqs_key.protocol_compiler.weights:weighted_family

Decimal and rational user weights may be converted to binary64 by the declaring entry point before they reach this module, with the moves recorded and normalisation and bound directions preserved. The preparation asserts no bound on the resulting change of key rate.

This module reads only binary64 weights (Requirement 12.2); ordinary user inputs reach that type through a preparation that belongs to the declarer (Requirement "Snapping by the declarer only" (foundations/req:who-snaps)).

Requirement 12.3 (Preparation of user weights before the module boundary). Ordinary decimal and rational user inputs for the weights of Definition 4.1 may be converted and prepared by the declaring entry point before they cross the module boundary into this module. An entry point that does so records the supplied and the prepared values and their displacement, preserves normalisation and every required bound direction, and leaves this module's exact tests unchanged. The preparation asserts no bound on a resulting change of the key rate.

Motivated by Def. 4.1, Req. 12.2, foundations/req:who-snaps · implemented (effective: accepted) · *Code* certiqs_key.runs.inputs:prepare_probabilities

A setting-dependent weighting presented through anything but the station's setting, such as the outcome alphabet or the device outcome, is refused by type. A value whose weight varied with the device outcome would be inferred from it and would not be a choice.

A value f whose weight varied with the device outcome would be inferred from that outcome and would not be a choice (Assumption 10.4, Remark 4.8); so the weighting reaches an outcome only through its setting component, as Theorem 4.4 (viii) records.

Requirement 12.4 (Setting-dependent weighting read through the setting only). The engine refuses by type, with `choice_weight_depends_on_outcome`, a weighting of the setting-dependent form of Definition 4.1 that is presented indexed by the outcome alphabet $\mathcal{V}$, by the device outcome, or by any component of the outcome other than its setting; in the general format, one presented indexed by a measured factor's whole outcome, by another factor's outcome, or by any component other than that factor's setting.

Motivated by Def. 4.1, Thm. 4.4, Rem. 4.8, Ass. 10.4 · implemented (effective: accepted) · *Code* certiqs_key.protocol_compiler.weights: refuse_outcome_dependent_weighting

The declared weights of the setting-independent form are decided exactly: each positive by its sign, and their sum equal to one in the declared order without rounding. A weight that is not positive is refused by type, and a list whose sum is not one or rounds is refused with one named reason.

The weights of the setting-independent form are declared scalars, and Definition 4.1 reads each of its conditions on them exactly.

Requirement 12.5 (Decision on the declared weight list). The engine decides the conditions of Definition 4.1 on the declared weights $(q_t)_{t\in\mathcal{T}_{\text{pub}}}$ of a choice-weighted presentation: each $q_t > 0$ by its sign, and $\sum_t q_t = 1$ exactly, with the sum accumulated in the declared order of $\mathcal{T}_{\text{pub}}$ — the order in which the declared public-choice record lists it (Requirement "Summation order declared with the data" (foundations/req:declared-order)) — and required to round nowhere. A declared weight that is zero or negative is refused by type. A weight list whose exact sum is not 1, or whose declared-order accumulation rounds at some addition, is refused with `choice_weights_not_normalized`, and no declaration is emitted.

Motivated by Def. 4.1, foundations/req:declared-order · implemented (effective: accepted) · *Code* certiqs_key.protocol_compiler.weights:weighted_family

The declared matrix of a setting-dependent weighting is decided exactly: every entry non-negative by its sign and every row sum equal to one in the declared order without rounding. A negative entry or a row not summing exactly to one is refused with its own reason.

The setting-dependent form is built from a stochastic matrix P, and Theorem 4.4 (viii) states that a P that is not one is refused; its proof gives the reason.

Requirement 12.6 (Stochastic matrix of a setting-dependent weighting). The engine decides the conditions of Definition 4.1 on the declared matrix P of the setting-dependent form: every entry $P(f \mid t) \geq 0$ by its sign, and every row sum $\sum_f P(f \mid t) = 1$ exactly, accumulated in the declared order of the choice alphabet (Requirement "Summation order declared with the data" (foundations/req:declared-order)) and required to round nowhere; the support of the weighting is decided by the sign of each entry. A negative entry, or a row whose declared-order sum is not exactly 1, is refused with `choice_weight_not_stochastic`.

Motivated by Def. 4.1, Thm. 4.4, foundations/req:declared-order · implemented (effective: accepted) · *Code* certiqs_key.protocol_compiler.weights:setting_weighted_family

Every product a weighting forms is decided binary64, emitted or handed on, and the emitted family exactly admissible, before anything is emitted. A failure refuses and emits nothing; an exact product that is binary64 but off the instance's lattice is emitted as it is.

A product of two binary64 numbers need not be one, and a family whose entries are all binary64 may still have a declared-order sum that rounds; both are decided before a weighted family leaves the compiler (Requirement "Exact verification before emission" (foundations/req:exact-emission)).

Requirement 12.7 (Emission decisions on the weighted products). The engine forms the products $q_t N_{u|t}$ and $P(f \mid t(v))F_v$ of Definition 4.1 exactly,

on the declared dyadic numbers, and takes the two emission decisions of that definition before emitting anything. The first, that every entry of every product it forms is a binary64 number, is taken on every product the map forms, whether the product is emitted or handed on: the intermediate families of an iterated presentation are products, and a presentation on Bob's side before the chain produces the physical family that (C2) and (C4) of Definition 7.1 read and that, in Band A, (C3) decides, so no condition of the chain is decided on a product whose entries were not decided first. The second is that the emitted family satisfies Definition 1.3. If either decision fails, the engine refuses — with `emitted_value_not_representable` in the first case and with the predicate that failed in the second — and emits no declaration. `emitted_value_not_representable` covers exactly an exact product of declared numbers that is not a binary64 number: an entry that is binary64 but lies off whatever lattice the instance declared its own data on is emitted, and the five conditions the consumer states are decided on it exactly and on it alone.

Motivated by Def. 1.3, Def. 4.1, Thm. 4.4, Def. 7.1, foundations/req:exact-emission · implemented (effective: accepted) · *Code* certiqs_key.protocol_compiler.weights:decide_emission

The compiler declares no lattice exponent for what it emits, so it never rounds or snaps its emission and never declares a nearby family: a failed emission condition refuses. It owes its consumer the five exact conditions, not a lattice.

Requirement "Exact verification before emission" (foundations/req:exact-emission) binds every producer, and the choice-weighted families are this module's products. Five of its items bind every producer, and for these families they are the five conditions of Definition 1.3 — the sector map, exact Hermiticity, exact positive semidefiniteness, exact completeness per block, and condition (S); a sixth, (D-LAT) of Requirement "The declared lattice condition" (foundations/req:declared-lattice), binds only a producer that declares a lattice exponent $k \leq 53$. That requirement makes a producer that cannot establish the conditions that bind it refuse, naming the predicate; only a producer that declares k may instead declare a nearby family established under all six, carrying three disclosures, and Requirement "Snapping by the declarer only" (foundations/req:who-snaps) lets only such a producer snap its own emission. (D-LAT) is a *sufficient* condition for the five to be establishable and is not among the conditions the consuming engine states, so this module states no (D-LAT) obligation of its own.

Requirement 12.8 (No lattice exponent and no snapping of the emission). The engine declares no lattice exponent for the families it emits. It establishes the five conditions of Definition 1.3 of those families directly, on the numbers it is about to emit, and refuses where one fails (Requirement 12.7); it neither rounds nor snaps its own emission, and it never declares a nearby

family (`no_snapping`). A specification whose weighting does not land exactly on the lattice is a specification to be re-declared at the instance. What the engine owes its consumer is the five conditions, and it decides them rather than inheriting them from a lattice.

Motivated by Def. 1.3, Def. 4.1, Req. 12.7, foundations/req:declared-lattice, foundations/req:exact-emission, foundations/req:who-snaps · implemented (effective: accepted) · *Code* certiqs_key.protocol_compiler.weights:decide_emission

The emitted declaration carries the output-state test-statistics model and never a fixed fraction. A specification requesting a state-independent fraction applied outside the operators is refused.

A state-independent fraction applied outside the operators is unsound, since the terms that carry the weight depend on the state (Remark 4.5), and by Theorem 4.4 (ii) the weight inside the operators reproduces the declared choice statistics exactly wherever the weighted family is the emitted one, so nothing is lost by refusing the fraction; in front of a receiver segment that squashes, the weight sits inside the physical family handed to the chain, and no fraction is emitted either. This is the rule the encoding exists to enforce.

Requirement 12.9 (No fixed fraction outside the measurement operators). The emitted declaration carries the output-state test-statistics model (Assumption "Output-state test statistics" (reductions/ass:tsm)) and never the fixed-fraction model: no path of the engine produces an artefact carrying a fixed-fraction model, and no weight q_t is emitted outside a POVM element. A specification requesting the fixed-fraction encoding of Theorem 4.4 (iv) — the conditional families together with a separate state-independent fraction q_t to be applied outside the operators — is refused with `fixed_fraction_requested`.

Motivated by Thm. 4.4, Rem. 4.5, reductions/ass:tsm · implemented (effective: accepted) · *Code* certiqs_key.protocol_compiler.weights:refuse_fixed_fraction

13 Requirements: validation, feasibility and the chain

The engine refuses a failing check with a reason naming it, repairs nothing, and reports in a check's transcript only the decisions it took. It tests the default feasibility candidate before a supplied witness, emits neither, and never reports a refusal for want of a witness as a defect of the protocol. It decides the presented physical receiver family exactly admissible on every route, and refuses a chain failing one of its conditions where the chain is consumed, before a declaration exists. It takes the Band-A route only for one measured factor whose receiver system, and transmitted system where the source is trusted, is two-dimensional and whose declared family is projective, and only where that factor's one link supplies the identity channel as data,

keeps the outcomes as they are and delivers the declared family. It emits both segments of the chain record and both slots of every link, empty or not, and omits the link-soundness row from its ledger only on Band A and on a single exact instance link the reductions module emitted in the same run, a provenance it never reads from the specification. Where the chain carries a reference companion, it compiles a companion declaration beside the declaration by the same map, decides none of its family's predicates and emits it evaluation-only, with no run of its own.

Imports foundations/def:factor-enclosure, reductions/cor:entries, reductions/cor:reference-companion, reductions/lem:linear, reductions/lem:row-relaxation, reductions/thm:chain, reductions/thm:emit, reductions/thm:emitted-product, reductions/thm:licence, reductions/thm:transfer

A candidate declaration failing a check of the validator is refused, and no declaration is emitted, with a reason naming the check and what that check names. Nothing is repaired and no table is completed with a default value.

A pass of every check of Definition 6.1 is what establishes the target conditions (Theorem 6.2 (i)), and the compiler, as a consumer of what it is handed, decides and never repairs (Requirement "Exact decision by every consumer" (foundations/req:consumer-decides-exactly)).

Requirement 13.1 (Refusals of the validator's checks). The engine runs checks V1 to V8 of Definition 6.1 on every candidate declaration and refuses a candidate that fails one, emitting no declaration, with a reason naming the failing check. It repairs nothing and completes no declared table with a default value. Where a check has a reason of its own, the refusal carries that reason and what the check names with it:

(i) V1: `sector_map_mismatch`, naming the first position at which the compared lists differ, and in the general format, for the block kind across the factors, the first factor at which the kind differs;

(ii) V2: `alice_marginal_trace_rounds` where an addition of the marginal's trace rounds; in the general format `alice_sector_marginal_zero`, naming the sector, where a block of the marginal is zero beside a measurement slot in ensemble form;

(iii) V3: `alice_povm_summation_rounds`, in the general format carrying the Alice sector; `bob_povm_summation_rounds`, carrying the sector and the outcome index at which the first inexact addition occurred, and in the general format the factor as well; in the general format `alice_ensemble_marginal_mismatch`, carrying the sector and the first differing entry, `alice_ensemble_sum_rounds`, carrying the sector and the index of the first inexact addition, and `ensemble_form_in_untrusted_mode` for a slot in ensemble form beside a marginal slot holding `none`;

(iv) V6: in the general format `constraint_scope_not_a_factor`, naming the entry;

(v) V7: `no_feasibility_witness`, naming, where a supplied witness was rejected on its map, the first position at which the two lists differ.

Motivated by Def. 6.1, Thm. 6.2, foundations/req:consumer-decides-exactly · accepted · *Code* certiqs_key.protocol_compiler.validation:validate

A check may stop at a failed prerequisite or predicate. Its transcript then reports only the decisions it took, in their execution order, and no outcome is inferred for a predicate it did not execute.

The counts of Theorem 6.2 (iv) are the complete applicable inventory of the decisions the checks take on well-typed data, not the number an execution that stops early performs.

Requirement 13.2 (Transcript of a check that stops early). A check of Definition 6.1 may stop after a failed prerequisite or predicate. Its transcript reports only the decisions the check actually took, in their execution order, and the engine infers no outcome for a predicate it did not execute.

Motivated by Def. 6.1, Thm. 6.2 · implemented (effective: accepted) · *Code* certiqs_key.protocol_compiler.validation:CheckOutcome

At a non-empty constraint list the engine tests the default candidate first and, if it fails, a supplied witness, emits neither, and refuses exactly when the default candidate fails and no supplied witness verifies.

Theorem 5.5 (ii) reduces D(vii) at a non-empty constraint list to exact decisions on at most two candidates, and check V7 of Definition 6.1 is where they are taken.

Requirement 13.3 (Default candidate before a supplied witness). Where the constraint list is non-empty, on a candidate declaration satisfying D(i)–D(iii) and D(vi), which checks V1–V3 and V6 of Definition 6.1 decide (a candidate failing one of them is refused there), the engine decides the conditions of (2.1), in the general format those of (2.3), first on the default candidate (5.2), in the general format (5.3), and, if it fails, on a supplied witness $\omega^\star$, read from the input `feasibility_witness`. It forms the default candidate only to test it, and it emits neither candidate. It refuses with `no_feasibility_witness` exactly when the default candidate fails and either no witness is supplied or the supplied one fails.

Motivated by Def. 2.1, Thm. 5.5, Def. 6.1 · implemented (effective: accepted) · *Code* certiqs_key.protocol_compiler.feasibility:check_feasibility

A refusal at the feasibility check is never reported to a user as a defect of the protocol: it means only that no feasible state was shown, and a well-formed declaration can be refused there.

Check V7 is the one check that can refuse a well-formed declaration (Theorem 6.2 (iii)): its refusal means that no feasible state was shown, not that none exists (Remark 6.3).

REQUIREMENT 13.4 (Reporting a refusal for want of a feasibility witness). The engine never reports a refusal with `no_feasibility_witness` to a user as a defect in the protocol.

Motivated by Thm. 6.2, Rem. 6.3 · accepted

The physical receiver family as presented is decided exactly admissible on every route, as one block on its physical receiver space, before any condition on the chain is decided. A failure is refused with its own reason, naming the first failure.

Clause (iv) of Definition 3.2 reads the physical receiver family as presented like every other declared family, so nothing of its exact admissibility is left to whoever presents the chain. On the Band-A route the same family is the one the link delivers, emitted as it is unless a setting-dependent choice weighting is declared, and check V3 decides it, or its weighting, again (Theorem 8.2 (ii)).

REQUIREMENT 13.5 (Decision on the presented physical receiver family). On every route the engine decides, on the declared numbers, the condition clause (iv) of Definition 3.2 places on the physical receiver family as presented, in the general format on each measured factor's: presented directly, (S), (H), (P) and (C) of Definition 1.3 on its physical receiver space as one block, in the declared order of its alphabet; presented as a choice-weighted family, condition (S) on its own declared-order sum, after the decision of Requirement 12.1 on its conditional families and the representability decision on the products formed from them. It decides it before any condition of Definition 7.1 is decided on the chain, and refuses a failure with `physical_family_not_admissible`, naming the measured factor, the predicate and the index of the first failure (for (S), the index of the first inexact addition). The decision stops at the first failure.

Motivated by Def. 1.3, Def. 3.2, Def. 4.1, Def. 7.1, Thm. 8.2, Req. 12.1 · accepted · *Code* certiqs_key.protocol_compiler.compile:decide_physical_family

A chain failing a condition on it is refused where it is consumed, before a declaration exists, with a reason naming the condition; the licensed-only and source-mode rules have reasons of their own. No refusal emits a declaration.

Theorem 7.5 (i) and (i′) make (C1)–(C5), (S1)–(S4), (A1)–(A3), the licensed-only rule and the declared-source-mode rule decidable by finite computation on the presented record. The grounds of the two source-segment rules are

Remark 7.3.

Requirement 13.6 (Refusals of the interface conditions on a chain). The engine decides the interface conditions on a chain of Definition 7.1 where the chain is consumed, before a declaration exists, and refuses a chain that fails one; no refusal emits a declaration.

(i) A chain failing one of (C1)–(C5) or (S1)–(S4) is refused with `chain_interface_condition_failed`, naming the condition and, for (C1)–(C5) in the general format, the factor, and, for (C4) and (S3), the outcome or signal index at which the first entry disagreed. A non-empty source segment failing one of (A1)–(A3) is refused.

(ii) Beside a non-empty source segment, an entry declared general is refused with `general_entry_beside_source_segment`, naming the entry.

(iii) A declaration carrying a non-empty source segment in untrusted-source mode is refused with `source_segment_in_untrusted_mode`.

Motivated by Def. 7.1, Rem. 7.3, Thm. 7.5 · accepted · *Code* certiqs_key.protocol_compiler.chain:check_chain

A specification on the Band-A route is refused unless it has one measured factor, a two-dimensional receiver system and transmitted system, and a projective declared receiver measurement. The route then never rests on an ideal-receiver premise its own declared data contradict.

On the Band-A route the residual physical content is Assumption 10.8, an ideal projective measurement on a qubit (Theorem 8.2 (iv)), while Theorem 8.2 itself holds for any declared dimension and any exactly admissible family. Part of that assumption is a property of the declared data: the number of measured factors, the dimensions and the projectivity of the declared family. The engine decides that part, so that the route is never taken for a declaration that contradicts the premise it rests on. Exact admissibility of the same family is decided first, on every route (Requirement 13.5). For a family presented as a choice-weighted family, Assumption 10.8 (i) is read of each conditional family: a public choice of known bias followed by an ideal projective measurement. The weighted family itself is not projective when a weight is below one, and the same measurement presented directly, as a passive basis choice, is refused on purpose: a passive choice is made by an optical element, which the idealisation does not describe.

Requirement 13.7 (Band A only within the ideal qubit receiver). The engine recognises the Band A route on the record, reading the shape that condition (b) of Definition 8.1 gives the chain and the sector clause of its (a), for a specification whose source segment is empty, whose emitted sector set (in the general format, the joint one) is a singleton, and in which, for some measured factor, the receiver segment is one link with status exact, equal

source and target descriptors, a channel that, where the link supplies one as data, is exactly the identity map on its system, an empty constraint slot and an empty licence slot; it decides the link condition on the chain record of every measured factor. The clauses of (b) this recognition leaves open — that the link supplies its channel as data, its outcome correspondence and its target POVM — are decided by Requirement 13.8. It refuses such a specification, before a declaration is emitted, with `band_a_outside_ideal_receiver`, naming the first clause below that fails and, for (iii), the choice value where there is one and the outcome index of the first element that is not a projector, unless

(i) the specification declares exactly one measured factor;

(ii) that factor's physical receiver system has dimension two and, in trusted-source mode, the transmitted system of the transmitter artefact (Definition 3.1 (a)) has dimension two; and

(iii) that factor's physical receiver family as presented is projective, decided exactly on the declared numbers: presented directly, every element satisfies $F_w^{\mathrm{ph}} F_w^{\mathrm{ph}} = F_w^{\mathrm{ph}}$; presented as a choice-weighted family, every element of every declared conditional family satisfies $N_{u|t} N_{u|t} = N_{u|t}$ (Definition 4.1).

Reading: The idealised Band-A route is taken only for a receiver that can be the ideal qubit measurement its premise A-IDEAL describes: one measuring station, a two-dimensional measured system and, with a trusted source, a two-dimensional transmitted system, and a declared measurement made of projectors (per basis, where a basis choice is declared). Anything else is refused rather than analysed under a premise its own numbers already contradict; such a receiver is declared through the receiver module and a squashing reduction.

Motivated by Def. 3.1, Def. 4.1, Def. 8.1, Thm. 8.2, Ass. 10.8, Req. 13.5, Req. 13.8 · *Used by* Req. 13.8 · accepted · *Code* certiqs_key.protocol_compiler.declared_premises:decide_band_a_ideal_receiver

On the Band-A route the one receiver link must supply the identity channel as data and keep the outcomes as they are, and the pullback identity then decides that the measurement the link delivers is the declared one. A specification failing any of this is refused before a declaration exists.

Theorem 8.2 (ii) and (v) read the family the link delivers as the declared one and the outcome correspondence as the identity, and Requirement 13.10 (i) omits the link-soundness row of Assumption 10.6 on that strength. A link record carries its channel, its outcome correspondence and its target POVM as separate data, and where the link only names its channel the pullback identity (C4) is not decidable (Definition 7.1); on such a record nothing would tie the measurement the key length is computed for to the one declared. So the route is taken only where all three are decided on the record.

Requirement 13.8 (Band A only on an identity link that delivers the declared family). For a specification that Requirement 13.7 recognises on

the Band A route and does not refuse, the engine decides on the chain record of its one measured factor, before a declaration is emitted, that

(i) the one link of that factor's receiver segment supplies its channel as data, which that recognition has then decided to be exactly the identity map on its system; and

(ii) the outcome correspondence of that segment is the identity: the physical receiver alphabet $\mathcal{V}^{\mathrm{ph}}$ and the alphabet $\mathcal{V}$ of the link's target POVM are one set, and $\gamma(w) = w$ for every $w \in \mathcal{V}^{\mathrm{ph}}$;

and refuses a specification failing either, before a declaration is emitted, with `band_a_link_not_identity`, naming the first clause that fails. On a specification passing both, condition (C4) of Definition 7.1 is decided, as on every chain whose links supply their channel as data, and reads $F_v = F_v^{\mathrm{ph}}$ for every $v \in \mathcal{V}$, entry for entry, $\{F_w^{\mathrm{ph}}\}$ being the physical receiver family as presented and, for a choice-weighted family, the family of products Definition 4.1 forms; a failure of it is refused as every failure of (C4) is (Requirement 13.6). A declaration emitted from a specification so recognised therefore satisfies Definition 8.1 (b) on its record — its one link the identity, supplied as data, its outcome correspondence the identity and its target POVM the declared family — and the rules that read the route, the ledger's among them (Requirement 13.10), read it of such a declaration alone.

Reading: The idealised Band-A route is taken only when its one reduction step is the identity map given as numbers, keeps your outcomes as they are, and hands on exactly the measurement you declared. The engine checks all three on the record and refuses anything else, so on this route the key length is always computed for the measurement your declaration names, with any declared public choice applied to it, and the premise that the reduction step is sound is proved rather than left to you.

Motivated by Def. 4.1, Def. 7.1, Def. 8.1, Thm. 8.2, Ass. 10.6, Req. 13.6, Req. 13.7, Req. 13.10 · implemented (effective: accepted) · *Code* certiqs_key.protocol_compiler.declared_premises:decide_band_a_identity_link

The emitted chain record carries the source segment, the receiver segment and, on every link, the constraint slot and the licence slot, whether or not they are empty, Band A included.

The order rule (X1) of Definition 7.1 is carried by the shape of the record, two segments, and the constraint entries each link transports are read off its slots (C5); the reason a source link may not follow a receiver link is Remark "Recorded stage order, immaterial to soundness for licensed lists" (reductions/rem:ordering).

Requirement 13.9 (Emission of both chain segments and every link slot). The engine emits the chain record (`reduction_chain_record`) with both of its segments, the source segment and the receiver segment, and on every link with both its constraint slot and its licence slot, each whether or not

it is empty. This includes Band A, where the source segment is empty and the receiver segment is one identity link with both slots empty.

Motivated by Def. 7.1, reductions/rem:ordering · implemented · *Code* certiqs_key. protocol_compiler.compile:chain_record

The ledger omits A-LINK only on Band A and where every measured factor's receiver segment is the single exact instance link of a reductions artefact of the same run, with the identity outcome correspondence. That provenance is the run's, never read from the specification.

Theorem 7.5 (v) names the soundness of each link as an obligation of the chain's producer, and Remark 7.7 records the chains for which the producer's own theorems discharge it. A specification is a record (Definition 3.1 (c)): its link types and status tags are data that a specification written by hand can carry equally, so whether a chain is one the reductions module emitted cannot be read off it. Condition (ii) below is the shape that the theorems this requirement uses discharge. An instance link followed by a further link squashes another model (a link of Definition "The domination link" (reductions/domination/def:domination-link)), and on such a chain the row stays. At several measured factors whose segments are each that factor's instance link alone, the joint transfer is Theorem "The emission of several measuring parties" (reductions/thm:emitted-product) (ii) and (iii), the apparatus measuring the product of the factors' families by A-MEAS (Assumption "Each measuring party's device as one POVM per round" (reductions/ass:meas)) (iv). A setting-dependent weighting applied after a factor's segment leaves the chain record as it is (Remark 4.8), so such a factor meets condition (ii) as an unweighted one does, and the weighted relation is Lemma "Linearity: post-processing, private settings and mixtures" (reductions/lem:linear) (i). At Band A the discharge is Theorem 8.2 (v), whose hypothesis, Definition 8.1 (b), is decided on the record (Requirement 13.8).

Requirement 13.10 (Omission of the link-soundness row only on a proved chain). The engine omits the row of Assumption 10.6 from the assumption ledger of a compilation only (i) on the Band A route, for a declaration whose record satisfies Definition 8.1 (b) as Requirements 13.7 and 13.8 decide it — the one link the identity, supplied as data, its outcome correspondence the identity and its target POVM, by the decided (C4), the presented physical family — where Theorem 8.2 (v) proves every clause of it; or (ii) where the source segment is empty and, for every measured factor of the specification, all of the following hold: its receiver segment consists of one link, the instance link of an artefact that the reductions module emitted in the same run, with status *exact* and no channel supplied as data; the outcome correspondence of the segment is the identity on that factor's target alphabet; and that factor's physical receiver family as presented is exactly the compression $\{\iota^\dagger M'^{\mathrm{ph}}_v \iota\}_v$ of that artefact's declared model onto the presented physical receiver space, ι its isometric inclusion into the physical system, or

the declared model itself. At several measured factors the omission rests in addition on A-MEAS (Assumption "Each measuring party's device as one POVM per round" (reductions/ass:meas)) (iv), that the round's physical measurement is the product of the factors' measurements. On either route, at a factor whose delivered family is weighted after its segment, in the setting-dependent form of Definition 4.1, the omission rests in addition on Lemma "Linearity: post-processing, private settings and mixtures" (reductions/lem:linear) (i): the weighting of the family the segment's link delivers and the identically weighted physical family stand in the squashing relation under the same channel, so the transfer applies to the weighted protocol as to the unweighted one; on route (i) the channel is the identity and the two weighted families are the same matrices (Theorem 8.2 (v)). That each link is that artefact's instance link and that each presented family is that compression is established by the part of the engine that ran the reductions module and docked its artefacts, only for an instance for which Remark 7.7 names the claim showing the docked family to be that compression, and is passed to the compiler beside the specification (`chain_provenance`); the compiler never reads it from the specification. The compiler decides, on the presented chain, the parts of (ii) that are data: for every measured factor the length of its segment, the status tag, the channel slot and the outcome correspondence (Definition 7.1, (C2)), and the length of the source segment. Where condition (ii) is not passed, or a part of it decided on the chain fails, the row is emitted.

Motivated by Def. 3.1, Def. 4.1, Rem. 4.8, Def. 7.1, Thm. 7.5, Rem. 7.7, Def. 8.1, Thm. 8.2, Ass. 10.6, Req. 13.7, Req. 13.8, reductions/cor:entries, reductions/domination/def:domination-link, reductions/lem:linear, reductions/thm:chain, reductions/thm:emit, reductions/thm:emitted-product, reductions/thm:licence, reductions/thm:transfer · *Assumes* A-MEAS · implemented (effective: accepted) · *Code* certiqs_key.protocol_compiler.compile:link_row_discharged

Where the chains of one or more measured factors carry a reference companion, the compiler forms beside the declaration one companion replacing each, by the same map, weighted exactly with its factor enclosures, decides none of its predicates and emits it evaluation-only, with no run of its own.

The companion a reductions emission carries beside its artefact (Requirement "Emitting the reference companion beside the artefact" (reductions/req:reference-companion)) is the same receiver segment run on the reference. Compiling it by the same map, from the same specification, gives a tuple that differs from the declaration in the measured families of the factors it replaces and the entries of their last links and in nothing else (Proposition "The compiled reference companion is an admissible declaration" (meat-core/prop:reference-companion)). Those families are known through enclosures and are off the binary64 grid, so the checks this module decides on emitted binary64 numbers, the validator's (Definition 6.1) and the chain conditions' (Definition 7.1), do not apply to them; that each is a POVM

is imported from the reductions corollary, and the consumer that evaluates it admits it under its own rule (Requirement "Admission of the reference companion beside its declaration" (meat-core/req:companion-admission)).

Requirement 13.11 (The reference companion compiled beside the declaration). Where the last link of the receiver segment of one or more measured factors was emitted with a reference companion (Requirement "Emitting the reference companion beside the artefact" (reductions/req:reference-companion)), the engine emits, beside the declaration $\mathfrak{D}$ it compiles, one *compiled reference companion* $\mathfrak{D}^{\mathrm{ref}}$, in `reference_companion`:

(i) it is the tuple the compilation map (Definition 3.3) gives from the same specification with, for every measured factor whose last link was so emitted, that link's target family replaced by its companion's and that link's entries replaced by its companion's relaxed entries (Lemma "A rounded constraint entry with its constant relaxed" (reductions/lem:row-relaxation)), those of the entries requested of the link included where the companion carries them, and none in place of a requested entry the companion does not carry (Requirement "Emitting the reference companion beside the artefact" (reductions/req:reference-companion) (iii)); the engine forms it by the code path that forms $\mathfrak{D}$, every other slot read from the same data, the family and the entries of every other measured factor's last link among them;

(ii) where a measured factor's family is weighted after its chain, by that factor's weighting (Definition 4.1), every product $P(f \mid t(v))\, F_v$ is formed exactly from the companion's enclosure: the centre multiplied by the weight and every radius by its modulus, so that the result encloses the product of the weight and the companion's element. Where the companion carries a factor enclosure (Definition "Factor enclosure of a block" (foundations/def:factor-enclosure)) of a block of F_v (Requirement "Emitting the reference companion beside the artefact" (reductions/req:reference-companion) (vii)), the block of the product carries, beside its entrywise enclosure, the one of Lemma 4.3: the same box, its centres and radii unaltered, and the core multiplied by the weight in exact rational arithmetic. Where the family is not weighted after its chain, every factor enclosure the companion carries is passed to $\mathfrak{D}^{\mathrm{ref}}$ unaltered;

(iii) the engine decides on $\mathfrak{D}^{\mathrm{ref}}$ none of the predicates it decides on the binary64 numbers of $\mathfrak{D}$: no Hermiticity, positivity or completeness of the family, no completeness sum in a declared order (Definition 1.3), no chain condition and no check of the validator; it passes the companion's entries through as Requirement 11.3 passes every entry;

(iv) it emits no bundle of a run of its own from $\mathfrak{D}^{\mathrm{ref}}$ and marks it evaluation-

only, a declaration with no run of its own, with a reference to the declaration $\mathfrak{D}$ it is the companion of and the companion's record carried unchanged.

Where no measured factor's last link carries a companion, nothing of this requirement applies, and the run forms its first-order number on $\mathfrak{D}$ alone, its blocks resting as the records of the artefact say.

Motivated by Def. 1.1, Def. 1.3, Def. 3.3, Def. 4.1, Lem. 4.2, Lem. 4.3, Def. 6.1, Def. 7.1, Req. 11.3, foundations/def:factor-enclosure, meat-core/prop:reference-companion, meat-core/req:companion-admission, reductions/cor:reference-companion, reductions/lem:row-relaxation, reductions/req:reference-companion · *Used by* meat-core/req:companion-admission · implemented (effective: accepted) · *Code* certiqs_key.protocol_compiler.companion:compile_reference_companion

14 Requirements: the emission gates

The engine emits the eligibility flag true only for a key-length-eligible declaration and produces, for each gate failure, the outcome the gate outcome table fixes. It tests attestations for presence and never for truth, and records a vacuous pass of the licensed-class gate as vacuous.

Imports transmitter/req:licence-scope

The emitted flag reads true only for a declaration that passes all three gates of Definition 9.1. A declaration marked false may be explored numerically but backs no key-length claim.

Theorem 9.4 fixes, for each gate failure, whether the outcome is a refusal or an emission that is not key-length-eligible; the flag an emitted declaration carries never reads true for one that fails a gate.

Requirement 14.1 (The key-length eligibility flag). The engine emits the flag `key_length_eligible` true for a declaration only if that declaration is key-length-eligible (Definition 9.1).

Motivated by Def. 9.1, Thm. 9.4 · implemented (effective: accepted) · *Code* certiqs_key.protocol_compiler.gates:resolve_gates

A failing check or a failing certificate is refused with the check or the condition named, whatever else fails; an unlicensed list or an absent attack-set licence alone is emitted marked not key-length-eligible. The engine never supplies a missing attestation or proceeds as though it had one.

The gate outcome table (Definition 9.3) fixes which of the two outcomes each failure has, as Theorem 9.4 (ii) states; this requirement names the strings and the flag that carry them. A check that fails, so that no declaration is shown to be well-formed, or a certificate that does not verify, is a refusal; a well-formed declaration on verified data whose key-length claim is unsupported is emitted and marked. The transmitter refuses a request whose

certificate fails a condition of its form (Requirement "Refusal of a certificate that fails a decided condition" (transmitter/req:failing-certificate)); gate (G2) re-decides the conditions all the same, a consumer deciding the data it is handed (Requirement "Exact decision by every consumer" (foundations/req:consumer-decides-exactly)).

REQUIREMENT 14.2 (Outcome of each gate failure). For each gate failure the engine produces the outcome the gate outcome table (Definition 9.3) fixes for it, and no other; where the failures of one declaration meet the premises of a refusal row and of an emission row both, it produces the refusal:

(i) row 1, a check that fails: a refusal naming the check, which in the general format covers `ensemble_form_in_untrusted_mode`, `alice_ensemble_marginal_mismatch`, `alice_ensemble_sum_rounds`, `alice_sector_marginal_zero` and `constraint_scope_not_a_factor`;

(ii) row 3, trusted-source mode at any d_A with a certificate present and failing a condition of gate (G2) for its form: a refusal naming the condition, an ensemble presentation beside a POVM-form declaration being refused with `ensemble_presentation_without_ensemble_form`;

(iii) rows 5 and 6, a constraint list not in the licensed class or the attack-set licence record `attack_set_license` absent: an emission with `key_length_eligible` false.

A trusted-mode specification at $d_A \geq 2$ whose certificate is absent does not reach this module. The mode is read off the transmitter artefact's marginal slot (Definition 3.3), and the transmitter refuses a trusted-mode request at $d_A \geq 2$ that carries no certificate before any artefact is emitted, with `trusted_mode_without_certificate` (Requirement "Certificate and attestation demanded only at $d_A \geq 2$" (transmitter/req:licence-scope); the modes are defined in Definition "Source specification" (transmitter/def:source-spec)). So an artefact in trusted mode at $d_A \geq 2$ carries the record gate (G2) tests for, and the two rows of the table that describe a certificate-absent case — rows 2 and 4, at $m = 0$ and at $m \geq 1$ — are *unreachable through the transmitter artefact.* At $d_A = 1$ the gate demands no certificate at all, the slot holds the [1] the transmitter filled it with (Requirement "The unique marginal served at $d_A = 1$" (transmitter/req:marginal-fill)), and one supplied there and failing is refused by row 3, its scope being every d_A. Rows 2 and 4 are unreachable in the general format for the same reason, which reads the transmitter's refusal and no property of the general format. The engine never supplies a missing attestation and never proceeds as though it had one.

Motivated by Def. 3.3, Def. 9.1, Def. 9.3, Thm. 9.4,
foundations/req:consumer-decides-exactly, transmitter/def:source-spec,
transmitter/req:failing-certificate, transmitter/req:licence-scope, transmitter/req:marginal-fill · accepted · *Code* certiqs_key.protocol_compiler.gates:resolve_gates

The retention attestation (SR3) behind gate (G2) and the attack-set licence behind gate (G3)(b) are tested for presence and never for truth. The engine never supplies one and never verifies one.

Neither attestation is a property of the emitted matrices, so no function of them decides it (Theorem 9.4 (i)); what the engine can do is record whether it was supplied.

REQUIREMENT 14.3 (Attestations tested for presence only). The engine tests the attestations of Definition 9.1 for presence only, never for truth: at gate (G2) the clause (SR3) of the source-replacement licence record `source_replacement_license` (Definition "Source-replacement certificate" (transmitter/def:srcert)), in every form of the certificate, and at gate (G3)(b) the attack-set licence record `attack_set_license`, the attestation of Assumption 10.7. It never supplies an attestation and never verifies one.

Motivated by Def. 9.1, Thm. 9.4, Ass. 10.7,
transmitter/def:srcert · accepted · *Code* certiqs_key.protocol_compiler.gates:resolve_gates

In untrusted-source mode gate (G3)(a) passes by construction, so the gate report records it as passed vacuously and not as passed. A vacuous pass is never reported as evidence of anything.

In untrusted-source mode the constraint gate rests on the attack-set licence alone (Remark 9.5); the gate report must not make its type-check half look like a second check.

REQUIREMENT 14.4 (A vacuous pass recorded as vacuous). Where gate (G3)(a) of Definition 9.1 holds vacuously, in untrusted-source mode at any d_A, the engine records it in the gate report `gate_report` as *passed vacuously* rather than *passed*, and it never reports the vacuous pass as evidence of anything.

Motivated by Def. 9.1,
Rem. 9.5 · accepted · *Code* certiqs_key.protocol_compiler.gates:resolve_gates

Interface

The declared inputs and outputs of the module and its submodules: each key with its symbol, when it is present and the claims that fix it. The domains, types and refusal rules are in the interface files themselves.

Inputs: Protocol compiler (protocol-compiler)

From `interface.yaml`.

Key and symbol	Presence	Fixed by
`transmitter_artifact` Spec_src	always	Def. 3.1, Def. 3.3, transmitter/rem:da1, Def. 9.1, Thm. 9.4, Req. 14.2
`receiver_povm` {F^ph_w}_{w in V^ph}	always; in Band A it IS the family the link delivers, the link's 'target_povm' being decided equal to it entry for entry (req:band-a-identity-link), and so the emitted family unless a 'choice_weighting' is declared, which then weights it; it is declared directly (thm:band-a)	Def. 3.1, Def. 3.2, Req. 13.5, Def. 6.1, Thm. 7.5, Thm. 8.2, Req. 13.8
`public_choices` (T_pub, q, {N_{u|t}})	optional; one such record per party, either possibly absent	Def. 4.1, Def. 3.3, Ass. 10.4, Def. 3.2
`choice_weighting` (F, t: V -> T, P(f | t))	OPTIONAL, and per measured factor: at most one per factor, of either party (def:spec (b), (b')); its choice is drawn by a party whose readout records that factor's setting component (ass:coin (ii)). Absent is the ordinary case: with no such record the emitted family is the chain's {F_v} unchanged (rem:relabel, the paragraph "With no setting-dependent choice declared").	Def. 4.1, Def. 3.1, Def. 3.3, Thm. 4.4, Rem. 4.8, Ass. 10.4
`reduction_chain` Chain = (Ch_src, Ch_rec)	always	Def. 3.1, Def. 7.1, transmitter/def: source-segment, Thm. 7.5, Def. 8.1, Thm. 8.2
`target_sectors` Smap = (S, (d_s)_s, block kind)	always	Def. 1.1, Def. 6.1, Def. 7.1, Def. 1.5

Key and symbol	Presence	Fixed by
`target_povm` {F_v}_{v in V}	always	Def. 3.1, Def. 7.1, Def. 3.3, Def. 1.3, Def. 6.1, Thm. 7.5, Def. 8.1, Req. 13.8
`public_structure` (Party, a_A, a_B, sigma, r^0_h, h, phi, Chat)	always	Def. 3.1, Rem. 5.2, Thm. 5.1, Def. 3.3
`constraint_entries` {(hat O_k, bowtie_k, rhs_k)}	always; m >= 0, EMPTY IS THE DEFAULT ROUTE	Def. 3.1, Def. 2.5, Def. 1.1, Def. 6.1, Def. 3.3
`attack_set_license`	optional; REQUIRED by gate (G3)(b) whenever m >= 1 and a key-length-eligible declaration is wanted	Def. 9.1, Thm. 9.4
`feasibility_witness` omega*	optional; required by branch (c) of check V7 when m >= 1 and the default candidate fails	Def. 6.1, Thm. 5.5, Def. 3.1
`chain_provenance`	optional; passed only by the part of the engine that ran the reductions module and docked its artefacts into the chain, and absent on every other route	Req. 13.10, Rem. 7.7, Ass. 10.6
`alice_sector_factor` (T, (d_A_t)_t) with a party label	general format only; absent is |T| = 1 with A_t0 = A, which is the basic format	Def. 1.1, Def. 3.1, Def. 6.1, Def. 1.5
`alice_form` ‘povm‘ or ‘ensemble‘	general format only; absent is ‘povm‘, which is the basic format	Def. 1.1, Def. 3.1, Def. 3.3, Def. 6.1
`alice_ensemble` {N_x}_{x in X}	general format, ensemble form only; in place of ‘alice_povm‘	Def. 1.1, Def. 1.3, Def. 6.1, Thm. 5.1
`receiver_parts` ((party_f, physical system_f, {F^ph,f_w}, optional P_f))_{f in F}	general format only; a one-entry list IS the pair ‘receiver_povm‘ plus ‘choice_weighting‘ of the rows above, which is the basic format	Def. 3.1, Def. 3.3, Def. 4.1
`reduction_chain.receiver_segments` (R^f_rec)_{f in F}	general format only; one segment IS the ‘reduction_chain‘ row’s Ch_rec, which is the basic format	Def. 3.1, Def. 7.1, Thm. 7.5

Key and symbol	Presence	Fixed by
`reduction_chain.source_segment` Ch_src with its obligation records	always present, and EMPTY for every matrix-declared specification; a NON-EMPTY segment may be presented, and is then decided as ‘domain‘ and ‘what_this_module_decides_on_it‘ say	Def. 3.1, Def. 7.1, transmitter/def: source-segment, Thm. 7.5
`public_structure.transcript` ((pi_m, C_m, a_m))_{m = 1..L}	general format only; L = 2 with pi = (A, B) and a_2 independent of kappa_1 IS the pair ‘announce_alice‘, ‘announce_bob‘, which is the basic format	Def. 1.1, Def. 3.1, Thm. 5.1, Def. 1.5
`constraint_entries.scope_and_tag` (O_k, rel_k, rhs_k; f_k)	general format only; one measured factor and no ‘party_local‘ tag is the basic format, where the scope is that factor and the two tags are the two of def:homog	Def. 1.1, Def. 2.5, Def. 3.1, Def. 6.1, Def. 9.1
`feasibility_witness.joint` omega*	general format only; |T| = 1 is the ‘feasibility_witness‘ row above	Def. 6.1, Thm. 5.5, Def. 3.1

Outputs: Protocol compiler (protocol-compiler)

From `interface.yaml`.

Key and symbol	Presence	Fixed by
`alice_dim` d_A	always	Def. 1.1, Def. 3.1
`alice_marginal` tau_A, or null	always	Def. 1.1, Def. 1.5, Def. 6.1, Def. 3.3
`source_replacement_license`	emitted exactly when trusted mode is emitted at d_A >= 2	Def. 9.1, Thm. 9.4, transmitter/ass:src, transmitter/def: srcert, Req. 11.7
`party_partition` Party	always	Def. 1.1, Def. 3.1, Def. 1.5
`sector_index_set` S	always	Def. 1.1, Req. 11.1, Def. 1.5, Def. 6.1
`sector_dims` (d_s)_s	always	Def. 1.1, Def. 6.1
`sector_block_kind`	always	Def. 1.1, Req. 11.1
`alice_povm` {M_x}_{x in X}	always	Def. 1.1, Def. 1.5, Def. 6.1, Def. 1.3

Key and symbol	Presence	Fixed by
`bob_povm` {F_v}_{v in V}, or {N_(v,f)}_{(v,f) in V_P}	always	Def. 1.1, Def. 1.5, Def. 1.3, Def. 6.1, Def. 4.1, Thm. 4.4, Def. 3.3
`reference_companion` D^ref	exactly where the last link of the receiver segment of one or more measured factors was emitted with a reference companion (reductions/req:reference-companion); one companion for all of them	Req. 13.11, Def. 3.3, Def. 4.1, Lem. 4.2, Lem. 4.3, meat-core/prop: reference-companion
`announce_alice` a_A	always	Def. 1.1, Def. 1.5
`announce_bob` a_B	always	Def. 1.1, Def. 1.5, Def. 3.3, Rem. 4.9
`accept_statistic` phi: Kann -> Chat	always	Def. 3.6, Def. 1.1, Def. 1.5, Rem. 3.8
`key_map` r: X x V -> Z	always	Def. 3.3, Thm. 5.1, Def. 1.5, Def. 6.1
`key_holder` h	always	Def. 1.1, Def. 1.5, Def. 6.1, Thm. 5.1
`constraint_list` CList	always; EMPTY IS THE DEFAULT ROUTE	Def. 1.1, Def. 3.3, Def. 1.5, Def. 2.5, Def. 6.1
`constraint_rhs_tag`	one per entry of CList	Def. 2.5, Def. 1.1, Def. 6.1
`attack_set_license`	present only if supplied	Def. 9.1, Thm. 9.4
`acceptance_alphabet` Chat	always	Def. 3.6, Def. 1.1
`label_index_map` idxm	always	Def. 3.3, Def. 1.5, Thm. 3.4, Def. 6.1, Def. 1.3
`reduction_chain_record` Chain	always	Def. 3.1, Def. 7.1, Thm. 7.5, Rem. 4.8, Req. 13.9
`choice_weighting_record` (F, t, P, the orders)	emitted exactly when a ‘choice_weighting‘ was declared at some measured factor	Def. 4.1, Def. 3.3, Req. 11.9, Rem. 4.8
`key_length_eligible`	always	Def. 9.1, Thm. 9.4
`gate_report`	always	Def. 9.1, Thm. 9.4, Rem. 9.5, Def. 1.3, Def. 6.1, Def. 7.1

Key and symbol	Presence	Fixed by
`refusal`	present exactly when the compiler refuses	Thm. 9.4, Def. 6.1, Def. 7.1
`assumption_ledger` A-...	always	—
`assumption_bindings`	always; one entry per ledger row, in its order; empty on a refusal	Ass. 10.1, Ass. 10.2, Ass. 10.3, Ass. 10.4, Ass. 10.6, Ass. 10.7, Ass. 10.8, transmitter/ass:ens, transmitter/ass:src, transmitter/decoy/ ass:wcp
`mathematical_spec_version`	always	—
`sector_factorisation` the ordered factor list of (DX1)	general format only; absent is one measured factor and no A factor	Def. 1.1, Def. 3.3, Def. 6.1
`alice_form` ‘povm‘ or ‘ensemble‘	general format only; absent is ‘povm‘	Def. 1.1, Def. 3.3
`alice_ensemble` {N_x}_{x in X}	general format, ensemble form only; in place of ‘alice_povm‘	Def. 1.1, Def. 1.3
`alice_marginal.blocks` (tau_A^(t))_t	general format with an A factor; |T| = 1 is the ‘alice_marginal‘ row	Def. 1.5, Def. 6.1
`bob_povm.per_factor` ({F^f_v_f})_{f in F}	general format only; one factor is the ‘bob_povm‘ row	Def. 1.1, Def. 3.3, Thm. 7.5
`transcript` ((pi_m, C_m, a_m))_{m = 1..L}	general format only; L = 2 is the pair ‘announce_alice‘, ‘announce_bob‘, which stays the emitted form of that case	Def. 1.1, Def. 3.3, Thm. 5.1
`constraint_scope` f_k	general format only; one measured factor is the scope of every entry	Def. 1.1, Def. 3.1, Def. 3.3
`constraint_rhs_tag.party_local` the third tag value	general format only	Def. 2.5, Def. 9.1, Thm. 9.4

Claim index

Claim	Title and identifier	Page
Definition 1.1	Protocol declaration protocol-compiler/def:declaration	16
Remark 1.2	Party labels in the general format protocol-compiler/rem:party-label-restriction	21
Definition 1.3	Exact admissibility of declared operator data protocol-compiler/def:exact	22
Remark 1.4	The summation condition beside completeness protocol-compiler/rem:summation-condition	24
Definition 1.5	Target conditions of a well-formed declaration protocol-compiler/def:target	25
Remark 1.6	Relation to the consuming engine's admission conditions protocol-compiler/rem:engine-admission	27
Definition 2.1	Feasible set protocol-compiler/def:feasible	28
Lemma 2.2	Hermiticity of the constraint values protocol-compiler/lem:hermitian-constraint-values	29
Remark 2.3	Hermiticity as a declared condition protocol-compiler/rem:wellposed	30
Remark 2.4	Restriction of the feasible set to block-diagonal states protocol-compiler/rem:block-restriction	31
Definition 2.5	Licensed constraint list protocol-compiler/def:homog	32
Remark 2.6	Soundness of a party-local entry protocol-compiler/rem:party-local-soundness	34
Example 2.7	Declaration of a two-party protocol with an untrusted source protocol-compiler/ex:two-party-shape	34
Definition 3.1	Protocol specification protocol-compiler/def:spec	36
Definition 3.2	Well-formed specification protocol-compiler/def:well-formed-spec	40
Definition 3.3	The compilation map protocol-compiler/def:compile	41
Theorem 3.4	Typing of an emitted declaration protocol-compiler/thm:typing	44
Remark 3.5	The raw-key map built from sifting and the key rule protocol-compiler/rem:rbuilt	47
Definition 3.6	Declared statistics functional protocol-compiler/def:phi	47

Claim	Title and identifier	Page
Theorem 3.7	Exact measurement families and the statistics functional protocol-compiler/thm:measurements-and-statistics	47
Remark 3.8	Totality of the acceptance-statistic map protocol-compiler/rem:phitotal	52
Definition 4.1	Choice-weighted families protocol-compiler/def:weighted	53
Lemma 4.2	Properties of the choice-weighted families protocol-compiler/lem:weighted	58
Lemma 4.3	A factor enclosure through a setting-dependent weighting protocol-compiler/lem:weighted-factor	60
Theorem 4.4	Encoding of public choices in the measurement operators protocol-compiler/thm:choice-encoding	61
Remark 4.5	Scope of the exact encoding protocol-compiler/rem:exactness	65
Remark 4.6	Deciding the conditional families protocol-compiler/rem:condfamily	66
Example 4.7	An exact weighted family built from incomplete conditional families protocol-compiler/ex:condfamily	68
Remark 4.8	The setting-dependent relabelling after the chain protocol-compiler/rem:relabel	69
Remark 4.9	The outcome alphabet is chosen upstream protocol-compiler/rem:alphabet	70
Theorem 5.1	Announcement locality, sifting and party computability protocol-compiler/thm:sifting-and-key	72
Remark 5.2	Necessity of the key-rule side condition protocol-compiler/rem:sidecond	74
Remark 5.3	Party computability read against the readouts protocol-compiler/rem:vacuous	74
Example 5.4	A three-message transcript protocol-compiler/ex:transcript	74
Theorem 5.5	Feasibility by an explicit or a verified candidate protocol-compiler/thm:feasibility	75
Remark 5.6	Feasibility by witness rather than by search protocol-compiler/rem:nosearch	79
Definition 6.1	The emitted validator protocol-compiler/def:validator	80
Theorem 6.2	Soundness, completeness and decidability of the validator protocol-compiler/thm:validator-soundness	86

Claim	Title and identifier	Page
Remark 6.3	The two directions of the validator protocol-compiler/rem:completeness	94
Definition 7.1	Interface conditions on a chain protocol-compiler/def:chaincond	96
Remark 7.2	Chain conditions here and in the reductions module protocol-compiler/rem:chaincond-reductions	100
Remark 7.3	Grounds of the source-segment rules protocol-compiler/rem:source-segment-rules	100
Remark 7.4	Chain conditions and validator checks protocol-compiler/rem:chain-and-validator	102
Theorem 7.5	Decided identities and named obligations of a consumed chain protocol-compiler/thm:chain-consumption	103
Remark 7.6	Direction of the outcome correspondence protocol-compiler/rem:coarse	109
Remark 7.7	Discharge of the chain obligations by the reductions module protocol-compiler/rem:obligations-discharged	110
Definition 8.1	Band-A specification protocol-compiler/def:banda	112
Theorem 8.2	The Band-A route protocol-compiler/thm:band-a	113
Definition 9.1	Key-length eligibility protocol-compiler/def:eligible	117
Remark 9.2	The source gate on each certificate form protocol-compiler/rem:gate-by-certificate-form	119
Definition 9.3	The gate outcome table protocol-compiler/def:gate-outcome-table	121
Theorem 9.4	The emission gates and their outcomes protocol-compiler/thm:gates	122
Remark 9.5	The licensed-class gate in untrusted-source mode and at $d_A = 1$ protocol-compiler/rem:degenerate	127
Proposition 9.6	Dropping a trusted marginal enlarges the feasible set protocol-compiler/prop:conservative	127
Remark 9.7	Dropping a marginal, never replacing it protocol-compiler/rem:notfallback	129
Assumption 10.1	Specification faithful to the apparatus protocol-compiler/ass:spec	130
Assumption 10.2	Alice's declared measurement where no source certificate is demanded protocol-compiler/ass:alice-meas	131
Assumption 10.3	Round homogeneity of the declaration protocol-compiler/ass:rhom	132

Claim	Title and identifier	Page
Assumption 10.4	Public choices as private coins of declared bias protocol-compiler/ass:coin	132
Remark 10.5	Scope of the private-coin assumption protocol-compiler/rem:coin-scope	133
Assumption 10.6	Soundness of every reduction link protocol-compiler/ass:link	134
Assumption 10.7	Attack-set licence protocol-compiler/ass:atk	135
Assumption 10.8	Ideal qubit receiver on the Band-A route protocol-compiler/ass:ideal	135
Requirement 11.1	Machine keys of the emitted declaration protocol-compiler/req:declaration-keys	137
Requirement 11.2	Operators on the measured system held as block lists protocol-compiler/req:operator-form	138
Requirement 11.3	Declared data passed through unchanged protocol-compiler/req:pass-through	138
Requirement 11.4	Per-factor decisions on the declared blocks protocol-compiler/req:derived-objects	139
Requirement 11.5	Reading the tag of a constraint entry protocol-compiler/req:tag-reading	139
Requirement 11.6	Exact decisions without a tolerance protocol-compiler/req:no-window	140
Requirement 11.7	The source-replacement licence passed on with the declaration protocol-compiler/req:source-licence-passed-on	140
Requirement 11.8	Letter-independent emission at a one-dimensional trusted A protocol-compiler/req:da1-source-identity	141
Requirement 11.9	Emission of the weighted families and the weighting record protocol-compiler/req:choice-weighting-record	142
Requirement 12.1	Decision on each declared conditional family protocol-compiler/req:conditional-family-decision	143
Requirement 12.2	Declared weights as binary64 input protocol-compiler/req:binary64-weights	143
Requirement 12.3	Preparation of user weights before the module boundary protocol-compiler/req:weight-preparation	144
Requirement 12.4	Setting-dependent weighting read through the setting only protocol-compiler/req:setting-only-weighting	144
Requirement 12.5	Decision on the declared weight list protocol-compiler/req:weight-list	145

Claim	Title and identifier	Page
Requirement 12.6	Stochastic matrix of a setting-dependent weighting protocol-compiler/req:stochastic-weights	145
Requirement 12.7	Emission decisions on the weighted products protocol-compiler/req:weighted-family-emission	145
Requirement 12.8	No lattice exponent and no snapping of the emission protocol-compiler/req:no-lattice-exponent	146
Requirement 12.9	No fixed fraction outside the measurement operators protocol-compiler/req:no-fixed-fraction	147
Requirement 13.1	Refusals of the validator's checks protocol-compiler/req:validator-refusals	148
Requirement 13.2	Transcript of a check that stops early protocol-compiler/req:validation-transcript	149
Requirement 13.3	Default candidate before a supplied witness protocol-compiler/req:feasibility-witness	149
Requirement 13.4	Reporting a refusal for want of a feasibility witness protocol-compiler/req:witness-refusal-meaning	150
Requirement 13.5	Decision on the presented physical receiver family protocol-compiler/req:physical-family-decision	150
Requirement 13.6	Refusals of the interface conditions on a chain protocol-compiler/req:chain-refusals	151
Requirement 13.7	Band A only within the ideal qubit receiver protocol-compiler/req:band-a-ideal-receiver	151
Requirement 13.8	Band A only on an identity link that delivers the declared family protocol-compiler/req:band-a-identity-link	152
Requirement 13.9	Emission of both chain segments and every link slot protocol-compiler/req:chain-record-slots	153
Requirement 13.10	Omission of the link-soundness row only on a proved chain protocol-compiler/req:link-row-provenance	154
Requirement 13.11	The reference companion compiled beside the declaration protocol-compiler/req:reference-companion-compiled	156
Requirement 14.1	The key-length eligibility flag protocol-compiler/req:eligibility-flag	157
Requirement 14.2	Outcome of each gate failure protocol-compiler/req:gate-outcomes	158
Requirement 14.3	Attestations tested for presence only protocol-compiler/req:attestation-presence	159
Requirement 14.4	A vacuous pass recorded as vacuous protocol-compiler/req:vacuous-gate-report	159

Index of defined terms

announcement map, 17

Band A, 112
basic format, 18

choice alphabet, 54
choice-weighted family, 53
compilation map, 41

default party partition, 26

ensemble form, 19

factor, 18
feasible set, 28

gate outcome table, 121
general, 32

homogeneous, 32

index map, 42
interface conditions on a chain, 96

joint sector, 19

key-length-eligible, 117
key-live, 18

licensed for the declared source mode, 32

lifted operator, 20

marginal slot, 17
measured factor, 18
message, 20

party partition, 17
party-local, 32
POVM form, 19
protocol declaration, 16
protocol specification, 36
public-choice alphabet, 53

readout map, 17

scope, 20
setting alphabet, 53
setting component, 54
setting-dependent form, 54
setting-independent form, 53
soundness condition of a party-local entry, 33
support of the weighting, 54
system tag, 18

transcript protocol, 20

well-formed declaration, 25
well-formed specification, 40

References

[1] Devashish Tupkary, Shlok Nahar, Amir Arqand, Ernest Y.-Z. Tan, and Norbert Lütkenhaus. A rigorous and complete security proof of decoy-state BB84 quantum key distribution, 2026.